Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

В.В. Смелов, Л. С. Мороз

Microsoft SQL Server 2008: основы Transact-SQL

Минск 2014

Оглавление

ПРЕДИСЛОВИЕ 3
Глава 1. ВВЕДЕНИЕ В СУБД MICROSOFT SQL SERVER 2008 5
Глава 2. СРЕДА SQL SERVER MANAGEMENT STUDIO 15
Глава 3. ОСНОВЫ ЯЗЫКА SQL 25
Глава 4. СОЗДАНИЕ И НАСТРОЙКА БАЗЫ ДАННЫХ 36
Глава 5. СОЗДАНИЕ, УДАЛЕНИЕ И МОДИФИКАЦИЯ ТАБЛИЦ 40
Глава 6. ПРИМЕНЕНИЕ ОПЕРАТОРА INSERT 76
Глава 7. ПРИМЕНЕНИЕ ОПЕРАТОРА SELECT 84
Глава 8. КОМБИНИРОВАНИЕ РЕЗУЛЬТАТОВ SELECT-ЗАПРОСОВ 142
Глава 9. ПРИМЕНЕНИЕ ОПЕРАТОРОВ DELETE И TRUNCATE 150
Глава 10. ПРИМЕНЕНИЕ ОПЕРАТОРА UPDATE 156
Глава 11. ПРИМЕНЕНИЕ ИНДЕКСОВ 163
Глава 12. ПРИМЕНЕНИЕ ПРЕДСТАВЛЕНИЙ 185
Глава 13. ОСНОВЫ ПРОГРАММИРОВАНИЯ НА T-SQL 193
Глава 14. ПРИМЕНЕНИЕ XML 259
ЛИТЕРАТУРА 278
Приложение. Таблицы базы данных BSTU 279

ПРЕДИСЛОВИЕ

Центральной компонентой современной информационной системы является база данных, представляющая собой специальным образом организованное на электронных носителях (как правило, это дисковые устройства) хранилище данных. В простейшем случае база данных может быть простым набором файлов операционной системы, но в большинстве случаев управление хранилищем осуществляется с помощью специализированного программного обеспечения – системы управления базами данных.

С момента появление первых систем управления базами данных прошло уже более полувека. За это время менялись подходы к модели представления данных (иерархическая, сетевая, реляционная, объектно-реляционная и объектно-ориентированная модели), к способу доступа к данным (файл-серверные, клиент-серверные, распределенные системы), развивались программные интерфейсы (ODBC, OLEDB, ADO, ADO.NET, JDBC), появился развитый инструментарий.

Современные системы управления базами данных представляют собой сложные программные комплексы, обеспечивающие надежное централизованное или распределенное хранение данных, одновременное обслуживание многих пользовательских запросов, допускающие непрерывный цикл доступа к данным. Усложнение систем управления базами данных повлекло появление новых профессий: проектировщиков, разработчиков, администраторов баз данных. Поддержка работоспособности базы данных является важнейшей самостоятельной задачей современной информационной системы.

Еще одной важной особенностью современных систем управления базами данных является наличие встроенных систем программирования, позволяющих в рамках базы данных создавать программные объекты: процедуры, функции, триггеры, типы данных и пр. Возможность программирования «на стороне базы данных» привела к новой тенденции в структуре программного обеспечения информационных систем, которая заключается в смещении программирования бизнес-логики в сторону систем управления базами данных. Другими словами, программный код, который раньше размещался в клиентской (по отношению к серверу базы данных) части приложения, теперь может быть размещен в его серверной части.

Последние пять лет лидерство в области разработки систем управления базами данных прочно удерживает «большая тройка» –Oracle Corp. (продукт Oracle RDBMS), IBM (DB/2) и Microsoft Corp. (Microsoft SQL Server). Именно в этих трех компаниях на настоящий момент разрабатываются и реализуются современные технологии управления данными.

Все три компании предлагают системы управления реляционными базами данных, в основе которых лежит табличное представление данных. Для доступа к таблицам с данными в таких системах применяется специальный язык запросов – SQL (Structured Query Language).

Язык SQL – формальный декларативный (отсутствуют операторы управления: переходов, ветвления и циклов) язык. Впервые SQL был разработан компанией IBM в начале 1970-х гг. В настоящий момент является общепризнанным стандартом (ISO/IEC9075) доступа к реляционным данным. При этом наиболее устоявшейся версией стандарта является редакция 1992 г., которую часто символически обозначают SQL92. Более подробно с теорией реляционных баз данных можно ознакомиться в источнике [1].

Предлагаемое учебно-методическое пособие посвящено основам программирования на языке Transact-SQL, представляющем собой процедурное расширение SQL. Интерпретация операторов программы (чаще ее называют сценарием или скриптом) на Transact-SQL выполняется сервером Microsoft SQL Server. Изложение ориентировано на версию сервера 2008 г. Microsoft SQL Server 2008 не различает языки SQL и Transact-SQL, но программист при написании сценария всегда может использовать такое подмножество языка Transact-SQL, которое оставит его в рамках SQL.

Пособие состоит из пятнадцати глав и приложения.

Первая глава является введением. Здесь разбирается архитектура системы управления базами данных Microsoft SQL Server 2008, кратко описываются ее основные компоненты, рассматривается набор инструментальных средств.

Вторая глава посвящена Microsoft SQL Server Management Studio – важнейшему инструментальному средству, представляющему собой графическое приложение-утилиту. Утилита позволяет конфигурировать и управлять всеми компонентами Microsoft SQL Server, и что особенно важно, включает в себя текстовый графический редактор, предоставляющий возможность создавать и выполнять сценарии на Transact-SQL.

В главах с третьей по двенадцатую описывается язык SQL, являющийся подмножеством языка Transact-SQL. Язык включает четыре подмножества операторов, позволяющих управлять (создавать, изменять и удалять) объектами сервера и баз данных, разграничивать к ним доступ, манипулировать табличными данными, а также управлять параллельной обработкой общих данных.

Тринадцатая глава посвящена основам программирования на Transact-SQL. В главе разбираются операторы языка Transact-SQL, расширяющие возможности SQL. Кроме того, здесь рассматриваются курсоры (механизм обработки результирующего набора, сформированного SELECT-запросом), транзакции (механизм управления параллельной обработкой), механизм TRY/CATCH обработки ошибок, а также разработка и применение программных объектов базы данных (процедур, функций и триггеров).

В четырнадцатой главе рассматриваются вопросы применения языка XML в базах данных Microsoft SQL Server 2008. Материал главы знакомит с методами преобразования результирующего набора в формат XML, методами преобразования XML-документа в последовательность строк таблицы. Кроме того, здесь разбираются принципы использования столбцов таблицы XML-типа.

Последняя пятнадцатая глава является практикумом, задания которого направлены на закрепление изученного читателем материала. Практикум состоит из четырнадцати практических работ. Каждая работа включает несколько заданий. Последнее задание каждой работы содержит перечень контрольных вопросов. Для выполнения любого задания вполне достаточно знаний, изложенных в данном пособии.

В приложении пособия расположен материал, необходимый для создания и заполнения таблиц базы данных, используемой при выполнении заданий практикума.

Синтаксис языка Transact-SQL не делает различия между большими и малыми буквами. Поэтому для удобства все ключевые слова Transact-SQL, встречающиеся в тексте пособия, пишутся большими буквами, а имена объектов базы данных при этом еще выделяются жирным шрифтом. В приведенных примерах ключевые слова пишутся малыми буквами, а имена объектов сервера и базы данных большими. Следует также обратить внимание на то, что в данном издании читатель не обнаружит материала, связанного с решением задач администрирования сервера Microsoft SQL Server 2008: настроек безопасности, копирования и восстановления баз данных, трассировки, аудита, мониторинга и анализа производительности и пр. С этой информацией можно ознакомиться в источниках литературы [3–5]. Не рассматриваются в пособии и принципы проектирования логической структуры реляционной БД. Наиболее подробно процесс проектирования описывается в уже ставшем классическим труде К. Дейта [1].

В тексте пособия используются следующие сокращения:

БД	база данных
СУБД	система управления базами данных
MSBO	Microsoft SQL Server Books Online
MSCM	Microsoft SQL Server Configuration Manager
MSS	Microsoft SQL Server 2008
MSMS	Microsoft SQL Server 2008 Management Studio
SNAC	SQL Native Access Client
SQL	Structured Query Language
T-SQL	Transact-SQL
XML	Extensible Markup Language

Глава 1. ВВЕДЕНИЕ В СУБД MICROSOFT SQL SERVER 2008

1.1. Общие сведения

Первая версия СУБД SQL Server 1.0 была представлена в 1989 г. как совместный проект компаний Ashton-Tate и Microsoft. Сервер работал в среде операционной системы MS-DOS.

После версии SQL Server 7.0 было принято другое обозначение версий – по году выпуска. На сегодняшний день единственным владельцем продукта является компания Microsoft, а наиболее распространены версии 2000, 2005, 2008, 2010 (версия 2008 R2) годов выпуска. Революционное изменение в архитектуре СУБД осуществилось при появлении сервера 2005, который стал использовать возможности платформы .NET Framework. Начиная с этой версии, все изменения сервера являются эволюционными (расширяющими возможности или повышающими производительность) и не требуют значительных усилий при миграции на более высокую версию.

Microsoft SQL Server 2008 поставляется в десяти редакциях. Наиболее простыми являются бесплатные редакции с названиями **Compact** и **Express**. Наиболее сложными – **Parallel Data Warehouse** и **Datacenter**. Самыми распространенными – **Standard** и **Enterprise**. Кроме того есть специализированные редакции: **Workgroup**, **Web**, **Developer** и **SQL Azure**. Кроме того, каждая редакция предполагает два выпуска, соответствующих выпускам (32/64) операционной системы Microsoft Windows. Возможности каждой редакции, совместимость выпусков Microsoft SQL Server 2008 с версиями и выпусками операционных систем Microsoft Windows описывается в пособии [3].

Дальнейшее изложение ориентировано на сервер Microsoft SQL Server 2008 R2 Enterprise Edition (MSS), который может быть установлен на компьютер, минимальные требования к которому отражены в табл. 1.1. На компьютере должна быть проинсталлирована операционная система Windows Server 2003 Standard SP2 или выше.

Таблица 1.1

Минимальные требования к оборудованию

Оборудование	Выпуск 32	Выпуск 64
Процессор	Pentium III и выше	Itanium, Opteron, Athlon, Xeon/Pentium EM64T
Тактовая частота	1,0 ГГц и выше	1,6 ГГц и выше
Оперативная память	512 МБ	512 МБ

Кроме того, для успешной инсталляции программного обеспечения MSS требуются предустановленные .NET Framework 3.5, Windows Installer 4.5 и Internet Explorer 6.

1.2. Архитектура

С точки зрения операционной системы MSS представляет собой набор Windows-сервисов. Просмотреть перечень установленных сервисов (служб) MSS можно с помощью консоли «Диспетчер сервера» (рис. 1.1) операционной системы Windows.

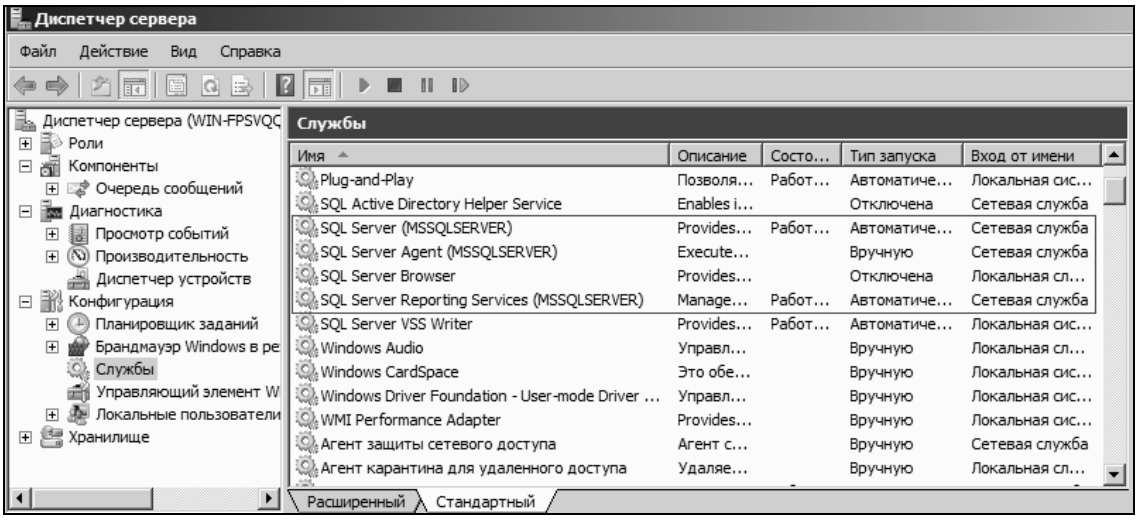


Рис. 1.1. Окно консоли «Диспетчер сервера», отражающее установленные сервисы MSS

В табл. 1.2 перечислены основные службы, входящие в состав MSS.

Таблица 1.2

Основные службы MSS

Наименование службы	Назначение
Database Engine	Управление реляционными БД
Analysis Services	Управление OLAP-кубами и интеллектуальный анализ данных
Integration Services	Поддержка решений по извлечению, преобразованию и загрузке данных
Reporting Services	Управление отчетами, построенными на основе SQL-запросов к реляционным БД
Full-Text Search	Управление полнотекстовым поиском
SQL Server Agent	Автоматизация административных задач
SQL Server Browser	Управление соединениями

Более подробно с назначением служб и других компонент MSS можно ознакомиться в издании [5]. Дальнейшее изложение в этом пособии будет посвящено службе **Database Engine**, представляющей собой ядро системы управления реляционной БД MSS. Более того, далее не будем различать понятия **Database Engine**, СУБД и MSS, если это специально не оговаривается.

Использование MSS подразумевает применение или разработку клиент-серверных приложений. При этом в качестве сервера выступает сам MSS, а в качестве клиента может быть либо утилита (специализированная программа), входящая в состав MSS, либо программа, разработанная на одном из языков программирования. В обоих случаях взаимодействие клиента и сервера может быть представлено так, как это изображено на следующей схеме (рис. 1.2).

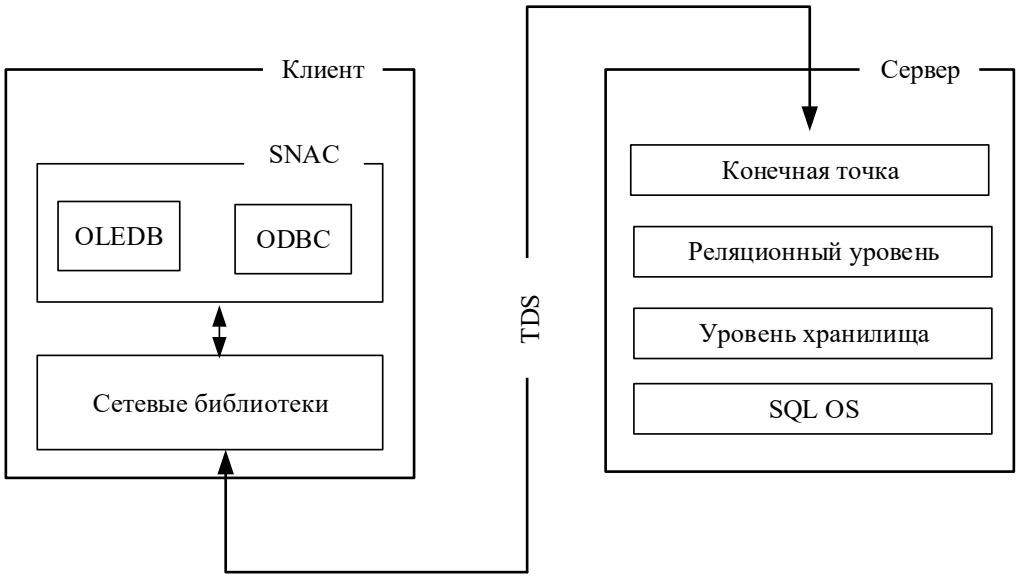


Рис. 1.2. Взаимодействие клиента и сервера MSS

Для взаимодействия клиентского приложения с сервером MSS служит специальная DLL-библиотека, называемая SNAC (SQL Native Access Client) и включающая в себя программные интерфейсы: провайдер OLEDB (Object Linking and Embedding, Database) и драйвер ODBC (Open Data Connectivity). Для передачи данных по компьютерной сети SNAC использует сетевые библиотеки MSS, с помощью которых обеспечивается передача данных в соответствии с протоколом прикладного уровня TDS (Tabular Data Stream).

Приложения, предназначенные для работы в среде .NET, должны использовать специальный программный интерфейс – ADO.NET, основанный на провайдерах данных (программная компонента, обеспечивающая соединение с сервером БД, выполнение команд и получение результатов их выполнения) платформы .NET Framework [5].

Для доступа Java-приложения в MSS служит программный интерфейс JDBC (Java Database Connectivity), в основе работы которого лежит JDBC-драйвер, обеспечивающий соединение и взаимодействие с сервером MSS [5].

В независимости от используемого программного интерфейса клиентское приложение осуществляет соединение со специальным объектом MSS – конечной точкой. При инсталляции MSS автоматически создается несколько конечных точек, позволяющих установить соединение с сервером с помощью нескольких сетевых протоколов (TCP/IP, Named Pipes, Shared Memory, HTTP и пр.). Более подробно о контрольных точках написано в источниках [3, 4, 5].

Программное обеспечение сервера условно можно разбить на три уровня: реляционный уровень, уровень хранилища данных и уровень взаимодействия с операционной системой (SQLOS).

Благодаря реляционному уровню база данных представляется пользователю (или программному интерфейсу) в виде набора реляционных объектов (таблиц, представлений, ограничений целостности и пр.).

Размещение, надежное хранение реляционных объектов и доступ (синхронизация, блокировка, кеширование и пр.) к ним регулируется программным обеспечением уровня хранилища.

Слой SQLOS обеспечивает взаимодействие MSS с операционной системой Windows. В этом смысле SQLOS можно представить как специализированную виртуальную машину, на которой работает MSS.

Для более детального знакомства с архитектурой MSS рекомендуется воспользоваться изданиями [3, 4, 5].

1.3. Инструментарий

В состав программного обеспечения СУБД MSS включен набор специализированных программ (обычно их называют утилитами), позволяющих различным категориям пользователей (системным администраторам, проектировщикам БД, разработчикам программного обеспечения, конечным пользователям) соединяться и взаимодействовать с MSS. Некоторые из них имеют графический интерфейс, другие работают в режиме командной строки.

1.3.1. Утилита SQL Server Books Online

Microsoft SQL Server Books Online (MSBO) – программное средство поддержки электронной документации MSS. На рис. 1.3 представлен скриншот главного окна MSBO.

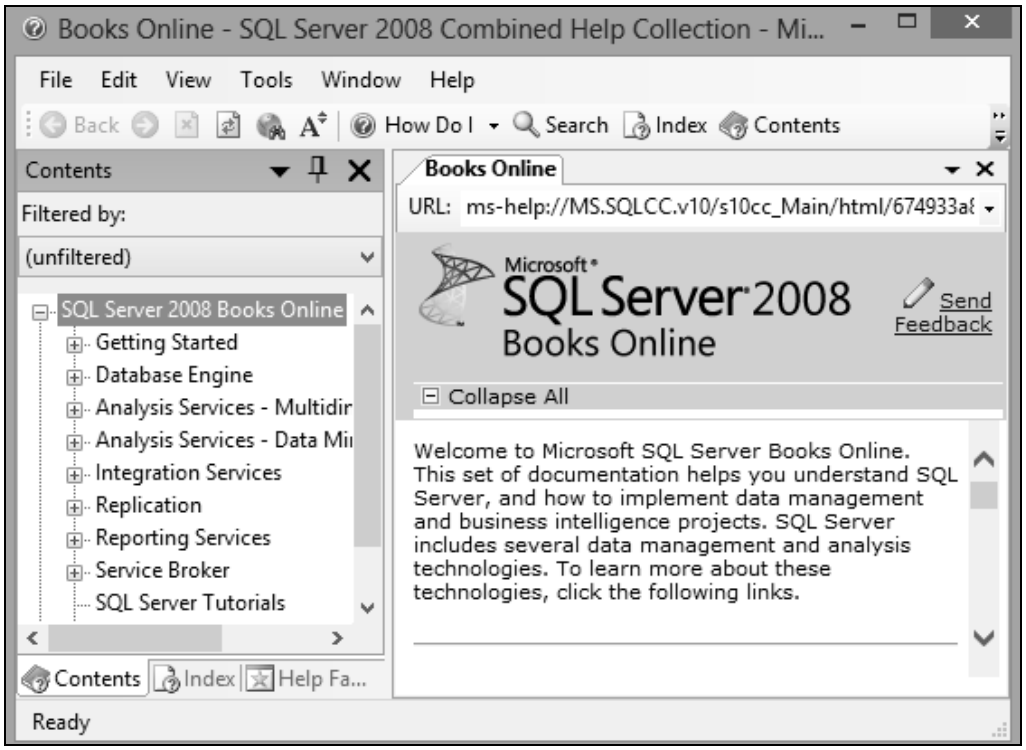


Рис. 1.3. Скриншот окна Microsoft SQL Server Books Online

Доступ к документации MSBO предоставляет несколькими способами: через оглавление, представленное в виде дерева; с помощью индекса, позволяющего отыскивать страницы документации по заданным ключевым словам. Кроме того MSBO может быть автоматически вызван с помощью меню или клавиши **F1** в других инструментах MSS для отображения страницы документации, соответствующей контексту выполнения.

1.3.2. Утилита SQL Server Management Studio

Утилита SQL Server Management Studio (MSMS) занимает особое место среди инструментов MSS. Она предоставляет пользователю графический интерфейс, позволяющий конфигурировать и администрировать все компоненты MSS. Кроме того утилита включает редактор, позволяющий разрабатывать, отлаживать и выполнять сценарии на языке Transact-SQL.

На рис. 1.4 представлен скриншот главного окна MSMS. Графический интерфейс утилиты выполнен в стандартном стиле Microsoft.

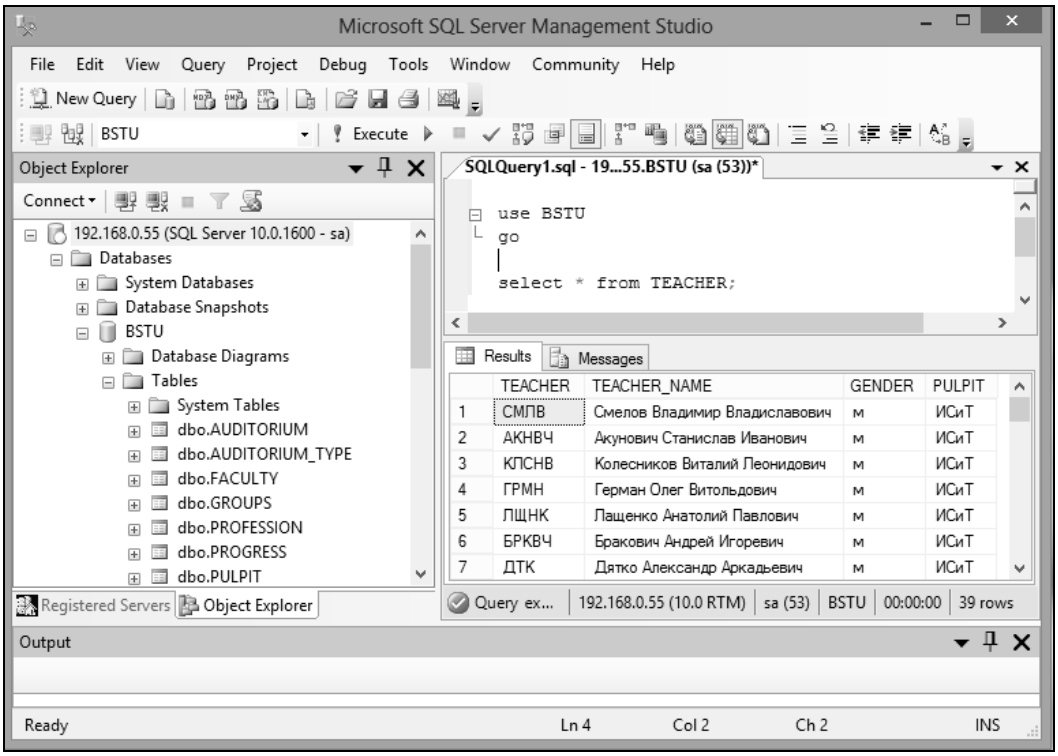


Рис. 1.4. Скриншот окна Microsoft SQL Server Management Studio

Как правило, при работе с MSMS используется дочернее окно **Object Explorer**, позволяющее просматривать, создавать, модифицировать и удалять объекты MSS и БД. С помощью окна текстового редактора пользователь создает и сохраняет файлы, содержащие сценарии на языкеTransact-SQL. Вызов специальных команд позволяет выполнять эти сценарии, а также просматривать результат их выполнения. Более подробно применение MSMS описывается в следующей главе пособия.

1.3.3. Утилита SQLCMD

Утилита SQLCMD имеет интерфейс командной строки и предназначена для выполнения сценариев Transact-SQL. Для отображения краткой инструкции по применению утилиты необходимо ее выполнить с ключом **-?** (рис. 1.5).

Утилиту можно запустить в диалоговом режиме, позволяющем построчно вводить операторы Transact-SQL и выполнять таким образом подготовленный сценарий с помощью оператора GO. На рис. 1.6 представлен скриншот окна консоли с запущенной в диалоговом режиме утилитой SQLCMD. Пронумерованные стоки, отображенные в окне консоли, соответствуют введенным пользователем данным.

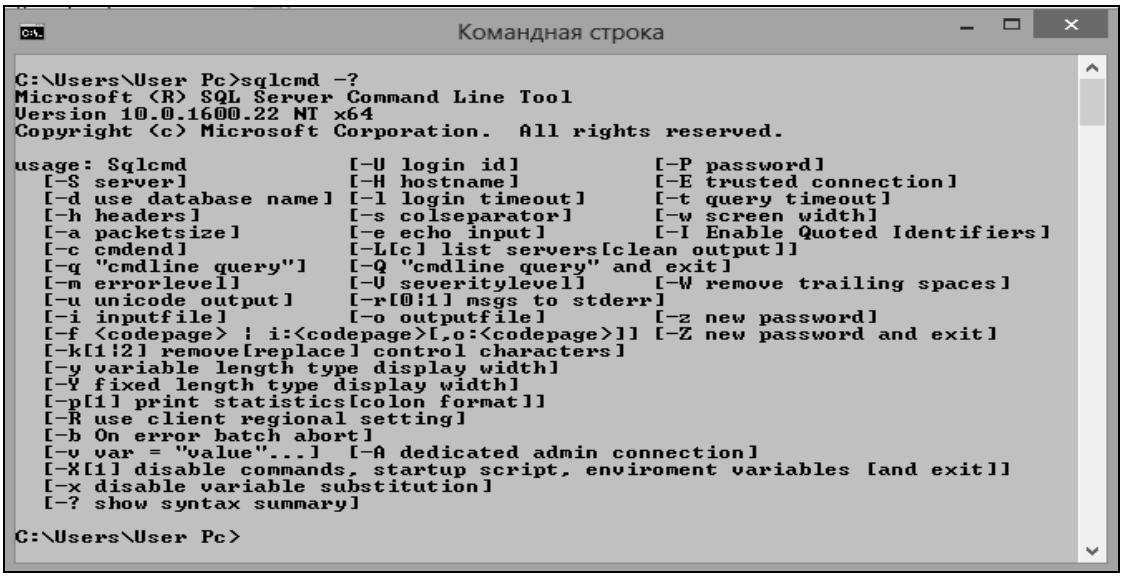


Рис. 1.5. Скриншот консоли с краткой инструкцией по использованию утилиты SQLCMD

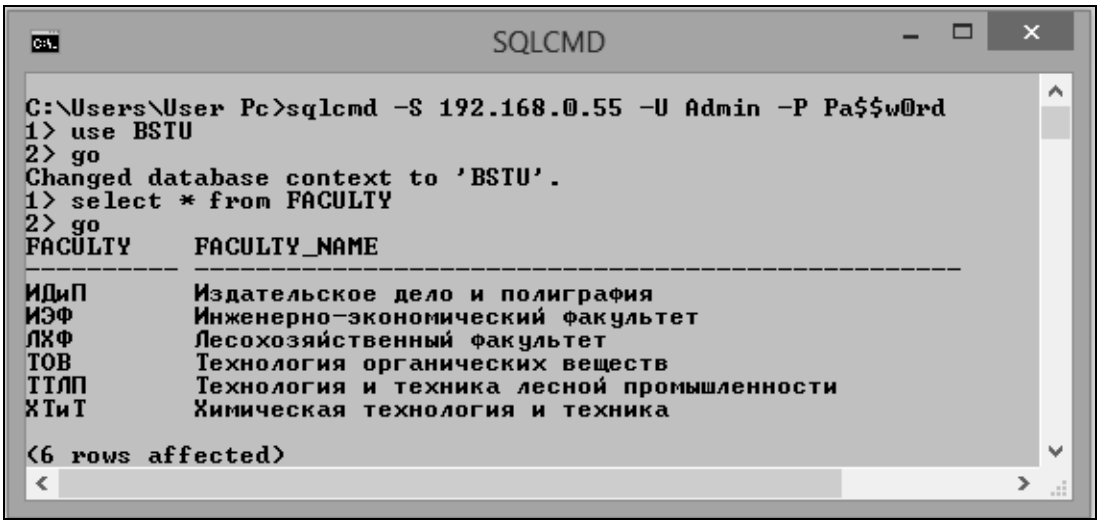


Рис. 1.6. Применение утилиты SQLCMD в диалоговом режиме

Чаще SQLCMD используют для выполнения уже готовых сценариев на Transact-SQL. При этом есть возможность передать параметры, заданные в командной строке. На рис. 1.7 представлен сценарий, который может быть подготовлен с помощью любого текстового редактора (например, Notepad). Сценарий содержит инструкции утилиты SQLCMD и операторы Transact-SQL. Обратите внимание на инструкцию :Setvar, задающую значение для системной переменной **SQLCMDMAXFIXEDTYPEWIDTH**, устанавливающей максимальный размер (в байтах) выводимых в столбцах данных. Кроме того, в сценарии используется переменная с именем **P**, значение которой должно быть задано в командной строке при вызове утилиты.

```
:Setvar SQLCMDMAXFIXEDTYPEWIDTH 25  -- максимальный размер столбца
use BSTU
go
select TEACHER [Код], TEACHER_NAME [Фамилия, имя, отчество]
        from TEACHER where PULPIT= $(P);  -- P - параметр, заданный в командной строке
go
```

Рис. 1.7. Сценарий, выполняемый утилитой SQLCMD

Если сценарий, представленный на рис. 1.7, размещен в файле с именем **D:\scr.sql**, то он может быть выполнен с помощью утилиты SQLCMD следующим образом (рис. 1.8).

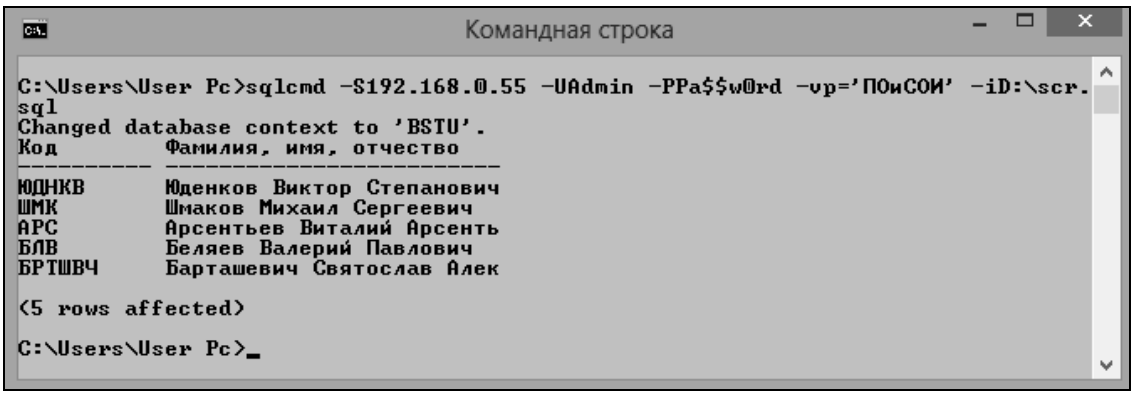


Рис. 1.8. Запуск SQLCMD для выполнения сценария,
размещенного в файле **D:\scr.sql**

Обратите внимание: 1) с помощью ключа **–i** задано имя файла со сценарием; 2) с помощью ключа **–v** задано значение переменной **p**, используемое в сценарии (рис. 1.7); 3) вывод результата осуществляется в окно консоли командной строки.

Обычно результат выполнения утилиты SQLCMD не выводится в окно консоли, а записывается в файл операционной системы. Имя выходного файла может быть задано с помощью ключа **–o**. На рис. 1.9 приведен пример командной строки вызова улиты SQLCMD с заданным именем (**D:\scr.txt**) выходного файла.

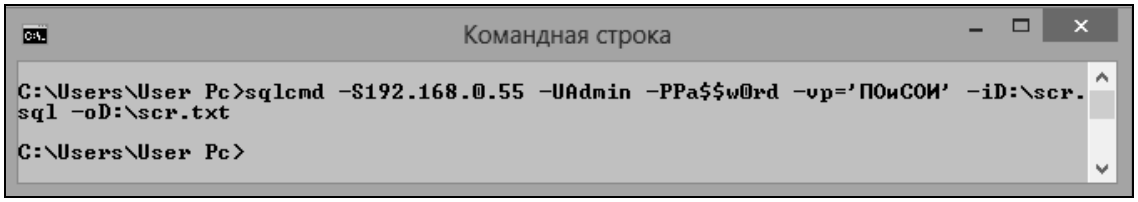


Рис. 1.9. Запуск SQLCMD с заданным выходным файлом

Сформированный утилитой SQLCMD результат выполнения сценария, может быть просмотрен с помощью любого текстового редактора.

Более подробно с возможностями утилиты SQLCMD можно ознакомиться в электронной документации, отображаемой утилитой MSBO или воспользовавшись источником [5].

1.3.4. Утилита SQL Server Configuration Manager

Microsoft SQL Configuration Manager (MSCM) – специальная программа (утилита), входящая в состав программного обеспечения СУБД MSS и предназначенная для конфигурирования, запуска и остановки служб сервера, а также настройки сетевых протоколов доступа к серверу. Кроме того, утилита может применяться на стороне клиента для конфигурирования SNAC.

Обычно MSCM запускается сразу после завершения процесса инсталляции сервера MSS для проверки работоспособности его служб.

На рис. 1.10 представлен скриншот главного окна MSCM. После выбора в меню левого окна пункта **SQL Server Services** в левом окне отобразятся все установленные на данном компьютере службы MSS, а также их состояние (Stopped – служба остановлена, Running – служба работает).

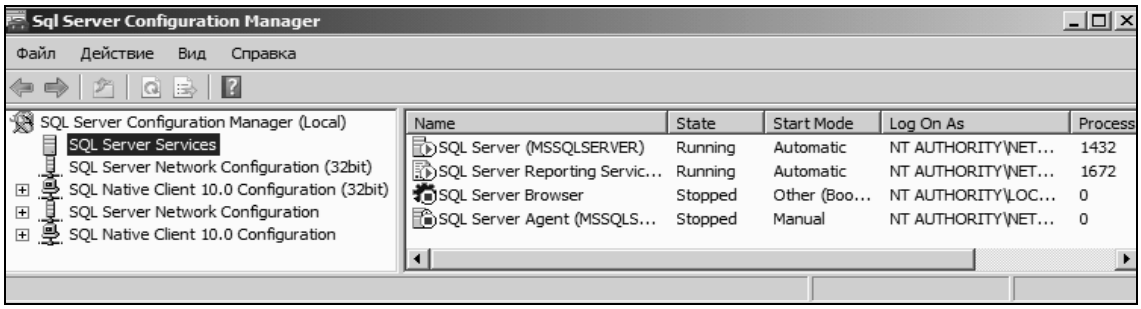


Рис. 1.10. Главное окно утилиты Microsoft SQL Server Configuration Manager

После выбора в правом окне необходимой службы можно с помощью контекстного меню ее остановить, стартовать или перезапустить (рис. 1.11).

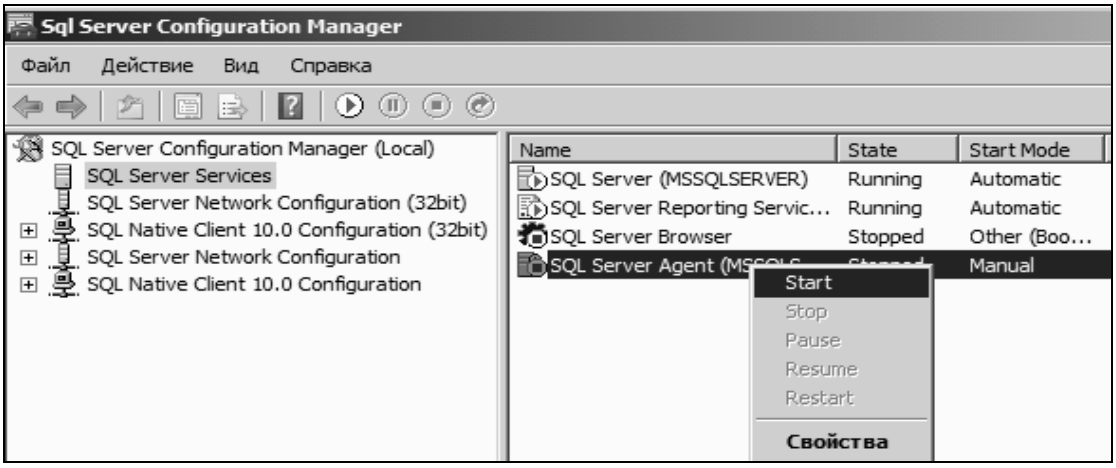


Рис. 1.11. Контекстное меню MSCM для управления состоянием сервисов

С помощью пункта контекстного меню **Properties** (**Свойства**) можно конфигурировать выбранной службой MSS (рис. 1.12).

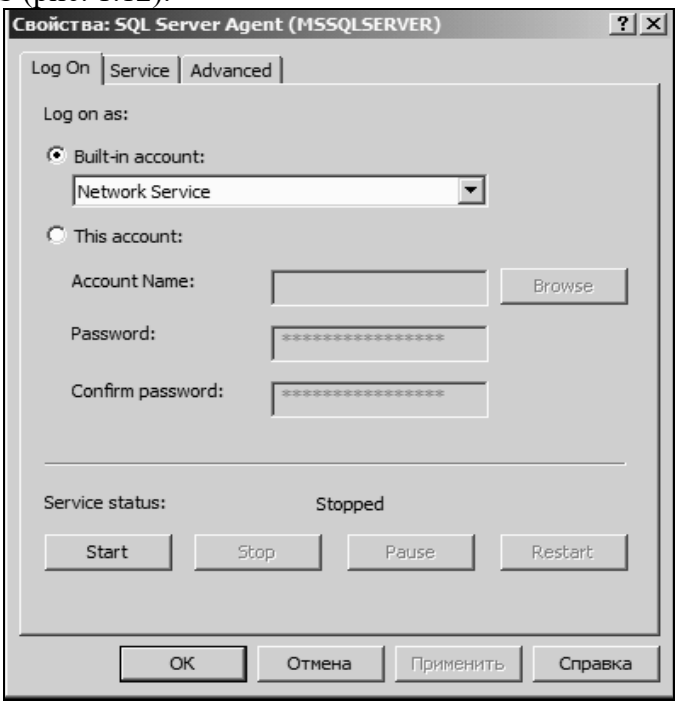


Рис. 1.12. Окно MSCM для конфигурирования службы SQL Server Agent

Конфигурирование службы сводится к установке параметров безопасности (учетная запись, от лица которой функционирует служба), режима старта (автоматический или ручной), а также ряда параметров, характерных для конкретной службы.

С помощью пункта меню SQL Server Network Configuration настраиваются сетевые протоколы службы (рис. 1.13).

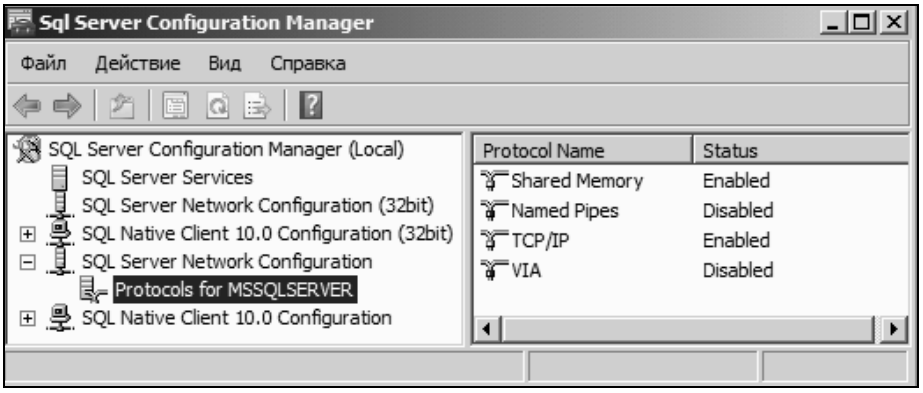


Рис. 1.13. Окно MSCM для настройки сетевых протоколов

На клиентском компьютере с помощью MSCM может быть сконфигурирован SNAC (рис. 1.14), который сводится к установке параметров сетевых протоколов, а также созданию псевдонимов (Aliases) – символических наименований совокупности сетевых настроек, которые потом могут быть применены при подключении к MSS (например, в MSMS).

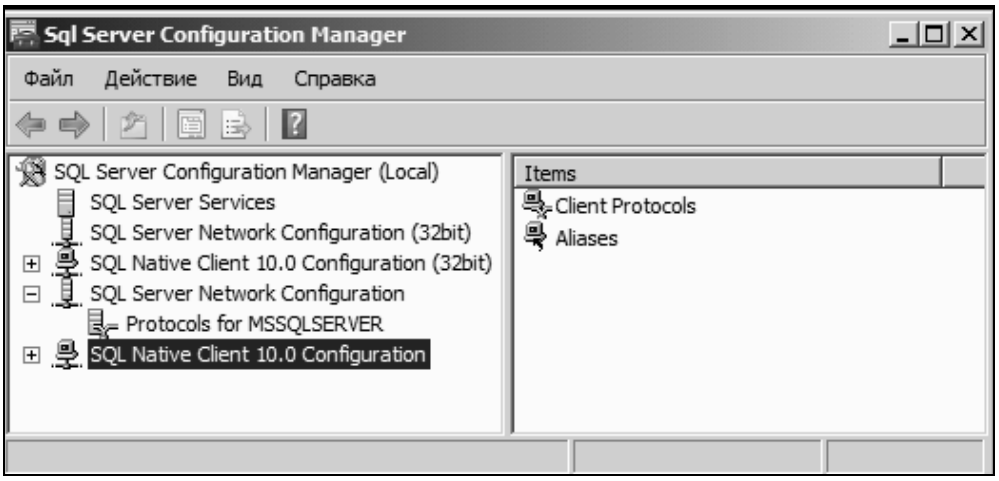


Рис. 1.14. Окно MSCM для конфигурирования SNAC

Более подробно с возможностями утилиты MSCM можно ознакомиться в электронной документации MSBO или в издании [5].

1.3.5. Остальные инструментальные средства

Помимо описанных выше инструментальных средств, в состав программного обеспечения СУБД MSS включен ряд дополнительных утилит. Краткая информация о наиболее важных из них сведена в табл. 1.3.

Таблица 1.3

Инструментальные средства в составе СУБД MSS

Наименование утилиты	Краткое описание
SQL Server Profiler	Приложение, обеспечивающее графический интерфейс для создания и управления трассировками, а также для анализа их результата
Database Engine Tuning Advisor	Приложение, обеспечивающее графический интерфейс для анализа рабочей нагрузки (процесса выполнения сценария T-SQL) и автоматической генерации рекомендаций по добавлению или изменению физической организации БД
SQL Server Business Intelligence Development Studio	Приложение, обеспечивающее графический интерфейс для создания приложений служб Reporting Services, Integration Services и Analysis Services
Import and Export Data	Приложение, обеспечивающее графический интерфейс для выполнения импорта данных в БД и экспорта данных из БД
BCP	Консольное приложение, предназначенное для массового копирования данных из файла операционной системы в таблицу БД

1.4. Служба Database Engine

Служба Database Engine является ядром системы управления реляционной БД. На одном компьютере может быть установлено несколько экземпляров (instance) службы Database Engine. При этом только один экземпляр может быть службой (сервером) по умолчанию (с именем **MS SQL SERVER**), другие экземпляры должны быть именованными службами и иметь уникальные имена. Каждый экземпляр службы Database Engine требует отдельной инсталляции, конфигурации и настройки безопасности.

Один сервер Database Engine может обеспечить доступ к нескольким БД. При этом каждый сервер в своей работе использует пять системных БД, перечисленных в табл. 1.4.

Таблица 1.4

Системные базы данных экземпляра Database Engine

Системная база данных	Назначение
master	Хранит все системные данные экземпляра Database Engine, а также информацию обо всех других БД, управляемых экземпляром
msdb	Используется службами SQL Server Agent (выполнение заданий по расписанию), Database Mail (формирование уведомлений по электронной почте), а также хранит информацию о резервном копировании БД
tempdb	Пространство для временных объектов экземпляра Database Engine и пользовательских временных таблиц. База данных пересоздается при каждой перезагрузке экземпляра
model	Шаблон, используемый при создании всех БД, управляемых экземпляром Database Engine
resource	БД, используемая только для чтения. Содержит системные объекты экземпляра Database Engine. Файлы БД являются скрытыми и не отображаются в MSMS

Основные параметры экземпляра Database Engine устанавливаются при инсталляции или с помощью MSCM. Как правило, требуется, чтобы сервер Database Engine стартовал автоматически при загрузке операционной системы. Для этого необходимо установить с помощью MSCM для параметра StartMode значение Automatic в свойствах экземпляра (рис. 1.15).

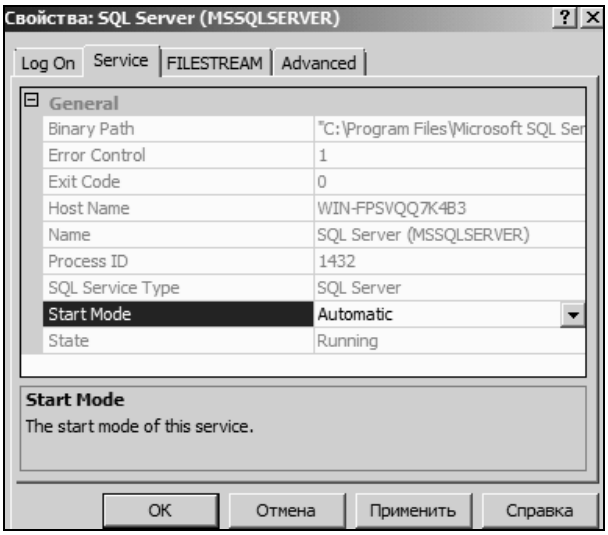


Рис. 1.15. Установка параметра StartMode для экземпляра Database Engine

Глава 2. СРЕДА SQL SERVER MANAGEMENT STUDIO

SQL Server Management Studio (SSMS) – это интегрированная среда для доступа, настройки, администрирования, разработки всех компонентов Microsoft SQL Server и управления ими. Она объединяет простые в использовании графические средства и богатые возможности для создания сценариев, позволяет пользователям выполнять разнообразные задачи в одном интерфейсе.

В состав среды SSMS входят следующие средства:

1. Редактор кода – богатый возможностями редактор скриптов для написания и редактирования скриптов.
2. Обозреватель объектов для отображения всех объектов, принадлежащих экземплярам SQL Server.
3. Обозреватель шаблонов для размещения и написания сценариев шаблонов.
4. Обозреватель решений для организации и хранения связанных скриптов как части проекта.
5. Окна свойств для отображения текущих свойств выбранных объектов.

SSMS позволяет писать и изменять скрипты, не соединяясь с экземпляром SQL Server, а также создавать сценарии из любого диалогового окна.

2.1. Запуск SQL Server Management Studio и подключение к серверу

При запуске SSMS (Пуск / SQL Server Management Studio) отображается диалоговое окно **Connect to Server** (рис. 2.1).



Рис. 2.1. Диалоговое окно **Connect to Server**

В нем задаются необходимые параметры для сервера:

тип сервера (в нашем случае это Database Engine);

имя сервера (имя компьютера, на котором установлен MS SQL Server);

вариант аутентификации (может быть Windows Authentication, когда используется учетная запись Windows, и SQL Server Authentication, когда Database Engine использует собственную систему аутентификации);

логин (sa – системный администратор);

пароль.

2.2. Основные окна

На рис. 2.2. представлены основные окна SSMS. Слева расположено окно обозревателя объектов, в центре вверху – окно редактора запросов, в центре внизу – окно результатов, справа – окно обозревателя решений.

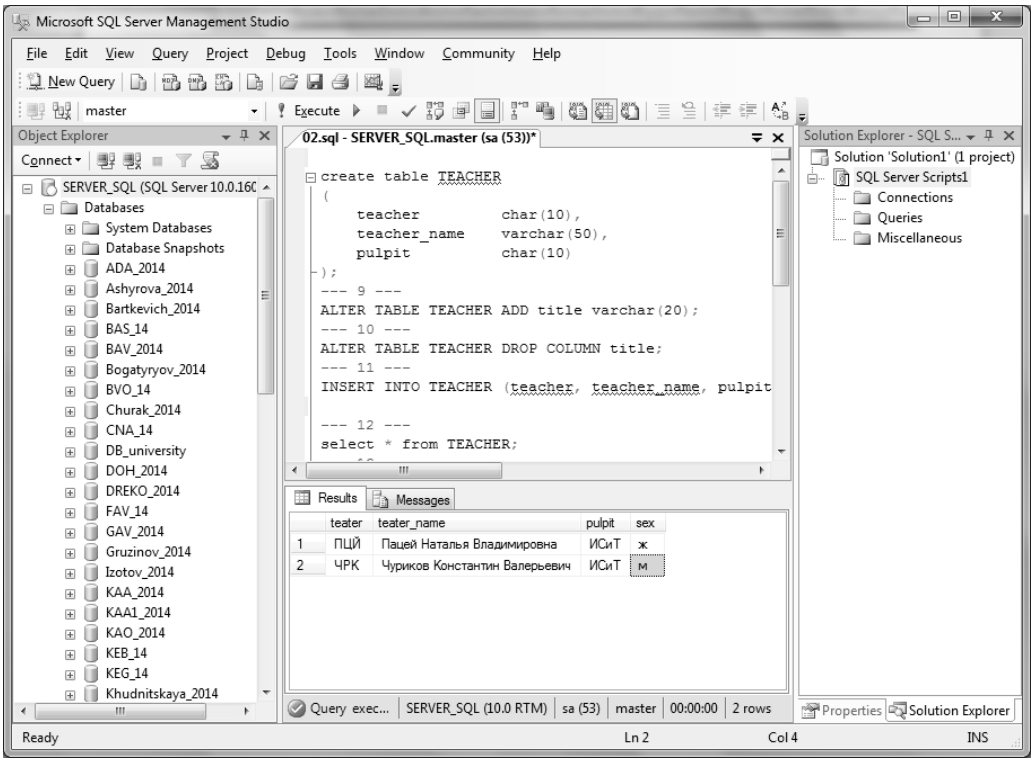


Рис 2.2. Основные окна SSMS

В SSMS используются 2 типа окон – окна компонентов и окна документов. Окна компонентов представляют такие элементы, как Object Explorer, Solution Explorer, Properties. Они перечислены в меню View. Окна документов позволяют просматривать и редактировать сценарии, XML-документы и любые другие текстовые документы.

Окна компонентов можно расположить по своему усмотрению. Они отображаются в одном из трех режимов: плавающего окна, стыкуемого окна или в виде вкладок. Плавающие окна можно разместить в любом месте, стыкуемые прикреплены хотя бы к одному краю границы интерфейса.

Для изменения типа и конфигурации окна компонентов можно щелкнуть правой кнопкой мыши вкладку или заголовок окна и выбрать подходящий режим из контекстного меню.

Окна документов всегда размещены в основной области интерфейса. Доступ к любому из документов осуществляется посредством вкладок в верхней части окна.

Для восстановления конфигурации по умолчанию в меню **Window** необходимо выбрать команду **Reset Window Layout**.

SSMS предоставляет множество параметров настройки окон, которые можно установить в окне **Options** (Tools / Options).

2.3. Обозреватель объектов

Обозреватель объектов (**Object Explorer** (рис. 2.3.)) представляет в виде древовидной структуры все компоненты подключенного экземпляра SQL Server, которыми можно управлять с помощью SSMS.

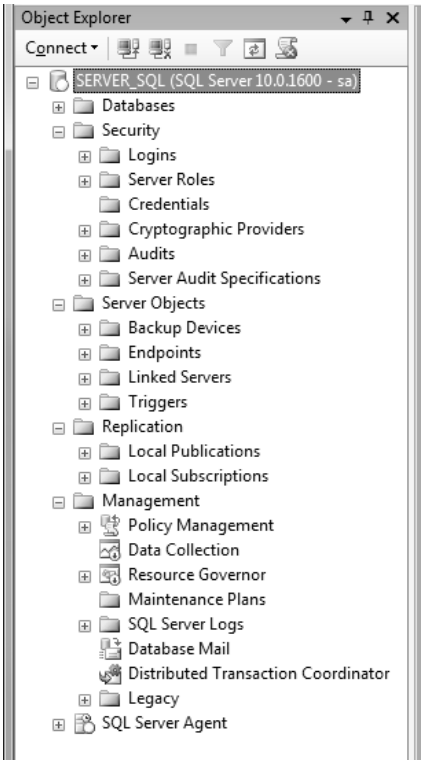


Рис 2.3. Окно Object Explorer в SSMS

По умолчанию отображаются следующие папки:

1. Базы данных (Databases), включая системные базы данных, моментальные снимки баз данных и пользовательские базы данных.
2. Объекты безопасности (Security), такие как имена входов, серверные роли и удостоверения.
3. Серверные объекты (Server Objects), в том числе устройства резервного копирования, конечные точки (SOAP и T-SQL), связанные серверы и триггеры.
4. Компоненты репликации (Replication), включая локальные публикации и локальные подписки.
5. Компоненты управления (Management), в том числе планы обслуживания, журналы SQL Server, монитор активности, компонент для отправки почтовых сообщений, координатор распределенных транзакций и полнотекстовый поиск.
6. Компоненты службы уведомлений со списком созданных экземпляров.

Фильтры Object Explorer. В реальных приложениях количество объектов баз данных может достигать сотен тысяч. Поиск единственного нужного объекта среди этого множества займет немало времени. В SSMS можно установить критерий фильтрации для объектов в папках Object Explorer.

Допускаются различные критерии фильтрации для разных типов объектов, например, для таблиц задается один набор условий, а для хранимых процедур – другой. Фильтры применимы

для большинства типов объектов, но все-таки не для всех. Например, можно установить фильтр для папки **Security / Users**, а для папки **Security / Roles** – нельзя. Фильтры доступны для всех тех типов объектов, которые действительно в них нуждаются.

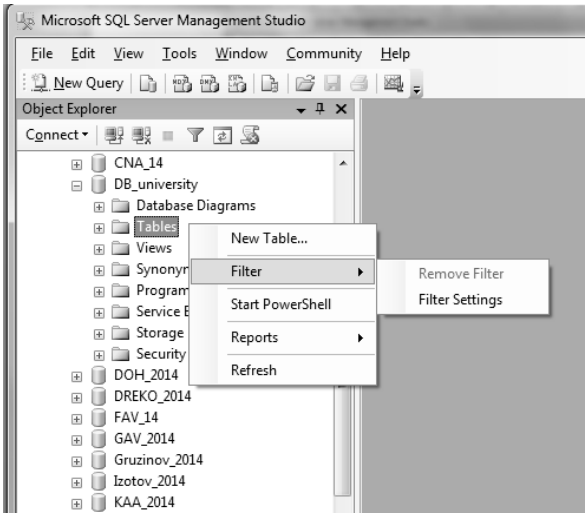


Рис. 2.4. Добавление фильтра в Object Explorer

Для создания фильтра (рис. 2.4) необходимо щелкнуть правой кнопкой мыши тип объекта, для которого требуется задать фильтр, и выбрать пункт **Filter**. Откроется подменю из двух пунктов, первый из которых, **Remove Filter**, удаляет существующий фильтр (этот пункт меню доступен только в том случае, если фильтр существует), и второй, **Filter Settings**, позволяющий добавить фильтр или изменить критерии текущего фильтра.

2.4. Редактор запросов

Редактор запросов **(Query Editor)** представляет собой многофункциональный редактор текста и сценариев, поддерживающий цветную подсветку синтаксиса, индикаторы отслеживания изменений и настраиваемые шаблоны. Он также содержит средства для проектирования, выполнения и анализа запросов и хранимых процедур.

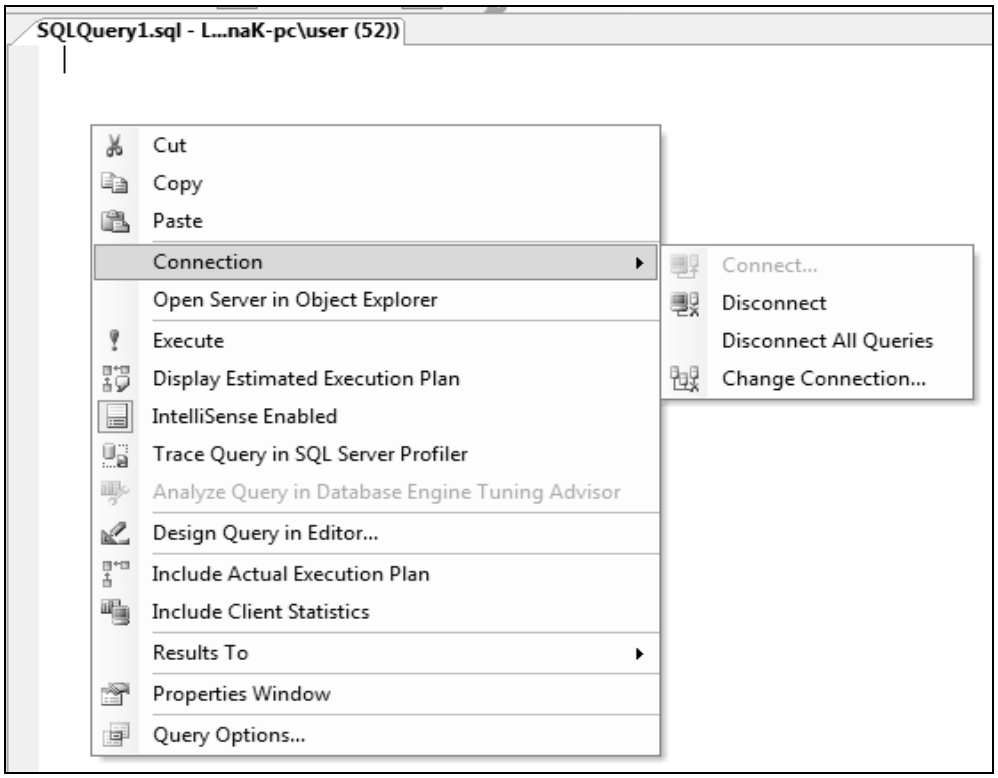


Рис. 2.5. Контекстное меню редактора запросов

Для создания нового запроса надо выбрать команду **New Query** на панели инструментов Standard (View / Toolbars / Standard) или нажать **Ctrl+N**. По умолчанию команда **New Query** откроет для редактирования новое окно сценария SQL. Если щелкнуть правой кнопкой мыши в рабочей области окна, то откроется меню редактора запросов, содержащее средства проектирования и выполнения запросов (рис. 2.5).

Окно запросов может быть подключенным к экземпляру SQL Server или полностью отключенным. Это удобно при редактировании запросов, когда разработчик находится за пределами сети или не может подключиться к серверу. Кроме того, можно отключить окно запросов от одного экземпляра SQL Server и подключить к другому. Это позволяет, к примеру, не закрывая и не открывая лишний раз файлы со сценариями, тестировать сценарии на сервере, предназначенном для разработки, а затем запускать их на производственном сервере.

Подменю **Connection** в меню Query Editor (рис. 2.5) содержит четыре команды: Connect, Disconnect, Disconnect All Queries и Change Connection.

Команда **Connect** позволяет выбрать сервер, с которым необходимо установить соединение. Если текущее окно уже подключено, эта команда неактивна. Команда **Disconnect** отсоединяет текущее окно от сервера, а команда **Disconnect All Queries** выполняет ту же операцию для всех открытых окон запросов. Команда **Change Connection** позволяет подключиться к другому серверу или изменить параметры подключения или аутентификации.

Команда **Execute** выполняет запрос и отображает результаты.

Команда **Display Estimated Execution Plan** выполняет синтаксический анализ текущего запроса и показывает предполагаемый план выполнения.

Пункт меню **Design Query in Editor** предназначен для запуска Query Designer – нового инструмента разработки SQL Server, который будет рассмотрен далее.

Команда **Include Actual Execution Plan** выполняет запрос и показывает план, который использовался на самом деле.

Пункт меню **Results To** позволяет задать способ представления результатов, полученных при выполнении запроса – Results to Text (в виде текста), Results to Grid (в виде сетки) и Results to File (сохранить в файл).

Создание и выполнение запросов с помощью дизайнера. Создавать запросы можно также с помощью дизайнера запросов **Query Designer**, входящего в состав SSMS.

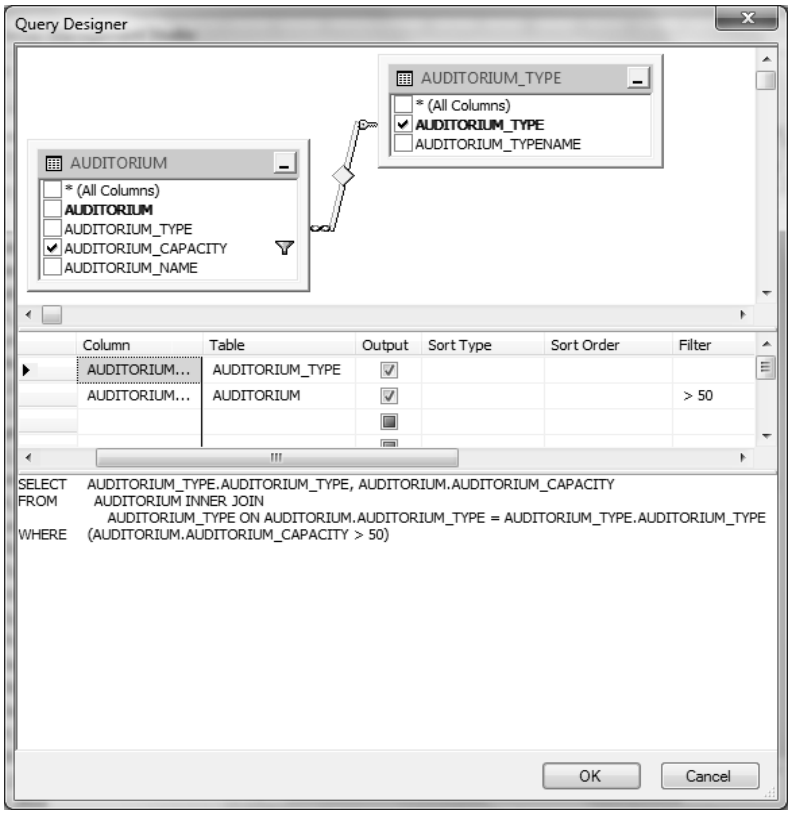


Рис. 2.6. Создание запроса с помощью Query Designer

В окне **Query Designer** (рис. 2.6) можно выбирать и добавлять в запрос таблицы, представления, функции и синонимы, относящиеся к текущему соединению На рис. 2.6 видно, что Query Designer также отображает связи между выбранными таблицами. Эти связи определяют типы соединений таблиц в запросе. Чтобы изменить связь и указать требуемый тип соединения, достаточно щелкнуть правой кнопкой линию, обозначающую связь, и в открывшемся меню выбрать пункт свойств **Properties**.

Если установить флажок рядом с именем поля, то поле добавится в запрос. Чтобы добавить в запрос все доступные поля, установите флажок рядом с элементом * (All Columns) вверху списка.

Когда поле добавляется в запрос, дизайнер вносит его в сетку с полями, а также соответствующим образом корректирует исходный текст запроса на языке SQL. Работая с сеткой, можно добавлять псевдонимы полей, менять порядок сортировки и задавать критерии отбора. Сетка и текстовое поле с SQL-кодом связаны между собой, при изменении содержимого одного из этих элементов управления другой меняется автоматически.

2.5. Создание базы данных

Создать базу данных, а также объекты базы данных (таблицы, хранимые процедуры, индексы, запросы и т. п.) можно при помощи дизайнеров и диалоговых окон среды разработки, а также при помощи сценариев SQL.

Для создания базы с помощью дизайнера необходимо в окне **Object Explorer** щелкнуть правой кнопкой узел **Databases** и выбрать **New Database**. Далее в открывшемся окне **New Database** указать имя новой базы данных и щелкнуть OK. База данных создана.

Если в окне **New Database** после указания имени щелкнуть кнопку Script, то в окне запросов сформируется программный код, позволяющий создать базу данных с указанным именем.

2.6. Создание таблиц

Для создания таблицы с помощью дизайнера надо перейти в окно **Object Explorer** и раскрыть узел базы данных, в которой требуется создать таблицу. Далее щелкнуть правой кнопкой узел **Tables** и выбрать **New Table**.

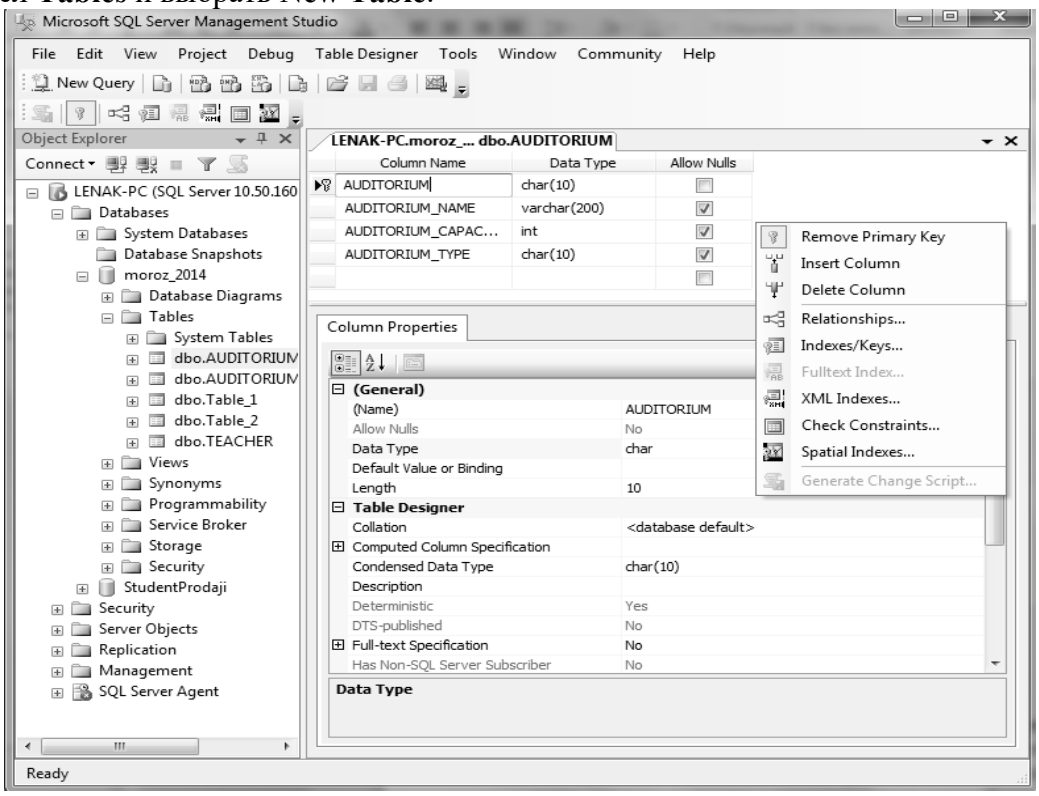


Рис. 2.7. Дизайнер таблиц

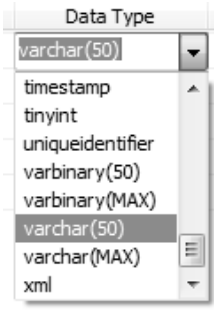


Рис. 2.8. Раскрывающийся список Data Type

В дизайнере (рис. 2.7.) используется сетка, содержащая названия столбцов таблицы (Column Name), типы данных (Data Type) и флажки, определяющие допустимость NULL-значений (Allow Nulls).

В раскрывающемся списке **Data Type** (рис. 2.8.) для некоторых типов данных присутствует сразу несколько элементов, в которые уже добавлены наиболее часто используемые значения длины.

Чтобы изменить размер типа данных переменной длины, можно ввести новое значение непосредственно в поле Data Type или поменять длину в расположенном под сеткой разделе свойств столбца Column Properties.

В дизайнере таблиц также есть контекстное меню, появляющееся по щелчку правой кнопкой мыши в верхней части (рис. 2.7.). При помощи этого меню удобно создавать первичные ключи, связи между таблицами (внешние ключи), индексы и ограничения.

При выборе любого из пунктов меню открывается соответствующий дизайнер. Хотя в каждом дизайнере настраиваются разные параметры, окна всех дизайнеров похожи на диалоговое окно Foreign Key Relationships (рис. 2.9.). Для создания нового объекта указанного типа всегда присутствует кнопка Add, а свойства этого объекта задаются в списке, расположенном справа. Удалить выбранный объект можно при помощи кнопки Delete.

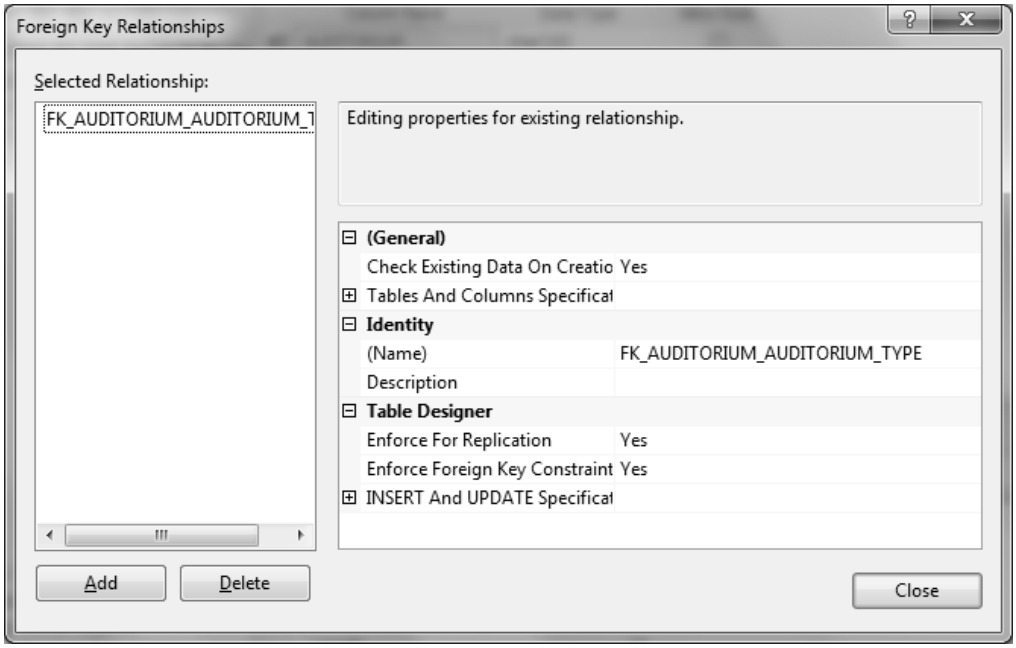


Рис. 2.9. Диалоговое окно Foreign Key Relationships

Просмотр содержимого таблиц. Если открыть контекстное меню таблицы в Обозревателе объектов (рис. 2.10), то можно получить всю информацию о данной таблице, а также произвести некоторые действия с таблицей (переименовать, удалить и др.).

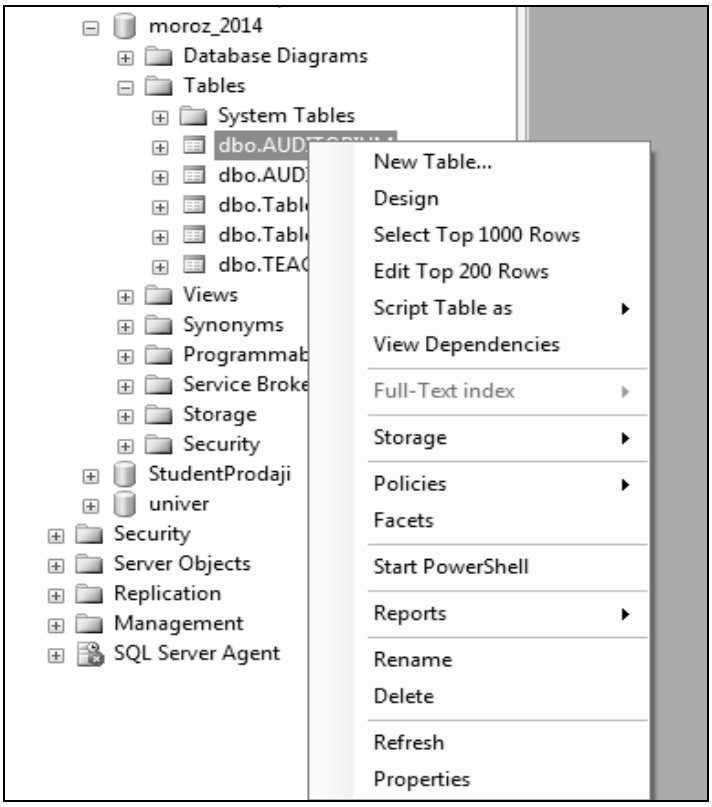


Рис. 2.10. Контекстное меню таблиц

Далее перечислены некоторые команды контекстного меню и их назначение.
New Table – создать новую таблицу;
Design – просмотреть структуру таблицы, открыв ее в дизайнере;

Select Top 1000 Rows – в окне редактора запросов получить скрипт запроса для вывода первых 1000 строк таблицы, а также результат его выполнения;

Edit Top 200 Rows – просмотреть содержимое первых 200 строк таблицы.

2.7. Создание объектов, относящихся к таблицам

Если в окне **Object Explorer** раскрыть узел любой из таблиц, то появится список объектов, относящихся к таблице. С каждым элементом списка связано контекстное меню, позволяющее создавать объект соответствующего типа (рис. 2.11.).

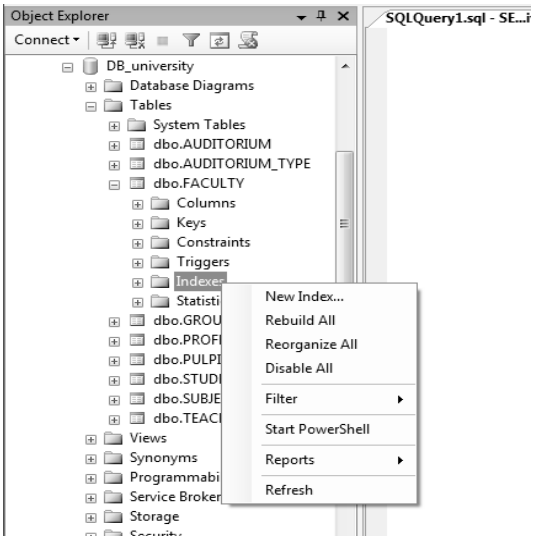


Рис. 2.11. Контекстное меню индексов таблицы

При выборе пункта меню **New <объект>** для объектов **Columns**, **Keys** или **Constraints** открывается дизайнер таблиц, а для объектов **Keys** или **Constraints** открывается еще и соответствующий дизайнер объектов. Команда **New Trigger** открывает окно сценария с шаблоном для создания нового триггера.

2.8. Автоматическое формирование скриптов

В верхней части большинства диалоговых окон для создания и изменения объектов, например, в верхней части диалогового окна для создания индексов **New Index**, находится меню генерации сценариев **Script** (рис. 2.12.).

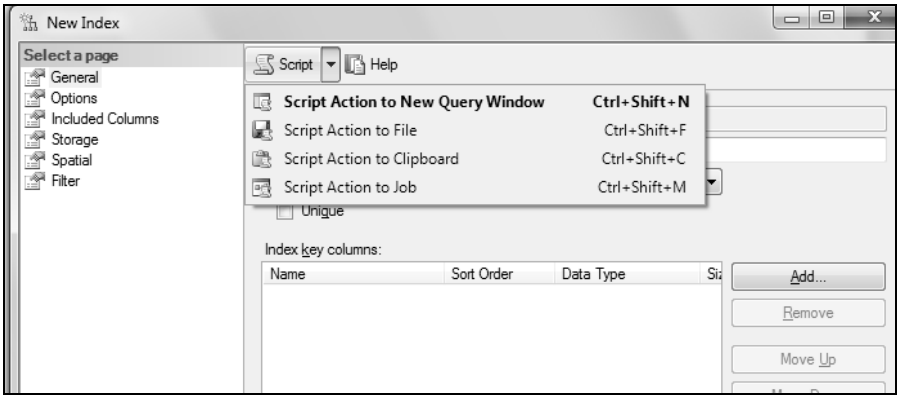


Рис. 2.12. Меню Script в окне создания индексов

С помощью меню Script можно

- открыть новое окно запроса с уже созданным скриптом для текущего объекта,
- сохранить сценарий в файле,
- поместить его в буфер обмена,
- создать соответствующее задание.

Автоматически создать скрипт также можно используя контекстное меню объекта в Обозревателе объектов (рис. 2.10.).

2.9. Работа с шаблонами

Среда SSMS предлагает большое число шаблонов, предназначенных для решения многих типовых задач. *Шаблоны* – это стандартные заготовки сценариев, которые можно использовать для создания собственных сценариев SQL.

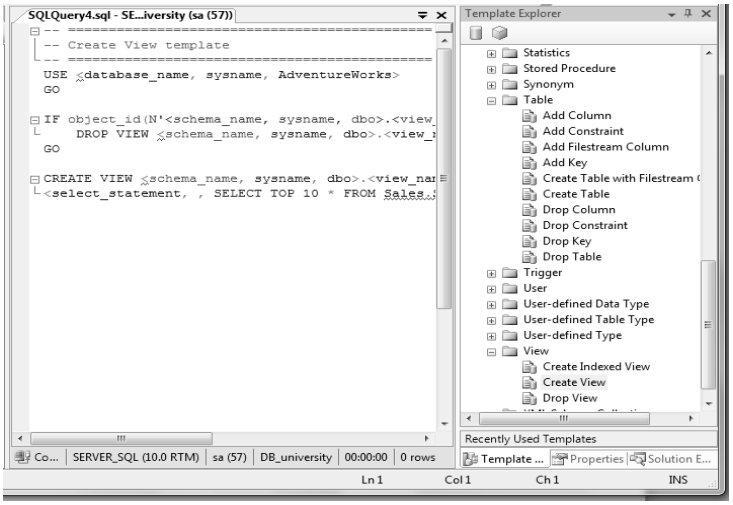


Рис. 2.13. Список шаблонов

Шаблоны содержат параметры, значения которых вводятся пользователем, такие, например, как имя таблицы. Используя эти параметры, можно ввести имя только один раз, а затем автоматически скопировать его во все требуемые позиции в скрипте. Также можно создавать собственные пользовательские шаблоны.

Доступ к шаблонам можно получить, открыв окно проводника шаблонов **Template Explorer**. По двойному щелчку любого узла с шаблоном открывается окно для редактирования запроса **Query Edit** с SQL-кодом соответствующего шаблона.

2.10. Решения, проекты и файлы Management Studio

SSMS обеспечивает интегрированное управление файлами при помощи проектов и решений.

Проект – это контейнер, который содержит логически связанные подключения и файлы со сценариями SQL. *Решение* – это контейнер, который содержит логически связанные файлы проектов. В каждом проекте есть также папка **Miscellaneous** (Разное) для файлов других типов. Любой файл, добавленный в эту папку, будет связан с проектом. Например, если имеется сценарий SQL, выводящий результат в файл, то этот выходной файл можно хранить как часть проекта в папке **Miscellaneous**, и при необходимости ссылаться на него.

По умолчанию при создании нового файла запроса создается решение. Для создания проекта следует выбрать **File** / **New** / **Project**. В открывшемся диалоговом окне **New Project** можно указать тип проекта, его имя и его местонахождение, а также определить, добавить ли проект к уже существующему решению или создать для него новое решение.

Чтобы создать в проекте новое подключение или запрос, выберите в меню **Project** (Проект) пункт **New Connection** (Создать соединение) или **New Query** (Создать запрос). Для добавления к проекту существующего файла выберите в меню **Project** пункт **Add Existing Item** (Добавить

существующий элемент) (рис. 2.14). В открывшемся диалоговом окне найдите и выберите файл, а затем щелкните кнопку **Add** (Добавить). Management Studio автоматически разместит файл в папке, соответствующей типу файла.

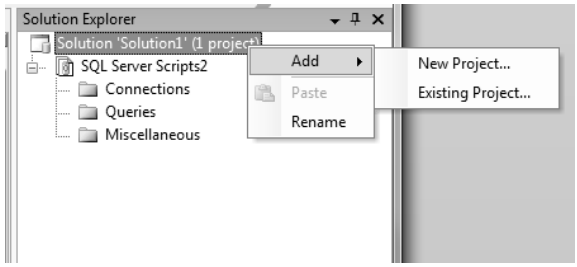


Рис. 2.14. Добавление проекта в Решение

Необходимо отметить, что проекты предназначены для упорядочения файлов, а не объектов баз данных. Элементы, относящиеся к проекту, хранятся в папках на жестком диске, как обычные файлы. Созданная хранимая процедура будет находиться в базе данных, как и раньше. Но если занести сценарий SQL, применявшийся при создании хранимой процедуры, в файл, то этот файл, хранящийся на жестком диске, можно добавить к проекту вместе с другими относящимися к проекту файлами, правильным образом сгруппировав их.

Такая организация файлов предоставляет удобный способ хранения сценариев, необходимых для восстановления базы данных.

Глава 3. ОСНОВЫ ЯЗЫКА SQL

3.1. Назначение, структура и стандарты языка SQL

Язык **SQL** (Structured Query Language, язык структурированных запросов) – специализированный язык, предназначенный для написания запросов к реляционной БД. Основной единицей этого языка является SQL-оператор. Запрос к БД представляет собой один или несколько операторов языка, интерпретируемых СУБД и позволяющих создать, модифицировать, удалить саму БД, а также ее объекты.

Прототипом SQL является язык SEQUEL, который был разработан в начале 1970-х гг. компанией IBM для экспериментальной СУБД SystemR. В 1986 г. ANSI (American National Standards Institute, Американский национальный институт стандартизации) принял первый стандарт языка SQL. Позже было опубликовано еще несколько стандартов, но наибольшую известность среди них получил SQL-92 (другое наименование SQL-2), названный так по году (1992) выпуска, который по сей день является ядром SQL-диалектов большинства современных реляционных СУБД. На сегодняшний день действует еще ряд стандартов: SQL-2003, SQL-2006, SQL-2008, которые в той или иной степени поддерживаются различными СУБД.

Операторы SQL делятся на несколько непересекающихся групп: **DDL** (Data Definition Language, язык определения данных), **DCL** (Data Control Language, язык управления данными), **DML** (Data Manipulation Language, язык манипулирования данными) и **TCL** (Transaction Control Language, язык управления транзакциями) (рис. 3.1).

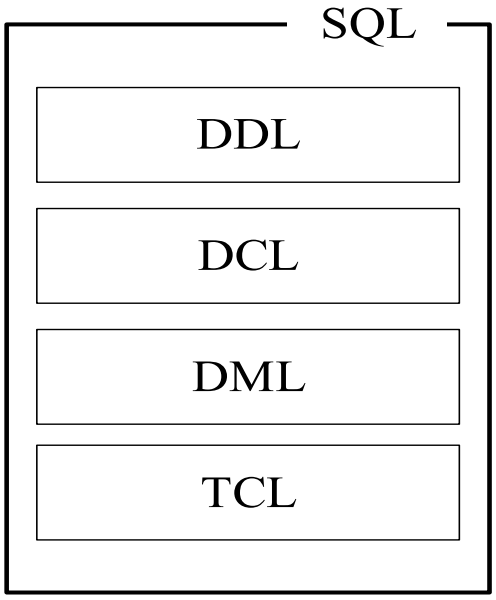


Рис. 3.1. Структура языка SQL

3.2. Операторы DDL SQL

Операторы DDL предназначены для создания, удаления и изменения объектов БД или сервера СУБД. В этой книге будут в основном рассматриваться операторы уровня БД. DDL включает три оператора: **CREATE**, **ALTER**, **DROP** (рис. 3.2).

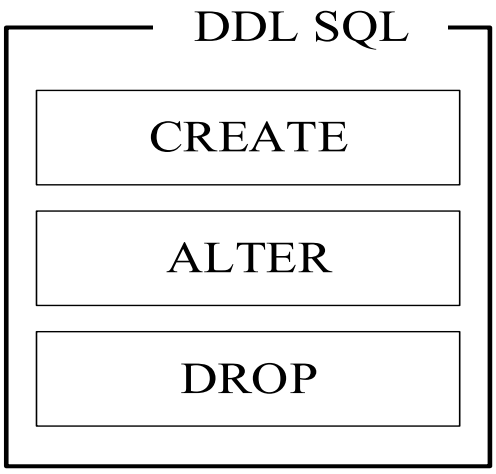


Рис. 3.2. Операторы DDL SQL

Оператор **CREATE** предназначен для создания объектов БД или сервера СУБД. Структура оператора представлена на рис. 3.3.



Рис. 3.3. Структура оператора CREATE

Каждый объект БД имеет тип (например, процедура, таблица, представление и т. д.), имя, а также ряд дополнительных параметров (на рис. 3.3 обозначены словом «дополнение»), характеризующих данный тип объекта. В некоторых случаях дополнительные параметры могут не указываться – в этом случае для объекта автоматически устанавливаются значения параметров объекта по умолчанию.

На рис. 3.4 представлен пример создания БД (тип **database**) с именем **BSTU** – объекта сервера СУБД. БД **BSTU** располагается в двух файлах операционной системы с именами, указанными параметрами **filename,** имеющими логические имена (параметр с именем **name**) соответственно **BSTU** и **BSTU_log**. Параметры **size**, **maxsize** и **filegrowth** указывают на начальный размер, максимальный размер, а также приращение размера файла.

```
create database BSTU on  primary
( name = N'BSTU', filename = N'C:\BSTU.mdf' ,
  size = 3072KB , maxsize = unlimited, filegrowth = 1024KB)
log on
( name  = N'BSTU_log', filename = N'C:\BSTU_log.ldf',
  size  = 1024KB , maxsize = 2048GB , filegrowth  = 10%)
```

Рис. 3.4. Пример использования оператора CREATE
для создания базы данных

На рис. 3.5 приведен пример создания таблицы (тип **table**) – объекта БД. Таблица имеет имя **AUDITORIUM** и состоит из пяти столбцов. Для столбца с именем **AUDITORIUM** указывается ограничение целостности первичный ключ (primary key) с именем **AUDITORIUM_PK**.

```
create  table AUDITORIUM
(
 AUDITORIUM char(20) not null,
 AUDITORIUM_TYPE char(10) null,
 AUDITORIUM_CAPACITY int null,
 AUDITORIUM_NAME varchar (50) null,
 constraint AUDITORIUM_PK  primary key clustered (AUDITORIUM)
 )
```

Рис. 3.5. Пример использования оператора CREATE
для создания таблицы

Создание программного объекта БД – скалярной функции с именем **CALC_CAPACITY** представлено на рис. 3.6. Функция возвращает единственное целочисленное значение – сумму значений в столбце **AUDITORIUM_ CAPACITY** всех строк таблицы **AUDITORIUM**.

```
 create function CALC_CAPACITY() returns int
 begin
   return (select sum(AUDITORIUM_CAPACITY)from AUDITORIUM)
 end;
```

Рис. 3.6. Пример использования оператора CREATE
для создания скалярной функции

Для модификации существующих объектов БД или сервера СУБД применяется оператор **ALTER**. Структура оператора представлена на рис. 3.7.



Рис. 3.7. Структура оператора ALTER

На рис. 3.8 приведен пример трехкратного применения оператора **ALTER** к базе данных (тип **database**) с именем **BSTU**. В первом случае оператор добавляет (ключевое слово **add**) новую файловую группу (**filegroup**) с именем **G1**. Во втором добавляет файл (**add file**) в файловую группу **G1**. Третий оператор изменяет (**modify**) свойство файловой группы **G1**, устанавливая признак того, что эта группа подразумевается в некоторых операторах SQL по умолчанию, если имя группы не указано явно.

```
alter database BSTU add filegroup G1;
alter database BSTU add file
( name = N'BSTU1', filename = N'C:\BSTU1.ndf' ,
  size = 3072KB , maxsize = unlimited, filegrowth = 1024KB) to filegroup G1;
alter database BSTU modify filegroup G1 default;
```

Рис. 3.8. Пример использования нескольких операторов ALTER
для изменения базы данных

Пример на рис. 3.9 демонстрирует применение оператора **ALTER** к таблице (тип **table**) с именем **AUDITORIUM**. Оператор изменяет свойства столбца (**alter column**) с именем **AUDITORIUM_NAME**.

```
alter table AUDITORIUM
   alter column  AUDITORIUM_NAME varchar(50) not null;
```

Рис. 3.9. Пример использования оператора ALTER
для изменения свойства столбца таблицы

На рис. 3.10 демонстрируется применение оператора ALTER к функции (тип **function**) с именем **CALC_CAPACITY.** Такое использование оператора позволяет изменить программный код функции, не меняя других ее свойств.

```
alter function CALC_CAPACITY() returns int  as
begin
     return (select avg(AUDITORIUM_CAPACITY) from AUDITORIUM)
end;
```

Рис. 3.10. Пример использования оператора ALTER
для изменения функции

Удалить (иногда применяют слова «уничтожить», «разрушить») существующий объект сервера или БД можно с помощью оператора DROP. Структура оператора представлена на рис. 3.11.



Рис. 3.11. Структура оператора DROP

Пример на рис. 3.12 демонстрирует двукратное применение оператора **DROP**.

```
drop table TEACHER;
drop function CALC_CAPACITY;
```

Рис. 3.12. Пример использования операторов DROP
для удаления таблицы и функции

В первом случае удаляется таблица (тип **table**) с именем **TRACHER**. Во втором удаляется функция (**function**) с именем **CALC_CAPACITY**.

3.3. Операторы DML SQL

Операторы DML предназначены для работы с одним (наиболее важным) типом объектов БД – таблицами. DML включает четыре оператора: **SELECT**, **INSERT**, **DELETE**, **UPDATE** (рис. 3.13). Иногда к этой группе относят оператор **TRUNCATE**.

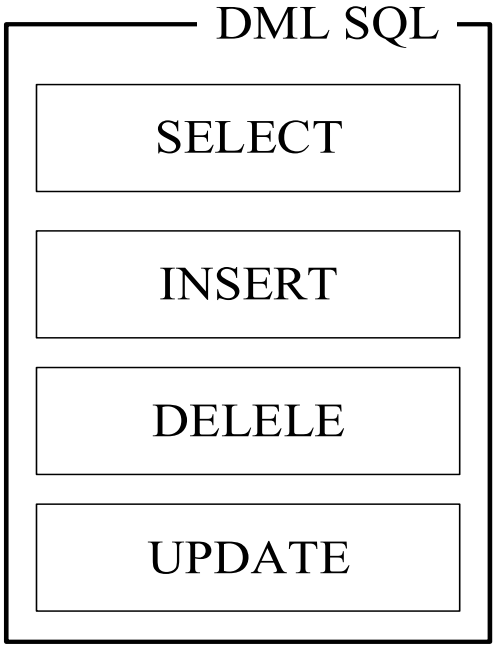


Рис. 3.13. Операторы DML SQL

Наиболее мощным DML-оператором является **SELECT**. Он позволяет выбрать множество строк из одной или нескольких таблиц. При успешном выполнении этого оператора формируется результирующий набор, представляющий собой множество однотипных (с одинаковыми

столбцами) строк. В общем случае результирующий набор может содержать ни одной, одну или более строк. Структура оператора SELECT достаточно сложная и детально будет разобрана позже, но в первом приближении ее можно представить как на рис. 3.14.



Рис. 3.14. Структура оператора SELECT

Любой оператор SELECT содержит список (на рис. 3.14 обозначен словом «список»), определяющий перечень столбцов (в общем случае и содержимое) результирующего набора. Дополнительная часть оператора (на рис. 3.14 обозначена словом «дополнение») описывает множество строк из одной или нескольких таблиц, служащее источником для формирования результирующего набора.

На рис. 3.15 приведены примеры использования оператора SELECT.

```
select * from dbo.TEACHER;
select PULPIT_NAME from PULPIT;
select PN, FY into NEWTABLE from  (select PULPIT_NAME PN, FACULTY FY from PULPIT)t;
select 'Hello World!!!';
select 60* 60*24 'секунд в сутках';
```

Рис. 3.15. Примеры использования операторов SELECT

В первом случае в результате выполнения оператора SELECT будет сформирован результирующий набор, состоящий из всех строк таблицы **dbo.TEACHER**, причем строки будут содержать все столбцы этой таблицы.

Второй оператор SELECT сформирует результирующий набор, состоящий из одного столбца с именем **PULPIT_NAME** таблицы **PULPIT**.

Третий оператор SELECT демонстрирует создание новой таблицы с именем **NEWTABLE**, включающей два столбца с именами **PN** и **FY**. Содержимое новой таблицы будет определяться результирующим набором, сформированным в результате выполнения другого (внутреннего) оператора **SELECT**, записанного в скобках после ключевого слова **FROM**.

Четвертый и пятый операторы формируют результирующие наборы, состоящие из одной строки, содержимое которой определяется значением выражений, указанных после ключевого слова **SELECT**.

Добавить одну или несколько строк в существующую таблицу можно с помощью оператора **INSERT**. Структура этого оператора представлена на рис. 3.16.

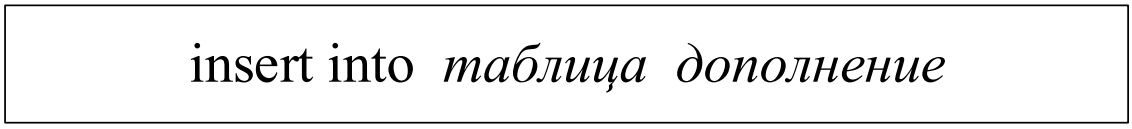


Рис. 3.16. Структура оператора INSERT

Ключевое слова **INTO** указывает на то, что далее следует имя таблицы, в которую будут добавляться строки.

На рис. 3.17 приведены четыре примера использования оператора INSERT.

```
insert into AUDITORIUM_TYPE (AUDITORIUM_TYPE, AUDITORIUM_TYPENAME )
       values ('ЛК', 'Лекционная');
insert into GROUPS   (FACULTY,  PROFESSION, YEAR_FIRST )
       values ('ИДиП','1-40 01 02', 2013),
              ('ИДиП','1-40 01 02', 2012),
              ('ИДиП','1-40 01 02', 2011);
insert into TTTT(PN, FY)
       select PULPIT_NAME, FACULTY from PULPIT;
insert AUDITORIUM_TYPE values ('ЛБ-Ф', 'Лаборатория физики');
```

Рис. 3.17. Примеры использования операторов INSERT

В первом примере оператор **INSERT** добавляет одну строку в таблицу с именем **AUDITORIUM_TYPE.** Добавляемая строка содержит столбцы с именами **AUDITORIUM_TYPE** и **AUDITORIUM_TYPENAME,** а соответствующие им значения указаны в скобках после ключевого слова **VALUES**.

Во втором примере добавляется три строки в таблицу **GROUPS**.

Третий пример на рис. 3.17 демонстрирует добавление в таблицу **GROUPS** строки из результирующего набора, сформированного оператором SELECT.

Последний пример на рис. 3.18 добавляет одну строку. При этом предполагается, что порядок в списке значений, указанных в скобках за ключевым словом VALUES, соответствует исходному (указанному в операторе CREATE) порядку столбцов таблицы **AUDITORIUM_TYPE**.

Для удаления строк из таблицы предназначен оператор DELETE. Структура этого оператора приведена на рис. 3.18.

delete from *таблица дополнение*

Рис. 3.18. Структура оператора DELETE

В результате выполнения оператора DELETE из таблицы удаляется ноль, одна или несколько строк.

На рис. 3.19 приведены три примера применения оператора DELETE.

В первом случае удаляются все строки из таблицы с именем **TTTT**.

Во втором примере из таблицы SUBJECT (ключевое слово FROM может быть опущено) удаляются строки, которые в столбце PULPIT имеют значение, указанное в кавычках после символа =.

```
delete from TTTT;
delete SUBJECT where PULPIT ='ЛЗиДВ'
delete from TEACHER where TEACHER_NAME like '%см%'
```

Рис. 3.19. Примеры использования операторов DELETE

Третий пример демонстрирует удаление из таблицы **TEACHER** строк, в столбце **TEACHER_NAME** которых содержится подстрока, обрамленная символами % в выражении, следующем за ключевым словом LIKE.

Для изменения строк таблицы предназначен оператор UPDATE. Структура этого оператора приведена на рис. 3.20.



Рис. 3.20. Структура оператора UPDATE

В результате выполнения оператора UPDATEв таблице изменяется ноль, одна или несколько строк.

На рис. 3.21 приведены два примера применения оператора UPDATE.

```
update AUDITORIUM set AUDITORIUM_CAPACITY *=1.15 where AUDITORIUM_TYPE = 'ЛБ-К';
update TEACHER set PULPIT = 'ИСиТ';
```

Рис. 3.21. Примеры использования операторов UPDATE

В первом случае изменяются строки таблицы **AUDITORIUM**. Во всех строках таблицы, значение столбца **AUDITORIUM_TYPE** которых равно строке **ЛБ-К,** числовое значение в столбце **AUDITORIUM_CAPACITY** умножается на 1.15.

Во втором примере во всех строках таблицы **TEACHER** значение в столбце **PULPIT** изменяется на строку **ИСиТ**.

3.4. Операторы TCL SQL

Операторы TCL предназначены для создания транзакций. Что такое транзакция подробно будет разобрано позже, но в первом приближении отметим, что транзакция – это несколько DML-операторов, которые либо все успешно выполнятся, либо все не выполняются. TCL SQL включает четыре оператора: BEGIN TRAN, SAVE TRAN, COMMIT TRAN и ROLLBACK TRAN (рис. 3.22).

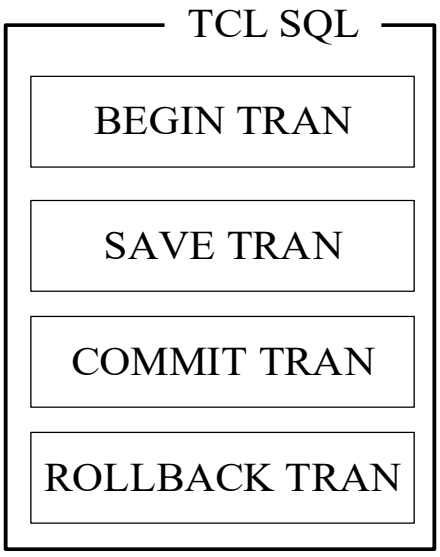


Рис. 3.22. Операторы TCL SQL

Оператор BEGIN TRAN указывает на начало транзакции. Структура оператора представлена на рис. 3.23.



Рис. 3.23. Структура оператора BEGIN TRAN

Предполагается, что за оператором BEGIN TRAN будут следовать один или несколько DML-операторов, входящих в транзакцию.

Снизу группу DML-операторов, входящую в транзакцию, ограничивает один из операторов: COMMIT TRAN или ROLLBACK TRAN.

Оператор COMMIT TRAN фиксирует транзакцию – подтверждает ее успешное завершение. Структура оператора COMMIT TRAN представлена на рис. 3.24.



Рис. 3.24. Структура оператора COMMIT

Оператор ROLLBACK TRAN откатывает транзакцию – отменяет все изменения, произведенные DML-операторами в таблицах БД в рамках данной транзакции. Структура оператора ROLLBACK TRAN представлена на рис. 3.25.



Рис. 3.25. Структура оператора ROLLBACK

Оператор SAVE TRAN применяется для формирования контрольной точки – промежуточного состояния транзакции, к которому может быть осуществлен откат. В тексте SQL-скрипта оператор SAVE TRAN должен находиться между операторами BEGIN TRAN и ROLLBACK TRAN. Структура оператора SAVE TRAN представлена на рис. 3.26.



Рис. 3.26. Структура оператора SAVETRAN

На рис. 3.27 представлен пример, демонстрирующий применение TCL-операторов.

```
begin try
 begin tran T1;
   insert into FACULTY values('ИСиТ', 'Информационных систем и технологий');
   update PULPIT set FACULTY ='ИСиТ' where PULPIT = 'ИСиТ';
   insert into PULPIT  values('ОС', 'Операционных систем', 'ИСиТ');
   insert into [SUBJECT] values ('ОС', 'Операционных систем', 'ИСиТ');
   insert into PULPIT  values('ВМ', 'Вычислительных машин','ИСиТ');
   insert into PULPIT  values('БД', 'Баз данных','ИСиТ');
   update [SUBJECT] set PULPIT ='БД' where [SUBJECT] = 'БД';
 commit tran T1;
end try
begin catch
   print 'Ошибка, выполняется откат';
   rollback tran T1;
end catch;
```

Рис. 3.27. Пример использования операторов BEGIN, COMMIT и ROLLBACK

Транзакция с именем T1 начинается оператором BEGIN TRAN, включает в себя пять операторов INSERT и два оператора UPDATE. В случае успешного выполнения всех DML-операторов, входящих в транзакцию, осуществляется фиксация (оператор COMMIT), иначе управление передается в блок BEGIN CATCH в котором выполняется откат.

3.5. Операторы DCL SQL

Операторы DCL предназначены для управления процессом авторизации. Авторизация – это процедура проверки разрешений (permission) на выполнения определенных операций. Авторизация может выполняться на двух уровнях: на уровне сервера или на уровне БД. Каждому уровню соответствует свой перечень операций, которыми можно управлять с помощью DCL.

Разрешениями могут обладать специальные объекты сервера или БД, которые объединены под общим названием – принципалы. Или иначе, принципал – это объект сервера или БД, которому может быть выдано разрешение на выполнение операции, а также отобрано или запрещено разрешение.

Все операции на уровне сервера или БД выполняются от имени принципалов. Например, при подключении к серверу любой пользователь сервера обязан пройти процедуру аутентификации (проверку подлинности), которая в простейшем случае сводится к вводу имени и пароля. После этого все действия пользователя на уровне сервера выполняются от имени объекта сервера, имеющего тип Login (учетная запись), а на уровне БД от имени объекта, имеющего тип User (пользователь БД). Объекты типов Login и User – принципалы, которым могут быть выданы разрешения.

Управление разрешениями осуществляется тоже от имени принципала, обладающего соответствующими разрешениями. Таким образом, принципалы могут быть связаны между собой отношением «выдал разрешение – получил разрешение». В начале цепочки этих связей находятся предопределенные принципалы, обладающие исключительными разрешениями и соответствующие учетным записям администраторов сервера.

DCL включает в себя три оператора GRANT, REVOKE и DENY (рис. 3.26).

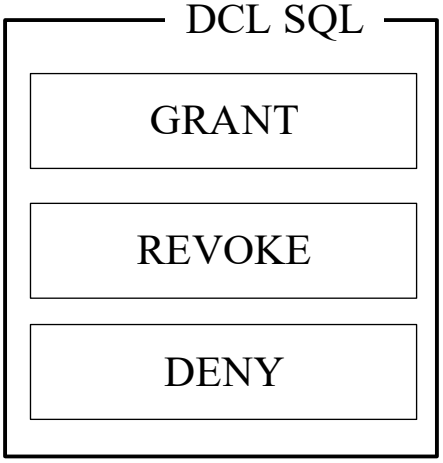


Рис. 3.28. Операторы DCL SQL

Оператор GRANT предназначен для выдачи разрешений. Структура этого оператора представлена на рис. 3.29.



Рис. 3.29. Структура оператора GRANT

После ключевого слова GRANT следует список разрешений, который будет назначен принципалу, имя которого указывается после ключевого слова TO.

Отобрать разрешение у принципала можно с помощью оператора REVOKE, структура которого представлена на рис. 3.30.



Рис. 3.30. Структура оператора REVOKE

После ключевого слова REVOKE следует список разрешений, который будет отобран у принципала, имя которого указывается после ключевого слова FROM.

Оператор DENY не входит в состав стандарта SQL и может применяться только в рамках Microsoft SQL Server. Он позволяет запретить разрешения, выданные ранее. Структура этого оператора представлена на рис. 3.31.



Рис. 3.31. Структура оператора DENY

После ключевого слова DENY следует список разрешений, который будет запрещен у принципала, имя которого указывается после ключевого слова TO.

На рис. 3.32 представлены примеры использования DCL-операторов. Инструкция USE, применяемая в начале скрипта, переключает текущий контекст на системную БД с именем **MASTER**. Далее с помощью оператора CREATE LOGIN создается объект типа Login (учетная запись сервера, принципал) с именем **BSTULogin**.

Следующий далее оператор USE переключает текущий контекст на пользовательскую БД с именем **BSTU**, а потом с помощью оператора CREATE USER создается объект типа USER (пользователь БД, принципал) с именем **BSTUUser**, который с помощью ключевых слов FOR LOGIN ссылается на объект типа Login с именем **BSTULogin**.

```
use MASTER;
create login BSTULogin
       with password = '1234567',
       check_policy = off,
       default_database = BSTU;
use BSTU;
create user BSTUUser
       for login BSTULogin
       with default_schema = dbo;

grant insert, update, delete, select on schema::DBO to BSTUUser with grant option;
deny  insert, update, delete on DBO.FACULTY to  BSTUUser;
```

Рис. 3.32. Пример использования операторов GRANT и DENY

Оператор GRANT на рис. 3.32 разрешает выполнять пользователю **BSTUUser** все DML-операторы применительно ко всем объектам схемы с именем **DBO**.

Следующий за оператором GRANT оператор DENY запрещает выполнять DML-операторы применительно к таблице **DBO.FACULTY**, находящейся в схеме **DBO**.

Таким образом, в результате пользователь **BSTUUser** имеет право применять DML-операторы ко всем объектам схемы **DBO** за исключением таблицы с именем **DBO.FACULTY**.

Следует обратить внимание на конструкцию WITH GRANT OPTION, применяемую в операторе GRANT, указывающую на то, что пользователь **BSTUUser** вместе с разрешением на выполнение DML-операций приобретает административные права на полученные разрешения. Другими словами, пользователь **BSTUUser** с этого момента имеет право выдавать разрешения (с помощью оператора GRANT) на выполнение DML-операторов применительно к объектам схемы **DBO** (за исключением таблицы **BSTUUser**) другим пользователям, а также отбирать и запрещать эти разрешения (REVOKE и DENY).

На рис. 3.33 представлен пример использования оператора REVOKE.

```
use BSTU;
revoke insert, update, delete on schema::DBO to BSTUUser cascade;
```

Рис. 3.33. Пример использования оператора REVOKE

Оператор REVOKE применяется к пользователю **BSTUUser** БД **BSTU**. Оператор отбирает право у пользователя на выполнение DML-операторов применительно к объектам схемы **DBO**.

Ключевое слово CASCADE используется в тех случаях, когда вместе с разрешениями пользователь обладает административным правом на отбираемые разрешения. После выполнения оператора REVOKE CASCADE пользователь утрачивает не только разрешения, но и административные права на них. Более того, разрешения и административные права отбираются у всех цепочек принципалов, которые унаследовали от пользователя **BSTUUser** эти разрешения.

Операторы DCL напрямую связаны с архитектурой информационной безопасности СУБД. Более подробное изучение операторов DCL выходит за рамки данного пособия. Для самостоятельного изучения архитектуры безопасности Microsoft SQL Server 2008, а также возможностей, предоставляемых DCL-операторами, рекомендуется ознакомиться с изданиями [4, 5].

Глава 4. СОЗДАНИЕ И НАСТРОЙКА БАЗЫ ДАННЫХ

4.1. Создание базы данных

Все БД, управляемые сервером Microsoft SQL Server 2008, можно разбить на две группы: системные и пользовательские. Системные БД создаются при инсталляции сервера и используются сервером в процессе его работы. Особую роль среди них играет БД **MASTER**, хранящая информацию обо всех остальных БД, управляемых сервером.

Для создания пользовательской БД применяется оператор CREATE DATABASE, который должен выполняться в контексте системной БД **MASTER**. В простейшем случае создание БД выглядит так, как это представлено на рис. 4.1.

```
use  master                                  -- переключить в контекст БД MASTER
go

create  database BSTU                        -- создать БД BSTU
on  primary                                  -- первичная файловая группа
  (
    name = N'BSTU_mdf',                      -- логическое имя файла
    filename  = N'C:\DB_BSTU\BSTU_mdf.mdf', -- физический файл ОС
    size  = 10240KB,                         -- первоначальный размер файла
    maxsize = UNLIMITED,                     -- максимальный размер файла
    filegrowth = 1024KB                      -- приращение
  )
 log  on                                     -- журнал транзакций
  (
    name = N'BSTU_log',                      -- логическое имя файла
    filename = N'C:\DB_BSTU\BSTU_log.ldf', -- физический файл ОС
    size = 10240KB,                          -- первоначальный размер файла
    maxsize = 2048GB,                        -- максимальный размер файла
    filegrowth = 10%                         -- приращение
  )
go
```

Рис. 4.1. Пример создания базы данных

С помощью кода на рис. 4.1 создается база данных с именем **BSTU**. БД представляет собой набор файлов операционной системы трех типов: первичный файл (расширение **mdf**), вторичные файлы (**ndf**) и файлы журнала транзакций (**log**).

Все файлы БД, кроме файлов журнала транзакций, распределены по файловым группам. Файловые группы – это поименованный набор файлов БД. Журнал транзакций в операторе CREATE DATABASE описывается отдельно в секции, обозначенной ключевыми словами LOG ON. Обязательной является файловая группа, называемая первичной. Для ее обозначения используются ключевые слова ON PRIMARY.

Описание каждого файла БД включает несколько атрибутов: логическое имя (заданное ключевым словом **name**), файл операционной системы (**filename**), а также первоначальный размер (**size**), максимальный размер (**maxsize**), приращение размера (**filegrowth**), заданное абсолютной величиной или в процентах по отношению к первоначальному размеру. Обязательными являются только первичный файл БД, всегда расположенный в первичной файловой группе, и файл журнала транзакций.

Первичный файл содержит всю информацию, необходимую для нормального функционирования БД, а также данные (например, строки таблиц).

Файл журнала транзакций используется для восстановления данных БД в случае аварийного завершения работы сервера.

Разработчик БД помимо первичной файловой группы может создавать дополнительные файловые группы, называемые вторичными. Во вторичных файловых группах могут располагаться только вторичные файлы. В первичной файловой группе помимо обязательного первичного файла тоже могут быть расположены вторичные файлы (рис. 4.2).

```
create  database BSTU         -- создать БД BSTU
on  primary        -- первичная файловая группа
  (   -- первичный файл (mdf)
    name = N'BSTU_mdf',filename  = N'C:\DB_BSTU\BSTU_mdf.mdf',
    size  = 10240KB, maxsize = UNLIMITED, filegrowth = 1024KB
   ),
   ( -- вторичный файл (nfd)
    name = N'BSTU_ndf',filename  = N'C:\DB_BSTU\BSTU_ndf.ndf',
    size  = 10240KB, maxsize = 1GB, filegrowth = 25%
   ),
 filegroup FG1      -- вторичная файловая группа
 ( -- вторичный файл (nfd)
    name = N'BSTU_fg1_1',filename  = N'C:\DB_BSTU\BSTU_fg1-1.ndf',
    size  = 10240KB, maxsize = 1GB, filegrowth = 25%
  ),
  ( -- вторичный файл (nfd)
    name = N'BSTU_fg1_2',filename  = N'C:\DB_BSTU\BSTU_fg1-2.ndf',
    size  = 10240KB, maxsize = 1GB, filegrowth = 25%
  )
 log  on           -- журнал транзакций
   ( --  файл журнала транзакций (log)
     name = N'BSTU_log',filename = N'C:\DB_BSTU\BSTU_log.ldf',
     size = 10240KB, maxsize = 2048GB, filegrowth = 10%
   )
```

Рис. 4.2. Пример создания базы данных, использующей вторичную группу и вторичные файлы

На рис. 4.2 приведен пример оператора CREATE DATABASE, включающего пять файлов: один первичный, три вторичных файла и один файл журнала транзакций. Файлы (кроме файла журнала транзакций) распределены по двум файловым группам: в первичной файловой группе находятся один первичный и один вторичный файлы; во вторичной файловой группе с именем **FG1** – два вторичных файла.

Файловые группы используются в основном для упрощения администрирования БД. Существуют операторы (например, копирования и восстановления БД), позволяющие рассматривать файловую группу как единое целое и выполнять операции не для каждого файла по отдельности, а сразу для файловой группы. Кроме того, файловые группы применяются при секционировании таблиц – технологии, позволяющей разместить фрагменты таблицы в разных файловых группах, что при некоторых дополнительных условиях позволяет повысить производительность запросов к этой таблице.

При создании таблиц и индексов дисковая память для них автоматически отводится в файловой группе по умолчанию. Для размещения этих объектов БД в другой файловой группе следует явно указывать ее имя в операторе CREATE, создающем таблицу или индекс. Кроме того, с помощью оператора ALTER DATABASE можно сделать другую файловую группу, которая будет использоваться по умолчанию (рис. 4.3).

```
use  master            -- переключить в контекст БД MASTER
go

alter database BSTU
     modify  filegroup FG1 default -- FG1  по умолчанию
go
```

Рис. 4.3. Назначение вторичной файловой группы FG1 файловой группой по умолчанию

На рис. 4.3 приведен пример использования оператора ALTER DATABASE, позволяющего назначить для БД **BSTU** файловой группой по умолчанию вторичную группу с именем **FG1**.

Возможности оператора CREATE DATABASE значительно шире, чем описаны в этом пособии. Для знакомства с их полным перечнем рекомендуется воспользоваться пособием [5].

4.2. Изменение базы данных

Изменить существующую БД можно с помощью оператора ALTER DATABASE.

Установка параметров базы данных. БД характеризуется не только структурой ее файлов, определяемой при ее создании с помощью оператора CREATE DATABASE, но и набором параметров. Начальные значения (по умолчанию) параметров устанавливаются при создании БД. Изменить их значения можно с помощью оператора ALTER DATABASE SET. Все параметры можно разбить на несколько групп: параметры автоматических действий, параметры курсора, параметры внешних действий, параметры восстановления, параметры SQL, параметры моментальных снимков, параметры компонента Service Broker и параметры доступности БД.

На рис. 4.4 представлен пример установки (изменения значений, установленных ранее) некоторых параметров, относящихся к группе параметров автоматических действий.

```
use MASTER
go

alter database BSTU set AUTO_CLOSE off            -- автоматическое закрытие БД
alter database BSTU set AUTO_CREATE_STATISTICS on -- автоматический сбор статистики
alter database BSTU set AUTO_SHRINK on            -- автоматичекое сжатие файлов
go
```

Рис. 4.4. Установка параметров группы автоматических действий

Каждый из устанавливаемых параметров может принимать только два значения ON (включен) и OFF (выключен).

Выключение параметра AUTO_CLOSE приводит к предотвращению закрытия БД при отсутствии с ней соединений. При включении этого параметра (ON) происходит автоматическое закрытие БД BSTU (освобождение всех динамических ресурсов сервера, связанных с БД BSTU) при отключении последнего пользователя. С одной стороны включение этого параметра позволяет экономить ресурсы сервера, с другой – снижает производительность работы, т. к. каждое первое после закрытия БД соединение будет приводить к трудоемкой операции сервера – новому открытию (выделению ресурсов сервера).

Параметр AUTO_CREATE_STATISTICS позволяет включить или отключить автоматический сбор статистики. В процессе своей работы сервер собирает статистику, которую использует специальная компонента сервера – оптимизатор запросов. Собранная сервером статистика позволяет оптимизатору запросов принимать решения о способах выполнения SQL-запросов к БД.

Включение параметра AUTO_SHRINK указывает серверу на необходимость периодического сжатия файлов БД.

С помощью встроенной функции DATABASEPROPERTYEX всегда можно получить текущие установки параметров БД. На рис. 4.5 приведен пример использования функции DATABASEPROPERTYEX в SELECT-запросе, а также результат выполнения этого запроса.

```
use MASTER
go

select  case DATABASEPROPERTYEX('BSTU', 'IsAutoClose')
        when 0 then 'off'      -- если = 0, то вернуть 'off'
        else 'on'              -- иначе  вернуть 'on'
        end   IsAutoClose,
        case DATABASEPROPERTYEX('BSTU', 'IsAutoShrink')
        when 0 then 'off'
        else 'on'
        end  IsAutoShrink,
        case DATABASEPROPERTYEX('BSTU', 'IsAutoCreateStatistics')
        when 0 then 'off'
        else 'on'
        end  IsAutoCreateStatistics
  go
```

IsAutoClose	IsAutoShrink	IsAutoCreateStatistics
off	on	on

Рис. 4.5. Отображение текущих значений параметров БД с помощью встроенной функции DATABASEPROPERTYEX

Функция DATABASEPROPERTYEX может возвращать два значения: 1 (соответствует включенному состоянию) и 0 (выключенное состояние). В SELECT-запросе используется оператор CASE, который будет подробно рассматриваться позже. Здесь же поясним: с его помощью в результате запроса, представленного на рис. 4.5, осуществляется замена числа 1 на строку ON, а числа 0 на OFF.

Для изучения параметров рекомендуется пособие [5].

Глава 5. СОЗДАНИЕ, УДАЛЕНИЕ И МОДИФИКАЦИЯ ТАБЛИЦ

5.1. Принципы проектирования логической схемы базы данных

На рис. 5.1 изображена трехуровневая модель БД, отражающая три взгляда на понятие «база данных».

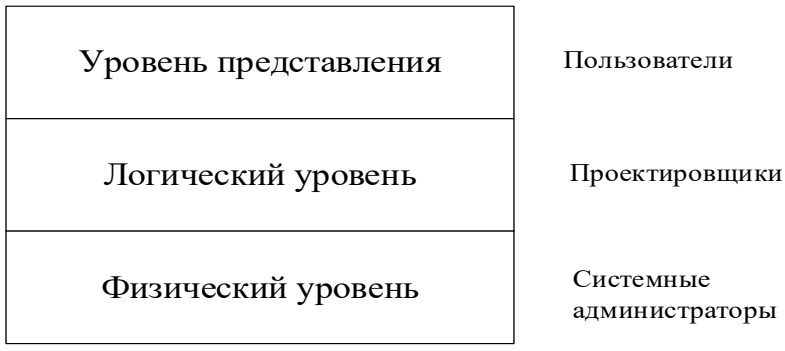


Рис. 5.1. Трехуровневая модель базы данных

С точки зрения системного администратора БД – это набор файлов операционной системы, размещенных на магнитных носителях и имеющих определенную физическую структуру, формат и организацию.

С точки зрения проектировщика БД – это логическая схема данных, представляющая собой набор специальным образом типизированных и структурированных данных с заданными

ограничениями и связями. Вид логической схемы зависит от выбранной проектировщиком модели данных. Наиболее известны три модели: иерархическая, сетевая и реляционная. Наиболее распространенной является реляционная модель, использующая в качестве структур данных таблицы, а для доступа к данным – язык SQL. СУБД Microsoft SQL Server 2008 поддерживает реляционную модель данных.

Пользователю БД может быть представлена различными способами: в простейшем случае в виде подмножества таблиц или представлений (объекты БД, о которых будет рассказано позже), набора процедур и функций на языке **T-SQL** и пр. В общем случае, пользователь БД видит то, что предоставляет используемый им для доступа к данным инструментарий и применяемый интерфейс с БД. Если пользователь – разработчик программного приложения, то его представление во многом зависит от применяемого им программного интерфейса. Если пользователь работает с табличным редактором **Excel**, то он может применить для доступа к данным **PowerPivot** – специальное программное средство, встраиваемое в **Excel**. Кроме того, представление пользователя зависит от настроек системы безопасности СУБД, регулирующей (часто говорят авторизующей) область видимости данных и порядок доступа к ресурсам БД.

Теория реляционных БД предполагает независимость трех уровней друг от друга. Другими словами физическое размещение данных (физический уровень) никак не должно ограничивать возможности проектирования логической схемы данных, а представления пользователей о БД не зависят от структуры логической схемы. В реальности современные СУБД обеспечивают лишь частичную независимость уровней друг от друга.

Центральное место в рассматриваемой на рис. 5.1 модели занимает логический уровень. Именно здесь определяется состав данных и связи между ними. Процесс проектирования логической схемы реляционной БД заключается в представлении данных в виде набора логически связанных таблиц.

Теория реляционных БД предлагает формализованную процедуру разбиения данных на таблицы. Процедура называется нормализацией и представляет собой поэтапное преобразование логической схемы БД. В результате каждого этапа логическая схема преобразует таблицы (обычно путем вертикального разбиения одной таблицы на несколько) к нормальной форме. Теория различает шесть нормальных форм реляционной таблицы. В практике проектирования применятся только три. Поэтому нормализация заключается в приведении таблиц логической схемы БД к третьей нормальной форме.

Процесс нормализации осуществляется в три этапа: исходными являются таблицы в первой нормальной форме, на втором этапе эти таблицы преобразовываются ко второй, а на третьем – к третьей.

Таблицы в первой нормальной форме построить просто – достаточно все данные разместить в таблицах с неповторяющимися строками. Преобразование таблиц ко второй и затем к третьей нормальным формам основывается на фундаментальном понятии теории реляционных БД – функциональной зависимости данных, изучение которой не предусматривается в данном пособии. Здесь лишь поясним, что процесс нормализации – это процесс устранения избыточности данных (хотя и сопровождается увеличением количества таблиц), позволяющий построить логическую схему БД, предотвращающую неоднозначные (противоречивые) результаты SELECT-запросов.

Для знакомства с основами теории реляционных БД, а также с процессом нормализации рекомендуется прочитать ставшую классической книгу К. Дейта [2].

5.2. Создание таблиц

Таблица в БД создается с помощью DDL-оператора CREATE TABLE.

На рис. 5.2 приведен пример создания таблицы **AUDITORIUM**, предназначенной для хранения данных об аудиторном фонде вуза.

```
create table AUDITORIUM  -- аудитории вуза
(
     AUDITORIUM          char(20)   -- идентификатор аудитории (304-1, 401-4,...)
                         constraint AUDITORIUM_PK  primary key,
     AUDITORIUM_TYPE     char(10)   -- тип аудитории  (ЛК, ЛБ-К, ...)
                         constraint  AUDITORIUM_AUDITORIUM_TYPE_FK foreign key
                         references AUDITORIUM_TYPE(AUDITORIUM_TYPE),
     AUDITORIUM_CAPACITY int        -- вместимость (макс. кол. слушателей)
                         constraint  AUDITORIUM_CAPACITY_CHECK default 1
                         check (AUDITORIUM_CAPACITY between 1 and 300),
     AUDITORIUM_NAME     varchar(50) -- текстовое наименование аудитории (комментарий)
) on FG1; -- в файловой группе FG1
```

Рис. 5.2. Пример создания таблицы

Таблица **AUDITORIUM** включает четыре столбца с именами: **AUDITORIUM**, **AUDITORIUM_TYPE, AUDITORIUM_CAPACITY, AUDITORIUM_NAME**.

Следует обратить внимание на то, что таблица будет размещена в файловой группе с именем **FG1**.

5.2.1. Типы данных

Каждый столбец таблицы, создаваемой с помощью оператора CREATE TABLE, характеризуется типом данных, для хранения которых он предназначен. Microsoft SQL Server 2008 поддерживает следующие типы данных: числовые, символьные, для хранения даты и времени, денежные, двоичные и специальные.

5.2.1.1. Числовые данные. Числовые данные могут быть точными и приближенными. В табл. 5.1 сведены точные числовые типы. Все, кроме DECIMAL и NUMERIC, типы являются целочисленными. Тип NUMERIC используется для хранения данных с фиксированной точкой точности p (максимальное количество цифр в числе) и масштабом s (количество цифр после десятичной точки). Тип DECIMAL является синонимом NUMERIC.

Таблица 5.1

Точные числовые типы

Тип данных	Диапазон значений	Количество байт
tinyint	0–255	1
smallint	–32 768–32 768	2
int	-2^{31}–$(2^{31}-1)$	4
bigint	-2^{63}–$(2^{63}-1)$	8
bit	0 или 1	1
decimal(p,s) numeric(p,s) $1 \le p \le 38$, $0 \le s < p$	$(-10^{38}+1)-(10^{38}+1)$	5–17

На рис. 5.3 приведен пример использования точных числовых типов для столбцов таблицы. После создания таблицы с помощью операторов INSERT в нее добавляются три строки, а затем с помощью SELECT содержимое таблицы выводится в результирующий набор (рис. 5.4).

```
use TEMPDB
go
create table EXACT_NUMBER
(
 N_BIT      bit,
 N_TINYINT  tinyint,
 N_SMALLINT smallint,
 N_INT      int,
 N_BIGINT   bigint,
 N_DECIMAL  decimal(10,3)
 );
 go
 insert into EXACT_NUMBER values (1, 123, 12345, 1234567890, 123456789012345678, 1234567.890);
 insert into EXACT_NUMBER values (-1, 123, -12345, -1234567890, -123456789012345678, -1234567.123);
 insert into EXACT_NUMBER values (0, 123, 12345, 1234567890, 123456789012345678, 1234567.1234);
 go
 select * from EXACT_NUMBER;
 go
```

Рис. 5.3. Пример использования точных числовых типов

N_BIT	N_TINYINT	N_SMALLINT	N_INT	N_BIGINT	N_DECIMAL
1	123	12345	1234567890	123456789012345678	1234567.890
1	123	-12345	-1234567890	-123456789012345678	-1234567.123
0	123	12345	1234567890	123456789012345678	1234567.123

Рис. 5.4. Результат выполнения SELECT-запроса на рис. 5.3

Следует обратить внимание, что ввод любых отличных от нуля (положительных и отрицательных) значений в столбец типа BIT интерпретируется как единица, а ввод числа с четырьмя знаками после точки в столбец DECIMAL(10,3) приводит к округлению до трех знаков.

В табл. 5.2 сведены приближенные числовые типы. Для типа FLOAT можно указать точность p, но поддерживается только два значения точности: 24 (для всех $1 \le p \le 24$) и 53 ($p > 24$). По сути, приближенный тип только один – FLOAT, тип REAL является синонимом типа FLOAT(24).

Таблица 5.2

Приближенные числовые типы

Тип данных	Диапазон значений	Количество байт
float(p) $1 \le p \le 53$	От $-1{,}79 \cdot 10^{308}$ до $-2{,}23 \cdot 10^{-308}$; 0; от $2{,}23 \cdot 10^{-308}$ до $1{,}79 \cdot 10^{308}$	4 или 8
real float(24)	От $-3{,}4 \cdot 10^{38}$ до $-1{,}18 \cdot 10^{-38}$; 0; от $1{,}18 \cdot 10^{-38}$ до $3{,}4 \cdot 10^{38}$	4

На рис. 5.5 приведен пример использования приближенных числовых типов для столбцов таблицы. После создания таблицы с помощью операторов INSERT в нее добавляются шесть строк, а затем с помощью SELECT содержимое таблицы выводится в результирующий набор (рис. 5.6).

```
use TEMPDB
go
create table APPROXIMATE_NUMBER
(
 N_FLOAT    float(53),
 N_REAL     real
 );
 go

 insert into APPROXIMATE_NUMBER values (1.1,2.2);
 insert into APPROXIMATE_NUMBER values (1.123456789012E10,2.123456789012E12);
 insert into APPROXIMATE_NUMBER values (112.3456789012*power(10.0,10),2.123456789012*power(10.,12));
 insert into APPROXIMATE_NUMBER values (1.123E-8,2.12345E-10);
 insert into APPROXIMATE_NUMBER values (-1.123E-8,-2.12345E-10);
 insert into APPROXIMATE_NUMBER values (0.0000123, .12345);
 select * from APPROXIMATE_NUMBER;
```

Рис. 5.5. Пример использования приближенных числовых типов

Следует обратить внимание на формат представления чисел, применяемых в операторах INSERT. Символы 1.123E-8 следует интерпретировать как $1{,}123 \cdot 10^{-8}$. Кроме того, в третьем операторе INSERT применяется встроенная функция POWER(X,Y), результатом которой является число равное X^Y.

N_FLOAT	N_REAL
1,1	2,2
11234567890,12	2,123457E+12
1123456789012	2,123457E+12
1,123E-08	2,12345E-10
-1,123E-08	-2,12345E-10
1,23E-05	0,12345

Рис. 5.6. Результат выполнения SELECT-запроса на рис. 5.5

5.2.1.2. Символьные данные. Для хранения символьных данных в таблицах БД применяется шесть типов, перечисленных в табл. 5.3.

Таблица 5.3

Символьные типы

Тип данных	Размер в символах	Количество байт
char(n)	1–8000	*n*
varchar(n)	1–4000	Количество символов + 2
varchar(max)	$1–(2^{31}-1)$	Количество символов + 2
nchar(n)	1–8000	2*n*
nvarchar(n)	1–4000	2·количество символов + 2
nvarchar(max)	$1–(2^{30}-1)$	2·количество символов + 2

Все символьные типы данных можно разбить на две группы: обычные символьные данные (CHAR, VARCHAR), символьные данные в формате Юникода (NCHAR, NVARCHAR). Обычные символьные данные для хранения одного символа используют один байт. Символьные данные в формате Юникода для хранения одного символа использует два байта.

Типы данных VARCHAR(MAX) и NVARCHAR(MAX) применяются для хранения больших символьных данных объемом до 4 Гб.

На рис. 5.7 приведен пример использования символьных типов. Таблица CHARACTER_DATA состоит из шести столбцов, имеющих различные символьные типы (табл. 5.3). С помощью оператора INSERT в таблицу добавляется одна строка. Следует обратить

внимание на литералы, используемые в списке VALUES оператора INSERT: одинарные кавычки ограничивают содержимое строки; символ N, примененный в трех последних элементах списка, обозначает, что далее следует строка в формате Юникода. Обратите внимание, что во все столбцы строки вводятся строки, имеющие по пять символов.

```
use TEMPDB
go

create table CHARACTER_DATA
(
 C_CHAR       char(100),
 C_VARCHAR    varchar(200),
 C_VARCHARM   varchar(max),
 C_NCHAR      nchar(100),
 C_NVARCHAR   nvarchar(200),
 C_NVARCHARM nvarchar(max)
 );
 go

 insert into CHARACTER_DATA values ('12345', '12345', '12345', N'12345', N'12345', N'12345')

 select * from  CHARACTER_DATA
 select datalength(C_CHAR)     [C_CHAR (байты)],  datalength(C_VARCHAR) [C_VARCHAR (байты)],
        datalength(C_VARCHARM) [C_VARCHARM(байты)],
        datalength(C_NCHAR) [C_NCHAR (байты)],   datalength(C_NVARCHAR) [C_NVARCHAR (байты)],
        datalength(C_NVARCHARM) [C_NVARCHARM(байты)] from  CHARACTER_DATA;

 select len(C_CHAR)     [C_CHAR (симв)],   len(C_VARCHAR) [C_VARCHAR (симв)],
        len(C_VARCHARM) [C_VARCHARM(симв)],
        len(C_NCHAR)     [C_NCHAR (симв)], len(C_NVARCHAR) [C_NVARCHAR (симв)],
        len(C_NVARCHARM) [C_NVARCHARM(симв)] from  CHARACTER_DATA;
```

Рис. 5.7. Пример использования символьных типов

За оператором создания таблицы следует три оператора SELECT: первый выводит строку из таблицы, второй – длину в байтах (встроенная функция DATALENGTH) каждого элемента этой строки и третий оператор выводит длину в символах (встроенная функция LEN). Результат выполнения трех SELECT-запросов представлен на рис. 5.8.

C_CHAR	C_NCHAR	C_VARCHAR	C_VARCHARM	C_NVARCHAR	C_NVARCHARM
12345	12345	12345	12345	12345	12345

C_CHAR (байты)	C_VARCHAR (байты)	C_VARCHARM(байты)	C_NCHAR (байты)	C_NVARCHAR (байты)	C_NVARCHARM(байты)
100	5	5	200	10	10

C_CHAR (симв)	C_VARCHAR (симв)	C_VARCHARM(симв)	C_NCHAR (симв)	C_NVARCHAR (симв)	C_NVARCHARM(симв)
5	5	5	5	5	5

Рис. 5.8. Результат выполнения SELECT-запросов на рис. 5.7

При использовании «широких» типов данных следует помнить об ограничении, которое накладывается на таблицы: указанная при создании суммарная длина строки таблицы не должна превышать 8060 байтов. В расчет не включаются столбцы с типами VARCHAR(MAX) и NVARCHAR(MAX).

5.2.1.3. Типы для хранения даты и времени. Для хранения даты и/или времени применяются типы данных, представленные в табл. 5.4. Для каждого типа указывается диапазон,

точность и формат строки, который может быть автоматически преобразован к соответствующему типу.

Таблица 5.4

Типы для даты и времени

Тип данных	Диапазон, точность, формат	Количество байт
date	01.01.1753–31.12.9999; 1 день; YYYYMMDD	3
time(*p*) $0 \le p \le 7$	00:00:00.0000000–23:59:59.9999999; 100 нс; hh:mm:ss.nnnnnnn	3–5
smalldatetime	01.01.1900 00:00–06.06.2079 23:59, 1 мин; YYYYMMDD hh:mm	4
datetime	01.01.1753 00:00:00.000–31.12.9999 23:59:59.999; 0.003 с; YYYYMMDD hh:mm:ss.nnn	8
datetime2(*p*) $0 \le p \le 7$	01.01.0001 00:00:00.00000000–31.12.9999 23:59:59.9999999; 100 нс; YYYYMMDD hh:mm:ss.nnnnnnn	6–8
datetimeoffset(*p*) $0 \le p \le 7$	01.01.0001.00:00:00:00000000+00:00–31.12.9999.23:59:59:9999999+23:59; 100 нс; YYYYMMDDhh:mm:ss:nnnnnnn ± hh:mm	8–10

На рис. 5.9 и 5.10 приведен пример использования типов для хранения даты и времени в таблицах БД. Для некоторых типов можно указать точность (в табл. 5.4 обозначается символом *p*), которая задает количество знаков после десятичной точки в секундах.

```
create table DATE_TIME_DATA
(
 D_DATE            date,
 D_TIME            time(7),
 D_SMALLDATETIME   smalldatetime,
 D_DATETIME        datetime,
 D_DATETIME2       datetime2(7),
 D_DATETIMEOFFSET  datetimeoffset(7)
 );
 go
 insert into DATE_TIME_DATA
        values (
                  '20140107',                          -- date
                  '23:59:55.1234567',                  -- time(7)
                  '20140107 23:59',                    -- smalldatetime
                  '20140107 23:59:59.123',             -- datetime
                  '20140107 23:59:59.1234567',         -- datetime2(7)
                  '20140107 23:59:59.1234567+02:30'    -- datetimeoffset(7)
                );
 select * from  DATE_TIME_DATA
```

Рис. 5.9. Пример применения типов для хранения даты и времени

D_DATE	D_TIME	D_SMALLDATETIME	D_DATETIME	D_DATETIME2	D_DATETIMEOFFSET
2014-01-07	23:59:55.1234567	2014-01-07 23:59:00	2014-01-07 23:59:59.123	2014-01-07 23:59:59.1234567	2014-01-07 23:59:59.1234567 +02:30

Рис. 5.10. Результат использования SELECT-запроса на рис. 5.9

Следует обратить внимание на формат данных в строковых литералах, используемых при вводе или корректировке (операторы INSERT и UPDATE) даты и времени. Приведенные в табл. 5.4 и на рис. 5.9 форматы не являются единственно возможными. В пособии [5] можно ознакомиться с другими формами представления даты и времени в виде строки. Кроме того, можно воспользоваться встроенными функциями преобразования **CAST** или **CONVERT** (будут рассматриваться ниже), с помощью которых можно явно преобразовать строку к одному типу из строки в дату и/или время.

5.2.1.4. Двоичные данные. В некоторых случаях в БД необходимо хранить просто последовательность битов безотносительно их дальнейшего применения. Другими словами Microsoft SQL Server 2008 ничего не «знает» о формате таких данных и рассматривает их как последовательность (часто говорят – поток) битов. О том, как использовать (интерпретировать) эти данные, решает пользователь, извлекающий эти данные из соответствующего столбца таблицы. Типы для хранения двоичных данных (говорят также – бинарных данных) часто применяются для хранения изображений (например, фотографий, фильмов) и звука. В то же время в двоичном виде можно хранить любые данные, например, файлы форматов **PDF**, **DOC**, **XLS**.

Типы, которые используются для хранения бинарных данных, перечислены в табл. 5.5.

Таблица 5.5

Типы для хранения двоичных данных

Тип данных	Размер в байтах	Количество байт
binary(n)	1–8000	n
varbinary(n)	1–8000	Количество символов + 2
varbinary(max)	$1–(2^{31} – 1)$	Количество символов + 2

Тип BINARY применяется в том случае, если размер двоичных данных фиксирован, заранее известен и не превышает 8000 байт. В тех случаях, когда нет предположений относительно размера двоичных данных, применяют тип VARBINARY.

На рис. 5.11 и 5.12 приведен пример использования типов для хранения двоичных данных.

```
use TEMPDB
go
create table BINARY_DATA
(
 B_BINARY      binary(400),
 B_VARBINARY   varbinary(400),
 B_VARBINARYM  varbinary(max)
 );
 go
 insert into BINARY_DATA
      values(
             0x123456789012345,
             0x123456789012345,
             (select *  from openrowset(bulk N'c:\SQLLoad\picture.jpg', SINGLE_BLOB)pic)
             );
 select datalength(B_BINARY), datalength(B_VARBINARY), datalength(B_VARBINARYM) from  BINARY_DATA
```

Рис. 5.11. Пример использования типов для хранения двоичных данных

binary(байты)	varbinary(байты)	varbinary(max)(байты)
400	8	8549

Рис. 5.12. Результат выполнения SELECT-запроса на рис. 5.9

5.2.1.5. Типы данных TIMESTAMP и ROWVERSION. По определению, строки в таблицах БД не имеют порядка. Пользователь, который выбирает строки из одной или нескольких таблиц с помощью оператора SELECT, может установить порядок строк в результирующем наборе только с помощью секции ORDER BY этого оператора.

В процессе обработки данных бывает необходимым установить хронологию изменения данных. Другими словами, необходима возможность отсортировать строки в порядке их ввода или последнего изменения или просто выяснить, какая из двух строк была введена или откорректирована позже, чем другая. Для решения такой задачи могут быть использованы специальный тип данных ROWVERSION и его синоним TIMESTAMP.

Столбец типа ROWVERSION занимает 8 байт и хранит номер (двоичное представление) изменения БД. Другими словами, если в БД были выполнены две операции, изменяющие (INSERT или UPDATE) одну или две разные таблицы, имеющие столбцы типа ROWVERSION, то значение, хранящееся в таком столбце, будет меньшим в строке, которая была введена или откорректирована раньше, чем введена или откорректирована другая строка.

Значения в столбцах типа ROWVERSION может вводиться и изменяться только сервером, пользователь может только считывать это значение в SELECT-запросе.

На рис. 5.13 и 5.14 представлен пример использования типов ROWVERSION и TIMESTAMP.

```
use TEMPDB
go
create table TIMESTAMP_DATA1
(
  ID    int,
        timestamp     --  имя по умолчанию TIMESTAMP
 );
 go
 create table TIMESTAMP_DATA2
 (
  ID            int,
  T_ROWVERSION  rowversion  --  имя обязательно   ,
 );
 go
 insert into TIMESTAMP_DATA1(ID)    values(1);
 insert into TIMESTAMP_DATA2(ID)    values(1);
 select * from TIMESTAMP_DATA1
 select * from TIMESTAMP_DATA2
```

Рис. 5.13. Пример применения типов ROWVERSION и TIMESTAMP

ID	timestamp
1	0x00000000000007EF

ID	T_ROWVERSION
1	0x00000000000007F0

Рис. 5.14. Результаты выполнения SELECT-запросов на рис. 5.13

5.2.1.6. Тип данных UNIQUEIDENTIFIER. Тип данных UNIQUEIDENTIFIER используется для хранения идентификаторов стандарта **UUID** (Universally Unique Identifier). Идентификаторы типа UNIQUEIDENTIFIER занимают 128 бит и могут быть получены с помощью встроенной функции **NEWID**. Главная особенность алгоритма генерации UUID-идентификаторов (описан в RFC 4122) – способность генерировать уникальные значения, которые с очень малой вероятностью могут быть независимо получены с помощью этого же алгоритма еще раз.

Пример использования типа UNIQUEIDENTIFIER приведен на рис. 5.15 и 5.16.

```
use TEMPDB
go
create table ID_DATA
(
  ID    uniqueidentifier,
  V     varchar(10)
 );
 go

 insert into ID_DATA  values(newid(), 'ABCD');

 select *, datalength(ID)[datalength(ID)] from ID_DATA
```

Рис. 5.15. Пример применения типов ROWVERSION и TIMESTAMP

ID	V	datalength(ID)
0E947E7A-CEDC-4253-8732-4A52A9024CF7	ABCD	16

Рис. 5.16. Результат выполнения SELECT-запроса на рис. 5.15

5.2.1.7. Остальные типы данных. Типы данных, которые используются относительно редко, сведены в табл. 5.6. XML-тип будет рассматриваться в этом пособии позже. С полным описанием остальных типов можно ознакомиться в издании [5].

Таблица 5.6

Редко используемые типы данных

Тип данных	Назначение
smallmoney, money	Денежный тип, разновидность числового типа с фиксированной точкой, предназначен для хранения значения денежных величин
xml	XML-тип предназначен для хранения данных, представленных в формате XML
hierarhied	Иерархический тип данных, используется для отражения иерархических (древовидных) связей между строками таблицы
geography, geometry	Пространственные данные, предназначенные для хранения географических координат, геометрических координат и геометрических (2D) объектов
sqlvariant	Универсальный тип, способный хранить числовые, символьные данные, а также данные для хранения даты и времени. Непосредственно перед использованием, данные могут быть преобразованы к необходимому типу
text, ntext, image	Устаревшие типы данных, поддерживаются для совместимости с предыдущими версиями сервера, заменены на varchar(max) и varbinary(max)

5.2.1.8. Преобразование типов. Для преобразования типов предусмотрены две встроенные функции **CAST** и **CONVERT**. По своим возможностям функции практически не отличаются, но функция **CAST** соответствует стандарту SQL-92, поэтому ее применение предпочтительнее.

На рис. 5.17 и 5.18 приведены примеры использования функций **CAST** и **CONVERT**. Обратите внимание, что в предпоследнем SELECT-запросе в функции **CONVERT** есть третий параметр, который задается числовым значением. Этот параметр применяется при преобразованиях, использующих типы данных для хранения даты и времени, и задает стиль представления этих данных.

```
select cast('-123.123' as numeric(7,3)),   -- символы --> число
       convert(numeric(7,3), '-123.123')
select cast( -123.123  as varchar(12)),    -- число --> символы
       convert(varchar(12),  -123.123)
select cast(getdate() as char(30)),        -- дата --> символы
       convert(char(30),  getdate()),
       convert(char(30),  getdate(), 112) -- 112 - стиль
select cast('20140107 22:00:01.123' as datetime), -- символы --> дата
       convert(datetime,  '20140107 22:00:01.123')
```

Рис. 5.17. Пример использования встроенных функций преобразования типов

(No column name)	(No column name)
-123.123	-123.123

(No column name)	(No column name)
-123.123	-123.123

(No column name)	(No column name)	(No column name)
Jan 2 2014 10:55PM	Jan 2 2014 10:55PM	20140102

(No column name)	(No column name)
2014-01-07 22:00:01.123	2014-01-07 22:00:01.123

Рис. 5.18. Результаты выполнения SELECT-запросов на рис. 5.17

Для более подробного знакомства с функциями преобразования типов данных рекомендуется пособие [5].

5.2.2. Ограничения целостности

Рассмотрим на рис. 5.19 упрощенный пример создания таблицы с именем **S_AUDITORIUM**. Обратим внимание, что столбец с именем **AUDUTORIUM_CAPACITY** имеет тип INT (целочисленное значение).

Попытка ввода в этот столбец значения, которое не может быть автоматически преобразовано к целому числу (рис. 5.20), приводит к ошибке (рис. 5.21), сопровождаемой сообщением сервера.

```
create table S_AUDITORIUM  -- аудитории вуза
    (
         AUDITORIUM          char(20),
         AUDITORIUM_TYPE     char(10),
         AUDITORIUM_CAPACITY int,
         AUDITORIUM_NAME     varchar(50)
    )
```

Рис. 5.19. Таблица, имеющая столбец с типом данных INT

```
insert into S_AUDITORIUM(AUDITORIUM, AUDITORIUM_NAME, AUDITORIUM_TYPE, AUDITORIUM_CAPACITY)
          values  ('206-1', '206-1','ЛБ-К', 'XXX');
```

Рис. 5.20. Добавление строки в таблицу S_AUDITORIUM

```
Msg 245, Level 16, State 1, Line 1
Conversion failed when converting the varchar value 'XXX' to data type int.
```

Рис. 5.21. Сообщение об ошибке при попытке преобразования строки к типу INT

Аналогичный результат был бы получен при попытке с помощью оператора UPDATE заменить существующее целочисленное значение в столбце **AUDITORIUM_CAPACITY** на значение другого типа.

Ограничения, накладываемые на столбцы таблицы, предотвращающие появление в БД данных, не соответствующих предварительно заданным свойствам таблицы, называются ограничениями целостности.

В простейшем случае ограничения целостности не допускают появление в БД данных, не соответствующих типу, как в приведенном выше (рис. 5.19–5.20) примере.

Помимо проверки типа данных при создании таблицы БД имеется возможность описать более сложные ограничения. Все типы таких ограничений целостности перечислены в табл. 5.7.

Таблица 5.7

Ограничения целостности

Условное обозначение ограничения целостности	Действие ограничения целостности
data type тип данных	Предотвращает появление в столбце значений, не соответствующих типу данных
not null запрет значений null	Предотвращает появление в столбце значений null
default значение по умолчанию	Устанавливает значение в столбце по умолчанию при выполнении операции INSERT
primary key первичный ключ	Предотвращает появление в столбце (группе столбцов) повторяющихся значений (комбинации значений) и пустого значения (комбинации пустых значений)
foreign key внешний ключ	Устанавливает связь между таблицей со столбцом, имеющим свойство foreign key (FK) и таблицей, имеющей столбец со свойством primary key (PK); предотвращает не согласованные операции между PK и FK
unique уникальное значение	Аналогично primary key, но допускает пустые значения и не может быть использован для связи с foreign key
check проверка значений	Предотвращается появление в столбце значения, не удовлетворяющего логическому условию

Для ограничений целостности PRIMARY KEY, FOREIGN KEY, UNIQUE и CHECK может быть задано имя, которое при возникновении ошибки, связанной с этим ограничением, будет указано в сообщении сервера. В том случае, если это имя не задано, при создании таблицы сервер назначает ограничениям этих типов собственные имена.

5.2.2.1. Первичный ключ. Одним из требований, предъявляемых к нормализованной таблице, является наличие одного или нескольких столбцов, значения в которых однозначно определяет каждую строку таблицы. Другими словами, в любой нормализованной таблице должен быть столбец (несколько столбцов), в котором значения (комбинация значений) являются разными в разных строках. Столбец или группа столбцов, обладающих таким свойством, называется **ключом**. Причем во втором случае, как правило, говорят о составном ключе.

На рис. 5.22 представлен пример SQL-сценария, демонстрирующий таблицу без первичного ключа. Обратите внимание, что в таблицу с помощью оператора INSERT добавляются две одинаковые строки. Результат SELECT-запроса (рис. 5.23) подтверждает наличие в одной таблице **FACULTY** двух одинаковых строк.

Сценарий на рис. 5.24 отличается от сценария на рис. 5.22 наличием свойства PRIMARY KEY, указанным для столбца **FACULTY.**

```
use BSTU

create table FACULTY  -- факульт
(
   FACULTY      char(10),
   FACULTY_NAME varchar(50)
);
go

insert into FACULTY   (FACULTY,   FACULTY_NAME )
       values  ('ХТиТ','Химическая технология и техника');
insert into FACULTY   (FACULTY,   FACULTY_NAME )
       values  ('ХТиТ','Химическая технология и техника');
select * from FACULTY;
```

Рис. 5.22. Создание таблицы, не имеющей первичного ключа

FACULTY	FACULTY_NAME
ХТиТ	Химическая технология и техника
ХТиТ	Химическая технология и техника

Рис. 5.23. Результат выполнения SELECT-запроса на рис. 5.22

При выполнении второго оператора INSERT, с помощью которого осуществляется попытка ввода строки с таким же значением в столбце **FACULTY**, что и в первом операторе, возникает ошибка, сообщение о которой формируется сервером (рис. 5.25).

```
use BSTU

create table FACULTY  -- факульт
(
   FACULTY      char(10) primary key,
   FACULTY_NAME varchar(50)
);
go

insert into FACULTY   (FACULTY,   FACULTY_NAME )
       values  ('ХТиТ','Химическая технология и техника');
insert into FACULTY   (FACULTY,   FACULTY_NAME )
       values  ('ХТиТ','Химическая технология и техника');
select * from FACULTY;
```

Рис. 5.24. Создание таблицы с первичным ключом

В сообщении об ошибке содержится: код ошибки (2627); уровень серьезности ошибки (14); метка (1); номер строки сценария (4), в которой находится оператор, вызвавший ошибку; сообщение об ошибке.

Следует обратить внимание, что в сообщении после слова CONSTRAINT следует непрерывная строка символов, обозначающих имя нарушенного ограничения, присвоенное сервером при создании таблицы **FACULTY**.

```
Msg 2627, Level 14, State 1, Line 4
Violation of PRIMARY KEY constraint 'PK__FACULTY__81E78829173876EA'. Cannot insert duplicate key in object 'dbo.FACULTY'.
The statement has been terminated.
```

Рис. 5.25. Сообщение об ошибке, сформированное сервером при выполнении сценария на рис. 5.23

Результат выполнения SELECT-запроса (рис. 5.26) подтверждает, что после выполнения двух операторов INSERT была добавлена только одна строка.

FACULTY	FACULTY_NAME
ХТиТ	Химическая технология и техника

Рис. 5.26. Результат выполнения SELECT-запроса на рис. 5.24

Для того, чтобы задать собственное имя ограничению, следует использовать ключевое слово CONTSTRAINT в синтаксической конструкции PRIMARY KEY (рис. 5.27, 5.28).

```
use BSTU

create table FACULTY  -- факульт
(
   FACULTY      char(10) constraint PK_FACULTY_FACULTY primary key,
   FACULTY_NAME varchar(50)
);
go

insert into FACULTY   (FACULTY,   FACULTY_NAME )
       values  ('ХТиТ','Химическая технология и техника');
insert into FACULTY   (FACULTY,   FACULTY_NAME )
       values  ('ХТиТ','Химическая технология и техника');
select * from FACULTY;
```

Рис. 5.27. Применение ключевого слова CONSTRAINT

```
Msg 2627, Level 14, State 1, Line 3
Violation of PRIMARY KEY constraint 'PK_FACULTY_FACULTY'. Cannot insert duplicate key in object 'dbo.FACULTY'.
```

Рис. 5.28. Сообщение об ошибке, содержащее имя ограничения, указанное при создании таблицы на рис. 5.27

Синтаксис оператора CREATE TABLE допускает другую форму записи ограничения (рис. 5.29).

```
create table FACULTY  -- факульт
(
   FACULTY      char(10),     -- идентификатор факультета
   FACULTY_NAME varchar(50), -- полное наименование
   constraint PK_FACULTY_FACULTY primary key(FACULTY)
);
```

Рис. 5.29. Другой способ записи ограничения PRIMARY KEY

На рис. 5.30 представлен пример создания таблицы с составным первичным ключом. В этом случае вторая форма записи ограничения является единственно возможной.

```
create table SHEDULE_TEACHER   --  расписание преподавателя
 (
  [DATETIME]  smalldatetime,   -- дата и время занятий
  TEACHER     char(10),        -- преподаватель
  [SUBJECT]   char(10),        -- дисциплина
  AUDITORIUM  char(10),        -- аудитория
  constraint PK_SHEDULE_TEACHER primary key([DATETIME],TEACHER)
 ) on FG2
```

Рис. 5.30. Создание таблицы, имеющей составной первичный ключ

Обратите внимание, что имена двух столбцов в создаваемой таблице (рис. 5.30) обрамлены квадратными скобками. Такой синтаксис применяется в том случае, если имя столбца совпадает с каким-нибудь ключевым словом или содержит специальные символы (в том числе пробелы).

Все примеры, приведенные выше, демонстрировали действие ограничения целостности PRIMARY KEY при выполнении операторов INSERT, но ограничение действует и для операторов UPDATE.

На рис. 5.31 приведен результат выполнения SELECT-запроса к таблице **FACULTY**. В таблице содержится две строки, имеющие разные значения в столбце **FACULTY**. Попытка выполнить оператор UPDATE (рис. 5.32), нарушающий заданное ограничение PRIMARY KEY (рис. 5.29), приводит к ошибке аналогичной той, что возникала при INSERT (рис. 5.28)

FACULTY	FACULTY_NAME
ИДиП	Издателькое дело и полиграфия
ХТиТ	Химическая технология и техника

Рис. 5.31. Содержимое таблицы FACULTY до выполнения оператора UPDATE

```
use BSTU

update FACULTY set FACULTY = 'ХТиТ' where  FACULTY = 'ИДиП';
```

Рис. 5.32. Оператор UPDATE, нарушающий ограничение PRIMARY KEY таблицы **FACULTY**

Кроме того, что ограничение PRIMARY KEY запрещает повторение значений в столбце (или комбинации значений при составном ключе), оно запрещает ввод пустых (NULL) значений в столбцы, входящие в первичный ключ. На рис. 5.33 приведены операторы, устанавливающие пустое значение (NULL) в столбце FACULTY. Результат выполнения сценария представлен на рис. 5.34.

```
use BSTU
go

update FACULTY set FACULTY = NULL where  FACULTY = 'ИДиП';
insert into FACULTY   (FACULTY,   FACULTY_NAME )
        values  (NULL,  'Лесохозяйственный факультет');
insert into FACULTY (FACULTY_NAME)
        values  ('Лесохозяйственный факультет');
go
```

Рис. 5.33. Операторы UPDATE и INSERT, устанавливающие пустое значение (NULL) в столбце FACULTY

Следует обратить внимание, что в сообщениях об ошибках на рис. 5.34 отсутствует имя ограничения **PK_FACULTY_FACULTY**. Дело в том, что свойство столбца PRIMARY KEY автоматически включает свойство (и соответствующее ему ограничение) NOT NULL, которое будет рассматриваться ниже. Именно это ограничение приводит к ошибкам при выполнении операторов INSERT и UPDATE из сценария на рис. 5.33.

```
Msg 515, Level 16, State 2, Line 1
Cannot insert the value NULL into column 'FACULTY', table 'BSTU.dbo.FACULTY'; column does not allow nulls. UPDATE fails.
The statement has been terminated.
Msg 515, Level 16, State 2, Line 2
Cannot insert the value NULL into column 'FACULTY', table 'BSTU.dbo.FACULTY'; column does not allow nulls. INSERT fails.
The statement has been terminated.
Msg 515, Level 16, State 2, Line 4
Cannot insert the value NULL into column 'FACULTY', table 'BSTU.dbo.FACULTY'; column does not allow nulls. INSERT fails.
The statement has been terminated.
```

Рис. 5.34. Результат выполнения сценария, представленного на рис. 5.33

5.2.2.2. Значение NULL. Столбцы любых типов данных могут содержать значение, обозначаемое символом NULL и интерпретируемое как неизвестная величина или пустое значение.

В строке таблицы значение NULL может появиться в результате выполнения операторов INSERT или UPDATE. На рис. 5.35 представлены примеры операторов INSERT, формирующих значения NULL в столбце **FACULTY_NAME**.

```
use BSTU
go

create table FACULTY  -- факультеты
(
   FACULTY      char(10),     -- идентификатор факультета
   FACULTY_NAME varchar(50),  -- полное наименование
   constraint FACULTY_PK primary key(FACULTY)
);
go

insert into FACULTY  (FACULTY, FACULTY_NAME)
         values  ('ИДиП', 'Издателькое дело и полиграфия');
insert into FACULTY  (FACULTY)
         values  ('ХТиТ');       -- неявный ввод значения NULL
insert into FACULTY  (FACULTY,  FACULTY_NAME)
         values  ('ЛХФ', NULL); -- явный ввод значения NULL

select * from FACULTY;
```

Рис. 5.35. Операторы INSERT, формирующие значение NULL

На рис. 5.36 отображен результат выполнения SELECT-запроса, выводящего в результирующий набор все строки таблицы **FACULTY**. Во второй и третьей строках в столбце с именем **FACULTY_NAME** содержатся значения NULL.

FACULTY	FACULTY_NAME
ИДиП	Издателькое дело и полиграфия
ЛХФ	NULL
ХТиТ	NULL

Рис. 5.36. Результат выполнения SELECT-запроса на рис. 5.35

Обратите внимание: оператором INSERT значение NULL формируется в столбцах в тех случаях, если значение в столбец не вводится (второй оператор INSERT) или NULL вводится явно (третий оператор INSERT).

Значение NULL может быть явно записано в столбцы существующих строк с помощью оператора UPDATE (рис. 5.37, 5.38).

```
use BSTU
go

update FACULTY set FACULTY_NAME = NULL where FACULTY = 'ИДиП';
select * from FACULTY;
```

Рис. 5.37. Явная запись значения NULL с помощью оператора UPDATE

FACULTY	FACULTY_NAME
ИДиП	NULL
ЛХФ	NULL
ХТиТ	NULL

Рис. 5.38. Результат выполнения SELECT-запроса на рис. 5.37

В тех случаях, когда требуется запретить в столбце таблицы значение NULL, применяется ограничение NOT NULL. На рис. 5.39 представлен пример использования этого ограничения целостности для столбца **FACULTY_NAME**.

Сразу после создания таблицы выполняются три оператора INSERT и один оператор UPDATE. С помощью второго и третьего операторов INSERT выполняется попытка ввода значения NULL в столбец **FACULTY_NAME.** Следующий далее оператор UPDATE делает попытку заменить содержимое этого же столбца на значение NULL. В результате выполнения этих трех операторов формируется три сообщения об ошибках (рис. 5.40). Обратите внимание, что сообщения об ошибке аналогичны сообщениям, формируемым при попытке установить значение NULL в столбец, для которого установлено ограничение PRIMARY KEY.

```
use BSTU
go

create table FACULTY  -- факультеты
(
   FACULTY      char(10),              --  идентификатор факультета
   FACULTY_NAME varchar(50)  not null,  -- полное наименование
   constraint FACULTY_PK primary key(FACULTY)
);
go

insert into FACULTY  (FACULTY, FACULTY_NAME)
         values  ('ИДиП', 'Издателькое дело и полиграфия');
insert into FACULTY  (FACULTY)
         values  ('ХТиТ');       -- неявный ввод значения NULL
insert into FACULTY  (FACULTY,  FACULTY_NAME)
         values  ('ЛХФ', NULL); -- явный ввод значения NULL
update FACULTY set FACULTY_NAME = NULL where FACULTY = 'ИДиП';
select * from FACULTY;
```

Рис. 5.39. Пример использования ограничения NOT NULL

```
Msg 515, Level 16, State 2, Line 4
Cannot insert the value NULL into column 'FACULTY_NAME', table 'BSTU.dbo.FACULTY'; column does not allow nulls. INSERT fails.
The statement has been terminated.
Msg 515, Level 16, State 2, Line 6
Cannot insert the value NULL into column 'FACULTY_NAME', table 'BSTU.dbo.FACULTY'; column does not allow nulls. INSERT fails.
The statement has been terminated.
Msg 515, Level 16, State 2, Line 8
Cannot insert the value NULL into column 'FACULTY_NAME', table 'BSTU.dbo.FACULTY'; column does not allow nulls. UPDATE fails.
```

Рис. 5.40. Сообщения об ошибках, возникших при выполнении сценария на рис. 5.39

В тоже время первый оператор INSERT (рис. 5.39) выполнен успешно, т. к. он не нарушает ограничения целостности NOT NULL, что подтверждается результатом выполнения оператора SELECT.

FACULTY	FACULTY_NAME
ИДиП	Издателькое дело и полиграфия

Рис. 5.41. Результат выполнения SELECT-запроса в сценарии на рис. 5.39

Напомним: ограничение PRIMARY KEY неявно подразумевает ограничение NOT NULL.

5.2.2.3. Значение по умолчанию. Иногда значение NULL целесообразно заменить некоторым другим предопределенным значением. В таких случаях используют свойство столбца (ограничение целостности) DEFAULT.

На рис. 5.42 приведен пример создания таблицы БД, имеющей столбец **FACULTY_NAME** со свойством DEFAULT.

```
create table FACULTY  -- факультеты
(
   FACULTY      char(10),                    --  идентификатор факультета
   FACULTY_NAME varchar(50)  default '???',  -- полное наименование
   constraint FACULTY_PK primary key(FACULTY)
);
go

insert into FACULTY  (FACULTY, FACULTY_NAME)
         values  ('ИДиП', 'Издателькое дело и полиграфия');
insert into FACULTY  (FACULTY)
         values  ('ХТиТ');       -- неявный ввод значения default
insert into FACULTY  (FACULTY,  FACULTY_NAME)
         values  ('ЛХФ', NULL); -- явный ввод значения NULL
insert into FACULTY   (FACULTY,   FACULTY_NAME )
         values  ('ТТЛП', NULL); -- явный ввод значения NULL
insert into FACULTY  (FACULTY,  FACULTY_NAME)
         values  ('ИЭФ', default); -- явный ввод значения default

update FACULTY set FACULTY_NAME = NULL where FACULTY = 'ИДиП';
update FACULTY set FACULTY_NAME = default where FACULTY = 'ЛХФ';

select * from FACULTY;
```

Рис. 5.42. Пример использования ограничения DEFAULT

Результат выполнения SELECT-запроса в сценарии на рис. 5.42 представлен на рис. 5.43.

FACULTY	FACULTY_NAME
ИДиП	NULL
ИЭФ	???
ЛХФ	???
ТТЛП	NULL
ХТиТ	???

Рис. 5.43. Результат выполнения SELECT-запроса в сценарии на рис. 5.42

При создании таблицы **FACULTY** (рис. 5.42) для столбца **FACULTY_NAME** с помощью ключевого слова DEFAULT указывается значение, которое будет вводиться в этот столбец оператором INSERT, если в списке столбцов для вводимой строки имя столбца **FACULTY_NAME** отсутствует (на рис. 5.42 оператор INSERT с комментарием «неявный ввод значения default»). Кроме того, можно явно указать на необходимость применения значения по умолчанию, как это сделано с помощью оператора INSERT с комментарием «явный ввод значения default».

Следует обратить внимание: свойство DEFAULT не отменяет возможности явного ввода значения NULL.

Операторы UPDATE (рис. 5.42) демонстрируют возможность изменения значений в столбце со свойством DEFAULT на значение NULL и значение по умолчанию.

Допускается комбинация свойств DEFAULT и NOT NULL столбца таблицы БД (рис. 5.44) . В этом случае для такого столбца станет невозможным установить значение NULL.

```
use BSTU
go

create table FACULTY  -- факультеты
(
   FACULTY      char(10),                              --  идентификатор факультета
   FACULTY_NAME varchar(50) not null default '???',  -- полное наименование
   constraint FACULTY_PK primary key(FACULTY)
);
go
```

Рис. 5.44. Комбинация свойств NOT NULL и DEFAULT

5.2.2.4. Внешний ключ. Внешний ключ – ограничение целостности, основанное на асимметричной связи (часто говорят – на отношении), установленной между двумя таблицами БД. Связь может быть установлена между строками таблицы (для краткости FK-таблицы), имеющей внешний ключ (FOREIGN KEY), и строками таблицы (PK-таблицы), имеющей первичный ключ (PRIMARY KEY). Часто говорят, что строки FK-таблицы ссылаются на строки PK-таблицы.

На одну строку PK-таблицы могут ссылаться несколько строк FK-таблицы. Поэтому говорят, что между PK-таблицей и FK-таблицей установлено отношение «один ко многим». Отметим, что PK- и FK-таблицы не обязательно являются разными.

Часто для обозначения такого отношения применяется символьное обозначение **1:*n** (читается «один к n»), где символ * указывает на то, что допускается нулевое значение ***n***, или другими словами, в PK-таблице допускаются строки, на которые не ссылается ни одна строка из FK-таблицы.

Ограничение FOREIGN KEY может быть задано в таблице только в том случае, если существует таблица, имеющая соответствующее ограничение PRIMARY KEY (PK-таблица).

На рис. 5.45 представлен пример создания FK-таблицы с именем **PULPIT**, имеющей внешний ключ.

```
create table  PULPIT  -- кафедра
(
     PULPIT      char(20)           -- идентификатор кафедры
                 constraint PULPIT_PK  primary key,
     PULPIT_NAME varchar(100)       -- полное наименование  кафедры
                 default '???',
     FACULTY     char(10)           -- идентификатор факультета
                 constraint PULPIT_FACULTY_FK foreign key
                 references FACULTY(FACULTY)
);
```

Рис. 5.45. Пример создания таблицы, имеющей ограничение FOREIGN KEY

Ограничение целостности FOREIGN KEY применяется для столбца **FACULTY**, ключевое слово REFERENCES указывает на PK-таблицу (в нашем случае – **FACULTY**) и столбец (**FACULTY**) со свойством PRIMARY KEY. Для краткости в дальнейшем столбец FK-таблицы,

для которого указано свойство FOREIGN KEY, будем называть FK-ключом, а столбец PK-таблицы – PK-ключом.

Структура таблицы **FACULTY** представлена на рис. 5.44, а на рис. 5.46 – содержимое этой таблицы.

FACULTY	FACULTY_NAME
ИДиП	Издателькое дело и полиграфия
ИЭФ	Инженерно-экономический факультет
ЛХФ	Лесохозяйственный факультет
ТОВ	Технология органических веществ
ТТЛП	Технология и техника лесной промышленности
ХТиТ	Химическая технология и техника

Рис. 5.46. Содержимое PK-таблицы **FACULTY**

Рассмотрим, каким образом ограничение FOREGN KEY проявляет себя при применении DML-операторов к FK-таблице **PULPIT** и PK-таблице **FACULTY**. При этом все сформулированные правила касаются лишь двух пар PK-ключей, PK-таблиц и FK-ключей, FK-таблиц и не принимают во внимание других ограничений, которые могут быть установлены для этих таблиц.

FK-таблица, оператор INSERT. Сценарий на рис. 5.47 выполняет шесть операторов INSERT, один из которых (отмечен комментарием) завершился с ошибкой (рис. 5.48, 5.49).

```
insert into PULPIT(PULPIT, PULPIT_NAME, FACULTY)  -- OK
        values('ИСиТ','Информационных систем и технологий ','ИДиП' );
insert into PULPIT(PULPIT, PULPIT_NAME, FACULTY)  -- OK
        values('ПОиСОИ','Полиграфического оборудования и систем обработки информации ', NULL);
insert into PULPIT(PULPIT, PULPIT_NAME, FACULTY)  -- OK
        values('ЛУ','Лесоустройства','ЛХФ');
insert into PULPIT(PULPIT, PULPIT_NAME, FACULTY)  -- error 547
         values('ЛВ','Лесоводства','XXXX');
insert into PULPIT(PULPIT, PULPIT_NAME, FACULTY)  -- OK
             values('ЛВ','Лесоводства','ЛХФ');
insert into PULPIT(PULPIT, PULPIT_NAME, FACULTY)  -- OK
        values  ('ТЛ','Транспорта леса',NULL);
select * from PULPIT  order by   FACULTY
```

Рис. 5.47. Добавление строк в FK-таблицу

```
(1 row(s) affected)
(1 row(s) affected)
(1 row(s) affected)
Msg 547, Level 16, State 0, Line 7
The INSERT statement conflicted with the FOREIGN KEY constraint "PULPIT_FACULTY_FK".
The conflict occurred in database "BSTU", table "dbo.FACULTY", column 'FACULTY'.
The statement has been terminated.
(1 row(s) affected)
(1 row(s) affected)
(5 row(s) affected)
```

Рис. 5.48. Сообщение об ошибке при выполнении сценария на рис. 5.47

PULPIT	PULPIT_NAME	FACULTY
ПОиСОИ	Полиграфического оборудования и систем обработки...	NULL
ТЛ	Транспорта леса	NULL
ИСиТ	Информационных систем и технологий	ИДиП
ЛВ	Лесоводства	ЛХФ
ЛУ	Лесоустройства	ЛХФ

Рис. 5.49. Результат выполнения SELECT-запроса на рис. 5.47

Сформулируем правила применения оператора INSERT при добавлении строки в FK-таблицу.

1. Оператор INSERT выполнится успешно, если для значения FK-ключа добавляемой строки в PK-таблице есть строка со значением PK-ключа, равным значению FK-ключа.
2. Оператор INSERT выполнится успешно, если значение FK-ключа будет NULL.

FK-таблица, оператор UPDATE. На рис. 5.50 сценарий содержит четыре оператора UPDATE, изменяющих FK-ключ в FK-таблице PULPIT. При выполнении четвертого оператора возникает ошибка с кодом 547 (рис. 5.51, 5.52).

```
update PULPIT set FACULTY = 'ИДиП' where PULPIT = 'ПОиСОИ';   -- OK
update PULPIT set FACULTY = 'ЛХФ'  where FACULTY is NULL;     -- OK
update PULPIT set FACULTY = NULL   where PULPIT = 'ИСиТ';     -- OK
update PULPIT set FACULTY = 'XXX'  where FACULTY = 'ЛХФ';     -- error 547

select * from PULPIT  order by   FACULTY
```

Рис. 5.50. Изменение строк FK-таблицы

```
(1 row(s) affected)
(1 row(s) affected)
(1 row(s) affected)

Msg 547, Level 16, State 0, Line 4
The UPDATE statement conflicted with the FOREIGN KEY constraint "PULPIT_FACULTY_FK".
The conflict occurred in database "BSTU", table "dbo.FACULTY", column 'FACULTY'.
The statement has been terminated.
```

Рис. 5.51. Сообщение об ошибке при выполнении сценария на рис. 5.50

PULPIT	PULPIT_NAME	FACULTY
ИСиТ	Информационных систем и технологий	NULL
ПОиСОИ	Полиграфического оборудования и систем обработки...	ИДиП
ТЛ	Транспорта леса	ЛХФ
ЛВ	Лесоводства	ЛХФ
ЛУ	Лесоустройства	ЛХФ

Рис. 5.52. Результат выполнения SELECT-запроса на рис. 5.50

Правила применения оператора UPDATE при изменении FK-таблицы следующие.

1. Оператор UPDATE выполнится успешно, если новое значение FK-ключа изменяемой строки таково, что в PK-таблице есть строка со значением PK-ключа, равным значению нового FK-ключа.

2. Оператор UPDATE выполнится успешно, если новое значение FK-ключа будет NULL.

FK-таблица, оператор DELETE. На рис. 5.53 представлен сценарий, состоящий из одного оператора DELETE, удаляющего все строки FK-таблицы **PULPIT**. На рис. 5.54 отображен результат выполнения сценария.

```
select count(*) 'количество строк до DELETE' from PULPIT;
delete PULPIT; -- удаление всех строк PULPIT
select count(*) 'количество строк после DELETE' from PULPIT;
```

Рис. 5.53. Удаление строк FK-таблицы

количество строк до DELETE
5

количество строк после DELETE
0

Рис. 5.54. Результат выполнения сценария на рис. 5.53

Правило применения оператора DELETE для удаления строк из FK-таблицы очень простое: оператор DELETE выполняется успешно всегда.

Для демонстрации применения DML-операторов к PK-таблице приведем FK-таблицу **PULPIT** в состояние, отображенное на рис. 5.55, при этом PK-таблица **FACULTY** осталась в прежнем состоянии (рис. 5.46).

PULPIT	PULPIT_NAME	FACULTY
ПиАХП	Процессов и аппаратов химических производств	NULL
ПОиСОИ	Полиграфического оборудования и систем обработки...	ИДиП
ИСиТ	Информационных систем и технологий	ИДиП
ЛВ	Лесоводства	ЛХФ
ОХ	Органической химии	ТОВ
ТЛ	Транспорта леса	ТТЛП

Рис. 5.55. Содержимое таблицы PULPIT

Заметим следующее:

1) на строку со значением PK-ключа **ИДиП** PK-таблицы **FACULTY** ссылается две строки из FK-таблицы **PULPIT**;

2) на строки со значениями PK-ключей **ИЭФ** и **ХТиТ** не ссылается ни одна строка из FK-таблицы **PULPIT**;

3) одна строка из FK-таблицы **PULPIT** не ссылается ни на одну строку в таблице (значение FK-ключа NULL).

PK-таблица, оператор INSERT. На рис. 5.56 представлен сценарий, состоящий из двух операторов INSERT, добавляющих строки в PK-таблицу **FACULTY**. При выполнении второго оператора возникла ошибка, связанная с нарушением ограничения PRIMARY KEY (рис. 5.57, 5.58).

```
insert into FACULTY    (FACULTY,    FACULTY_NAME ) -- OK
         values   ('ИСП',    'Информационных систем и программирования');
insert into FACULTY    (FACULTY,    FACULTY_NAME ) -- error 2627
         values   ('ИДиП',    'Изд. дело, полиграфии и сист. обр. инф.');
select * from FACULTY order by FACULTY
```

Рис. 5.56. Добавление строк в PK-таблицу **FACULTY**

```
(1 row(s) affected)
Msg 2627, Level 14, State 1, Line 3
Violation of PRIMARY KEY constraint 'FACULTY_PK'.
Cannot insert duplicate key in object 'dbo.FACULTY'.
The statement has been terminated.
```

Рис. 5.57. Сообщение об ошибке при добавлении строки в PK-таблицу **FACULTY**

FACULTY	FACULTY_NAME
ИДиП	Издателькое дело и полиграфия
ИСП	Информационных систем и программирования
ИЭФ	Инженерно-экономический факультет
ЛХФ	Лесохозяйственный факультет
ТОВ	Технология органических веществ
ТТЛП	Технология и техника лесной промышленности
ХТиТ	Химическая технология и техника

Рис. 5.58. Результат выполнения SELECT-запроса в сценарии на рис. 5.56

Правило применения оператора INSERT для добавления строк в PK-таблицу: оператор INSERT выполняется успешно всегда, если не нарушает ограничение PRIMARY KEY.

PK-таблица, оператор UPDATE. На рис. 5.59 представлен сценарий, состоящий из трех операторов UPDATE, изменяющих строки в PK-таблице **FACULTY**. При выполнении второго и третьего операторов возникли ошибки, связанные с нарушением ограничений FOREIGN KEY и PRIMARY KEY (рис. 5.60, 5.61).

```
update FACULTY set FACULTY = 'ИЭ'    where  FACULTY = 'ИЭФ' -- OK
update FACULTY set FACULTY = 'ЛХ'    where  FACULTY = 'ЛХФ' -- error 547
update FACULTY set FACULTY = 'ИДиП' where  FACULTY = 'ИСП' -- error 2627

select * from FACULTY order by FACULTY
```

Рис. 5.59. Изменение строк в PK-таблице **FACULTY**

```
(1 row(s) affected)
Msg 547, Level 16, State 0, Line 2
The UPDATE statement conflicted with the REFERENCE constraint "PULPIT_FACULTY_FK".
The conflict occurred in database "BSTU", table "dbo.PULPIT", column 'FACULTY'.
The statement has been terminated.
Msg 2627, Level 14, State 1, Line 3
Violation of PRIMARY KEY constraint 'FACULTY_PK'.
Cannot insert duplicate key in object 'dbo.FACULTY'.
The statement has been terminated.
```

Рис. 5.60. Сообщение об ошибках при изменении строк в PK-таблице **FACULTY**

FACULTY	FACULTY_NAME
ИДиП	Издателькое дело и полиграфия
ИСП	Информационных систем и программирования
ИЭ	Инженерно-экономический факультет
ЛХФ	Лесохозяйственный факультет
ТОВ	Технология органических веществ
ТТЛП	Технология и техника лесной промышленности
ХТиТ	Химическая технология и техника

Рис. 5.61. Результат выполнения SELECT-запроса в сценарии на рис. 5.60

Правило применения оператора UPDATE для изменения строк в PK-таблице: оператор UPDATE выполняется успешно, если применяется к строкам, на которые не ссылаются строки из FK-таблицы, и не нарушается ограничение PRIMARY KEY; строки FK-таблицы, не связанные со строками PK-таблицы (имеют значение NULL FK-ключа), никак не влияют на выполнение оператора UPDATE.

PK-таблица, оператор DELETE. На рис. 5.62 представлен сценарий, состоящий из двух операторов DELETE, удаляющих строки в PK-таблице **FACULTY**. При выполнении второго оператора возникла ошибка, связанная с нарушением ограничения FOREIGN KEY (рис. 5.63, 5.64).

```
select count(*) 'количество строк до DELETE' from FACULTY;
delete FACULTY  where  FACULTY = 'ИЭ' -- OK
delete FACULTY  where  FACULTY = 'ЛХФ' -- error 547
select count(*) 'количество строк после DELETE' from FACULTY;
```

Рис. 5.62. Удаление строк в PK-таблице **FACULTY**

```
(1 row(s) affected)
Msg 547, Level 16, State 0, Line 3
The DELETE statement conflicted with the REFERENCE constraint "PULPIT_FACULTY_FK".
The conflict occurred in database "BSTU", table "dbo.PULPIT", column 'FACULTY'.
The statement has been terminated.
```

Рис. 5.63. Сообщение об ошибках при удалении строк в PK-таблице **FACULTY**

количество строк до DELETE
7

количество строк после DELETE
6

Рис. 5.64. Результаты выполнения SELECT-запросов в сценарии на рис. 5.62

Правило применения оператора DELETE для удаления строк в PK-таблице: оператор DELETE выполняется успешно, если применяется к строкам, на которые не ссылаются строки из FK-таблицы; строки FK-таблицы, не связанные со строками PK-таблицы (имеют значение NULL FK-ключа), никак не влияют на выполнение оператора DELETE.

5.2.2.5. Проверка значений. При вводе или корректировке для проверки значений данных применяется ограничение целостности CHECK. На рис. 5.65 представлен пример использования CHECK для проверки значений в столбце **GENDER** таблицы **TEACHER**. После ключевого слова CHECK в описании свойств столбца **GENDER** следует логическое выражение, проверяющее значение в столбце.

В том случае, когда значение выражения принимает значение «истина», оно считается правильным. Оператор IN, используемый в логическом выражении, имеет значение «истина» в том случае, если значение в столбце, указанном слева от IN, будет совпадать хотя бы с одним значением, указанным в списке справа от IN.

```
use BSTU
go
create table TEACHER
(
 TEACHER       char(10)  constraint TEACHER_PK  primary key,
 TEACHER_NAME  varchar(100),
 GENDER        char(1)  constraint TEACHER_GENDER_CH check (GENDER in ('м', 'ж')),
 PULPIT        char(20) constraint TEACHER_PULPIT_FK foreign key
                        references PULPIT(PULPIT)
);
go
insert into  TEACHER    (TEACHER,   TEACHER_NAME, PULPIT )          -- OK
                      values  ('СМЛВ',    'Смелов Владимир Владиславович',  'ИСиТ');
insert into  TEACHER    (TEACHER,  TEACHER_NAME, PULPIT, GENDER )   -- OK
                      values  ('АКНВЧ','Акунович Станислав Иванович','ИСиТ', 'м');
insert into  TEACHER    (TEACHER,  TEACHER_NAME, PULPIT, GENDER )   -- OK
                      values  ('БРС',     'Брусенцова Татьяна Палладьевна', 'ИСиТ', 'м');
insert into  TEACHER    (TEACHER,  TEACHER_NAME, PULPIT, GENDER )   -- OK
                      values  ('МРЗ',     'Мороз Елена Станиславовна',     'ИСиТ','ж');
insert into  TEACHER    (TEACHER,  TEACHER_NAME, PULPIT, GENDER )   -- error 547
                      values  ('РМНК',     'Романенко Дмитрий Михайлович',  'ИСиТ','x');

update  TEACHER  set GENDER = 'ж'  where  TEACHER =  'БРС';         -- OK
update  TEACHER  set GENDER = NULL where  TEACHER =  'МРЗ';         -- OK
update  TEACHER  set GENDER = 'y'  where  TEACHER =  'СМЛВ';        -- error 547

select * from TEACHER
```

Рис. 5.65. Пример использования ограничения CHECK

В сценарии на рис. 5.65 выполняется пять операторов INSERT и три оператора UPDATE. Два оператора завершаются с ошибкой (рис. 5.66, 5.67), а остальные выполняются успешно.

```
(1 row(s) affected)
(1 row(s) affected)
(1 row(s) affected)
(1 row(s) affected)
Msg 547, Level 16, State 0, Line 9
The INSERT statement conflicted with the CHECK constraint "TEACHER_GENDER_CH".
The conflict occurred in database "BSTU", table "dbo.TEACHER", column 'GENDER'.
The statement has been terminated.
(1 row(s) affected)
(1 row(s) affected)
Msg 547, Level 16, State 0, Line 14
The UPDATE statement conflicted with the CHECK constraint "TEACHER_GENDER_CH".
The conflict occurred in database "BSTU", table "dbo.TEACHER", column 'GENDER'.
The statement has been terminated.
```

Рис. 5.66. Сообщения об ошибках в операторах INSERT и UPDATE, нарушающих ограничение CHECK в таблице TEACHER (рис. 5.65)

Следует обратить внимание на то, что значение NULL не подвергается проверке ограничением CHECK и может быть введено с помощью INSERT либо UPDATE.

TEACHER	TEACHER_NAME	GENDER	PULPIT
АКНВЧ	Акунович Станислав Иванович	м	ИСиТ
БРС	Брусенцова Татьяна Палладьевна	ж	ИСиТ
МРЗ	Мороз Елена Станиславовна	NULL	ИСиТ
СМЛВ	Смелов Владимир Владиславович	NULL	ИСиТ

Рис. 5.67. Результат выполнения SELECT-запроса в сценарии на рис. 5.65

Ограничение CHECK может быть скомбинировано с другими ограничениями, как это показано на рис. 5.68. В этом случае возможность ввести значение NULL с помощью INSERT или откорректировать в существующей строке значение столбца GENDER на NULL вызовет ошибку.

```
use BSTU
go
create table TEACHER
(
 TEACHER       char(10)  constraint TEACHER_PK  primary key,
 TEACHER_NAME  varchar(100),
 GENDER        char(1)
               not null
               default 'м'
               constraint TEACHER_GENDER_CH check (GENDER in ('м', 'ж')),
 PULPIT        char(20) constraint TEACHER_PULPIT_FK foreign key
                        references PULPIT(PULPIT)
);
```

Рис. 5.68. Совместное использование ограничений
CHECK, NOT NULL и DEFAULT

5.2.2.6. Уникальные значения. Ограничение целостности UNIQUE позволяет значения в столбце или комбинацию значений нескольких столбцов таблицы сделать уникальными в этой таблице. На рис. 5.69 представлен пример использования ограничения UNIQUE для столбца **PROFESSION_NAME** таблицы **PROFESSION**. Сценарий содержит пять операторов INSERT и один оператор UPDATE. Три оператора завершаются с ошибкой, имеющей код 2627 (рис. 5.70, 5.71). Ошибки возникают при повторном вводе значения в столбец **PROFESSION_NAME** или корректировке существующего значения на значение, которое уже в этом столбце есть.

```
use BSTU
go
create table PROFESSION  --  специальность
(
 PROFESSION      char(20)     --  специальность
                 constraint PROFESSION_PK  primary key,
 FACULTY         char(10)     -- факультет
                 constraint PROFESSION_FACULTY_FK
                 foreign key references FACULTY(FACULTY),
 PROFESSION_NAME varchar(100) --  наименование специальности
                 constraint PROFESSION_NAME_UQ unique,
 QUALIFICATION   varchar(50)  --  квалификация
);
go
insert into PROFESSION(FACULTY, PROFESSION,PROFESSION_NAME, QUALIFICATION) --OK
      values ('ИДиП','1-40 01 02', 'Информационные системы и технологии',
              'инженер-программист-системотехник' );
insert into PROFESSION(FACULTY, PROFESSION,PROFESSION_NAME, QUALIFICATION) --OK
      values ('ИДиП','1-47 01 01','Издательское дело', 'редактор-технолог' );
insert into PROFESSION(FACULTY, PROFESSION,PROFESSION_NAME, QUALIFICATION) -- error 2627
      values  ('ИДиП',  '1-36 06 01','Издательское дело', 'инженер-электромеханик' );
insert into PROFESSION(FACULTY, PROFESSION,PROFESSION_NAME, QUALIFICATION) --OK
       values ('ХТиТ',  '1-36 01 08',NULL,'инженер-механик' );
insert into PROFESSION(FACULTY, PROFESSION,PROFESSION_NAME, QUALIFICATION) -- error 2627
       values ('ХТиТ',  '1-36 07 01',NULL, 'инженер-механик' );
update PROFESSION set PROFESSION_NAME = 'Информационные системы и технологии' -- error 2627
       where PROFESSION = '1-36 01 08';
select * from PROFESSION;
```

Рис. 5.69. Применение ограничения целостности UNIQUE

```
(1 row(s) affected)
(1 row(s) affected)
Msg 2627, Level 14, State 1, Line 6
Violation of UNIQUE KEY constraint 'PROFESSION_NAME_UQ'.
Cannot insert duplicate key in object 'dbo.PROFESSION'.
The statement has been terminated.
(1 row(s) affected)
Msg 2627, Level 14, State 1, Line 10
Violation of UNIQUE KEY constraint 'PROFESSION_NAME_UQ'.
Cannot insert duplicate key in object 'dbo.PROFESSION'.
The statement has been terminated.
Msg 2627, Level 14, State 1, Line 12
Violation of UNIQUE KEY constraint 'PROFESSION_NAME_UQ'.
Cannot insert duplicate key in object 'dbo.PROFESSION'.
The statement has been terminated.
```

Рис. 5.70. Сообщения об ошибках в сценарии на рис. 5.69

PROFESSION	FACULTY	PROFESSION_NAME	QUALIFICATION
1-36 01 08	ХТиТ	NULL	инженер-механик
1-40 01 02	ИДиП	Информационные системы и технологии	инженер-программист-системотехник
1-47 01 01	ИДиП	Издательское дело	редактор-технолог

Рис. 5.71. Результат выполнения SELECT-запроса на рис. 5.69

Следует обратить внимание на то, что ограничение UNIQUE допускает значение NULL, но только в одной из строк, т. е. значение NULL этим ограничением рассматривается как полноправное значение.

5.2.3. Свойство IDENTITY столбца таблицы

Процесс нормализации таблиц приводит к вертикальному делению одной таблицы на несколько. При этом для обеспечения нормальной формы новых таблиц иногда приходится прибегать к суррогатным ключам.

Суррогатный ключ – это дополнительный столбец, добавленный к уже имеющимся столбцам таблицы и предназначенный служить первичным ключом.

СУБД Microsoft SQL Server 2008 позволяет создавать столбцы с автоматически формируемым целочисленным уникальным значением. На рис. 5.72 представлен сценарий создания таблицы **GROUPS**.

```
use BSTU
go
create table GROUPS --  учебные группы
(
 IDGROUP     int          -- идентификатор учебой группы
             identity(1,1)
             constraint GROUP_PK  primary key,
 FACULTY     char(10)     -- факультет
             constraint  GROUPS_FACULTY_FK foreign key
             references FACULTY(FACULTY),
 PROFESSION  char(20)      -- специальность
             constraint  GROUPS_PROFESSION_FK foreign key
             references PROFESSION(PROFESSION),
 YEAR_FIRST  smallint      -- год поступления
             check (YEAR_FIRST<=YEAR(GETDATE())),
 );
 go
 insert into GROUPS   (FACULTY,  PROFESSION, YEAR_FIRST ) -- OK
         values ('ИДиП','1-40 01 02', 2013),
                ('ИДиП','1-40 01 02', 2012),
                ('ИДиП','1-40 01 02', 2011),
                ('ИДиП','1-40 01 02', 2010),
                ('ИДиП','1-47 01 01', 2013);
 insert into GROUPS   ( IDGROUP, FACULTY,  PROFESSION, YEAR_FIRST) -- error 544
         values (99, 'ИДиП','1-47 01 01', 2013);
 select * from GROUPS;
```

Рис. 5.72. Применение свойства IDENTITY

Для столбца **IDGROUP** задано свойство IDENTITY, указывающее на необходимость автоматического формирования целочисленного значения в этом столбце при выполнении оператора INSERT. Свойство IDENTITY имеет два параметра, которые указываются в скобках после ключевого слова IDENTITY. Первый параметр задает стартовое значение идентификатора, второй – приращение. Попытка явно ввести значение в столбец со свойством IDENTITY с помощью INSERT приводит к ошибке (рис. 5.73, 5.74).

```
(5 row(s) affected)
Msg 544, Level 16, State 1, Line 7
Cannot insert explicit value for identity column in table 'GROUPS' when IDENTITY_INSERT is set to OFF.
```

Рис. 5.73. Сообщение о попытке явного ввода значения в столбец со свойством IDENTITY

IDGROUP	FACULTY	PROFESSION	YEAR_FIRST
1	ИДиП	1-40 01 02	2013
2	ИДиП	1-40 01 02	2012
3	ИДиП	1-40 01 02	2011
4	ИДиП	1-40 01 02	2010
5	ИДиП	1-47 01 01	2013

Рис. 5.74. Результат SELECT-запроса из сценария на рис. 5.72

Попытка выполнить корректировку значения в столбце, имеющем свойство IDENTITY, приводит к ошибке с кодом компиляции 8102.

```
update GROUPS set IDGROUP = 99 where  IDGROUP = 1;

Msg 8102, Level 16, State 1, Line 1
Cannot update identity column 'IDGROUP'.
```

Рис. 5.75. Сообщение об ошибке, формируемое при попытке выполнить оператор UPDATE применительно к столбцу со свойством IDENTITY

5.2.4. Вычисляемые столбцы

Столбец таблицы БД, значение которого может быть вычислено как выражение, содержащее в качестве аргументов другие столбцы таблицы, константы, вызовы встроенных функций и пр., называется вычисляемым.

Если для вычисляемого столбца не указано свойство PERSISTED, то значения такого столбца физически не хранятся в таблице, а вычисляются каждый раз при обращении к нему. Такие столбцы называются виртуальными.

В некоторых случаях (например, при создании индекса по вычисляемому столбцу) свойство PERSISTED является обязательным, оно обеспечивает физическое хранение в таблице вычисленного значения. Свойство PERSISTED к выражению вычисляемого столбца предъявляет требование детерминистичности. Детерминистическое выражение – выражение, значение которого всегда одинаково для каждой комбинации значений аргументов. Другими словами, значение функции не зависит от времени.

В дальнейшем рассматриваются только виртуальные столбцы. Для знакомства с возможностями, которые предоставляют вычисляемые PERSISTED-столбцы, рекомендуется изучить пособия [3, 5].

На рис. 5.76 представлен сценарий создания таблицы **GROUPS**, содержащей виртуальный вычисляемый столбец с именем **COURSE**.

```
use BSTU
go
create table GROUPS --  учебные группы
(
 IDGROUP      int          -- идентификатор учебой группы
              identity(1,1)
              constraint GROUP_PK  primary key,
 FACULTY      char(10)     -- факультет
              constraint  GROUPS_FACULTY_FK foreign key
              references FACULTY(FACULTY),
 PROFESSION   char(20)      -- специальность
              constraint  GROUPS_PROFESSION_FK foreign key
              references PROFESSION(PROFESSION),
 YEAR_FIRST   smallint      -- год поступления
              check (YEAR_FIRST<=YEAR(GETDATE())),
 COURSE    as  case          -- курс
               when   month(getdate()) between 8 and 12
                      then year(getdate())- YEAR_FIRST +1
               else   year(getdate())- YEAR_FIRST
               end
  );
go
insert into GROUPS   (FACULTY,  PROFESSION, YEAR_FIRST )
         values ('ИДиП','1-40 01 02', 2013), --1
                ('ИДиП','1-40 01 02', 2012),
                ('ИДиП','1-40 01 02', 2011),
                ('ИДиП','1-40 01 02', 2010),
                ('ИДиП','1-47 01 01', 2013),---5 гр
                ('ИДиП','1-47 01 01', 2012),
                ('ИДиП','1-47 01 01', 2011);
select * from GROUPS;
```

Рис. 5.76. Пример использования виртуального вычисляемого столбца

Описание вычисляемого столбца состоит из двух частей: имени и выражения, стоящего после ключевого слова AS. Результат SELECT-запроса, выполненного в рамках сценария на рис. 5.76, представлен на рис. 5.77. Обратите внимание на выводимые значения в вычисляемом виртуальном столбце **COURSE**.

IDGROUP	FACULTY	PROFESSION	YEAR_FIRST	COURSE
1	ИДиП	1-40 01 02	2013	1
2	ИДиП	1-40 01 02	2012	2
3	ИДиП	1-40 01 02	2011	3
4	ИДиП	1-40 01 02	2010	4
5	ИДиП	1-47 01 01	2013	1
6	ИДиП	1-47 01 01	2012	2
7	ИДиП	1-47 01 01	2011	3

Рис. 5.77. Результат SELECT-запроса из сценария на рис. 5.77

Если ввести вычисляемое значение (с помощью INSERT) или изменить (с помощью UPDATE), то это вызовет ошибку (рис. 5.78).

```
insert into GROUPS   (FACULTY,  PROFESSION, YEAR_FIRST, COURSE ) -- error 271
       values ('ТОВ','1-54 01 03', 2012, 2);
update GROUPS set COURSE = 1 where PROFESSION = '1-40 01 02';     -- error 271

Msg 271, Level 16, State 1, Line 1
The column "COURSE" cannot be modified because it is either a computed column or is the result of a UNION operator.
```

Рис. 5.78. Сообщение об ошибке при попытке добавить или откорректировать вычисляемое значение

5.3. Временные таблицы

Основное отличие временных таблиц от постоянных в том, что они хранятся в системной БД **TEMPDB** и не могут иметь внешние ключи. Как правило, временные таблицы создаются для временного хранения результатов SELECT-запросов. При применении временных таблиц следует помнить, что БД **TEMPDB** снова создается при каждом перезапуске сервера, поэтому сохранить или восстановить временную таблицу в случае сбоя, приведшего к перезапуску сервера СУБД, невозможно.

Существует два вида временных таблиц: локальные и глобальные. Они отличаются друг от друга форматом имени, областью видимости и жизненным циклом.

5.3.1. Локальные временные таблицы

Локальные временные таблицы имеют имена, начинающиеся с символа #, доступны только создавшему ее пользователю и могут быть удалены с помощью оператора DROP TABLE. Если пользователь временную таблицу не удалил сам, то она удалится автоматически при его отключении. На рис. 5.79 и 5.80 приведен пример, демонстрирующий создание, использование и удаление локальной временной таблицы с именем **#TEACHER**.

```
use BSTU
go
create table #TEACHER
(
 TEACHER       char(10)
               constraint [T_TEACHER_PK]  primary key,
 TEACHER_NAME  varchar(100) default '???',
 GENDER        char(1) default 'м'
               constraint [T_TEACHER_GENDER_CH] check (GENDER in ('м', 'ж')),
 PULPIT        char(20)
 );
 go
insert into  #TEACHER    (TEACHER,   TEACHER_NAME, PULPIT )
      values  ('СМЛВ',    'Смелов Владимир Владиславович',  'ИСиТ'),
              ('АКНВЧ',    default,                          'ИСиТ'),
              ('КЛСНВ',    null,                             'ИСиТ'),
              ('ГРМН',     'Герман Олег Витольдович',        'ИСиТ'),
              ('ЛЩНК',    'Лащенко Анатолий Павлович',       'ИСиТ');
update #TEACHER set TEACHER_NAME = TEACHER  where  TEACHER_NAME = '???';
delete #TEACHER  where TEACHER_NAME is null;
select * from  #TEACHER;
drop table #TEACHER;
```

Рис. 5.79. Пример использования локальной временной таблицы

TEACHER	TEACHER_NAME	GENDER	PULPIT
АКНВЧ	АКНВЧ	м	ИСиТ
ГРМН	Герман Олег Витольдович	м	ИСиТ
ЛЩНК	Лащенко Анатолий Павлович	м	ИСиТ
СМЛВ	Смелов Владимир Владиславович	м	ИСиТ

Рис. 5.80. Результат выполнения SELECT-запроса в сценарии на рис. 5.79

Следует обратить внимание: несмотря на то, что временная таблица **#TEACHER** создается в контексте БД **BSTU** (оператор USE), размещена она будет в системной базе данных **TEMPDB**.

Как правило, локальные временные таблицы применяются для временного хранения результатов трудоемких SELECT-запросов.

5.3.2. Глобальные временные таблицы

Глобальные временные таблицы имеют имена, начинающиеся с символа ##, доступны всем пользователям, подключенным к серверу, и могут быть удалены с помощью оператора DROP TABLE. Если глобальная временная таблица не удалена одним из пользователей, то она удалится автоматически при отключении всех пользователей, которые работали с этой таблицей. Если таблица использовалась только создавшим ее пользователем, то она будет удалена сразу после его отключения. Обычно глобальные временные таблицы применяются для обмена данными между несколькими сеансами.

5.4. Удаление таблиц

Таблицу можно удалить с помощью оператора DROP TABLE. Проблема с удалением таблицы может быть в трех следующих случаях.

1. Пользователь не имеет достаточных прав на удаление таблицы.
2. Таблица заблокирована транзакцией другого сеанса.
3. На первичный ключ удаляемой таблицы ссылается внешний ключ другой таблицы.

В первом случае выполняющий удаление таблицы пользователь должен быть наделен соответствующим разрешением (permission) с помощью оператора GRANT или должен быть включен в одну из системных фиксированных ролей БД.

В том случае, если таблица заблокирована транзакцией другого пользователя, выполнение оператора DROP TABLE приостанавливается до завершения транзакции. Если транзакция завершится в пределах допустимого времени, и пользователь обладает соответствующими разрешениями, то удаление таблицы будет выполнено.

Если существуют таблицы с внешними ключами, ссылающимися на удаляемую таблицу, то предварительно необходимо модифицировать (с помощью оператора ALTER TABLE удалить соответствующее ограничение FOREIGN KEY) или удалить эти таблицы.

На рис. 5.81 представлена диаграмма логической схемы БД **BSTU**. Прямоугольники обозначают таблицы БД, линии – соединяющие их связи между внешними и первичными ключами. Со стороны внешнего ключа изображен символ бесконечности, со стороны первичного ключа – ключ. Попытка удаления таблицы **FACULTY** приведет к ошибке (рис. 5.82), т. к. на ее первичный ключ ссылаются вторичные ключи таблиц **PULPIT**, **GROUPS** и **PROFESSION**. Правильная последовательность удаления таблиц БД **BSTU** представлена на рис. 5.83.

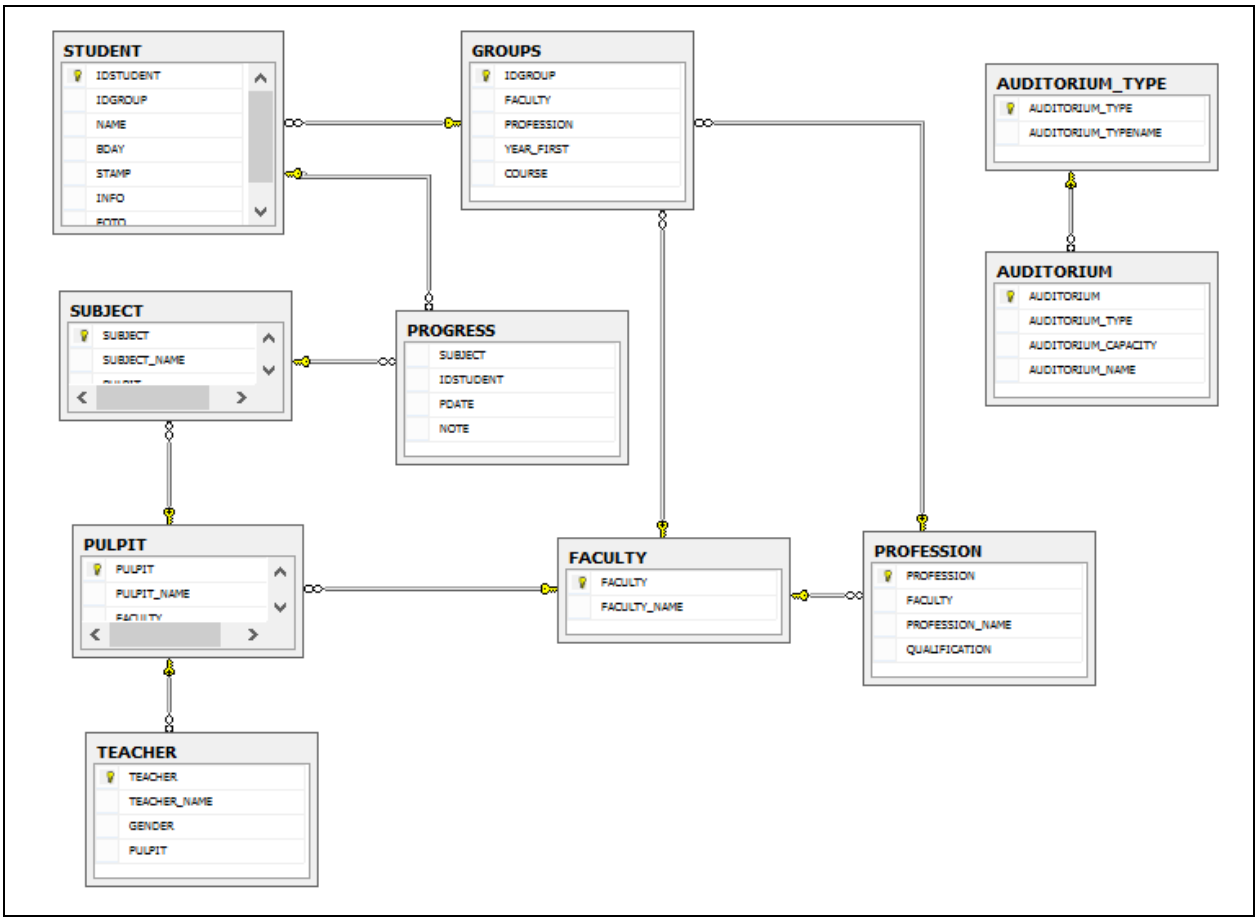


Рис. 5.81. Логическая схема БД **BSTU**

```
use BSTU
go

drop table FACULTY;

Msg 3726, Level 16, State 1, Line 2
Could not drop object 'FACULTY' because it is referenced by a FOREIGN KEY constraint.
```

Рис. 5.82. Сообщение об ошибке при попытке удаления связанной таблицы

```
use BSTU
go
drop table TEACHER, PROGRESS, [SUBJECT], PULPIT, STUDENT, GROUPS, PROFESSION, FACULTY;
drop table dbo.AUDITORIUM, dbo.AUDITORIUM_TYPE;
```

Рис. 5.83. Правильная последовательность удаления таблиц БД **BSTU**

5.5. Модификация таблиц

Под модификацией таблицы подразумевается изменение ее структуры: добавление или удаление столбцов, добавление или удаление ограничений, изменение точности или типа числовых данных, размерности символьных данных и т. п. В простейшем случае можно просто удалить (DROP TABLE) и создать ее (CREATE TABLE) с новой структурой. Но такой путь бывает не всегда рациональным, особенно, если для работы с таблицей уже распределены права

доступа или таблица содержит значительные объемы данных. Следует помнить, что не всякая модификация таблицы сохраняет данные в таблице.

Перед модификацией таблицы целесообразно ознакомиться с ее структурой. Обычно для этого применяют системную процедуру **SP_HELP**, при вызове которой указывают в качестве параметра имя исследуемой таблицы (рис. 5.84).

```
use BSTU
go
exec  SP_HELP TEACHER;
```

Name	Owner	Type	Created_datetime
TEACHER	dbo	user table	2014-01-09 15:24:54.093

Column_name	Type	Computed	Length	Prec	Scale	Nullable	TrimTrailingBlanks	FixedLenNullInSource
TEACHER	char	no	10			no	no	no
TEACHER_NAME	varchar	no	100			yes	no	yes
GENDER	char	no	1			yes	no	yes
PULPIT	char	no	20			yes	no	yes

constraint_type	constraint_name	delete_action	update_action	status_enabled	status_for_replication	constraint_keys
CHECK on column GENDER	TEACHER_GENDER_CH	(n/a)	(n/a)	Enabled	Is_For_Replication	([GENDER]='ж' OR [GENDER]='м')
PRIMARY KEY (clustered)	TEACHER_PK	(n/a)	(n/a)	(n/a)	(n/a)	TEACHER
FOREIGN KEY	TEACHER_PULPIT_FK	No Action	No Action	Enabled	Is_For_Replication	PULPIT
						REFERENCES BSTU.dbo.PULPI...

Рис. 5.84. Фрагмент результата работы системной процедуры **SP_HELP**

Во втором результирующем наборе вывода системной процедуры **SP_HELP** (рис. 5.84) можно выяснить, что столбец **TEACHER_NAME** (Column_name) таблицы **TEACHER** имеет тип VARCHAR(100) (Type, Length) и допускает значение NULL (Nullable).

В третьем результирующем наборе вывода системной процедуры **SP_HELP** перечислены все ограничения (constraint_type, constraint_name), установленные для таблицы **TEACHER**.

На рис. 5.85 показана часть строк содержимого таблицы **TEACHER**. Последний столбец результирующего набора указывает размер данных в столбце **TEACHER_NAME** каждой строки таблицы.

```
use BSTU
go
select *, len(TEACHER_NAME) 'len(TEACHER_NAME)' from TEACHER  order by TEACHER
```

TEACHER	TEACHER_NAME	GENDER	PULPIT	len(TEACHER_NAME)
АКНВЧ	Акунович Станислав Иванович	NULL	ИСиТ	27
АРС	Арсентьев Виталий Арсентьевич	NULL	ПОиСОИ	29
АТР	Атрощенко Олег Александрович	NULL	ЛУ	28
БЗБРДВ	Безбородов Владимир Степанович	NULL	ОХ	30
БЛВ	Беляев Валерий Павлович	NULL	ПОиСОИ	23
БРГ	Бурганская Татьяна Минаевна	NULL	ЛПиСПС	27
БРКВЧ	Бракович Андрей Игоревич	NULL	ИСиТ	24
БРНВСК	Барановский Станислав Иванович	NULL	ЭТиМ	30
БРС	Брусенцова Татьяна Палладьевна	NULL	ИСиТ	30
БРТШВЧ	Барташевич Святослав Александрович	NULL	ПОиСОИ	34
ГРМН	Герман Олег Витольдович	NULL	ИСиТ	23
ГРН	Гурин Николай Иванович	NULL	ИСиТ	22
ДЛН	Долинина Татьяна Николаевна	NULL	СБУАиА	27
ДМДК	Демидко Марина Николаевна	NULL	ЛПиСПС	25
ДТК	Дятко Александр Аркадьевич	NULL	ИСиТ	26
ЕЩНК	Ещенко Людмила Семеновна	NULL	ТНВиО...	24

Рис. 5.85. Содержимое таблицы **TEACHER**

На рис. 5.86 представлен сценарий, изменяющий структуру таблицы **TEACHER**. Первый оператор ALTER изменяет существующий столбец **TEACHER_NAME**: уменьшает максимальный размер столбца до 50 символов и добавляет ограничение NOT NULL. Второй оператор добавляет новый столбец с именем **BDATE** типа DATE, не допускающий значений NULL и с заданным значением по умолчанию.

```
use BSTU
go
alter table TEACHER  -- изменение столбца
  alter column  TEACHER_NAME varchar(50) not null;
alter table TEACHER   -- добавление нового столбца
  add BDATE  date not null default '19600101'
```

Рис. 5.86. Модификация таблицы **TEACHER**

На рис. 5.87 отображен результат SELECT-запроса к таблице **TEACHER** после ее модификации с помощью сценария на рис. 5.86. Сравнив рис. 5.85 и 5.87 не сложно заметить, что в таблице появился новый столбец с именем **BDATE**, полностью заполненный значением по умолчанию, а значения остальных столбцов не изменились. Обратим внимание на то, что значения в столбце **GENDER** равны NULL.

```
use BSTU
go
select *, len(TEACHER_NAME) 'len(TEACHER_NAME)' from TEACHER  order by TEACHER
```

TEACHER	TEACHER_NAME	GENDER	PULPIT	BDATE	len(TEACHER_NAME)
АКНВЧ	Акунович Станислав Иванович	NULL	ИСиТ	1960-01-01	27
АРС	Арсентьев Виталий Арсентьевич	NULL	ПОиСОИ	1960-01-01	29
АТР	Атрощенко Олег Александрович	NULL	ЛУ	1960-01-01	28
БЗБРДВ	Безбородов Владимир Степанович	NULL	ОХ	1960-01-01	30
БЛВ	Беляев Валерий Павлович	NULL	ПОиСОИ	1960-01-01	23
БРГ	Бурганская Татьяна Минаевна	NULL	ЛПиСПС	1960-01-01	27
БРКВЧ	Бракович Андрей Игоревич	NULL	ИСиТ	1960-01-01	24
БРНВСК	Барановский Станислав Иванович	NULL	ЭТиМ	1960-01-01	30
БРС	Брусенцова Татьяна Палладьевна	NULL	ИСиТ	1960-01-01	30
БРТШВЧ	Барташевич Святослав Александрович	NULL	ПОиСОИ	1960-01-01	34
ГРМН	Герман Олег Витольдович	NULL	ИСиТ	1960-01-01	23
ГРН	Гурин Николай Иванович	NULL	ИСиТ	1960-01-01	22
ДЛН	Долинина Татьяна Николаевна	NULL	СБУАиА	1960-01-01	27
ДМДК	Демидко Марина Николаевна	NULL	ЛПиСПС	1960-01-01	25
ДТК	Дятко Александр Аркадьевич	NULL	ИСиТ	1960-01-01	26
ЕЩНК	Ещенко Людмила Семеновна	NULL	ТНВиО...	1960-01-01	24

Рис. 5.87. Содержимое таблицы **TEACHER** после ее модификации
с помощью сценария на рис. 5.86

На рис. 5.88 демонстрируется изменение свойства столбца **GENDER**: запрещается значение NULL.

```
use BSTU
go
alter table TEACHER alter column GENDER char(1) not null;
Msg 515, Level 16, State 2, Line 1
Cannot insert the value NULL into column 'GENDER', table 'BSTU.dbo.TEACHER';
column does not allow nulls. UPDATE fails.
The statement has been terminated.
```

Рис. 5.88. Сообщение об ошибке при изменении свойства
столбца **GENDER** таблицы **TEACHER**

Попытка установить свойство, запрещающее значения NULL в столбце **GENDER,** приводит к ошибке из-за того, что среди строк таблицы **TEACHER** уже есть такие, которые содержат значение NULL. В этом случае необходимо предварительно изменить все NULL-значения на какое-нибудь допустимое значение (рис. 5.89).

```
use BSTU
go
update TEACHER set GENDER = 'м' where GENDER is null or GENDER not in ('м', 'ж');
alter table TEACHER  alter column GENDER char(1) not null;
```

Рис. 5.89. Замена недопустимых значений в столбце **GENDER**
на допустимое значение и модификация таблицы

Для более полного знакомства с возможностями оператора ALTER TABLE рекомендуется обратиться к источнику [5].

Глава 6. ПРИМЕНЕНИЕ ОПЕРАТОРА INSERT

6.1. Структура оператора INSERT

Оператор INSERT относится к группе операторов DML SQL и предназначен для добавления строк в таблицу.

Уточним описанную ранее (рис. 3.16) структуру оператора INSERT (рис. 6.1). Оператор INSERT может добавить ноль, одну или более строк в заданную (таблица-приемник) таблицу БД. Источником строк для добавления служит тоже таблица (таблица-источник), содержащая ноль, одну или более строк. Причем таблица-приемник – это таблица БД (временная или постоянная) или таблица-переменная (этот тип таблиц рассматривается позже), а таблица-источник – это результирующий набор данных (строки, полученные в результате SELECT-запроса) или просто заданный список строк.

insert into *таблица-приемник таблица-источник*

Рис. 6.1. Структура оператора INSERT

В зависимости от того, каким образом описывается таблица-приемник и таблица источник, различают несколько форм оператора INSERT.

6.2. Простейшая форма оператора INSERT

Простейшая форма оператора INSERT подразумевает ввод данных во все столбцы таблицы-приемника, при этом считается, что столбцы упорядочены в той последовательности, в которой они заданы оператором CREATE TABLE при создании таблицы-приемника.

На рис. 6.2 представлен фрагмент вывода информации о структуре таблицы **SUBJECT**, сформированный системной процедурой SP_HELP.

```
use BSTU
go
exec SP_HELP [SUBJECT]
```

Column_name	Type	Computed	Length	Prec	Scale	Nullable	TrimTrailingBlanks	FixedLenNullInSource
SUBJECT	char	no	10			no	no	no
SUBJECT_NAME	varchar	no	100			yes	no	yes
PULPIT	char	no	20			yes	no	yes

Рис. 6.2. Информация о структуре таблицы **SUBJECT**

Обратим внимание на порядок столбцов в таблице – процедура **SP_HELP** выводит их в стандартном порядке – в порядке их следования в операторе CREATE TABLE при создании таблицы.

Пример использования операторов INSERT в простейшей форме приведен на рис. 6.3.

```
use BSTU
go
insert into SUBJECT  values ('СУБД',   'Системы управления базами данных','ИСиТ');
insert into SUBJECT  values ('БД',     'Базы данных','ИСиТ'),
                            ('ИНФ',    'Информационные технологии','ИСиТ'),
                            ('ОАиП',  'Основы алгоритмизации и программирования','ИСиТ'),
                            ('ПЗ',     'Представление знаний в компьютерных системах','ИСиТ');
```

Рис. 6.3. Пример использования оператора INSERT простейшей формы

Первый оператор INSERT вводит одну строку в таблицу **SUBJECT**. Строка представляется в виде списка значений, перечисленных в скобках за ключевым словом VALUES. Количество элементов списка и их порядок соответствуют количеству столбцов в таблице и их стандартному порядку (рис. 6.2).

Рассмотрим структуру таблицы **GROUPS** (рис. 6.4).

```
use BSTU
go
exec SP_HELP GROUPS
```

Column_name	Type	Computed	Length	Prec	Scale	Nullable	TrimTrailingBlanks	FixedLenNullInSource
IDGROUP	int	no	4	10	0	no	(n/a)	(n/a)
FACULTY	char	no	10			yes	no	yes
PROFESSION	char	no	20			yes	no	yes
YEAR_FIRST	smallint	no	2	5	0	yes	(n/a)	(n/a)
COURSE	int	yes	4	10	0	yes	(n/a)	(n/a)

Identity	Seed	Increment	Not For Replication
IDGROUP	1	1	0

Рис. 6.4. Информация о структуре таблицы **GROUPS**

Обратим внимание на то, что таблица **GROUPS** содержит столбец **IDGROUP** со свойством IDENTITY (на это указывает строка во втором результирующем наборе на рис. 6.4) и вычисляемый столбец **COURSE** (отметка yes в столбце Computed первого набора). Значение обоих столбцов формируется автоматически, поэтому при применении оператора INSERT они просто игнорируются (рис. 6.5). Аналогично следует поступать в случае использования столбцов с типом TIMESTAMP и ROWVERSION (см. рис. 5.13 на с. 69).

```
use BSTU
go
insert into GROUPS values ('ИДиП','1-40 01 02', 2013),
                          ('ИДиП','1-40 01 02', 2012),
                          ('ИДиП','1-40 01 02', 2011),
                          ('ИДиП','1-40 01 02', 2010),
                          ('ИДиП','1-47 01 01', 2013),
                          ('ИДиП','1-47 01 01', 2012),
                          ('ИДиП','1-47 01 01', 2011),
                          ('ИДиП','1-36 06 01', 2010),
                          ('ИДиП','1-36 06 01', 2013);
```

Рис. 6.5. Применение оператора INSERT для добавления строк в таблицу **GROUPS**

В том случае, если требуется ввести значение NULL, то это надо указывать явно. Если необходимо ввести значение по умолчанию, то можно использовать ключевое слова DEFAULT (рис. 5.42).

6.3. Форма оператора INSERT с применением списка столбцов

В том случае, если требуется обозначить порядок значений в списке VALUES оператора INSERT, применяется список столбцов. Список с именами столбцов указывается в скобках сразу после имени таблицы в секции.

На рис. 6.6. представлен примеры нескольких операторов INSERT, применяющих список столбцов. Обратите внимание: ключевое слово INTO при написании оператора INSERT не является обязательным и может быть опущено.

```
use BSTU
go
insert  TEACHER(TEACHER,   TEACHER_NAME, PULPIT)
    values  ('СМЛВ',    'Смелов Владимир Владиславович',  'ИСиТ');
insert  TEACHER(TEACHER, PULPIT,  TEACHER_NAME)
    values  ('АКНВЧ',  'ИСиТ',  'Акунович Станислав Иванович');
insert  TEACHER(GENDER, TEACHER, TEACHER_NAME, PULPIT)
    values  (default,   'КЛСНВ',    'Колесников Виталий Леонидович',  'ИСиТ');
insert   TEACHER(PULPIT, TEACHER,  TEACHER_NAME)
     values  ('ИСиТ', 'ГРМН',    'Герман Олег Витольдович'),
             ('ИСиТ', 'ЛЩНК',    'Лащенко Анатолий Павлович'),
             ('ИСиТ', 'БРКВЧ',   'Бракович Андрей Игоревич');
```

Рис. 6.6. Примеры оператора INSERT со списком столбцов

В каждом операторе INSERT на рис. 6.6 применяется разный список столбцов. Кроме того, в первом операторе не предполагается ввод значения в столбец **GENDER** (его нет в списке столбцов), поэтому введется значение NULL или значение по умолчанию, в третьем операторе явно указывается, что в столбец вводится значение по умолчанию.

Как и в предыдущем случае, столбцы, имеющие тип, для которого значение формируется автоматически, в списке не указываются.

6.4. Конструкция INSERT … SELECT

В качестве таблицы-источника может выступать результирующий набор, сформированный оператором SELECT. На рис. 6.7 представлен многотабличный SELECT-запрос (детально такие запросы будут рассматриваться позже). Результирующий набор, сформированный после выполнения запроса (на рис. 6.7 представлен его фрагмент), содержит данные из трех таблиц: **STUDENT**, **SUBJECT** и **PROGRESS**.

```
use BSTU
go
select st.NAME,pr.PDATE,sb.SUBJECT_NAME, pr.NOTE
from STUDENT st join PROGRESS  pr on st.IDSTUDENT = pr.IDSTUDENT
                join [SUBJECT] sb on pr.[SUBJECT] = sb.[SUBJECT]
```

NAME	PDATE	SUBJECT_NAME	NOTE
Манакова Анастасия Владимировна	2013-01-10	Основы алгоритмизации и программирования	6
Хартанович Екатерина Александровна	2013-01-10	Основы алгоритмизации и программирования	8
Горбач Елизавета Юрьевна	2013-01-10	Основы алгоритмизации и программирования	7
Зыкова Кристина Дмитриевна	2013-01-10	Основы алгоритмизации и программирования	5
Медведева Мария Андреевна	2013-01-10	Основы алгоритмизации и программирования	4
Шенец Екатерина Сергеевна	2013-01-10	Основы алгоритмизации и программирования	4
Шитик Алина Игоревна	2013-01-10	Основы алгоритмизации и программирования	8
Силюк Валерия Ивановна	2013-06-12	Базы данных	4
Сергель Виолетта Николаевна	2013-06-12	Базы данных	9
Добродей Ольга Анатольевна	2013-06-12	Базы данных	5
Подоляк Мария Сергеевна	2013-06-12	Базы данных	7

Рис. 6.7. Многотабличный SELECT-запрос и результирующий набор, сформированный по итогам его выполнения

Иногда для последующей обработки выбранных данных бывает полезным сохранить их во временной таблице. На рис. 6.8 представлен сценарий, в котором создается временная таблица с именем [#Результаты экзамена], и с помощью оператора INSERT результат SELECT-запроса (рис. 6.7) помещается в эту таблицу. Следует обратить внимание, что для таблицы-приемника используется форма, подразумевающая стандартный порядок столбцов. На рис. 6.9 представлен SELECT-запрос к таблице [#Результаты экзамена], а также результат выполнения этого запроса. Не сложно убедиться, что соответствие между столбцами результирующего набора и столбами таблицы [#Результаты экзамена] устанавливается по порядку их перечисления.

```
use BSTU
go
if  exists (  --  если в TEMPDB есть таблица [#Результаты экзамена]
            select * from  TEMPDB.sys.objects
            where OBJECT_ID =  object_id(N'TEMPDB.DBO.[#Результаты экзамена]') and TYPE in (N'U')
            )
   drop  table [#Результаты экзамена]  -- то эта таблица удаляется
go
create table [#Результаты экзамена] -- создать таблицу [#Результаты экзамена]
   (
    [Ф.И.О. студента] varchar(50),
    [дата экзамена]   date,
    [дисциплина]      varchar(50),
    [оценка]          tinyint
    );
go
insert [#Результаты экзамена]  -- поместить результат SELECT-запроса в  таблицу [#Результаты экзамена]
     select st.NAME,pr.PDATE,sb.SUBJECT_NAME, pr.NOTE
     from STUDENT st join PROGRESS  pr on st.IDSTUDENT = pr.IDSTUDENT
                     join [SUBJECT] sb on pr.[SUBJECT] = sb.[SUBJECT];
```

Рис. 6.8. Оператор INSERT, помещающий результат SELECT-запроса во временную таблицу

```
select * from [#Результаты экзамена]
```

Ф.И.О. студента	дата экзамена	дисциплина	оценка
Манакова Анастасия Владимировна	2013-01-10	Основы алгоритмизации и программирования	6
Хартанович Екатерина Александровна	2013-01-10	Основы алгоритмизации и программирования	8
Горбач Елизавета Юрьевна	2013-01-10	Основы алгоритмизации и программирования	7
Зыкова Кристина Дмитриевна	2013-01-10	Основы алгоритмизации и программирования	5
Медведева Мария Андреевна	2013-01-10	Основы алгоритмизации и программирования	4
Шенец Екатерина Сергеевна	2013-01-10	Основы алгоритмизации и программирования	4
Шитик Алина Игоревна	2013-01-10	Основы алгоритмизации и программирования	8
Силюк Валерия Ивановна	2013-06-12	Базы данных	4
Сергель Виолетта Николаевна	2013-06-12	Базы данных	9
Добродей Ольга Анатольевна	2013-06-12	Базы данных	5
Подоляк Мария Сергеевна	2013-06-12	Базы данных	7

Рис. 6.9. SELECT-запрос к временной таблице [#Результаты экзамена] и результат его выполнения

При необходимости можно применить список столбцов для того, чтобы обеспечить соответствие порядку столбцов, указанному в SELECT-запросе. На рис. 6.10 приведен пример оператора INSERT со списком столбцов. Обратите внимание, что столбцы в SELECT-запросе переставлены в другом порядке, нежели в запросе на рис. 6.8, а порядок столбцов, указанный в INSERT, соответствует этому порядку.

```
use BSTU
go
insert [#Результаты экзамена]([оценка], [дисциплина],[дата экзамена],[Ф.И.О. студента])
       select pr.NOTE, sb.SUBJECT_NAME, pr.PDATE,st.NAME
       from STUDENT st join PROGRESS  pr on st.IDSTUDENT = pr.IDSTUDENT
                       join [SUBJECT] sb on pr.[SUBJECT] = sb.[SUBJECT];
```

Рис. 6.10. Пример оператора INSERTсо списком столбцов

При добавлении строк с помощью конструкции INSERT … SELECT можно ограничить количество добавляемых строк с помощью необязательной секции TOP, которая располагается сразу за ключевым словом INSERT(рис. 6.11).

```
use BSTU
go
select count(*) [количество строк в таблице] from [#Результаты экзамена]
insert top (7) into[#Результаты экзамена]([оценка], [дисциплина],[дата экзамена],[Ф.И.О. студента])
       select pr.NOTE, sb.SUBJECT_NAME, pr.PDATE,st.NAME
       from STUDENT st join PROGRESS  pr on st.IDSTUDENT = pr.IDSTUDENT
                       join [SUBJECT] sb on pr.[SUBJECT] = sb.[SUBJECT]
       order by pr.NOTE;
select count(*) [количество строк в таблице] from [#Результаты экзамена]
```

количество строк в таблице
0

количество строк в таблице
7

Рис. 6.11. Пример использования секции TOP в операторе INSERT

В примере на рис. 6.11 в операторе INSERT используется секция TOP, которая задает количество строк (в наше случае 7), добавляемых в таблицу [#Результаты экзамена]. Надо обратить внимание на то, что TOP отбирает из результирующего набора только заданное количество первых строк (в нашем случае 7 первых строк), поэтому важен порядок сортировки, указанный в операторе SELECT с помощью секции ORDER BY (подробно применение секции ORDER BY будет рассматриваться позже).

6.5. Конструкция INSERT … EXECUTE

SELECT-запросы, используемые в операторе INSERT, могут быть достаточно сложными и, кроме того, могут применяться и в других местах. Поэтому важно иметь механизм, позволяющий сохранить однажды написанный запрос для его повторного применения. Microsoft SQL Server 2008 имеет несколько таких механизмов: представления, хранимые процедуры и функции – все они будут подробно рассматриваться в этом пособии позже.

Конструкция INSERT … EXECUTE очень похожа на конструкцию INSERT … SELECT и таблица-источник тоже является результирующим набором, сформированным в результате SELECT-запроса. Отличие в том, что SELECT-запрос выполняется в рамках хранимой процедуры.

В первом приближении хранимая процедура – объект БД, представляющий собой поименованный код, написанный на T-SQL, который может принимать параметры, возвращать параметры и формировать результирующие наборы. В нашем случае интерес представляет именно последняя из перечисленных возможностей хранимой процедуры – способность формировать результирующие наборы.

На рис. 6.12 приведен пример сценария, в котором сначала создается хранимая процедура **GetExamResults** с помощью оператора CREATE PROCEDURE, а затем вызывается с помощью оператора EXECUTE.

```
use BSTU
go
create procedure GetExamResults  -- создать процедуру с именем GetExamResults
as                               --  хранится в БД BSTU
begin -- формирует результирующий набор
  select st.NAME,pr.PDATE,sb.SUBJECT_NAME, pr.NOTE
       from STUDENT st join PROGRESS  pr on st.IDSTUDENT = pr.IDSTUDENT
                       join [SUBJECT] sb on pr.[SUBJECT] = sb.[SUBJECT];
end;
go;
execute GetExamResults;     -- вызов процедуры
```

NAME	PDATE	SUBJECT_NAME	NOTE
Манакова Анастасия Владимировна	2013-01-10	Основы алгоритмизации и программирования	6
Хартанович Екатерина Александровна	2013-01-10	Основы алгоритмизации и программирования	8
Горбач Елизавета Юрьевна	2013-01-10	Основы алгоритмизации и программирования	7
Зыкова Кристина Дмитриевна	2013-01-10	Основы алгоритмизации и программирования	5
Медведева Мария Андреевна	2013-01-10	Основы алгоритмизации и программирования	4
Шенец Екатерина Сергеевна	2013-01-10	Основы алгоритмизации и программирования	4
Шитик Алина Игоревна	2013-01-10	Основы алгоритмизации и программирования	8
Силюк Валерия Ивановна	2013-06-12	Базы данных	4
Сергель Виолетта Николаевна	2013-06-12	Базы данных	9
Добродей Ольга Анатольевна	2013-06-12	Базы данных	5
Подоляк Мария Сергеевна	2013-06-12	Базы данных	7

Рис. 6.12. Пример создания и вызова процедуры, формирующей результирующий набор

Хранимая процедура **GetExamResults** не принимает никаких параметров, она только выполняет один SELECT-запрос (аналогичный запросу на рис. 6.8), который формирует результирующий набор.

На рис. 6.13 приведен сценарий, демонстрирующий применение конструкции INSERT … EXECUTE.

```
use BSTU
go
insert [#Результаты экзамена]  execute GetExamResults;
```

Рис. 6.13. Пример использования конструкции INSERT … EXECUTE с вызовом хранимой процедуры, созданной сценарием на рис. 6.12

Результат выполнения оператора INSERT на рис. 6.13 будет точно такой же, как при выполнении оператора INSERT в сценарии на рис. 6.8.

В общем случае, хранимая процедура может формировать несколько результирующих наборов. Однако при использовании такой процедуры в конструкции INSERT … EXECUTE выдвигаются требования к однородности (одинаковое количество столбцов и совместимые типы соответствующих столбцов) результирующих наборов, т. к. они используются как один общий последовательно составленный из фрагментов результирующий набор.

6.6. Применение секции OUTPUT

Операторы INSERT обновляют БД и поэтому часто являются частью транзакции. Механизм транзакций рассматривается в этом пособии позже, но здесь поясним, что транзакция в первом приближении – это набор операторов DML SQL, которые рассматриваются как единое целое и могут либо *все* полностью выполниться (и изменить БД) или *все* не выполниться (и БД не изменится), причем ключевое слово здесь «все».

Если разработчик БД собирается фиксировать DML-события (INSERT, UPDATE, DELETE), изменяющие БД (например, записывать информацию об изменениях в БД), то он должен в рамках изменяющей БД транзакции записывать эту информацию (обычно в другую таблицу, называемую журналом). Однако если транзакция не завершится успешно, то все изменения БД, которые были выполнены в рамках транзакции, аннулируются, в том числе данные, записанные в журнал. Таким образом, в журнале будет сохраняться информация только об успешных изменениях БД (в случае успешного завершения транзакции), а информация о неуспешных попытках изменить БД в журнале не сохранится, т. к. сама запись в журнал является частью транзакции, изменяющей БД.

Секция OUTPUT может использоваться во всех трех DML-операторах, изменяющих БД: INSERT, DELETE, UPDATE. Главной особенностью этой секции является то, что на ее работу не распространяется действие процедуры отката (ROLLBACK), устраняющей все изменения БД, выполненные DML-операторами в рамках транзакции в случае ее неуспешного завершения.

В рамках оператора INSERT секция OUTPUT позволяет получить и записать данные из INSERT-оператора в один из двух источников: в таблицу БД или в результирующий набор.

На рис. 6.14 представлен пример использования оператора INSERT с секцией OUTPUT, предназначенной для вывода информации в результирующий набор.

```
use BSTU
go
insert into  TEACHER    (TEACHER,   TEACHER_NAME, PULPIT)
       output   'INSERT', inserted.TEACHER , SYSDATETIME()
       values  ('АКНВЧ',     'Акунович Станислав Иванович',  'ИСиТ'),
               ('КЛСНВ',     'Колесников Виталий Леонидович','ИСиТ'),
               ('ГРМН',      'Герман Олег Витольдович',      'ИСиТ'),
               ('ЛЩНК',    'Лащенко Анатолий Павлович',      'ИСиТ'),
               ('БРКВЧ',     'Бракович Андрей Игоревич',     'ИСиТ'),
               ('ДТК',     'Дятко Александр Аркадьевич',     'ИСиТ');
```

(No column name)	TEACHER	(No column name)
INSERT	АКНВЧ	2014-01-12 01:09:01.0552968
INSERT	КЛСНВ	2014-01-12 01:09:01.0552968
INSERT	ГРМН	2014-01-12 01:09:01.0552968
INSERT	ЛЩНК	2014-01-12 01:09:01.0552968
INSERT	БРКВЧ	2014-01-12 01:09:01.0552968
INSERT	ДТК	2014-01-12 01:09:01.0552968

Рис. 6.14. Пример использования секции OUTPUT в операторе INSERT

Следует обратить внимание на префикс INSERTED, используемый в списке OUTPUT. Этот префикс обозначает, что далее через точку следует имя столбца таблицы, в которую добавляется строка.

Результирующий набор можно направить в предварительно созданную таблицу. На рис. 6.15 приведен пример оператора INSERT, использующий секцию OUTPUT для вывода информации в предварительно созданную временную таблицу [#Журнал]. Содержимое таблицы [#Журнал] выводится с помощью оператора SELECT в сценарии на рис. 6.16.

```
use BSTU
go
create table [#Журнал]
(
 [Идентификатор] bigint identity (1,1),
 [Операция]      char(8),
 [Дата и время]  datetime2,
 [Ключ]          varchar(100)
)
go
insert into  TEACHER     (TEACHER,    TEACHER_NAME, PULPIT)
       output   'INSERT', SYSDATETIME(), inserted.TEACHER into [#Журнал]
       values  ('АКНВЧ',     'Акунович Станислав Иванович',  'ИСиТ'),
               ('КЛСНВ',     'Колесников Виталий Леонидович','ИСиТ'),
               ('ГРМН',      'Герман Олег Витольдович',      'ИСиТ'),
               ('ЛЩНК',     'Лащенко Анатолий Павлович',     'ИСиТ'),
               ('БРКВЧ',     'Бракович Андрей Игоревич',     'ИСиТ'),
               ('ДТК',      'Дятко Александр Аркадьевич',    'ИСиТ');
```

Рис. 6.15. Пример использования в операторе INSERT секции OUTPUT, записывающей данные в таблицу

```
use BSTU
go
select * from [#Журнал];
```

Идентификатор	Операция	Дата и время	Ключ
1	INSERT	2014-01-12 01:29:30.9693593	АКНВЧ
2	INSERT	2014-01-12 01:29:30.9693593	КЛСНВ
3	INSERT	2014-01-12 01:29:30.9693593	ГРМН
4	INSERT	2014-01-12 01:29:30.9693593	ЛЩНК
5	INSERT	2014-01-12 01:29:30.9693593	БРКВЧ
6	INSERT	2014-01-12 01:29:30.9693593	ДТК

Рис. 6.16. Содержимое таблицы [#Журнал], сформированное секцией OUTPUT оператора INSERT в сценарии на рис. 6.15

Применение секции OUPUT не допускается в конструкции INSERT … EXECUTE, требует списка вводимых столбцов, а также накладывает ряд ограничений на таблицу, в которую секция OUTPUT записывает информацию. Более детально с применением секции OUTPUT можно ознакомиться в издании [5].

Глава 7. ПРИМЕНЕНИЕ ОПЕРАТОРА SELECT

7.1. Структура оператора SELECT

Среди DML-операторов SELECT является наиболее сложным и важным. Умение составлять SELECT-запросы – необходимое условие разработки приложений БД. В общем случае, SELECT-запрос может быть частью запроса к БД, выполненного с помощью любого другого DML-оператора.

Структура оператора SELECT представлена на рис. 7.1.

SELECT TOP DISTINCT список столбцов
INTO новая таблица
FROM источник строк
WHERE логическое условие – фильтр строк
GROUP BY выражение для группировки строк
HAVING логическое условие – фильтр строк
ORDER BY выражение для сортировки ASC DESC

Рис. 7.1. Структура оператора SELECT

Оператор SELECT состоит из нескольких секций: SELECT, INTO, FROM, WHERE, GROUP BY, HAVING, ORDER BY. Кроме того, в секции SELECT могут использоваться опции TOP и/или DISTINCT, а в секции ORDER BY опции ASC или DESC. Обязательной является только секция SELECT.

Для пояснения работы оператора SELECT уточним некоторые понятия, который до сих пор использовались интуитивно.

Строка – это упорядоченный список данных, состоящий из одного или нескольких элементов. Каждый элемент списка характеризуется типом данных и уникальным в рамках строки именем. Если имя элемента не задано, то именем считается его номер в списке. Упорядоченная последовательность типов и имен элементов строки называется структурой строки.

Набор однородных строк – набор строк, имеющих одинаковую структуру. Множество элементов из набора однородных строк, имеющих одно и то же имя, образуют столбец набора однородных строк. Набор однородных строк может содержать 0, 1 или более строк. Набор, содержащий 0 строк, называется пустым набором однородных строк. Набор однородных строк, состоящий из одной строки, в которой только один элемент, называется скалярным набором.

Результатом работы SELECT-запроса являются данные, организованные в виде набора однородных строк, называемого результирующим набором данных. Результирующий набор данных, не содержащий ни одной строки, называется пустым результирующим набором данных. Результирующий набор, состоящий из одной строки, в которой только один элемент, называется скалярным результирующим набором.

Основное назначение секции SELECT – указать порядок и имена столбцов результирующего набора данных. Основой для содержимого SELECT-секции является набор однородных строк, описанный во FROM. В простейшем случае SELECT без изменения выводит все столбцы однородного набора строк, сформированного в секции FROM. Опция DISTINCT позволяет избавиться от повторяющихся строк в результирующем наборе, а TOP – ограничить в нем их количество. В общем случае в этой секции указывается список выражений, содержащих имена столбцов набора строк из FROM, константы, а также скалярные результаты SELECT-запросов. Такой список будем называть SELECT-списком. Часто говорят, что SELECT-список определяет вертикаль результирующего набора.

Секция INTO позволяет на основе структуры результирующего набора (заданную SELECT-списком) создать новую таблицу и заполнить ее строками результирующего набора.

Секция FROM в общем случае содержит выражение, результатом которого является набор однородных строк с явно поименованными столбцами. В простейшем случае – это таблица БД, а в общем случае, может быть достаточно сложное выражение, содержащее несколько таблиц,

SELECT-запросов, представлений и табличных функций (представления и табличные функции рассматриваются позже).

Секция WHERE применяется для отбора (фильтрации) строк, сформированных в секции FROM. Как правило, секция содержит логическое выражение, которое вычисляется для каждой строки. В том случае, если выражение принимает истинное значение – строка отбирается. Логическое выражение может быть достаточно сложным и содержать в себе другой SELECT-запрос. Говорят, что секция WHERE определяет горизонталь результирующего набора.

GROUP BY – секция, позволяющая сгруппировать данные по значениям, хранящимся в столбцах однородного набора строк, сформированного в секции FROM. Секция GROUP BY ограничивает содержимое SELECT-списка.

Секция HAVING имеет такое же назначение, что и WHERE, но применяется после выполнения секции GROUP BY.

Для сортировки результирующего набора применяется секция ORDER BY. В этой секции может быть указано одно или несколько выражений над элементами столбцами набора однородных строк, сформированного в секции FROM. Выражения дают возможность задать порядок строк в результирующем наборе. Опции ASC и DESC позволяют указать порядок сортировки строк (по возрастанию или по убыванию). Применение опции DISTINCT в секции SELECT ограничивает возможности сортировки.

Очень важным для понимания работы оператора SELECT является последовательность применения сервером СУБД секций и опций SELECT-запроса (рис. 7.2).

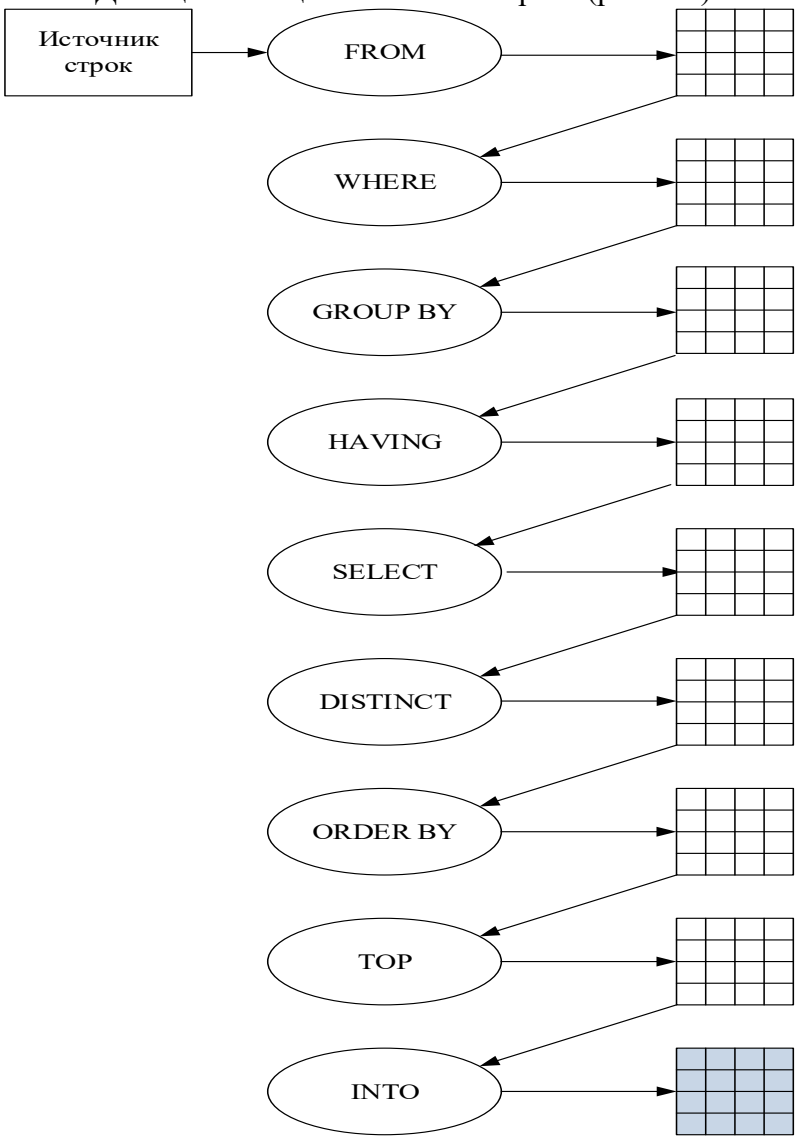


Рис. 7.2. Последовательность выполнения секций в SELECT-запросе

В результате выполнения каждой секции (на рис. 7.2 обозначены овалами) формируются промежуточные результирующие наборы (прямоугольники).

Первой выполняется секция FROM. В этой секции на основе источника строк формируется первый промежуточный результирующий набор данных.

Секция INTO выполняется последней. В ней на основе входящего промежуточного результирующего набора формируется и заполняется строками таблица.

На рис. 7.2 изображены все секции, но в реальном SELECT-запросе такое случается редко. Например, не всякий SELECT-запрос завершается построением новой таблицы. В этом случае, в результате выполнения SELECT-запроса будет сформирован выходной результирующий набор, являющийся выходным для опции TOP. Если не указан TOP, то результат будет сформирован секцией ORDER BY.

Некоторые секции являются зависимыми. Например, секция HAVING может обрабатывать только результат выполнения секции GROUP BY, а при наличии секции DISTINCT, запрещается использовать в секции ORDER BY имена столбцов, отсутствующих в списке SELECT.

Обязательной является только секция SELECT, все остальные секции могут быть опущены. На рис. 7.3 представлены примеры SELECT-запросов разной сложности.

```
use BSTU
-- только select -------
select  'Hello World';
select SYSDATETIME() ;
select 365*60*60;
--- select...from ---
select TEACHER_NAME from TEACHER;
select count(*) from AUDITORIUM;
--- select...from...order by ---
select TEACHER_NAME from TEACHER order by PULPIT;
select AUDITORIUM_NAME from AUDITORIUM order by AUDITORIUM_CAPACITY desc;
--- select...from...group by...order by ---
select AUDITORIUM_NAME from AUDITORIUM order by AUDITORIUM_CAPACITY desc;
select FACULTY, count(*)  from PULPIT
       group by FACULTY order by 1;
--- select ... from ... group by...having...order by ---
select FACULTY, count(*)  from PULPIT
       group  by FACULTY   having count(*)> 3 order by FACULTY;
--- select.. into ...top...from...group by...having...order by ---
select  top(2) PULPIT, count(*) CC into #SUBJECT from [SUBJECT]
        group  by PULPIT  having count(*)> 3 order by CC;
```

Рис. 7.3. Примеры SELECT-запросов

7.2. Список выбора

Список выбора (SELECT-список) определяет перечень и имена столбцов результирующего набора данных. В простейшем случае (рис. 7.4) SELECT-список может содержать константы или выражения независящие от строк, указанных в секции FROM (если она есть).

```
use BSTU
select  'Hello World';
```

(No column name)
Hello World

Рис. 7.4. Пример простейшего SELECT-запроса

Обратите внимание (рис. 7.4) на то, что, если секция FROM отсутствует, то в результирующем наборе формируется только одна строка. В SELECT-списке задана строчная константа – она и отобразилась в безымянном столбце результирующего набора (No column name). Имя столбца, может быть задано с помощью ключевого слова AS (рис. 7.5).

```
use BSTU
select  'Hello' as first_word,  'World' [второе слово];
```

first_word	второе слово
Hello	World

Рис. 7.5. SELECT-запрос с поименованными столбцами

SELECT-запрос на рис. 7.5 формирует результирующий набор, состоящий из двух поименованных столбцов. Заданные имена (их еще называют псевдонимами или алиасами) указываются через ключевое слово AS или сразу после элемента списка (AS может быть опущено). Если псевдоним не удовлетворяет правилам именования столбцов таблицы, то его можно указать с применением квадратных скобок.

Если в SELECT-запросе указан источник строк с помощью секции FROM, то в результирующем наборе формируется столько строк, сколько их сформировала секция FROM. В примере на рис. 7.6 в качестве источника используется таблица с именем **FACULTY**. Таблица содержит шесть строк, поэтому результирующий набор состоит из шести строк. SELECT-список применяется для каждой строки FROM-набора. В связи с тем, что единственное в SELECT-списке выражение (константа) не зависит от значений в строках, значение этого выражения повторяется шесть раз.

```
use BSTU
select  'Hello' [константа] from FACULTY;
```

константа
Hello
Hello
Hello
Hello
Hello
Hello

Рис. 7.6. SELECT-запрос с указанным источником строк

Специальное выражение * (звездочка) используется для обозначения списка с именами всех столбцов FROM-набора. В примерах на рис. 7.7 и 7.8 SELECT-запросы формируют результирующие наборы, аналогичные содержимому таблицы **FACULTY**, являющейся источником строк для FROM-набора.

```
use BSTU
select  * from FACULTY;
```

FACULTY	FACULTY_NAME
ИДиП	Издателькое дело и полиграфия
ИЭФ	Инженерно-экономический факультет
ЛХФ	Лесохозяйственный факультет
ТОВ	Технология органических веществ
ТТЛП	Технология и техника лесной промышленности
ХТиТ	Химическая технология и техника

Рис. 7.7. SELECT-запрос, выводящий все столбцы FROM-набора

```
use BSTU
select  SYSTEM_USER [пользователь сервера], USER [пользователь БД], * from FACULTY;
```

пользователь сервера	пользователь БД	FACULTY	FACULTY_NAME
sa	dbo	ИДиП	Издателькое дело и полиграфия
sa	dbo	ИЭФ	Инженерно-экономический факультет
sa	dbo	ЛХФ	Лесохозяйственный факультет
sa	dbo	ТОВ	Технология органических веществ
sa	dbo	ТТЛП	Технология и техника лесной промышленности
sa	dbo	ХТиТ	Химическая технология и техника

Рис. 7.8. SELECT-запрос, выводящий все столбцы FROM-набора

Обратите внимание на рис. 7.7: в результирующий набор выводятся строки, идентичные строкам таблицы **FACULTY**.

На рис. 7.8 помимо строк и столбцов таблицы **FACULTY** имеются два дополнительных столбца, значения в которых формируются для каждой строки FROM-набора, но они никак не зависят от их содержимого.

7.2.1. Список столбцов

Обычно при выполнении SELECT-запроса список столбцов указывается явно (рис. 7.9, 7.10). Если не указаны псевдонимы, то имена столбцов результирующего набора совпадают с именами столбцов источника.

```
use BSTU
select  TEACHER, PULPIT [кафедра] from TEACHER;
```

TEACHER	кафедра
АКНВЧ	ИСиТ
БРКВЧ	ИСиТ
ГРМН	ИСиТ
ДТК	ИСиТ
КПСНВ	ИСиТ
ЛЩНК	ИСиТ

Рис. 7.9. SELECT-запрос с явным списком столбцов

Таблица **TEACHER** имеет три столбца. В SELECT-запросе на рис. 7.9 в результирующий набор выводится только два столбца, причем второй столбец переименовывается (ему назначается псевдоним).

```
use BSTU
go
select TEACHER+ ' '+ PULPIT    [конкатенация строк],
       substring(TEACHER,1,2)  [подстрока],
       upper(TEACHER_NAME)     [верхний регистр],
       lower(TEACHER_NAME)     [нижений регистр]
from TEACHER;
```

конкатенация строк	подстрока	верхний регистр	нижений регистр
АКНВЧ ИСиТ	АК	АКУНОВИЧ СТАНИСЛАВ ИВАНОВИЧ	акунович станислав иванович
БРКВЧ ИСиТ	БР	БРАКОВИЧ АНДРЕЙ ИГОРЕВИЧ	бракович андрей игоревич
ГРМН ИСиТ	ГР	ГЕРМАН ОЛЕГ ВИТОЛЬДОВИЧ	герман олег витольдович
ДТК ИСиТ	ДТ	ДЯТКО АЛЕКСАНДР АРКАДЬЕВИЧ	дятко александр аркадьевич
КЛСНВ ИСиТ	КЛ	КОЛЕСНИКОВ ВИТАЛИЙ ЛЕОНИДОВИЧ	колесников виталий леонидович
ЛЩНК ИСиТ	ЛЩ	ЛАЩЕНКО АНАТОЛИЙ ПАВЛОВИЧ	лащенко анатолий павлович

Рис. 7.10. SELECT-запрос, применяющий выражения над содержимым столбцов

Пример на рис. 7.10 демонстрирует применение SELECT-списка, в котором используются выражения над содержимым явно указанных столбцов.

7.2.2. Ключевое слово DISTINCT

На рис. 7.11 представлен SELECT-запрос и фрагмент результирующего набора, сформированного этим запросом.

```
use BSTU
go
select  FACULTY, QUALIFICATION from PROFESSION order by FACULTY, QUALIFICATION;
```

FACULTY	QUALIFICATION
ИЭФ	экономист-менеджер
ЛХФ	инженер лесного хозяйства
ЛХФ	инженер садово-паркового строительства
ЛХФ	специалист в сфере туризма
ТОВ	инженер по сертификации
ТОВ	инженер-химик-технолог
ТОВ	инженер-химик-технолог
ТТЛП	инженер-механик
ТТЛП	инженер-технолог

Рис. 7.11. SELECT-запрос, формирующий результирующий набор
с повторяющимися строками

Обратим внимание на то, что среди строк результирующего набора есть повторяющиеся.

Ключевое слово DISTINCT позволяет избавиться от повторяющихся строк (рис. 7.11) в результирующем наборе.

```
use BSTU
go
select distinct FACULTY, QUALIFICATION from PROFESSION order by FACULTY, QUALIFICATION;
```

FACULTY	QUALIFICATION
ИЭФ	экономист-менеджер
ЛХФ	инженер лесного хозяйства
ЛХФ	инженер садово-паркового строительства
ЛХФ	специалист в сфере туризма
ТОВ	инженер по сертификации
ТОВ	инженер-химик-технолог
ТТЛП	инженер-механик
ТТЛП	инженер-технолог
ХТиТ	инженер-механик

Рис. 7.12. SELECT-запрос с применением ключевого слова DISTINCT

7.2.3. Ключевое слово TOP

Ключевое слово TOP позволяет ограничить количество строк в результирующем выражении.

На рис. 7.13 демонстрируется применение ключевого слова TOP для получения первых трех строк результирующего набора.

```
use BSTU
go
select top(3) IDSTUDENT, NAME  from STUDENT
```

IDSTUDENT	NAME
1000	Манакова Анастасия Владимировна
1001	Хартанович Екатерина Александровна
1002	Горбач Елизавета Юрьевна

Рис. 7.13. Применение ключевого слова TOP для выбора первых трех строк результирующего набора данных

Кроме абсолютного значения в TOP можно указать количество выбираемых строк в процентах (рис. 7.14).

```
use BSTU
go
select count(*)                        [количество строк],
       round(3.0*count(*)/100.0,1) [3% строк]
from STUDENT;
select top(3) percent  IDSTUDENT,NAME from STUDENT;
```

количество строк	3% строк
178	5.300000

IDSTUDENT	NAME
1000	Манакова Анастасия Владимировна
1001	Хартанович Екатерина Александровна
1002	Горбач Елизавета Юрьевна
1003	Зыкова Кристина Дмитриевна
1004	Борисевич Ольга Анатольевна

Рис. 7.14. Применение ключевого слова TOP для выбора трех процентов первых строк результирующего набора

Следует отметить, что обычно ключевое слово TOP применяется с секцией ORDER BY, предназначенной для сортировки. Опция TOP выполняется после сортировки – это позволяет отобрать строки, оказавшиеся в результирующем наборе на первых позициях после сортировки.

Допустимо применение TOP совместно с DISTINCT (рис. 7.15). В этом случае, сначала устраняются повторяющиеся строки (выполняется DISTINCT), а потом выбирается заданное количество первых строк (TOP).

```
use BSTU
go
select distinct top(3) IDSTUDENT, NAME  from STUDENT
```

IDSTUDENT	NAME
1000	Манакова Анастасия Владимировна
1001	Хартанович Екатерина Александровна
1002	Горбач Елизавета Юрьевна

Рис. 7.15. Совместное применение DISTINCT и TOP

7.2.4. Применение подзапросов в списке SELECT

Подзапрос – это SELECT-запрос, выполненный в рамках другого запроса. При этом основной SELECT-запрос называется внешним, а SELECT-запрос, который выполняется в рамках внешнего – внутренним.

На рис. 7.16 приведен пример SELECT-запроса, содержащего три подзапроса в списке SELECT.

```
use BSTU
go
select FACULTY.FACULTY   [факультет] ,
       (select count(*) from GROUPS
                        where GROUPS.FACULTY = FACULTY.FACULTY) [количество групп],
       (select count(*) from PROFESSION p
                        where p.FACULTY = FACULTY.FACULTY) [количество специальностей],
       (select count(*) from PROFESSION) [всего специальностей]
from FACULTY;
```

факультет	количество групп	количество специальностей	всего специальностей
ИДиП	11	3	15
ИЭФ	5	2	15
ЛХФ	5	3	15
ТОВ	4	3	15
ТТЛП	3	2	15
ХТиТ	4	2	15

Рис. 7.16. Применение подзапросов в списке SELECT

Внешний SELECT-запрос выбирает строки из таблицы **FACULTY**, три внутренних – из таблиц **GROUPS** и **PROFESSION**. Обратите внимание, что столбец с именем **FACULTY** присутствует во всех трех таблицах, поэтому для однозначности применяются двухкомпонентные имена столбцов в формате ТАБЛИЦА.СТОЛБЕЦ.

Подзапросы бывают двух видов: коррелируемые и независимые.

Коррелируемый подзапрос зависит от внешнего запроса и выполняется для каждой строки результирующего набора. На рис. 7.16 два первых подзапроса являются коррелируемыми. Обратите внимание: в логических выражениях секций WHERE (подробно рассматривается

позже) используется значение столбца **FACULTY** таблицы **FACULTY**, формируемое во внешнем запросе.

Независимый подзапрос не зависит от внешнего запроса и выполняется только один раз, но результат его выполнения подставляется в каждую строку результирующего набора. На рис. 7.16 третий подзапрос является независимым.

Важно понимать, что в SELECT-списке допускается применять только такие подзапросы, которые формируют скалярный результирующий набор (набор, состоящий из одной строки и одного столбца).

Отметим следующее.

Запись выражения для SELECT-запроса на рис. 7.16 можно сократить, если применить для имен таблиц псевдонимы.

Пример на рис. 7.16 носит исключительно учебное назначение. В реальности здесь не следовало бы применять коррелируемые запросы, а можно было использовать многотабличный запрос с JOIN-соединением таблиц **FACULTY**, **GROUPS** и **PROFESSION**.

Псевдонимы и многотабличные запросы рассматриваются позже при описании возможностей секции FROM оператора SELECT.

7.3. Секция FROM

Секция FROM содержит выражение, которое интерпретируется как набор однородных строк с явно поименованными столбцами.

В простейшем случае в секции FROM может быть указана таблица БД (рис. 7.17).

```
use BSTU
go
select * from [SUBJECT]
```

SUBJECT	SUBJECT_NAME	PULPIT
БД	Базы данных	ИСиТ
БЛЗиПсОО	Биология лесных зверей и птиц с осн. охотов.	ОВ
ВТЛ	Водный транспорт леса	ТЛ
ДМ	Дискретная математика	ИСиТ
ИГ	Инженерная геодезия	ЛУ
ИНФ	Информационные технологии	ИСиТ

Рис. 7.17. Источник строк – таблица

В качестве источника строк во FROM может выступать результат другого SELECT-запроса (7.18). Другими словами, в секции FROM можно выполнить подзапрос.

```
use BSTU
go
select * from (select [SUBJECT], PULPIT from [SUBJECT]) as T1
```

SUBJECT	PULPIT
БД	ИСиТ
БЛЗиПсОО	ОВ
ВТЛ	ТЛ
ДМ	ИСиТ
ИГ	ЛУ
ИНФ	ИСиТ
КГ	ИСиТ
КМС	ИСиТ

Рис. 7.18. Источник строк – SELECT-запрос

Если во FROM используется SELECT-запрос, то его результирующий набор, выступающий в качестве источника строк, должен быть поименован с помощью ключевого слова AS (может быть опущено), задающего псевдоним. На рис. 7.18 результирующий набор поименован символом **T1**. Обратите внимание, что SELECT-список внутреннего запроса, расположенного во FROM, содержит только два из трех столбцов таблицы **SUBJECT**, и имена этих столбцов унаследовал результирующий набор внешнего SELECT-запроса.

```
use BSTU
go
select  distinct [Кафедра] [Университетские кафедры]
from (select [SUBJECT] [Дисциплина], PULPIT [Кафедра] from [SUBJECT])  T1
```

Университетские кафедры
ИСиТ
ЛЗиДВ
ЛМиЛЗ
ЛПиСПС
ЛУ

Рис. 7.19. Применение псевдонимов столбцов в подзпросе

В подзапросе примера на рис. 7.19 для выбираемых столбцов указываются имена-псевдонимы. В этом случае внешний запрос «видит» только псевдонимы исходного набора. Обратите внимание, что окончательный результирующий набор имеет только один столбец с именем, заданным в псевдонимом SELECT-списка внешнего столбца.

Многотабличные запросы. Если секция FROM содержит выражение, включающее несколько таблиц, то говорят о многотабличных запросах. На рис. 7.20 представлен пример простейшего многотабличного запроса и фрагмент сформированного им результирующего набора.

```
use BSTU
select * from FACULTY, PULPIT
```

FACULTY	FACULTY_NAME	PULPIT	PULPIT_NAME	FACULTY
ИДиП	Издателькое дело и полиграфия	БФ	Белорусской филологии	ИДиП
ИДиП	Издателькое дело и полиграфия	ИСиТ	Информационных систем и технологий	ИДиП
ИДиП	Издателькое дело и полиграфия	ЛВ	Лесоводства	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛЗиДВ	Лесозащиты и древесиноведения	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛКиП	Лесных культур и почвоведения	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛМиЛЗ	Лесных машин и технологии лесозаготовок	ТТЛП
ИДиП	Издателькое дело и полиграфия	ЛПиСПС	Ландшафтного проектирования и садово-паркового с...	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛУ	Лесоустройства	ЛХФ

Рис. 7.20. Пример простейшего многотабличного запроса

Секция FROM содержит две таблицы: **FACLUTY** и **PULPIT**. Обратите внимание, что результирующий набор содержит два столбца с именем **FACULTY**. Это произошло из-за того, что таблицы имеют столбцы с одинаковыми именами. Поэтому, если в SELECT-списке потребуется явно указать столбец с именем **FACULTY**, то необходимо уточнить к какой именно таблице он относится (рис. 7.21).

```
use BSTU
select FACULTY.FACULTY, FACULTY_NAME, PULPIT.FACULTY from FACULTY, PULPIT
```

FACULTY	FACULTY_NAME	FACULTY
ИДиП	Издателькое дело и полиграфия	ИДиП
ИДиП	Издателькое дело и полиграфия	ИДиП
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ТТЛП
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ХТиТ

Рис. 7.21. Формирование результирующего набора с двумя столбцами, имеющими одинаковое имя

В SELECT-списке запроса (рис. 7.21) перечислены три столбца. Для двух столбцов с именем **FACULTY** явно указаны таблицы, к которым они относятся, а для **FACULTY_NAME** таблица не указана. Столбец **FACULTY_NAME** содержится только в таблице **FACULTY**, поэтому такая запись не вызывает двусмысленности. Этот же запрос можно записать с применением псевдонимов для используемых во FROM таблиц (7.22).

```
use BSTU
select f.FACULTY, FACULTY_NAME, p.FACULTY from FACULTY  as f, PULPIT p
```

FACULTY	FACULTY_NAME	FACULTY
ИДиП	Издателькое дело и полиграфия	ИДиП
ИДиП	Издателькое дело и полиграфия	ИДиП
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ТТЛП
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ЛХФ
ИДиП	Издателькое дело и полиграфия	ХТиТ

Рис. 7.22. Применение псевдонимов таблиц

Считается хорошей практикой при написании многотабличных запросов, во избежание путаницы, имя таблицы или псевдонима указывать для каждого столбца SELECT-списка.

Рассмотрим результирующий набор, сформированный запросом, представленным на рис. 7.22. В соответствии с SELECT-списком он состоит из трех столбцов: первые два относятся к таблице **FACULTY**, а третий – к **PULPIT**.

Результирующий набор создается по следующим правилам:

1) каждая строка таблицы **FACULTY** соединяется со всеми строками таблицы **PULPIT**, образуя промежуточный набор однородных строк, имеющий суммарное количество столбцов и количество строк, равное произведению количеств строк таблиц **FACULTY** и **PULPIT** (рис. 7.23);

2) из всех столбцов промежуточного набора «вырезаются» столбцы, указанные в списке SELECT.

```
use BSTU
select   (select count(*)  from FACULTY) [количество строк в FACULTY],
         (select count(*)  from PULPIT)  [количество строк в PULPIT],
         (select count(*)  from FACULTY, PULPIT)  [количество строк в FACULTY,PULPIT]
```

количество строк в FACULTY	количество строк в PULPIT	количество строк в FACULTY,PULPIT
6	26	156

Рис. 7.23. Количество строк в результирующем наборе запроса, представленного на рис. 7.22

Если в секции FROM перечислено через запятую более двух таблиц, то правило формирования результирующего набора применяется слева направо для всех таблиц. Сначала строится результирующий набор для первой пары таблиц, потом он соединяется с третьей таблицей, образуя очередной набор, и т. д. В конечном наборе количество столбцов равно суммарному количеству столбцов всех таблиц, а количество строк – произведению числа их строк.

Применение псевдонимов таблиц дает возможность дважды указать одну и ту же таблицу в секции FROM (рис. 2.24).

```
use BSTU
go
select * from FACULTY f1, FACULTY f2
```

FACULTY	FACULTY_NAME	FACULTY	FACULTY_NAME
ИДиП	Издателькое дело и полиграфия	ИДиП	Издателькое дело и полиграфия
ИЭФ	Инженерно-экономический факультет	ИДиП	Издателькое дело и полиграфия
ЛХФ	Лесохозяйственный факультет	ИДиП	Издателькое дело и полиграфия
ТОВ	Технология органических веществ	ИДиП	Издателькое дело и полиграфия
ТТЛП	Технология и техника лесной промышленности	ИДиП	Издателькое дело и полиграфия
ХТиТ	Химическая технология и техника	ИДиП	Издателькое дело и полиграфия

Рис. 7.24. Таблица FACULTY дважды указана в секции FROM

В многотабличных запросах в секции FROM можно использовать подзапросы (рис. 7.25).

```
use BSTU
go
select * from (select [SUBJECT] from [SUBJECT]) as T1,
              (select [PULPIT] from [PULPIT])  as T2
```

SUBJECT	PULPIT
БД	БФ
БПЗиПсОО	БФ
ВТЛ	БФ

Рис. 7.25. Применение двух подзапросов в секции FROM

Соединение таблиц CROSS JOIN. В примерах на рис. 7.20–7.22, 7.24, 7.25 выполнялась операция, которая в теории БД называется соединением таблиц. Действительно, в результате этих операций получался набор, строки которого представляют собой соединение строк двух или более таблиц. Способ соединения, рассмотренный в этих примерах, наиболее простой: каждая строка одной таблицы соединяется с каждой строкой другой таблицы. Такое соединение называют декартовым произведением таблиц.

Для выполнения декартова произведения двух или более таблиц следует просто перечислить их через запятую в секции FROM. Существует другой способ записи этой же операции с помощью выражения CROSS JOIN.

На рис. 7.26 приведены примеры записи декартова произведения таблиц с применением CROSS JOIN.

```
use BSTU
go
select *                                                    -- аналог рис.7.20
from FACULTY cross join  PULPIT;
select PULPIT.FACULTY, FACULTY_NAME,PULPIT.FACULTY          -- аналог рис.7.21
from FACULTY cross join PULPIT;
select f.FACULTY, FACULTY_NAME, p.FACULTY                   -- аналог рис.7.22
from FACULTY  as f cross join PULPIT p;
select *
from FACULTY f1 cross join  FACULTY f;                      -- аналог рис.7.24
select *
from (select [SUBJECT] from [SUBJECT]) as T1 cross join -- аналог рис.7.25
     (select [PULPIT]  from [PULPIT])  as T2;
select *
from [SUBJECT] cross join [PULPIT] cross join FACULTY    -- соединение 3х
```

Рис. 7.26. Примеры применения выражения CROSS JOIN

Соединение таблиц INNER JOIN. Соединение таблиц INNER JOIN (внутреннее соединение) – наиболее часто используемый вид соединения реляционных таблиц.

На рис. 7.27 приведен пример SELECT-запроса с использованием внутреннего соединения таблиц **FACULTY** и **PULPIT** и фрагмент результирующего набора.

```
use BSTU
go
select FACULTY.FACULTY, PULPIT.PULPIT
from FACULTY  inner join  PULPIT
              on FACULTY.FACULTY = PULPIT.FACULTY
```

FACULTY	PULPIT
ИДиП	БФ
ИДиП	ИСиТ
ЛХФ	ЛВ
ЛХФ	ЛЗиДВ
ЛХФ	ЛКиП
ТТЛП	ЛМиЛЗ
ЛХФ	ЛПиСПС

Рис. 7.27. Пример внутреннего соединения двух таблиц

Обратите внимание, что выражение, описывающее внутреннее соединение, состоит из двух частей: собственно выражение INNER JOIN, указываемое между двумя соединяемыми таблицами; логическое выражение, записываемое после ключевого слова ON.

Результирующий набор SELECT-запроса на рис. 7.27 создается по следующим правилам:

1) выполняется декартово произведение для таблиц **FACULTY** и **PULPIT**;

2) из результирующего набора, полученного на предыдущем шаге, выбираются строки, удовлетворяющие условию логического выражения, указанного после ON;

3) из всех столбцов результирующего набора «вырезаются» столбцы, указанные в списке SELECT.

Аналогичный результат может быть получен, если выполнить следующий SELECT-запрос (рис. 7.28). Проанализировав этот запрос, не сложно понять, что он в точности повторяет правила внутреннего соединения таблиц, описанные выше. Основное отличие в том, что отбор строк в этих запросах выполняется на разных этапах: в первом случае при формировании набора строк в секции FROM, во втором – при фильтрации строк в секции WHERE (подробно об этой секции позже).

```
use BSTU
go
select FACULTY.FACULTY, PULPIT.PULPIT
from FACULTY  cross join  PULPIT
where FACULTY.FACULTY = PULPIT.FACULTY
```

Рис. 7.28. Запрос, формирующий результат, аналогичный результату запроса на рис. 7.27

На рис. 7.29 приведен запрос, вычисляющий с помощью встроенной функции COUNT (рассматривается позже) количество строк в результирующих наборах, сформированных шестью запросами.

```
use BSTU
go
select  (select count(*)
         from FACULTY ) [FACULTY],
        (select count(*)
         from PULPIT )  [PULPIT],
        (select count(*)
         from FACULTY  cross join  PULPIT) [cross join],
        (select count(*)
         from FACULTY  cross join  PULPIT
         where FACULTY.FACULTY = PULPIT.FACULTY) [cross join, where],
        (select count(*)
         from FACULTY  inner join  PULPIT
         on FACULTY.FACULTY = PULPIT.FACULTY)  [inner join1],
        (select count(*)
         from PULPIT inner join FACULTY
         on PULPIT.FACULTY = FACULTY.FACULTY)  [inner join2]
```

FACULTY	PULPIT	cross join	cross join, where	inner join1	inner join2
6	26	156	26	26	26

Рис. 7.29. Количество строк в результирующих наборах, формируемых различными запросами

Обратите внимание на следующее: количество строк, формируемое при внутреннем соединении, не превышает количества строк декартова произведения таблиц; запросы на рис. 7.27 и 7.28 формируют одинаковое число строк; операция INNER JOIN является коммутативной – формируемый результирующий набор не зависит от порядка, в котором указаны таблицы.

Внутреннее соединение можно выполнить для двух и более таблиц. В примере на рис. 7.30 соединяются три таблицы: **FACULTY**, **PULPIT** и **SUBJECT**. Обратите внимание на использование псевдонимов таблиц.

```
use BSTU
go
select f.FACULTY, p.PULPIT, s.[SUBJECT]
from FACULTY  f inner join PULPIT    p on f.FACULTY = p.FACULTY
               inner join [SUBJECT] s on p.PULPIT = s.PULPIT;
```

FACULTY	PULPIT	SUBJECT
ИДиП	ИСиТ	БД
ЛХФ	ОВ	БПЗиПсОО
ТТЛП	ТЛ	ВТЛ
ИДиП	ИСиТ	ДМ
ЛХФ	ЛУ	ИГ
ИДиП	ИСиТ	ИНФ
ИДиП	ИСиТ	КГ
ИДиП	ИСиТ	КМС
ЛХФ	ПЗиДВ	ЛВ

Рис. 7.30. Внутреннее соединение трех таблиц

Допускается применение подзапросов. На рис. 7.31 представлен пример использование подзапроса в конструкции INNER JOIN.

```
use BSTU
go
select f.FACULTY, pp.PULPIT
from FACULTY  f inner join (
                            select PULPIT, FACULTY from PULPIT p
                            where not exists (
                                               select * from [SUBJECT] s
                                               where s.PULPIT = p.PULPIT
                                              )
                           )pp
                on f.FACULTY = pp.FACULTY;
```

Рис. 7.31. Применение подзапросов в конструкции INNER JOIN

Обратите внимание на то, что подзапрос, используемый в INNER JOIN, сам содержит подзапрос в секции WHERE. Такая иерархия подзапросов является типичной. Принцип применения подзапросов в секции WHERE будет рассматриваться позже.

Логическое условие, указываемое после ключевого слова ON, не обязательно должно содержать условие, записываемое в виде равенства – это может любое логическое выражение. Пример на рис. 7.32 демонстрирует применение условия в виде неравенства.

```
use BSTU
go
select p.PULPIT, s.[SUBJECT]
from PULPIT p inner join [SUBJECT] s on p.PULPIT != s.PULPIT;
```

PULPIT	SUBJECT
БФ	БД
БФ	БПЗиПсОО
БФ	ВТЛ
БФ	ДМ
БФ	ИГ
БФ	ИНФ
БФ	КГ
БФ	КМС

Рис. 7.32. Логическое условие после ON в виде неравенства

С помощью INNER JOIN можно соединять таблицу саму с собой. Обычно такие запросы применяются для получения информации об иерархических связях между строками одной таблицы. На рис. 7.33 создается и заполняется таблица с именем **[Руководители и Исполнители]**, с помощью которой демонстрируется внутреннее соединение таблицы с собой.

```
use TEMPDB
go
create table [Руководители и Исполнители]
(
 [Исполнитель]  varchar(10) primary key,
 [Фамилия]      varchar(50),
 [Руководитель] varchar(10)
                foreign key references [Руководители и Исполнители]([Исполнитель])
);
go
insert [Руководители и Исполнители]
       ([Исполнитель], [Фамилия],        [Руководитель] )
values ('ЖРС',         'Жарский И.М.',   NULL),
       ('УРБ',         'Урбанович П.П.', 'ЖРС'),
       ('СМЛ',         'Смелов В.В',     'УРБ'),
       ('ПЦЙ',         'Пацей Н.В.',     'УРБ'),
       ('РМН',         'Романенко Д.М',  'УРБ'),
       ('ШМК',         'Шмаков М.С.',    'ЖРС'),
       ('ЮДН',         'Юденков В.С.',   'ШМК'),
       ('АРС',         'Арсентьев В.А.', 'ШМК');
```

Рис. 7.33. Создание таблицы **[Руководители и Исполнители]**

Обратите внимание: в таблице **[Руководители и Исполнители]** установлена иерархическая связь между строками. Каждая строка описывает члена какой-то организации, в которой устанавливается отношение руководитель – исполнитель. При этом руководитель, может выступать исполнителем для руководителя более высокого уровня. Исполнитель, для которого нет руководителя (в столбце [Руководитель] значение NULL), является руководителем самого высокого уровня.

Кроме того, следует обратить внимание на ограничение FOREIGN KEY, указанное для таблицы **[Руководители и Исполнители]** – оно ссылается на эту же таблицу.

На рис. 7.34 демонстрируется запрос, использующий внутреннее соединение с собой таблицы **[Руководители и Исполнители]**.

```
use TEMPDB
go
select t1.[Фамилия] [Руководитель], t2.[Фамилия] [Исполнитель]
from  [Руководители и Исполнители] t1
      inner join
      [Руководители и Исполнители] t2
      on t2.[Руководитель] = t1.[Исполнитель]
order by [Руководитель], [Исполнитель]
```

Руководитель	Исполнитель
Жарский И.М.	Урбанович П.П.
Жарский И.М.	Шмаков М.С.
Урбанович П...	Пацей Н.В.
Урбанович П...	Романенко Д.М
Урбанович П...	Смелов В.В
Шмаков М.С.	Арсентьев В.А.
Шмаков М.С.	Юденков В.С.

Рис. 7.34. Внутреннее соединение с собой таблицы
[Руководители и Исполнители]

Соединение таблиц OUTER JOIN. Еще раз рассмотрим пример внутреннего соединения таблиц **PULPIT** и **TEACHER**. На рис. 7.35 представлены два SELECT-запроса с внутренним соединением этих таблиц. В первом запросе подсчитывается количество строк в результирующем наборе, во втором – формируется сам результирующий набор. Обратим внимание: результирующий набор содержит 13 строк.

Отметим, что в результирующий набор попали те строки из **PULPIT**, для которых в таблице **TEACHER** нашлись строки, в которых значение в столбце **FACULTY** совпадает со значением в одноименном столбце таблицы **TEACHER**.

```
use BSTU
go
select count(*) [количество строк]
from PULPIT p inner join TEACHER t on p.PULPIT = t.PULPIT

select p.PULPIT_NAME, t.TEACHER_NAME
from PULPIT p inner join TEACHER t on p.PULPIT = t.PULPIT
```

	количество строк
1	13

	PULPIT_NAME	TEACHER_NAME
1	Информационных систем и технологий	Акунович Станислав Иванович
2	Полиграфического оборудования и систем обработки...	Арсентьев Виталий Арсентьевич
3	Органической химии	Безбородов Владимир Степанович
4	Полиграфического оборудования и систем обработки...	Беляев Валерий Павлович
5	Информационных систем и технологий	Бракович Андрей Игоревич
6	Полиграфического оборудования и систем обработки...	Барташевич Святослав Алексан...

Рис. 7.35. Внутреннее соединение таблиц **PULPIT** и **TEACHER**

Запрос, представленный на рис. 7.36, формирует результирующий набор из таблицы **PULPIT** со строками, имеющими такие значения в столбце **FACULTY**, каких нет в одноименном столбце ни в одной из строк таблицы **TEACHER**. Другими словами, этот запрос формирует набор строк из **PULPIT**, которые не соединились ни с одной строкой из таблицы в запросе на рис. 7.35. Обратим внимание: количество не соединившихся строк таблицы **PULPIT** равно 22.

Внешнее соединение двух таблиц формирует набор строк, состоящий из двух частей:

1) результат внутреннего соединения двух таблиц;

2) строки из двух таблиц, которые не смогли соединиться; если в SELECT-списке указаны столбцы двух таблиц, то значения в столбцах, соответствующих незаполненной (несоединенной) части строки, будут NULL.

Имеется два вида внешнего соединения: LEFT OUTER JOIN – левое внешнее соединение и RIGHT OUTER JOIN – правое внешнее соединение.

Левое внешнее соединение, в соответствии со вторым пунктом правила, включает в набор несоединенные строки таблицы, имя которой записано слева от ключевых слов LEFT OUTER JOIN, а правое внешнее соединение – несоединенные строки таблицы, имя которой записано справа от RIGHT OUTER JOIN.

```
use BSTU
go
select count(*) [количество строк] from PULPIT p
where not exists (select * from TEACHER t where t.PULPIT = p.PULPIT);
select p.PULPIT_NAME from PULPIT p
where not exists (select * from TEACHER t where t.PULPIT = p.PULPIT);
```

	количество строк
1	22

	PULPIT_NAME
1	Белорусской филологии
2	Лесоводства
3	Лесных культур и почвоведения
4	Лесных машин и технологии лесозаготовок
5	Ландшафтного проектирования и садово-паркового с...
6	Лесоустройства
7	Машин и аппаратов химических и силикатных произво...
8	Менеджмента и экономики природопользования
9	Охотоведения
10	Процессов и аппаратов химических производств
11	Полиграфических производств
12	Редакционно-издательских тенологий
13	Статистики, бухгалтерского учета, анализа и аудита
14	Технологий деревообрабатывающих производств
15	Технологии и дизайна изделий из древесины
16	Туризма и природопользования
17	Транспорта леса
18	Технологии неорганических веществ и общей химиче...
19	Технологии нефтехимического синтеза и переработки...
20	Химической переработки древесины
21	Химии, технологии электрохимических производств и ...
22	Экономической теории и маркетинга

Рис. 7.36. Несоединенные строки таблицы **PULPIT** в запросе на рис. 7.35

На рис. 7.37 представлены три SELECT-запроса, содержащие левое внешнее соединение таблиц **PULPIT** и **TEACHER**: в первом запросе вычисляется количество строк в результирующем наборе; во втором запросе в результирующий набор выводятся строки, содержащие значение NULL в столбце **TEACHER_NAME**; в третьем запросе в результирующий набор выводятся строки, содержащие значение не NULL в столбце **TEACHER_NAME.**

```
use BSTU
go
select count(*) [количество строк]
from PULPIT p left outer join TEACHER t on p.PULPIT = t.PULPIT;

select p.PULPIT_NAME, t.TEACHER_NAME
from PULPIT p left outer join TEACHER t on p.PULPIT = t.PULPIT
where t.TEACHER_NAME is not null
order by p.PULPIT;

select p.PULPIT_NAME, t.TEACHER_NAME
from PULPIT p left  outer join TEACHER t on p.PULPIT = t.PULPIT
where t.TEACHER_NAME is null
order by p.PULPIT;
```

Рис. 7.37. SELECT-запросы, содержащие левое внешнее соединение двух таблиц: **PULPIT** и **TEACHER**

На рис. 7.38–7.40 продемонстрированы результирующие наборы, сформированные тремя запросами, представленными на рис. 7.37.

количество строк
35

Рис. 7.38. Результат выполнения первого SELECT-запроса на рис. 7.37

	PULPIT_NAME	TEACHER_NAME
1	Информационных систем и технологий	Акунович Станислав Иванович
2	Информационных систем и технологий	Бракович Андрей Игоревич
3	Информационных систем и технологий	Герман Олег Витольдович
4	Информационных систем и технологий	Дятко Александр Аркадьевич
5	Информационных систем и технологий	Колесников Виталий Леонидович
6	Информационных систем и технологий	Лащенко Анатолий Павлович
7	Лесозащиты и древесиноведения	Звягинцев Вячеслав Борисович
8	Органической химии	Безбородов Владимир Степано...
9	Полиграфического оборудования и с...	Беляев Валерий Павлович
10	Полиграфического оборудования и с...	Арсентьев Виталий Арсентьевич
11	Полиграфического оборудования и с...	Барташевич Святослав Алекса...
12	Полиграфического оборудования и с...	Шмаков Михаил Сергеевич
13	Полиграфического оборудования и с...	Юденков Виктор Степанович

Рис. 7.39. Результат выполнения второго SELECT-запроса на рис. 7.37

	PULPIT_NAME	TEACHER_NAME
1	Белорусской филологии	NULL
2	Лесоводства	NULL
3	Лесных культур и почв...	NULL
4	Лесных машин и техно...	NULL
5	Ландшафтного проект...	NULL
6	Лесоустройства	NULL
7	Машин и аппаратов хи...	NULL
8	Менеджмента и эконо...	NULL
9	Охотоведения	NULL
10	Процессов и аппарато...	NULL
11	Полиграфических про...	NULL
12	Редакционно-издатель...	NULL
13	Статистики, бухгалтер...	NULL
14	Технологий деревообр...	NULL
15	Технологии и дизайна ...	NULL
16	Туризма и природопол...	NULL
17	Транспорта леса	NULL
18	Технологии неорганич...	NULL
19	Технологии нефтехим...	NULL
20	Химической перераб...	NULL
21	Химии, технологии эле...	NULL
22	Экономической теори...	NULL

Рис. 7.40. Результат выполнения третьего SELECT-запроса на рис. 7.37

Необходимо обратить внимание на следующее: на рис. 7.39 результирующий набор содержит те же строки, что и результирующий набор на рис. 7.35; на рис. 7.40 результирующий набор содержит те же строки, что и результирующий набор на рис. 7.36; если у второго или третьего SELECT-запроса на рис. 7.37 убрать секцию WHERE, то результирующий набор такого запроса будет содержать 35 строк (рис. 7.38), являющихся объединением двух результирующих наборов, представленных на рис. 7.39 и 7.40; столбец **TEACHER_NAME** принимает значение NULL в строках результирующего набора, соответствующих несоединенным строкам (рис. 7.40).

Заметим, что, в общем случае, соединение OUTER JOIN не коммутативно. На рис. 7.41 представлен SELECT-запрос, в котором используется левое внешнее соединение между таблицами **TEACHER** и **PULPIT**. Обратите внимание: таблицы при соединении перечислены в обратном порядке, нежели в запросах на рис. 7.37; количество строк в результирующем наборе равно 13 (против 35 на рис. 7.37). Таким образом, внешнее соединение на рис. 7.41 соединяет все 13 строк (рис. 7.42) таблицы **TEACHER** с 13 строками таблицы **PULPIT**. Такое совпадение обусловлено еще и тем, что у таблицы **TEACHER** существует ограничение FOREIGN KEY, ссылающееся на таблицу **PULPIT** и поэтому, в принципе, нельзя добавить в таблицу **TEACHER** значения, которые не свяжутся по заданному логическому выражению в ON с таблицей **PULPIT**.

```
use BSTU
go
select p.PULPIT_NAME, t.TEACHER_NAME
from  TEACHER t left  outer join PULPIT p on p.PULPIT = t.PULPIT;
```

	PULPIT_NAME	TEACHER_NAME
1	Информационных систем и технологий	Акунович Станислав Иванович
2	Полиграфического оборудования и систем обработки...	Арсентьев Виталий Арсентьевич
3	Органической химии	Безбородов Владимир Степанович
4	Полиграфического оборудования и систем обработки...	Беляев Валерий Павлович
5	Информационных систем и технологий	Бракович Андрей Игоревич
6	Полиграфического оборудования и систем обработки...	Барташевич Святослав Александр
7	Информационных систем и технологий	Герман Олег Витольдович
8	Информационных систем и технологий	Дятко Александр Аркадьевич
9	Лесозащиты и древесиноведения	Звягинцев Вячеслав Борисович
10	Информационных систем и технологий	Колесников Виталий Леонидович
11	Информационных систем и технологий	Лащенко Анатолий Павлович
12	Полиграфического оборудования и систем обработки...	Шмаков Михаил Сергеевич
13	Полиграфического оборудования и систем обработки...	Юденков Виктор Степанович

Рис. 7.41. Левое внешнее соединение с обратным порядком таблиц по отношению к соединению в запросах на рис. 7.37

```
use BSTU
go
select count(*) [количество строк в TEACHER] from TEACHER
exec SP_HELP 'TEACHER'
```

количество строк в TEACHER
13

constraint_type	constraint_name	constraint_keys
DEFAULT on column GENDER	DF__TEACHER__GENDER__2AA05119	('м')
CHECK on column GENDER	TEACHER_GENDER_CH	([GENDER]='ж' OR [GENDER]='м')
PRIMARY KEY (clustered)	TEACHER_PK	TEACHER
FOREIGN KEY	TEACHER_PULPIT_FK	PULPIT
		REFERENCES BSTU.dbo.PULPIT (PULPIT)

Рис. 7.42. Количество строк в таблице **TEACHER** (равно 13), ограничение FOREIGN KEY таблицы **TEACHER**

Рассмотрим правое внешнее соединение между таблицами **PULPIT** и **TEACHER**. На рис. 7.43 представлен пример использования соединения RIGHT OUTER JOIN между таблицами **PULPIT** и **TEACHER**.

```
use BSTU
go
select p.PULPIT_NAME, t.TEACHER_NAME
from  PULPIT p  right  outer join TEACHER t on p.PULPIT = t.PULPIT;
```

	PULPIT_NAME	TEACHER_NAME
1	Информационных систем и технологий	Акунович Станислав Иванович
2	Полиграфического оборудования и систем обработки...	Арсентьев Виталий Арсентьевич
3	Органической химии	Безбородов Владимир Степанович
4	Полиграфического оборудования и систем обработки...	Беляев Валерий Павлович
5	Информационных систем и технологий	Бракович Андрей Игоревич
6	Полиграфического оборудования и систем обработки...	Барташевич Святослав Александрович
7	Информационных систем и технологий	Герман Олег Витольдович
8	Информационных систем и технологий	Дятко Александр Аркадьевич
9	Лесозащиты и древесиноведения	Звягинцев Вячеслав Борисович
10	Информационных систем и технологий	Колесников Виталий Леонидович
11	Информационных систем и технологий	Лащенко Анатолий Павлович
12	Полиграфического оборудования и систем обработки...	Шмаков Михаил Сергеевич
13	Полиграфического оборудования и систем обработки...	Юденков Виктор Степанович

Рис. 7.43. Пример правого внешнего соединения таблицы **PULPIT** и **TEACHER**

Обратите внимание: результат запроса на рис. 7.43 совпадает с результатом запроса на рис. 7.41.

Возьмем за основу сценарий на рис. 7.37, поменяем порядок таблиц, изменим вид внешнего соединения LEFT на RIGHT и получим сценарий, представленный на рис. 7.44.

```
use BSTU
go
select count(*) [количество строк]
from  TEACHER t right outer join PULPIT p on p.PULPIT = t.PULPIT;

select p.PULPIT_NAME, t.TEACHER_NAME
from TEACHER t  right outer join PULPIT p  on p.PULPIT = t.PULPIT
where  t.TEACHER_NAME is not null
order by p.PULPIT;

select p.PULPIT_NAME, t.TEACHER_NAME
from TEACHER t right  outer join PULPIT p on p.PULPIT = t.PULPIT
where  t.TEACHER_NAME is null
order by p.PULPIT;
```

Рис. 7.44. Модифицированный сценарий на рис. 7.37: изменен порядок таблиц в соединении и вид внешнего соединения LEFT на RIGHT

В результате выполнения сценария на рис. 7.44 формируется три результирующих набора (рис. 7.45–7.47).

количество строк
35

Рис. 7.45. Результат выполнения первого SELECT-запроса на рис. 7.44

	PULPIT_NAME	TEACHER_NAME
1	Информационных систем и технологий	Акунович Станислав Иванович
2	Информационных систем и технологий	Бракович Андрей Игоревич
3	Информационных систем и технологий	Герман Олег Витольдович
4	Информационных систем и технологий	Дятко Александр Аркадьевич
5	Информационных систем и технологий	Колесников Виталий Леонидович
6	Информационных систем и технологий	Лащенко Анатолий Павлович
7	Лесозащиты и древесиноведения	Звягинцев Вячеслав Борисович
8	Органической химии	Безбородов Владимир Степано...
9	Полиграфического оборудования и с...	Беляев Валерий Павлович
10	Полиграфического оборудования и с...	Арсентьев Виталий Арсентьевич
11	Полиграфического оборудования и с...	Барташевич Святослав Алекса...
12	Полиграфического оборудования и с...	Шмаков Михаил Сергеевич
13	Полиграфического оборудования и с...	Юденков Виктор Степанович

Рис. 7.46. Результат выполнения второго SELECT-запроса на рис. 7.44

Не сложно заметить, что результаты, полученные после выполнения сценариев на рис. 7.37 и 7.44., совпадают. Проведенные исследования позволяют сформулировать следующее правило. Если в SELECT-запросе с внешним соединением двух таблиц:

1) поменять порядок соединения этих таблиц;
2) изменить вид внешнего соединения на альтернативный, то получим SELECT-запрос, формирующий результирующий набор, совпадающий с результирующим набором исходного SELECT-запроса.

	PULPIT_NAME	TEACHER_NAME
1	Белорусской филологии	NULL
2	Лесоводства	NULL
3	Лесных культур и почв...	NULL
4	Лесных машин и техно...	NULL
5	Ландшафтного проект...	NULL
6	Лесоустройства	NULL
7	Машин и аппаратов хи...	NULL
8	Менеджмента и эконо...	NULL
9	Охотоведения	NULL
10	Процессов и аппарато...	NULL
11	Полиграфических про...	NULL
12	Редакционно-издатель...	NULL
13	Статистики, бухгалтер...	NULL
14	Технологий деревообр...	NULL
15	Технологии и дизайна ...	NULL
16	Туризма и природопол...	NULL
17	Транспорта леса	NULL
18	Технологии неорганич...	NULL
19	Технологии нефтехим...	NULL
20	Химической перерабо...	NULL
21	Химии, технологии эле...	NULL
22	Экономической теори...	NULL

Рис. 7.47. Результат выполнения третьего SELECT-запроса на рис. 7.44

Как и прежде, вместо таблицы допускается использование подзапроса. На рис. 7.48 представлен запрос с применением соединения трех таблиц **FACULTY**, **PULPIT** и **TEACHER**.

```
use BSTU
go
select f.FACULTY_NAME, p.PULPIT_NAME, t.TEACHER_NAME
from FACULTY f join PULPIT p on f.FACULTY = p.FACULTY right  outer join
     (select TEACHER_NAME, PULPIT from TEACHER  where TEACHER_NAME like 'A%' ) t
      on p.PULPIT = t.PULPIT
```

FACULTY_NAME	PULPIT_NAME	TEACHER_NAME
Издательское дело и полиграфия	Информационных систем и технологий	Акунович Станислав Иванович
Издательское дело и полиграфия	Полиграфического оборудования и систем ...	Арсентьев Виталий Арсентьевич

Рис. 7.48. SELECT-запрос, использующий внутреннее соединение
и внешнее соединение с подзапросом

Между **FACULTY** и **PULPUIT** установлено внутреннее соединение, результат которого с помощью правого внешнего соединения связан с результатом SELECT-запроса над таблицей **TEACHER** (выбираются строки, у которых значение столбца **TEACHER_NAME** начинается с символа **A**).

7.4. Секция WHERE

Исходным набором строк для секции WHERE является результирующий набор секции FROM. Секция WHERE предназначена для фильтрации строки, содержит логическое выражение, вычисляемое для каждой строки исходного набора. Если значение логического выражения принимает значение «истина», то строка помещается в результирующий набор секции WHERE. На рис. 7.48 представлен SELECT-запрос, выполняющий пять подзапросов, применяющих секцию WHERE для отбора (фильтрации) строк из таблицы **AUDITORIUM** (в данном случае результирующий набор секции FROM состоит из всех строк таблицы **AUDITORIUM**).

```
use BSTU
go
select
(select count(*) from AUDITORIUM) 'всего',
(select count(*) from AUDITORIUM
                 where AUDITORIUM_CAPACITY > 15) 'больше 15',
(select count(*) from AUDITORIUM
                where AUDITORIUM_CAPACITY > 50)  'больше 50',
(select count(*) from AUDITORIUM
                where AUDITORIUM_CAPACITY > 60)  'больше 60',
(select count(*) from AUDITORIUM
                where AUDITORIUM_CAPACITY > 90)  'больше 90'
```

всего	больше 15	больше 50	больше 60	больше 90
23	20	17	14	0

Рис. 7.49. Вычисление количества строк в результирующих
наборах подзапросов

Все подзапросы на рис. 7.49 в секции WHERE применяют логическое выражение, сравнивающее значение в столбце **AUDITORIUM_CAPACITY** с заданной величиной. Функция **COUNT** (более подробно рассматривается позже) подсчитывает количество строк, которое

сформировал бы SELECT-запрос (но в данных запросах выводит только одну строку), если бы не применялась агрегатная функция. Обратите внимание: последний запрос сформировал бы пустой результирующий набор (количество строк равно нулю).

Запрос на рис. 7.50 формирует и выводит результирующий набор, состоящий из 14 строк. Сравните с количеством строк, вычисленным в четвертом подзапросе на рис. 7.49.

```
use BSTU
go
select * from AUDITORIUM where AUDITORIUM_CAPACITY > 60
```

	AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
1	02Б-4	ЛК	90	02Б-4
2	103-4	ЛК	90	103-4
3	105-4	ЛК	90	105-4
4	107-4	ЛК	90	107-4
5	114-4	ЛК-К	90	114-4
6	132-4	ЛК	90	132-4
7	229-4	ЛК	90	229-4
8	304-4	ЛБ-К	90	304-4
9	305-4	ЛК-К	90	305-5
10	314-4	ЛК	90	314-4
11	320-4	ЛК	90	320-4
12	408-2	ЛК	90	408-2
13	423-1	ЛБ-К	90	423-1
14	429-4	ЛК	90	429-4

Рис. 7.50. Простейший SELECT-запрос, формирующий 14 строк в результирующий набор секции WHERE

7.4.1. Операции сравнения

Логические выражения, применяемые в секции WHERE, как правило, основываются на операциях сравнения (табл. 7.1).

Таблица 7.1

Операции сравнения

Символ	Операция сравнения
=	Равенство
<>	Неравенство
!=	Неравенство
>	Больше
>=	Больше или равно
<	Меньше
<=	Меньше или равно

Операции сравнения применимы ко всем основным типам данных: числовым, денежным, символьным, датам и времени.

На рис. 7.51–7.53 представлены примеры использования операций сравнения в секции WHERE.

```
use BSTU
go
select * from PROGRESS where NOTE <= 4
```

SUBJECT	IDSTUDENT	PDATE	NOTE
ОАиП	1005	2013-01-10	4
ОАиП	1006	2013-01-10	4
БД	1008	2013-06-12	4
СУБД	1017	2013-01-18	4
КГ	1018	2013-06-15	4
ОХ	1065	2013-01-19	4
ОХ	1069	2013-01-19	4
ЭТ	1158	2013-01-15	4

Рис. 7.51. Применение в секции WHERE операции «меньше или равно» для сравнения числовых значений

Применение операций сравнения для чисел не вызывает сложности понимания, т. к. сравнение осуществляется естественным для чисел способом. Аналогично сравниваются денежные данные.

```
use BSTU
go
select * from  TEACHER where TEACHER != 'БРКВЧ'
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
АКНВЧ	Акунович Станислав Иванович	м	ИСиТ
АРС	Арсентьев Виталий Арсентьевич	м	ПОиСОИ
БЗБРДВ	Безбородов Владимир Степанович	м	ОХ
БЛВ	Беляев Валерий Павлович	м	ПОиСОИ
БРТШВЧ	Барташевич Святослав Александрович	м	ПОиСОИ
ГРМН	Герман Олег Витольдович	м	ИСиТ
ДТК	Дятко Александр Аркадьевич	м	ИСиТ
ЗВГЦВ	Звягинцев Вячеслав Борисович	м	ЛЗиДВ
КЛСНВ	Колесников Виталий Леонидович	м	ИСиТ
ЛЩНК	Лащенко Анатолий Павлович	м	ИСиТ
ШМК	Шмаков Михаил Сергеевич	м	ПОиСОИ
ЮДНКВ	Юденков Виктор Степанович	м	ПОиСОИ

Рис. 7.52. Применение в секции WHERE операции «не равно» для сравнения строковых значений

Сравнение строк осуществляется общепринятым в программировании способом: если сравниваются две строки, состоящие из одинакового количества символов, то сравниваются два числа, образованные кодами символов, входящими в эти строки; если строки состоят из разного количества символов, то короткая строка дополняется необходимым количеством пробелов справа и сравнение сводится к первому варианту. Следует также отметить, что на сравнение строк влияет параметр **COLLATION**, значение которого может быть установлено на уровне БД, столбца таблицы или операции сравнения. Подробные сведения об этом системном параметре можно получить в пособии [5].

```
use BSTU
go
select * from PROGRESS where PDATE >  '20130601'
```

SUBJECT	IDSTUDENT	PDATE	NOTE
БД	1008	2013-06-12	4
БД	1009	2013-06-12	9
БД	1010	2013-06-12	5
БД	1011	2013-06-12	7
БД	1012	2013-06-12	5
КГ	1018	2013-06-15	4
КГ	1019	2013-06-15	7
КГ	1020	2013-06-15	7
КГ	1021	2013-06-15	9
КГ	1022	2013-06-15	5
КГ	1023	2013-06-15	6

Рис. 7.53. Применение в секции WHERE операции «больше» для сравнения даты

При сравнении двух дат большей считается та, которая хронологически на оси времени расположена правее. Аналогично сравнивается время. Если в операции сравнения дат или времени необходимо применить литералы, то применять их следует в соответствии с правилами, описанными в п. 5.2.1.3, или применять явное преобразование к соответствующему типу с помощью функций **CAST** или **CONVERT**.

7.4.2. Сравнение строк

Кроме общепринятых операций (табл. 7.1) для сравнения строк применяются предикаты: LIKE, CONTAINS и FREETEXT. Предикатом называется выражение, значениями которого могут быть только «истина» и «ложь».

Предикаты CONTAINS и FREETEXT чаще применяются при полнотекстовом поиске, поэтому здесь они не рассматриваются. Для ознакомления с их возможностями рекомендуется воспользоваться пособием [5].

Предикат LIKE позволяет осуществлять сравнение одной строки с множеством строк, заданным с помощью шаблона. Шаблон – это строка, которая с помощью специальных символов позволяет описать множество строк. При этом LIKE принимает значение «истина» в том случае, если сравниваемая строка будет найдена во множестве строк, описанном с помощью шаблона. Или говорят иначе – если строка будет соответствовать заданному шаблону.

Приведем несколько типичных примеров использования предиката LIKE. Запрос на рис. 7.54 демонстрирует содержимое столбца **QUALIFICATION** таблицы **PROFESSION**, а запросы на рис. 7.55–7.58 демонстрируют применение LIKE с шаблоном на основе специального символа **%.**

```
use BSTU
go
select QUALIFICATION from PROFESSION
```

QUALIFICATION
экономист-менеджер
экономист
инженер-механик
инженер-механик
инженер-электромеханик
инженер-механик
инженер-программист-системотехник
инженер-технолог
редактор-технолог
инженер-химик-технолог
инженер-химик-технолог
инженер по сертификации
инженер лесного хозяйства
инженер садово-паркового строительства
специалист в сфере туризма

Рис. 7.54. Содержимое столбца **QUALIFICATION** таблицы **PROFESSION**

```
use BSTU
go
select QUALIFICATION from PROFESSION
      where QUALIFICATION like 'экономист%'
```

QUALIFICATION
экономист-менеджер
экономист

Рис. 7.55. Выбор строк, начинающихся с цепочки символов **экономист** в столбце **QUALIFICATION** таблицы **PROFESSION**

```
use BSTU
go
select QUALIFICATION from PROFESSION
      where QUALIFICATION like '%технолог'
```

QUALIFICATION
инженер-технолог
редактор-технолог
инженер-химик-технолог
инженер-химик-технолог

Рис. 7.56. Выбор строк, завершающихся цепочкой символов **технолог** в столбце **QUALIFICATION** таблицы **PROFESSION**

```
use BSTU
go
select QUALIFICATION from PROFESSION
      where QUALIFICATION like '%химик%'
```

QUALIFICATION
инженер-химик-технолог
инженер-химик-технолог

Рис. 7.57. Выбор строк, содержащих цепочку символов **химик** в столбце **QUALIFICATION** таблицы **PROFESSION**

```
use BSTU
go
select QUALIFICATION from PROFESSION
      where QUALIFICATION like '%сад%парк%'
```

QUALIFICATION
инженер садово-паркового строительства

Рис. 7.58. Выбор строк, содержащих последовательно две цепочки символов **сад** и **парк** в столбце **QUALIFICATION** таблицы **PROFESSION**

Символ **%** обозначает любое (включая ноль) количество любых символов. Шаблон **экономист%** описывает множество строк, которые начинаются с цепочки символов **экономист**, **%технолог** – множество строк, заканчивающихся цепочкой символов **технолог**, а шаблон **%химик%** – множество строк, содержащих цепочку **химик**. В шаблоне можно указать несколько цепочек. Например, **%сад%парк%** описывает множество строк, содержащих последовательно две цепочки символов **сад** и **парк**.

На рис. 7.59 демонстрируется шаблон с применением символа _ (подчеркивание). Этот символ означает, что вместо него может стоять ровно один любой символ. Обратите внимание, что в сочетании с символом **%** символ подчеркивание позволяет указать, что требуется как минимум один любой символ.

```
use BSTU
go
select QUALIFICATION from PROFESSION
      where QUALIFICATION like 'экономист_%'
```

QUALIFICATION
экономист-менеджер

Рис. 7.59. Выбор строк, начинающихся с цепочки символов **экономист** и следующего за ней любого символа в столбце **QUALIFICATION** таблицы **PROFESSION**

Более подробно о возможностях описания множества строк с помощью шаблонов, можно узнать в издании [5].

7.4.3. Сравнение значений NULL

На рис. 7.60 представлен пример, демонстрирующий корректное и некорректное сравнение величин в секции WHERE.

```
use BSTU
go
select
     (select 'Y' where 0=0)              '0=0',
     (select 'Y' where 1!=2)             '1!=2',
     (select 'Y' where 'A'='A')          'A=A',
     (select 'Y' where 'A'='A       ') 'A=A',
     (select 'Y' where 'A'!='B')         'A!=B';
select
     (select 'Y' where NULL = NULL)   'NULL=NULL',
     (select 'Y' where NULL != NULL) 'NULL!=NULL',
     (select 'Y' where 0 = NULL)      '0=NULL',
     (select 'Y' where 0 != NULL)     '0!=NULL',
     (select 'Y' where 0 > NULL)      '0>NULL',
     (select 'Y' where 0 < NULL)      '0<NULL';
```

0=0	1!=2	A=A	A=A	A!=B
Y	Y	Y	Y	Y

NULL=NULL	NULL!=NULL	0=NULL	0!=NULL	0>NULL	0<NULL
NULL	NULL	NULL	NULL	NULL	NULL

Рис. 7.60. Сравнение с NULL в секции WHERE

Корректное сравнение демонстрируется с помощью пяти подзапросов в первом операторе SELECT, некорректное – во втором. Некорректность вызвана применением операторов сравнения к значениям NULL. Обратите внимание: результат второго SELECT-запроса содержит во всех столбцах значение NULL – признак неопределенности (в данном случае – отсутствия) значения.

Сравнения со значением NULL допускается только с помощью операции IS NULL, либо IS NOT NULL (рис. 7.61).

```
use BSTU
go
select
  (select 'Y' where NULL is NULL)      'NULL is  NULL',
  (select 'Y' where NULL is not NULL) 'NULL is not NULL',
  (select 'Y' where 0 = NULL)          '0 is NULL',
  (select 'Y' where 0 is not NULL)     '0 is not NULL';
```

NULL is NULL	NULL is not NULL	0 is NULL	0 is not NULL
Y	NULL	NULL	Y

Рис. 7.61. Сравнение с NULL с помощью операции IS NULL

На рис. 7.62 представлен пример использования операции IS NULL для поиска строк в таблице **PULPIT**, содержащих в столбце **FACULTY** значение NULL.

```
use BSTU
go
update PULPIT set FACULTY = NULL  where FACULTY = 'ЛХФ';
select * from PULPIT where FACULTY IS NULL;
```

PULPIT	PULPIT_NAME	FACULTY
ЛВ	Лесоводства	NULL
ЛЗиДВ	Лесозащиты и древесиноведения	NULL
ЛКиП	Лесных культур и почвоведения	NULL
ЛПиСПС	Ландшафтного проектирования и садово-паркового с...	NULL
ЛУ	Лесоустройства	NULL
ОВ	Охотоведения	NULL
ТиП	Туризма и природопользования	NULL

Рис. 7.62. Фильтрация строк в таблице **PULPIT** с помощью операции IS NULL

С помощью оператора UPDATE (подробно разбирается позже) во всех строках таблицы **PULPIT** со значением **ЛХФ** в столбце **FACULTY** это значение заменяется на NULL. Затем выполняется SELECT-запрос, использующий в секции WHERE операцию IS NULL для выбора строк со значением NULL в столбце **FACULTY**.

На рис. 7.63 представлен пример использования операций IS NULL и IS NOT NULL при подсчете строк в таблице **PULPIT**.

```
use BSTU
go
select
(select count(*) from PULPIT where FACULTY is NULL)       'is NULL',
(select count(*) from PULPIT where FACULTY is not NULL)   'is not NULL',
(select count(*) from PULPIT)                             'всего'
```

is NULL	is not NULL	всего
7	19	26

Рис. 7.63. Применение IS NULL и IS NOT NULL в секции WHERE подзапросов

При использовании значения NULL в столбцах таблицы следует помнить следующее.

1. Для поиска строк со значениями NULL необходимо применять только операции IS NULL или IS NOT NULL.
2. Результатом любой другой операции, использующей в качестве операнда значение NULL, является NULL.
3. В секции WHERE значение логического выражения равное NULL интерпретируется как «ложь».

Для удобства работы с NULL-значениями предназначены две встроенные функции IS NULL и COALESCE.

На рис. 7.64 приведен пример использования функции IS NULL в SELECT-списке, а также фрагмент результирующего набора.

```
use BSTU
go
select FACULTY, isnull(FACULTY, '???') [isnull(FACULTY)] from PULPIT
```

FACULTY	isnull(FACULTY)
ИДиП	ИДиП
ИДиП	ИДиП
NULL	???
NULL	???
NULL	???
ТТЛП	ТТЛП
NULL	???

Рис. 7.64. Применение встроенной функции IS NULL в SELECT-списке

Функция IS NULL принимает два параметра и проверяет их значения на NULL слева направо. Функция возвращает первое значение не равное NULL.

Функция COALESCE очень похожа на IS NULL, но может принимать любое количество параметров. Аналогично функции IS NULL она слева направо проверяет значения параметров и возвращает первое значение не равное NULL (рис. 7.65).

```
use BSTU
go
select FACULTY, PULPIT,
       coalesce(FACULTY, NULL, PULPIT, '???') [coalesce(FACULTY,...)] from PULPIT
```

FACULTY	PULPIT	coalesce(FACULTY,...)
ИДиП	БФ	ИДиП
ИДиП	ИСиТ	ИДиП
NULL	ЛВ	ЛВ
NULL	ЛЗиДВ	ЛЗиДВ
NULL	ЛКиП	ЛКиП
ТТЛП	ЛМиЛЗ	ТТЛП

Рис. 7.65. Применение встроенной функции COALESCE в SELECT-списке

Функции IS NULL и COALESCE можно применять в секции WHERE. На рис. 7.66 приведен пример SELECT-запроса, использующего функцию IS NULL в секции WHERE.

```
select distinct FACULTY  from PULPIT where isnull(FACULTY, '###') ='###'
```

FACULTY
NULL

Рис. 7.66. Применение встроенной функции ISNULL
в секции WHERE SELECT-запроса

7.4.4. Логические связки AND, OR и NOT

Логические выражения в SELECT-запросах могут состоять из более, чем одной операции сравнения. Для записи таких выражений применяются логические операции AND, OR и NOT. Операции AND и OR являются бинарными, т. е. предназначены для соединения двух логических выражений. Значение операции AND будет «истина», если оба операнда имеют значение «истина». Значение операции OR будет «истина» в том случае, если хотя бы один из операндов имеет значение «истина». Операция NOT – унарная, применяется к любому логическому выражению и меняет его значение на противоположное (рис. 7.67).

```
select isnull((select 'истина' where (1=1)and(1=1)), 'ложь') [истина and истина],
       isnull((select 'истина' where (1=1)and(1<1)), 'ложь') [истина and ложь],
       isnull((select 'истина' where (1<1)and(1=1)), 'ложь') [ложь and истина],
       isnull((select 'истина' where (1<1)and(1<1)), 'ложь') [ложь and ложь]

select isnull((select 'истина' where (1=1)or(1=1)), 'ложь') [истина or истина],
       isnull((select 'истина' where (1=1)or(1<1)), 'ложь') [истина or ложь],
       isnull((select 'истина' where (1<1)or(1=1)), 'ложь') [ложь or истина],
       isnull((select 'истина' where (1<1)or(1<1)), 'ложь') [ложь or ложь]

select isnull((select 'истина' where  not(1=1)), 'ложь')    [not истина],
       isnull((select 'истина' where  not(1<1)), 'ложь')    [not ложь]
```

истина and истина	истина and ложь	ложь and истина	ложь and ложь
истина	ложь	ложь	ложь

истина or истина	истина or ложь	ложь or истина	ложь or ложь
истина	истина	истина	ложь

not истина	not ложь
ложь	истина

Рис. 7.67. Таблица истинности для логических операций AND, OR, NOT

На рис. 7.68–7.72 представлено пять примеров использования логических операций AND, OR, NOT. Следует обратить внимание, что для операций принят следующий приоритет: NOT, AND, OR. Приоритет операций может быть изменен с помощью скобок (7.71, 7.72).

```
use BSTU
go
select * from TEACHER where   TEACHER_NAME like 'М%' and GENDER ='ж'  and PULPIT = 'ИСиТ'
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
МРЗ	Мороз Елена Станиславовна	ж	ИСиТ

Рис. 7.68. Применение операций AND в секции WHERE

```
use BSTU
go
select * from TEACHER where  not TEACHER_NAME like 'М%' and  GENDER ='ж'  and PULPIT = 'ИСиТ'
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
БРС	Брусенцова Татьяна Палладьевна	ж	ИСиТ
ЖЛК	Жиляк Надежда Александровна	ж	ИСиТ
КШК	Кишкурно Татьяна Вадимовна	ж	ИСиТ
ПСТВЛВ	Пустовалова Наталия Николаевна	ж	ИСиТ

Рис. 7.69. Применение операций NOT и AND в секции WHERE

```
use BSTU
go
select * from TEACHER where   TEACHER_NAME like 'М%' or GENDER ='ж'  or   PULPIT = 'ИСиТ'
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
АКНВЧ	Акунович Станислав Иванович	м	ИСиТ
БРКВЧ	Бракович Андрей Игоревич	м	ИСиТ
БРС	Брусенцова Татьяна Палладьевна	ж	ИСиТ
ГРМН	Герман Олег Витольдович	м	ИСиТ
ГРН	Гурин Николай Иванович	м	ИСиТ
ДЛН	Долинина Татьяна Николаевна	ж	СБУАиА
ДТК	Дятко Александр Аркадьевич	м	ИСиТ
ЕЩНК	Ещенко Людмила Семеновна	ж	ТНВиОХТ
ЖЛК	Жиляк Надежда Александровна	ж	ИСиТ
КБЛ	Кабайло Александр Серафимович	м	ИСиТ
КЛСНВ	Колесников Виталий Леонидович	м	ИСиТ
КШК	Кишкурно Татьяна Вадимовна	ж	ИСиТ
ЛЩНК	Лащенко Анатолий Павлович	м	ИСиТ
МРЗ	Мороз Елена Станиславовна	ж	ИСиТ
МХВ	Мохов Сергей Петрович	м	ЛМиЛЗ
МШКВСК	Машковский Владимир Петрович	м	ЛУ
ПСТВЛВ	Пустовалова Наталия Николаевна	ж	ИСиТ
РМНК	Романенко Дмитрий Михайлович	м	ИСиТ
СМЛВ	Смелов Владимир Владиславович	м	ИСиТ
УРБ	Урбанович Павел Павлович	м	ИСиТ
ЧРН	Черная Наталья Виеторовна	ж	ХПД

Рис. 7.70. Применение операций OR в секции WHERE

```
use BSTU
go
select * from TEACHER where  TEACHER_NAME like 'М%' or  GENDER ='ж' and PULPIT = 'ИСиТ'
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
БРС	Брусенцова Татьяна Палладьевна	ж	ИСиТ
ЖЛК	Жиляк Надежда Александровна	ж	ИСиТ
КШК	Кишкурно Татьяна Вадимовна	ж	ИСиТ
МРЗ	Мороз Елена Станиславовна	ж	ИСиТ
МХВ	Мохов Сергей Петрович	м	ЛМиЛЗ
МШКВСК	Машковский Владимир Петрович	м	ЛУ
ПСТВЛВ	Пустовалова Наталия Николаевна	ж	ИСиТ

Рис. 7.71. Применение операций OR и AND в секции WHERE

```
use BSTU
go
select * from TEACHER where  (TEACHER_NAME like 'М%' or  GENDER ='ж') and PULPIT = 'ИСиТ'
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
БРС	Брусенцова Татьяна Палладьевна	ж	ИСиТ
ЖЛК	Жиляк Надежда Александровна	ж	ИСиТ
КШК	Кишкурно Татьяна Вадимовна	ж	ИСиТ
МРЗ	Мороз Елена Станиславовна	ж	ИСиТ
ПСТВЛВ	Пустовалова Наталия Николаевна	ж	ИСиТ

Рис. 7.72. Применение операций скобок для изменения приоритета операций OR и AND в секции WHERE

7.4.5. Операция BETWEEN

На рис. 7.73 представлен запрос, позволяющий выбрать строки из таблицы **AUDITORIUM** с заданным диапазоном значений столбца **AUDITORIUM_CAPACITY**.

```
use BSTU
go
select * from  AUDITORIUM where  AUDITORIUM_CAPACITY >= 30 and  AUDITORIUM_CAPACITY <= 60
```

AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
110-4	ЛК	30	110-4
111-4	ЛК	30	111-4
137-4	ЛК-К	60	132-7
236-1	ЛК	60	236-1
313-1	ЛК-К	60	313-1
324-1	ЛК-К	50	324-1

Рис. 7.73. Запрос, задающий диапазон значений в столбце

Логическое выражение, заданное в секции WHERE запроса на рис. 7.73, может быть записано короче с помощью ключевого слова BETWEEN (рис. 7.74).

```
use BSTU
go
select * from  AUDITORIUM where  AUDITORIUM_CAPACITY between  30 and 60
```

AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
110-4	ЛК	30	110-4
111-4	ЛК	30	111-4
137-4	ЛК-К	60	132-7
236-1	ЛК	60	236-1
313-1	ЛК-К	60	313-1
324-1	ЛК-К	50	324-1

Рис. 7.74. Применение операции BETWEEN

Операция BETWEEN может быть использована совместно с другими операциями (рис. 7.75).

```
use BSTU
go
select * from  AUDITORIUM where  not AUDITORIUM_CAPACITY between  30 and 60
                                 and AUDITORIUM_TYPE = 'ЛК-К'
```

AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
114-4	ЛК-К	90	114-4
305-4	ЛК-К	90	305-5

Рис. 7.75. Применение операции BETWEEN совместно с другими операциями

7.4.6. Применение подзапросов

Напомним, подзапрос – это SELECT-запрос, который выполняется в рамках другого запроса. Подзапросы могут применяться в секции WHERE. В простейшем случае подзапрос некоррелированный и формирует скалярный результирующий набор (рис. 7.76).

```
use BSTU
go
-- наименование типа аудиторий,
-- которых больше всего
-- с вместимостью от 30 до 60
select AUDITORIUM_TYPENAME  from AUDITORIUM_TYPE
where AUDITORIUM_TYPE = (
        select top(1) AUDITORIUM_TYPE  from AUDITORIUM
                      where AUDITORIUM_CAPACITY  between  30 and 60
                      group by  AUDITORIUM_TYPE
                      order by count(*) desc)
```

AUDITORIUM_TYPENAME
Лекционная с уст. проектором

Рис. 7.76. Применение некоррелируемого подзапроса в секции WHERE

Подзапрос на рис. 7.76 формирует одно значение (достигается с помощью опции TOP и указания единственного столбца в SELECT-списке). Подзапрос некоррелированный, поэтому выполняется только один раз. В секции WHERE внешнего запроса значение столбца **AUDITORIUM_TYPE** каждой строки таблицы **AUDITORIUM_TYPE** сравнивается с

результатом выполнения подзапроса. При совпадении значений строка внешнего запроса копируется в окончательный результирующий набор.

```
use BSTU
go
-- студенты,
-- получившие оценки
-- выше среднего значения по дисциплине
select  s.NAME [Студент],  p.[SUBJECT] [Дисциплина] , p.NOTE [Оценка]
         from STUDENT s inner join PROGRESS p on s.IDSTUDENT = p.IDSTUDENT
         where p.NOTE > (select avg(NOTE) from PROGRESS pp  where pp.SUBJECT = p.SUBJECT)
```

Студент	Дисциплина	Оценка
Хартанович Екатерина Александровна	ОАиП	8
Горбач Елизавета Юрьевна	ОАиП	7
Шитик Алина Игоревна	ОАиП	8
Сергель Виолетта Николаевна	БД	9
Подоляк Мария Сергеевна	БД	7
Яцкевич Галина Иосифовна	СУБД	6
Акулова Елена Геннадьевна	СУБД	9

Рис. 7.77. Применение коррелируемого подзапроса в секции WHERE

На рис. 7.77 приведен пример использования коррелируемого подзапроса в секции WHERE и фрагмент результирующего набора. Внутренний запрос выполняется для каждой строки результирующего набора внешнего запроса и формирует единственное значение (скалярный результирующий набор), которое сравнивается со значением столбца **NOTE** таблицы **PROGRESS**, связанной внутренним соединением с таблицей **STUDENT**. Обратите внимание, что при сравнении применяется неравенство «больше».

7.4.7. Применение операций IN, ANY, ALL

Запросы, представленные на рис. 7.76 и 7.77, применяют подзапросы, формирующие скалярные результирующие наборы. Для работы с результирующими наборами, представляющими собой множество однородных значений (значений одного типа), можно использовать операции IN, ANY и ALL.

В простейшем случае множество значений может быть указано явно. На рис. 7.78 приведен пример использования оператора SELECT, в секции WHERE которого, используется операция IN.

```
use BSTU
go
select  s.NAME [Студент],  p.[SUBJECT] [Дисциплина] , p.NOTE [Оценка]
          from STUDENT s inner join PROGRESS p on s.IDSTUDENT = p.IDSTUDENT
          where p.NOTE  in  (7,9,10)
```

Студент	Дисциплина	Оценка
Горбач Елизавета Юрьевна	ОАиП	7
Сергель Виолетта Николаевна	БД	9
Подоляк Мария Сергеевна	БД	7
Акулова Елена Геннадьевна	СУБД	9
Буянова Мария Александровна	КГ	7
Харченко Елена Геннадьевна	КГ	7
Крученок Евгений Александрович	КГ	9
Кузмичева Анна Андреевна	ОХ	9
Навроцкий Ярослав Юрьевич	ЭТ	7
Хамицевич Вадим Николаевич	ЭТ	9

Рис. 7.78. Применение операции IN с явно указанным списком значений

Операция IN формирует логическое значение «истина» в том случае, если значение, указанное слева от ключевого слова IN (на рис. 7.78 – значение в столбце **NOTE** таблицы **PROGRESS**), равно хотя бы одному из значений списка, указанного справа от IN (на рис. 7.78 – одно из значений: 7, 9 или 10). Выражение IN на рис. 7.78 может быть заменено выражением, использующим операцию OR (рис. 7.79).

```
use BSTU
go
select  s.NAME [Студент],  p.[SUBJECT] [Дисциплина] , p.NOTE [Оценка]
          from STUDENT s inner join PROGRESS p on s.IDSTUDENT = p.IDSTUDENT
          where p.NOTE = 7 or p.NOTE = 9 or p.NOTE = 10
```

Студент	Дисциплина	Оценка
Горбач Елизавета Юрьевна	ОАиП	7
Сергель Виолетта Николаевна	БД	9
Подоляк Мария Сергеевна	БД	7
Акулова Елена Геннадьевна	СУБД	9
Буянова Мария Александровна	КГ	7
Харченко Елена Геннадьевна	КГ	7
Крученок Евгений Александрович	КГ	9
Кузмичева Анна Андреевна	ОХ	9
Навроцкий Ярослав Юрьевич	ЭТ	7
Хамицевич Вадим Николаевич	ЭТ	9

Рис. 7.79. Замена операции IN с явным списком значений (рис. 7.78), несколькими сравнениями, соединенными операциями OR

На рис. 7.80 приведен пример SELECT-запроса, в котором список для операции IN формируется как результат выполнения некоррелированного подзапроса. Обратите внимание на то, что в этом случае заменить операцию IN несколькими сравнениями не получится.

```
use BSTU
go
select PULPIT_NAME from PULPIT
       where PULPIT in (select distinct PULPIT from TEACHER
                                   where TEACHER_NAME like 'A%')
```

PULPIT_NAME
Информационных систем и технологий
Лесоустройства
Полиграфического оборудования и систем обработки...

Рис. 7.80. Формирование списка значений для операции IN с помощью подзапроса, формирующего результирующий набор из одного столбца

Операция IN может применяться совместно с другими логическими операциями. Пример на рис. 7.81 демонстрирует операцию IN в сочетании с операциями NOT и AND. При этом следует помнить, что IN и LIKE будут иметь наивысший приоритет: операция NOT будет применяться к результату выполнения операции IN, и лишь затем операция AND соединит отрицание IN и предварительно полученный результат LIKE-сравнения.

```
use BSTU
go
select PULPIT_NAME from PULPIT
       where not PULPIT in (select distinct PULPIT from TEACHER
                                where TEACHER_NAME like 'А%')
             and PULPIT_NAME like '%хим_%'
```

PULPIT_NAME
Машин и аппаратов химических и силикатных произво...
Органической химии
Процессов и аппаратов химических производств
Технологии неорганических веществ и общей химиче...
Технологии нефтехимического синтеза и переработки...
Химической переработки древесины
Химии, технологии электрохимических производств и ...

Рис. 7.81. Применение операции IN совместно с другими логическими операциями

Операция ALL используется совместно с операциями сравнения и тоже предназначена для сравнения со списком значений, который задается как результат выполнения подзапроса (рис. 7.82).

```
use BSTU
go
select s.NAME, p.[SUBJECT], p.NOTE
from STUDENT s inner join  PROGRESS p on s.IDSTUDENT = p.IDSTUDENT
where p.NOTE >all (select avg(NOTE)+1 from PROGRESS pp
                   group by pp.SUBJECT)
```

NAME	SUBJECT	NOTE
Хартанович Екатерина Александровна	ОАиП	8
Шитик Алина Игоревна	ОАиП	8
Сергель Виолетта Николаевна	БД	9
Акулова Елена Геннадьевна	СУБД	9
Крученок Евгений Александрович	КГ	9
Кузмичева Анна Андреевна	ОХ	9
Даниленко Максим Васильевич	ОХ	8
Ляхов Виталий Сергеевич	ЭТ	8
Хамицевич Вадим Николаевич	ЭТ	9

Рис. 7.82. Применение операции ALL совместно с операцией сравнения

В примере на рис. 7.82 операция ALL применяется совместно с операцией сравнения «больше». Обратите внимание: справа от символов **>ALL** (читается: больше каждого) в скобках записан подзапрос, возвращающий столбец значений. Операция >ALL сформирует истинное значение в том случае, если значение, стоящее слева (на рис. 7.82 – значение в столбце **NOTE** таблицы **PROGRESS**), больше каждого значения в списке, указанном справа (значения в столбце, сформированном подзапросом).

Операцию ALL можно применять со всеми операциями сравнения и в сочетании с другими логическими операциями (рис. 7.83).

```
use BSTU
go
select s.NAME, p.[SUBJECT], p.NOTE
from STUDENT s inner join  PROGRESS p on s.IDSTUDENT = p.IDSTUDENT
where p.NOTE <=all (select avg(NOTE)+1 from PROGRESS pp group by pp.SUBJECT) and
      p.[SUBJECT] not in (select [SUBJECT] from [SUBJECT] where PULPIT ='ИСиТ')
```

NAME	SUBJECT	NOTE
Бань Сергей Анатольевич	ОХ	6
Сечейко Илья Александрович	ОХ	4
Бурко Диана Францевна	ОХ	5
Зизюк Ольга Олеговна	ОХ	4
Федоров Александр Владимирович	ЭТ	4
Поль Виктория Михайловна	ЭТ	5

Рис. 7.83. Применение операции ALL совместно с другими логическими операциями

Операция ANY записывается и применяется аналогично операции ALL, но результат формируется иным образом (рис. 7.84).

```
use BSTU
go
select s.NAME, p.[SUBJECT], p.NOTE
from STUDENT s inner join  PROGRESS p on s.IDSTUDENT = p.IDSTUDENT
where p.NOTE <=any (select avg(NOTE)+1 from PROGRESS pp group by pp.SUBJECT)and
      p.[SUBJECT] not in (select [SUBJECT] from [SUBJECT] where PULPIT ='ИСиТ')
```

NAME	SUBJECT	NOTE
Бань Сергей Анатольевич	ОХ	6
Сечейко Илья Александрович	ОХ	4
Бурко Диана Францевна	ОХ	5
Зизюк Ольга Олеговна	ОХ	4
Навроцкий Ярослав Юрьевич	ЭТ	7
Федоров Александр Владимирович	ЭТ	4
Поль Виктория Михайловна	ЭТ	5

Рис. 7.84. Применение операции ANY совместно с операцией сравнения и другими логическими операциями

При выполнении ANY-сравнения (на рис. 7.84 – операция >= ANY) стоящее слева значение (столбец **NOTE** таблицы **PROGRESS**) сравнивается (операция >=) с каждым значением списка, записанным справа (содержимое столбца, сформированного подзапросом). Если хотя бы для одного значения из списка операция сравнения принимает значение «истина», то вся операция (>= ANY) принимает значение «истина».

7.4.8. Применение операции EXISTS

Часто в подзапросе, который применяется в секции WHERE, бывает достаточно выяснить только сам факт наличия строк в результирующем наборе. В таких случаях удобно использовать операцию EXISTS.

На рис. 7.85 представлен пример использования операции EXISTS.

```
use BSTU
go
select * from FACULTY f where   EXISTS (select * from PULPIT p where p.FACULTY = f.FACULTY)
```

FACULTY	FACULTY_NAME
ИДиП	Издательское дело и полиграфия
ИЭФ	Инженерно-экономический факультет
ТОВ	Технология органических веществ
ТТЛП	Технология и техника лесной промышленности
ХТиТ	Химическая технология и техника

Рис. 7.85. Пример использования операции EXISTS

В скобках, следующих за ключевым словом EXISTS, записывается подзапрос. Результатом операции является «истина», если результирующий набор подзапроса содержит хотя бы одну строку, в противоположном случае операция EXISTS формирует значение «ложь».

```
use BSTU
go
select * from FACULTY f where  not EXISTS (select * from PULPIT p where p.FACULTY = f.FACULTY)
                               or f.FACULTY = 'ИДиП'
```

FACULTY	FACULTY_NAME
ИДиП	Издательское дело и полиграфия
ЛХФ	Лесохозяйственный факультет

Рис. 7.86. Пример применения операции EXISTS в сочетании
с другими логическими операциями

Как и другие логические операции, EXISTS может использоваться в логических выражениях (рис. 7.86).

7.5. Секция GROUP BY

Если в SELECT-запросе есть секция GROUP BY, то она применяется после секции WHERE. Основное назначение GROUP BY – разбиение множества строк, сформированного секциями FROM и WHERE, на группы в соответствии со значениями в заданных операцией GROUP BY столбцах, а также выполнение вычислений над группами строк (подсчет количества строк; суммирование значений; вычисление средних, минимальных, максимальных значений и т. п.).

На рис. 7.87 изображена схема обработки строк секцией GROUP BY.

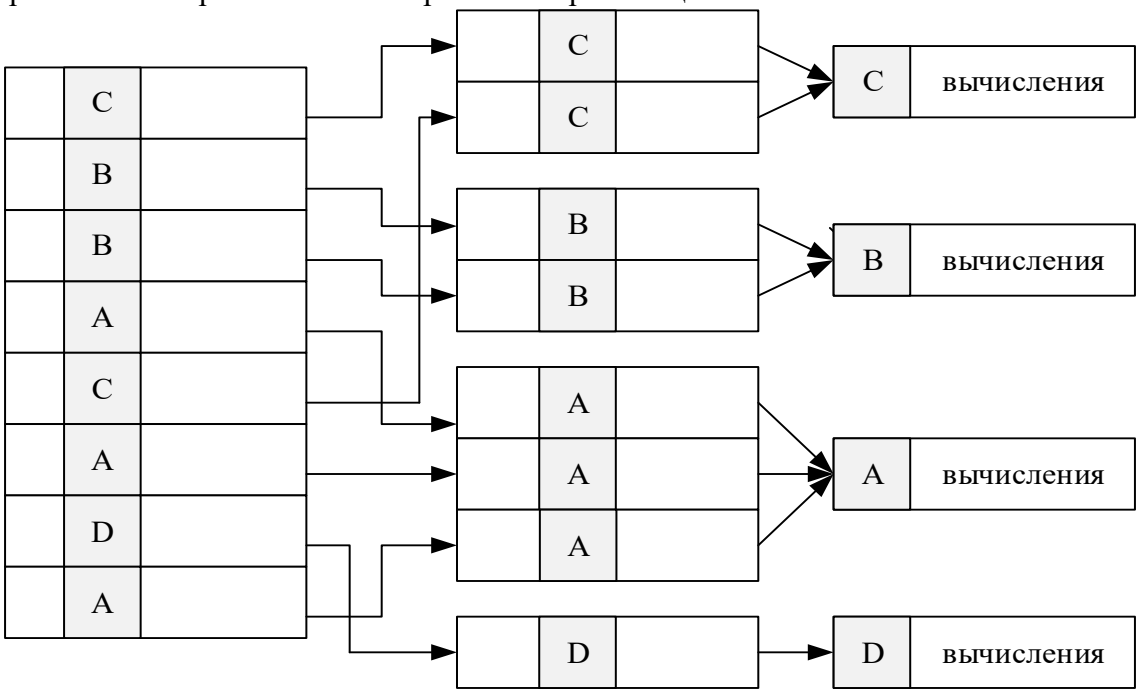


Рис. 7.87. Схема работы секции GROUP BY

Секция GROUP BY выполняет обработку строк в три этапа: 1) формирование строковых групп; 2) вычисление агрегатных функций над каждой группой строк; 3) формирование результирующего набора. В секции GROUP BY задается одно или несколько выражений (формирующих значения), задающих признак объединения строк в группы. На втором этапе для вычислений могут применяться агрегатные функции. Перечень наиболее часто используемых функций приведен в табл.7.2.

Таблица 7.2

Наиболее часто применяемые агрегатные функции

Наименование функции	Назначение
AVG	Вычисление среднего значения
COUNT	Вычисление количества строк
MAX	Вычисление максимального значения
MIN	Вычисление минимального значения
SUM	Вычисление суммы значений

7.5.1. Простейшая группировка

В простейшем случае группировка может быть выполнена без использования секции GROUP BY, а только с применением агрегатных функций. В этом случае предполагается, что все строки находятся в одной группе.

На рис. 7.88 представлен пример простейшей группировки.

```
use BSTU
go
select max(AUDITORIUM_CAPACITY) [Максимальная вместимость],
       min(AUDITORIUM_CAPACITY) [Минимальная вместимость],
       avg(AUDITORIUM_CAPACITY) [Средняя вместимость],
       count(*)                 [Количество аудиторий],
       sum(AUDITORIUM_CAPACITY) [Суммарная вместимость]
 from AUDITORIUM;
```

Максимальная вм...	Минимальная вме...	Средняя вм...	Количество ауд...	Суммарная вме...
90	15	69	23	1595

Рис. 7.88. Простейшая группировка с помощью агрегатных функций

Следует обратить внимание, что в запросе на рис. 7.88 в SELECT-списке нельзя указывать наименования столбцов, а только агрегатные функции или константы, т. к. результирующий набор представляет собой одну строку (строки сгруппировались в одну строку).

К простейшей группировке относят группировку по значениям одного столбца. На рис. 7.89 представлен пример SELECT-запроса, применяющего группировку строк таблицы **PULPIT** по значениям столбца **FACULTY**.

```
use BSTU
go
select FACULTY [Код факультета],
       count(*) [Количество кафедр] from PULPIT  group by FACULTY
```

Код факультета	Количество кафедр
NULL	7
ИДиП	5
ИЭФ	3
ТОВ	3
ТТЛП	4
ХТиТ	4

Рис. 7.89. Пример группировки по значениям одного столбца

Результирующий набор, полученный после выполнения SELECT-запроса, представленного на рис. 7.89, позволяет выявить, что в столбце **FACULTY** таблицы **PULPIT** содержится 6 различных значений (включая NULL); в столбце **Количество кафедр** с помощью агрегатной функции COUNTподсчитывается количество строк в таблице **PULPIT** с соответствующим значением (столбец **Код факультета**) столбца **FACULTY;** сумма чисел в столбце **Количество кафедр** равна количеству строк в таблице **PULPIT.** В последнем легко убедиться, если выполнить запрос, представленный на рис. 7.90.

```
use BSTU
go
select  count(*) [Количество кафедр] from PULPIT
```

Количество кафедр
26

Рис. 7.90. Вычисление количества строк в таблице PULPIT

Обратите внимание (рис. 7.89) на то, что при применении GROUP BY в SELECT-списке допускается указывать только те столбцы, по которым осуществляется группировка.

7.5.2. Группировка по выражению

В простейшем случае в GROUP BY указывается один столбец (рис. 7.89). В этом случае формируются группы, соответствующие каждому значению указанного столбца. Если указать два столбца, то группы будут сформированы для каждого сочетания значений этих столбцов (рис. 7.91).

```
use BSTU
go
select PDATE      [дата],
       [SUBJECT] [дисциплина],
       count(*)   [количество студентов],
       max(NOTE)  [максимальня оценка]
from PROGRESS  group by PDATE, [SUBJECT]
```

дата	дисциплина	количество студентов	максимальня оценка
2013-06-12	БД	5	9
2013-06-15	КГ	6	9
2013-01-10	ОАиП	7	8
2013-01-19	ОХ	6	9
2013-01-18	СУБД	5	9
2013-01-15	ЭТ	5	9

Рис. 7.91. Группировка строк таблицы **PROGRESS**
по двум столбцам **PDATE**, **SUBJECT**

Количество групп, сформированных секцией GROUP BY в SELECT-запросе на рис. 7.91, равно количеству сочетаний значений из столбцов **PDATA** и **SUBJECT** строк таблицы **PROGRESS**. Например, сочетание значений **2013-01-19** и **OX** встречается в таблице 6 раз (вычисляется с помощью агрегатной функции COUNT), а максимальное значение в столбце **NOTE** среди строк этой группы равно 9.

В некоторых случаях бывает удобным группировать не просто по значениям столбцов, а по выражениям над столбцами (7.92).

```
use BSTU
go
select month(BDAY)   [месяц],
       count(*)      [кол.родившихся в этот месяц],
       100*count(*)/(select count(*) from STUDENT) [%]
from STUDENT  group by  month(BDAY)
```

месяц	кол.родившихс...	%
1	6	3
2	14	7
3	11	6
4	18	10
5	12	6
6	8	4
7	18	10
8	30	16
9	15	8
10	11	6
11	16	8
12	19	10

Рис. 7.92. Группировка строк таблицы **STUDENT** по значению, возвращаемому функцией

Встроенная функция MONTH в секции GROUP BY применима к типам данных, используемых для хранения даты, и возвращает номер месяца. В запросе на рис. 7.92 группировка строк таблицы **STUDENT** осуществляется по номеру месяца. Кроме того, для каждой группы подсчитывается количество строк (COUNT) и доля (в процентах) количества строк.

При применении выражений в секции GROUP BY в списке SELECT помимо агрегатных функций допускается указывать точно такие же выражения.

Следует обратить внимание, что при вычислении процента (рис. 7.92) применяется целочисленная арифметика, которая приводит к значительной погрешности. Запрос на рис. 7.93 подсчитывает две суммы: сумму процентов, вычисленных с применением целочисленной арифметики, и сумму процентов, вычисленных на базе чисел с дробной частью.

```
use BSTU
go
select sum(t.[%]) [не точный %],  sum(t.[точный %]) [точный %]
from (select month(BDAY)   [месяц],
             count(*)      [кол.родившихся в этот месяц],
             100    *count(*)/(select count(*) from STUDENT) [%],
             100.0 *count(*)/(select count(*) from STUDENT) [точный %]
      from STUDENT  group by  month(BDAY))t
```

не точный %	точный %
94	99.999999999990

Рис. 7.93. Погрешность вычислений с помощью целочисленной арифметики (может быть значительной

На рис. 7.94 представлен пример еще одного SELECT-запроса, применяющего выражение в секции GROUP BY.

```
use BSTU
go
select rtrim(f.FACULTY)+'-'+rtrim(p.PULPIT) [факультет-кафедра],
       count(s.[SUBJECT]) [количество дисциплин]
from  FACULTY f inner join PULPIT p on f.FACULTY = p.FACULTY
                inner join [SUBJECT] s on p.PULPIT = s.PULPIT
group by rtrim(f.FACULTY)+'-'+rtrim(p.PULPIT);
```

факультет-кафедра	количество дисциплин
ИДиП-ИСиТ	14
ИДиП-ПОиСОИ	2
ИЭФ-МиЭП	1
ИЭФ-ЭТиМ	1
ТОВ-ОХ	1
ТОВ-ТНХСиППМ	1
ТТЛП-ЛМиЛЗ	1
ТТЛП-ТЛ	1
ХТиТ-ТНВиОХТ	1
ХТиТ-ХТЭПиМЭЕ	1

Рис. 7.94. Применение выражения в секции GROUP BY

Выражение в секции GROUP BY (рис. 7.94) применяет к значениям двух столбцов встроенную функцию RTRIM (удаление в строке пробелов справа) и выполняет конкатенацию (символ + для строк). Кроме того, для каждой сформированной таким образом строки осуществляется подсчет количества не равных NULL значений в столбце **SUBJECT**.

7.5.3. Группировка отфильтрованных данных

Следует помнить, что если в SELECT-запросе применяются секции WHERE и GROUP BY, первой исполняется секция WHERE. На рис. 7.95 и 7.96 приводятся примеры двух запросов: первый не использует секцию WHERE; а второй использует.

```
use BSTU
go
select  FACULTY [факультет],
        count(PROFESSION) [количество специальностей]
from PROFESSION
group by FACULTY
```

факультет	количество специальностей
ИДиП	3
ИЭФ	2
ЛХФ	3
ТОВ	3
ТТЛП	2
ХТиТ	2

Рис. 7.95. Применение секции GROUP BY без WHERE

```
use BSTU
go
select  FACULTY [факультет],
        count(PROFESSION) [количество специальностей]
from PROFESSION
where PROFESSION_NAME like '%хим%'
group by FACULTY
```

факультет	количество специальностей
ТОВ	3
ХТиТ	1

Рис. 7.96. Совместное применение секций GROUP BY и WHERE

В примере на рис. 7.96 запрос выполняется в два этапа: вначале осуществляется фильтрация строк в соответствии с логическим выражением в секции WHERE (операция LIKE проверяет вхождение подстроки **хим** в строку столбца **PROFESSION_NAME**); затем выполняется группировка результата фильтрации в соответствии с выражением, указанным в секции GROUP BY (столбец **PROFESSION**).

Если в SELECT-запросе указана опция TOP, ограничивающая количество строк в окончательном результирующем наборе, следует помнить, что действие опции осуществляется после группировки.

На рис. 7.97 приведен пример SELECT-запроса, аналогичный запросу, представленному на рис. 7.96, но с применением опции TOP.

```
use BSTU
go
select top(1) FACULTY [факультет],
        count(PROFESSION) [количество специальностей]
from PROFESSION
where PROFESSION_NAME like '%хим%'
group by FACULTY
```

факультет	количество специальностей
ТОВ	3

Рис. 7.97. Совместное применение секций GROUP BY, WHERE и опции TOP

Выполнение запроса на рис. 7.97 осуществляется в три этапа: фильтрация, группировка и выбор первой строки из результирующего набора, сформированного секцией GROUP BY.

7.5.4. Применение конструкций ROLLUP и CUBE

Конструкции ROLLUP и CUBE применяются в секции GROUP BY и служат для вычисления агрегатных значений над подмножествами строк.

На рис. 7.98 представлен пример использования конструкции ROLLUP.

```
use BSTU
go
select p.PULPIT      [кафедра],
       s.[SUBJECT]   [дисциплина],
       pr.PDATE      [дата экзамена],
       sum(pr.NOTE)  [сумма оценок]
from PULPIT p inner join [SUBJECT]s
               on p.PULPIT = s.PULPIT
              inner join PROGRESS pr
               on s.[SUBJECT] = pr.[SUBJECT]
where p.PULPIT_NAME like '%эконом%' or p.PULPIT_NAME like '%хим%'
group by rollup(p.PULPIT, s.[SUBJECT], pr.PDATE)
```

	кафедра	дисциплина	дата экзамена	сумма оценок
1	ОХ	ОХ	2013-01-19	36
2	ОХ	ОХ	NULL	36
3	ОХ	NULL	NULL	36
4	ЭТиМ	ЭТ	2013-01-15	33
5	ЭТиМ	ЭТ	NULL	33
6	ЭТиМ	NULL	NULL	33
7	NULL	NULL	NULL	69

Рис. 7.98. Применение конструкции ROLLUP

Все строки результирующего набора (рис. 7.98) разобьем на две группы: 1) строки, не содержащие значения NULL (строки 1, 4); 2) строки, содержащие одно значение NULL (2, 5); 3) строки, содержащие два значения NULL (3, 6); 4) строки, содержащие три значения NULL (7).

В первой группе находятся строки, которые могут быть сформированы с помощью аналогичного запроса, но без конструкции ROLLUP (рис. 7.99)

```
use BSTU
go
select p.PULPIT      [кафедра],
       s.[SUBJECT]   [дисциплина],
       pr.PDATE      [дата экзамена],
       sum(pr.NOTE)  [сумма оценок]
from PULPIT p inner join [SUBJECT]s
               on p.PULPIT = s.PULPIT
              inner join PROGRESS pr
               on s.[SUBJECT] = pr.[SUBJECT]
where p.PULPIT_NAME like '%эконом%' or p.PULPIT_NAME like '%хим%'
group by p.PULPIT, s.[SUBJECT], pr.PDATE
```

кафедра	дисциплина	дата экзамена	сумма оценок
ЭТиМ	ЭТ	2013-01-15	33
ОХ	ОХ	2013-01-19	36

Рис. 7.99. Запрос, аналогичный запросу на рис. 7.98, но без конструкции ROLLUP

Вторая группа строк представляет собой результаты группировок первой группы строк, но по двум столбцам (рис. 7.100).

```
use BSTU
go
select  t.[кафедра],t.[дисциплина],
        NULL [дата экзамена],sum(t.[сумма оценок]) [сумма оценок]
from  (select p.PULPIT  [кафедра], s.[SUBJECT]  [дисциплина],
              pr.PDATE     [дата экзамена], sum(pr.NOTE) [сумма оценок]
       from PULPIT p join [SUBJECT]s  on p.PULPIT = s.PULPIT
                     join PROGRESS pr on s.[SUBJECT] = pr.[SUBJECT]
       where p.PULPIT_NAME like '%эконом%' or p.PULPIT_NAME like '%хим%'
       group by p.PULPIT, s.[SUBJECT], pr.PDATE)t
group by  t.[кафедра], t.[дисциплина]
```

кафедра	дисциплина	дата экзамена	сумма оценок
ОХ	ОХ	NULL	36
ЭТиМ	ЭТ	NULL	33

Рис. 7.100. Группировка результата выполнения запроса на рис. 7.99

Третья группа строк представляет собой результаты группировок второй группы строк, но по одному столбцу (рис. 7.101).

```
use BSTU
go
select t1.[кафедра], NULL [дисциплина],
       NULL [дата экзамена], sum(t1.[сумма оценок]) [сумма оценок]
from  (select  t.[кафедра],t.[дисциплина],
               NULL [дата экзамена],sum(t.[сумма оценок]) [сумма оценок]
        from  (select p.PULPIT  [кафедра], s.[SUBJECT]  [дисциплина],
                      pr.PDATE     [дата экзамена], sum(pr.NOTE) [сумма оценок]
               from PULPIT p join [SUBJECT]s  on p.PULPIT = s.PULPIT
                             join PROGRESS pr on s.[SUBJECT] = pr.[SUBJECT]
               where p.PULPIT_NAME like '%эконом%' or p.PULPIT_NAME like '%хим%'
               group by p.PULPIT, s.[SUBJECT], pr.PDATE)t
        group by  t.[кафедра], t.[дисциплина])t1
group by  t1.[кафедра]
```

кафедра	дисциплина	дата экзамена	сумма оценок
ОХ	NULL	NULL	36
ЭТиМ	NULL	NULL	33

Рис. 7.101. Группировка результата выполнения запроса на рис. 7.100

Последняя строка (в четвертой группе только одна строка) результирующего набора на рис. 7.98 может быть получена с помощью агрегирующей функции SUM, примененной к результату запроса на рис. 7.101 без секции GROUP BY. В виду громоздкости этот запрос здесь не рассматривается, но проанализировав строку 7 результирующего набора на рис. 7.96, можно легко убедиться в том, что она может быть получена в результате группировки всех строк результирующего набора запроса на рис. 7.101.

Сформулируем правило формирования результирующего набора SELECT-запроса, применяющего в секции GROUP BY конструкцию ROLLUP.

1. Результирующий набор содержит $n + 1$ групп строк, где n – количество выражений для группировки, указанных в скобках за ключевым словом ROLLUP.
2. Первая группа строк является результатом выполнения группировки по всем n выражениям.

3. Вторая группа строк является результатом группировки первой группы строк по $n - 1$ первым выражениям. Причем столбцы, по которым не выполнялась группировка, заполняются значениями NULL.

4. Группа строк k является группировкой группы строк, полученной на предыдущем этапе по $n - k + 1$ первым выражениям.

5. Последняя $(n + 1)$-я группа содержит одну строку. Значения во всех столбцах, соответствующих выражениям в ROLLUP, равны NULL. Значение в столбце, соответствующем агрегатной функции, вычисляется по всем строкам исходного для группировки набора строк.

На рис. 7.102 представлен запрос и сформированный им результирующий набор строк. В секции GROUP BY запроса применяется конструкция CUBE. Обратите внимание, что список столбцов в скобках, следующих за ключевым словом CUBE, аналогичен ROLLUP-списку на рис. 7.98.

```
use BSTU
go
select p.PULPIT      [кафедра],
       s.[SUBJECT]   [дисциплина],
       pr.PDATE      [дата экзамена],
       sum(pr.NOTE)  [сумма оценок]
from PULPIT p inner join [SUBJECT]s
               on p.PULPIT = s.PULPIT
              inner join PROGRESS pr
               on s.[SUBJECT] = pr.[SUBJECT]
where p.PULPIT_NAME like '%эконом%' or p.PULPIT_NAME like '%хим%'
group by cube(p.PULPIT, s.[SUBJECT], pr.PDATE)
```

	кафедра	дисциплина	дата экзамена	сумма оценок
1	ЭТиМ	ЭТ	2013-01-15	33
2	NULL	ЭТ	2013-01-15	33
3	NULL	NULL	2013-01-15	33
4	ОХ	ОХ	2013-01-19	36
5	NULL	ОХ	2013-01-19	36
6	NULL	NULL	2013-01-19	36
7	NULL	NULL	NULL	69
8	ОХ	NULL	2013-01-19	36
9	ОХ	NULL	NULL	36
10	ЭТиМ	NULL	2013-01-15	33
11	ЭТиМ	NULL	NULL	33
12	ОХ	ОХ	NULL	36
13	NULL	ОХ	NULL	36
14	ЭТиМ	ЭТ	NULL	33
15	NULL	ЭТ	NULL	33

Рис. 7.102. Применение конструкции CUBE

Сравним результирующие наборы строк, полученные в результате запросов, представленных на рис. 7.98 и 7.102. Заметим, что все строки результирующего набора на рис. 7.98 имеются в результирующем наборе на рис. 7.102, но не наоборот.

Рассмотрим строки 2 и 5 результирующего набора на рис. 7.102. Этих строк нет в результирующем наборе на рис. 7.98. Эти строки получены в результате группировки по столбцам **SUBJECT** и **PDATE** исходного для секции GROUP BY набора строк.

Проанализировав результирующий набор на рис. 7.102, не сложно выяснить, что каждая строка относится к одной из групп строк, которые получаются как результат выполнения группировки по подмножеству выражений, указанных в CUBE-списке. Например: строки 1 и 4 – это результат группировки по столбцам (выражениям) **PULPIT**, **SUBJECT** и **PDATE**; строки 2 и

5 – результат группировки по столбцам **SUBJECT** и **PDATE**; строки 8 и 10 – результат группировки по столбцам **PULPIT** и **PDATE** и т. д.

Сформулируем правило формирования результирующего набора SELECT-запроса, применяющего в секции GROUP BY конструкцию CUBE.

1. Формируется множество всех подмножеств выражений, указанных в CUBE-списке.
2. Для каждого непустого подмножества, сформированного в п. 1 выполняется группировка. Если количество элементов подмножества меньше количества элементов CUBE-списка, то соответствующие значения в строках заполняются NULL (как это делалось конструкцией ROLLUP). Сформированные строки помещаются в результирующий набор секции GROUP BY.
3. Для пустого подмножества, сформированного в п. 1 выполняется группировка аналогичная той, что выполнялась в п. 5 правил для ROLLUP.

7.5.5. Секция HAVING

Для фильтрации результирующего набора строк, сформированного секцией GROUP BY, служит секция HAVING. На рис. 7.103 представлен SELECT-запрос? применяющий секцию GROUP BY, а на рис. 7.104 тот же запрос, но дополненный секцией HAVING.

```
use BSTU
go
select AUDITORIUM_TYPE           [тип аудитории],
       sum(AUDITORIUM_CAPACITY)  [суммарная вместимость]
from AUDITORIUM
group by AUDITORIUM_TYPE
```

тип аудитории	суммарная вместимость
ЛБ-К	225
ЛК	1020
ЛК-К	350

Рис. 7.103. Запрос с применением секции GROUP BY

```
use BSTU
go
select AUDITORIUM_TYPE           [тип аудитории],
       sum(AUDITORIUM_CAPACITY)  [суммарная вместимость]
from AUDITORIUM
group by AUDITORIUM_TYPE
having sum(AUDITORIUM_CAPACITY) > 300 -- фильтрация
```

тип аудитории	суммарная вместимость
ЛК	1020
ЛК-К	350

Рис. 7.104. Запрос с применением секции HAVING

Смысл секции HAVING такой же, как и у секции WHERE. Основное отличие – действие секции HAVING распространяется на результат выполнения секции GROUP BY, а действие секции WHERE – на результат секции FROM.

Логическое выражение, указанное в HAVING, вычисляется для каждой строки результирующего набора, сформированного секцией GROUP BY. Как и в случае с секцией WHERE, строка отбирается в результирующий набор секции, если логическое выражение принимает значение «истина». При этом следует помнить, что в логическом выражении могут

применяться лишь такие компоненты, которые основываются на выражениях из секции GROUP BY и/или значениях, которые могут быть вычислены. Применительно к рис. 7.104 в HAVING можно использовать выражения, построенные над значением из столбца **AUDITORIUM_TYPE**, и/или агрегатные функции. В случае, приведенном на рис. 7.104, отфильтровываются такие группы строк, в которых сумма значений в столбце **AUDITORIUM_CAPACITY** превышает 300. На рис. 7.105 представлен еще один пример использования секции HAVING.

```
use BSTU
go
select AUDITORIUM_TYPE           [тип аудитории],
       sum(AUDITORIUM_CAPACITY) [суммарная вместимость]
from AUDITORIUM
group by AUDITORIUM_TYPE
having sum(AUDITORIUM_CAPACITY) > 300  and
       min(AUDITORIUM_CAPACITY) > 30   or
       AUDITORIUM_TYPE ='ЛБ-К'
```

тип аудитории	суммарная вместимость
ЛБ-К	225
ЛК-К	350

Рис. 7.105. Секция HAVING, использующая составное логическое выражение

Следует обратить внимание на то, что в логическом выражении HAVING могут использоваться выражения, которых нет в SELECT-списке, как, например, это сделано в запросе на рис. 7.105: применяется агрегатная функция MIN.

7.6. Секция ORDER BY

В большинстве SELECT-запросов секция ORDER BY выполняется последней. Назначение секции – сортировка результирующего набора строк. Исходным набором для секции ORDER BY является результат выполнения одной из предшествующих секций (см. рис. 7.2 на рис. 122), результатом – этот же набор, но в заданном порядке строк. Следует обратить внимание на то, что в секции ORDER BY могут использоваться псевдонимы имен столбцов, назначенные в SELECT-списке. В общем случае в ORDER BY может быть указано несколько выражений, в простейшем – просто столбец (рис. 7.106).

```
use BSTU
go
select  TEACHER, TEACHER_NAME [имя преподавателя]  from TEACHER order by TEACHER_NAME
```

TEACHER	имя преподавателя
АКНВЧ	Акунович Станислав Иванович
АРС	Арсентьев Виталий Арсентьевич
АТР	Атрощенко Олег Александрович
БРНВСК	Барановский Станислав Иванович
БРТШВЧ	Барташевич Святослав Александрович
БЗБРДВ	Безбородов Владимир Степанович
БЛВ	Беляев Валерий Павлович
БРКВЧ	Бракович Андрей Игоревич
БРС	Брусенцова Татьяна Палладьевна
БРГ	Бурганская Татьяна Минаевна
ГРМН	Герман Олег Витольдович

Рис. 7.106. Пример SELECT-запроса, использующего секцию ORDER BY

Выполненная в запросе (рис. 7.106) сортировка формирует результирующий набор строк, упорядоченный в порядке возрастания значений из столбца **TEACHER_NAME**. При этом порядок сортировки в данном запросе задан неявно. Для того чтобы задать порядок явно, следует применять ключевые слова ASC (рис. 7.107, в порядке возрастания) или DESC (рис. 7.108, в порядке убывания).

```
use BSTU
go
select  TEACHER, TEACHER_NAME [имя преподавателя]  from TEACHER order by TEACHER_NAME asc
--- результат аналогичен, результату на рис. 7.106
```

Рис. 7.107. Применение ключевого слова ASC в секции ORDER BY

```
use BSTU
go
select  TEACHER, TEACHER_NAME [имя преподавателя]  from TEACHER order by TEACHER_NAME desc
```

TEACHER	имя преподавателя
ЮДНКВ	Юденков Виктор Степанович
ШМК	Шмаков Михаил Сергеевич
ЧРН	Черная Наталья Виеторовна
УРБ	Урбанович Павел Павлович
СМЛВ	Смелов Владимир Владиславович
РМНК	Романенко Дмитрий Михайлович
РЖК	Рожков Леонид Николаевич
РВКЧ	Ровкач Андрей Иванович
ПСТВЛВ	Пустовалова Наталия Николаевна
ПРКПЧК	Прокопчук Николай Романович
НВРВ	Неверов Александр Васильевич

Рис. 7.108. Применение ключевого слова DESC
в секции ORDER BY для сортировки в порядке убывания

В секции ORDER BY через запятую может быть перечислено несколько выражений, используемых для сортировки (рис. 7.109), причем для каждого из них может быть указан свой порядок сортировки (ключевые слова ASC или DESC).

```
use BSTU
go
select  count(*) [количество дисциплин],PULPIT [кафедра]
from [SUBJECT]
group by PULPIT
order by  [количество дисциплин] desc, [кафедра] asc
```

количество дисциплин	кафедра
14	ИСиТ
2	ПОиСОИ
1	ЛЗиДВ
1	ЛМиЛЗ
1	ЛПиСПС
1	ЛУ
1	МиЭП
1	ОВ
1	ОХ
1	ТЛ
1	ТНВиОХТ
1	ТНХСиППМ
1	ХТЭПиМЭЕ
1	ЭТиМ

Рис. 7.109. Применение нескольких выражений в секции ORDER BY

Исходным набором строк для секции ORDER BY в SELECT-запросе на рис. 7.109 является результирующий набор секции GROUP BY. Сортировка выполняется по двум выражениям: значениям столбца **количество дисциплин** и значениям столбца **кафедра**. Следует обратить внимание на следующее: в секции ORDER BY применяются псевдонимы столбцов; для каждого выражения в секции ORDER BY указан свой порядок сортировки.

Выражения в секции ORDER BY могут быть достаточно сложными. В примере на рис. 7.110 применяется оператор CASE (подробно рассматривается позже), который в зависимости от логических условий, следующих за ключевыми словами WHEN, формирует значения, указывающиеся за ключевыми словами THEN или ELSE.

```
use BSTU
go
select PROFESSION_NAME [специальность],
       QUALIFICATION   [квалификация]
from PROFESSION
order by  case
           when QUALIFICATION like '%химик%'     and
                QUALIFICATION like '%технолог%'  and
                QUALIFICATION like '%инженер%'  then 1
           when QUALIFICATION like '%технолог%'  and
                QUALIFICATION like '%инженер%'  then 2
           when QUALIFICATION like '%инженер%'  then 3
           else 4
           end  asc
```

специальность	квалификация
Химическая технология органических веществ, матер...	инженер-химик-технолог
Химическая технология переработки древесины	инженер-химик-технолог
Лесоинженерное дело	инженер-технолог
Конструирование и производство изделий из компози...	инженер-механик
Машины и оборудование лесного комплекса	инженер-механик
Полиграфическое оборудование и системы обработки ...	инженер-электромеханик
Машины и аппараты химических производств и предпр...	инженер-механик
Информационные системы и технологии	инженер-программист-системотехник
Физико-химические методы и приборы контроля каче...	инженер по сертификации
Лесное хозяйство	инженер лесного хозяйства
Садово-парковое строительство	инженер садово-паркового строительства
Туризм и природопользование	специалист в сфере туризма
Издательское дело	редактор-технолог
Экономика и управление на предприятии	экономист-менеджер
Бухгалтерский учет, анализ и аудит	экономист

Рис. 7.110. Применение сложных выражений в секции ORDER BY

В результате выполнения сортировки лидирующими в результирующем наборе окажутся строки, содержащие в значении столбца **QUALIFCATION** три подстроки: **химик**, **технолог** и **инженер**; затем будут следовать строки, содержащие в **QUALIFCATION** подстроки **технолог** и **инженер**; далее – содержащие подстроку **инженер**; и, наконец, последними будут строки, не удовлетворяющие перечисленным требованиям.

Следует помнить, что опция TOP применяется после ORDER BY и позволяет отобрать заданное количество лидирующих строк отсортированного результирующего набора (7.111).

```
use BSTU
go
select  top(5) count(*) [количество дисциплин],PULPIT [кафедра]
from [SUBJECT]
group by PULPIT
order by  [количество дисциплин] desc, [кафедра] asc
```

количество дисциплин	кафедра
14	ИСиТ
2	ПОиСОИ
1	ЛЗиДВ
1	ЛМиПЗ
1	ЛПиСПС

Рис. 7.111. Применение сложных выражений в секции ORDER BY

Кроме того, применение опции DISTINCT, избавляющей результирующий набор от повторяющихся строк, позволяет применять в выражениях ORDER BY только те столбцы, которые указаны в SELECT-списке запроса.

7.7. Секции COMPUTE и COMPUTE BY

Секции COMPUTE и COMPUTE BYпредназначены для формирования дополнительных результирующих наборов с итоговыми строками. Применение этих секций возможно только для БД, управляемых Microsoft SQL Server, так как они предусмотрены стандартами SQL.

На рис. 7.112 представлен пример использования секции COMPUTE для вычисления суммарного, минимального и максимального значений в столбце **AUDITORIUM_CAPACITY** таблицы **AUDITORIUM**. Исходным является результирующий набор, сформированный в любой завершающей секции SELECT-запроса. Секция COMPUTE формирует дополнительный результирующий набор строк, состоящий только из одной строки, столбцы которой содержат итоговые значения. Для вычисления итоговых значений допускается применение только агрегатных функций (табл. 7.2) и не допускается использование псевдонимов столбцов. При выполнении секции формируются столбцы, имена которых задаются автоматически и соответствуют агрегатным функциям, используемым в секции COMPUTE.

```
use BSTU
go
select AUDITORIUM, AUDITORIUM_CAPACITY
from AUDITORIUM where AUDITORIUM_TYPE = 'ЛК-К'
compute sum(AUDITORIUM_CAPACITY), min(AUDITORIUM_CAPACITY),  max(AUDITORIUM_CAPACITY)
```

AUDITORIUM	AUDITORIUM_CAPACITY
114-4	90
137-4	60
305-4	90
313-1	60
324-1	50

sum	min	max
350	50	90

Рис. 7.112. Применение секции COMPUTE

Допускается использование нескольких секций COMPUTE для одного SELECT-запроса (7.113). Каждой секции COMPUTE соответствует свой отдельный результирующий набор строк.

```
use BSTU
go
select AUDITORIUM, AUDITORIUM_CAPACITY
from AUDITORIUM where AUDITORIUM_TYPE = 'ЛК-К'
compute sum(AUDITORIUM_CAPACITY), count(AUDITORIUM_CAPACITY)
compute min(AUDITORIUM_CAPACITY), max(AUDITORIUM_CAPACITY)
compute avg(AUDITORIUM_CAPACITY)
```

AUDITORIUM	AUDITORIUM_CAPACITY
114-4	90
137-4	60
305-4	90
313-1	60
324-1	50

sum	cnt
350	5

min	max
50	90

avg
70

Рис. 7.113. Применение нескольких секций COMPUTE в одном SELECT-запросе

На рис. 7.114 приведен пример применения секции COMPUTE в SELECT-запросе, использующем группировку (GROUP BY).

```
use BSTU
go
select AUDITORIUM_TYPE, count(*) [количество аудиторий данного типа]
from AUDITORIUM
group by AUDITORIUM_TYPE
compute sum(count(*))
```

AUDITORIUM_TYPE	количество аудиторий данного типа
ЛБ-К	5
ЛК	13
ЛК-К	5

sum
23

Рис. 7.114. Совместное применение секций GROUP BY и COMPUTE

Обратите внимание на принцип вычисления суммарного количества строк в секции COMPUTE.

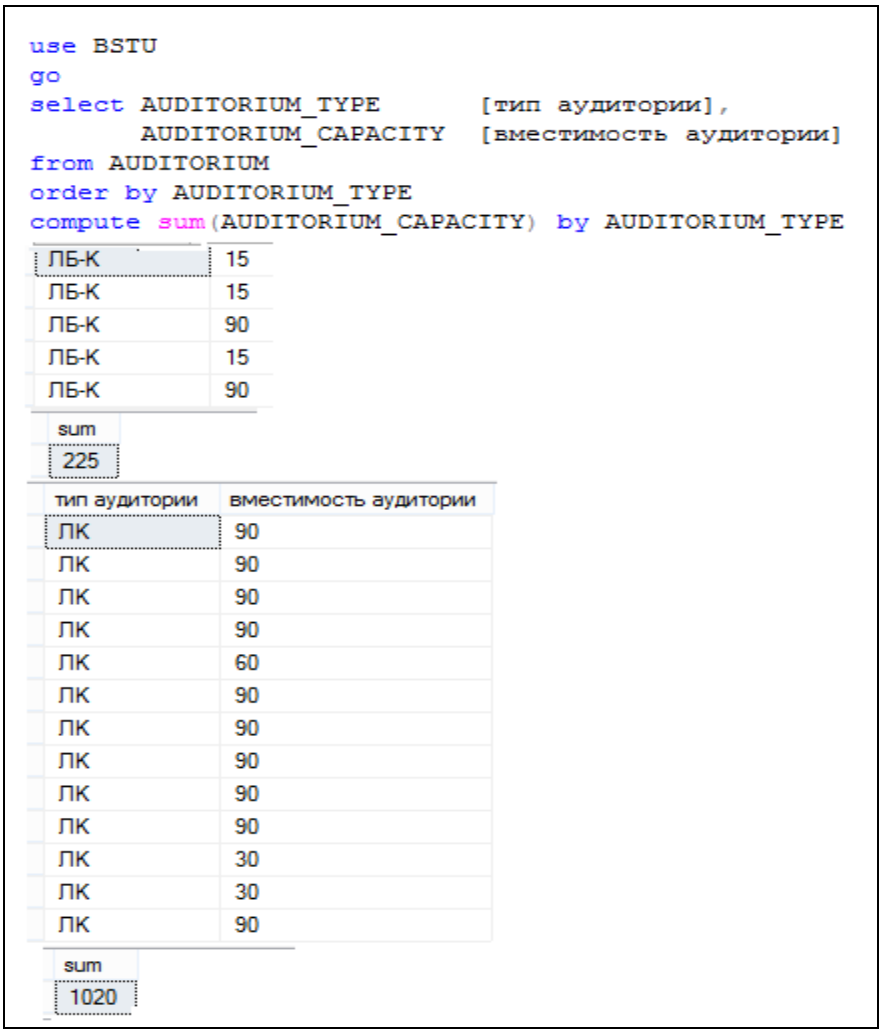
Применение секции COMPUTE BY несколько отличается от применения COMPUTE. Назначение этой секции – вычисление промежуточных итогов. Секция может быть применена только совместно с секцией ORDER BY. Причем за ключевым словом BY секции COMPUTEBY может быть записана такая же последовательность выражений (ASC и DESC не указывается), как

в секции ORDER BY. Если в ORDER BY через запятую указано несколько выражений, то в COMPUTE BY допускается указывать только первые слева, без пропуска.

На рис. 7.115 приведен пример SELECT-запроса с применением секции COMPUTE BY и первые четыре результирующих набора, сформированных этим запросом. При использовании COMPUTE BY формируется несколько результирующих наборов, каждый из которых можно отнести к одному из двух типов.

Первый тип – это набор строк, структура которых формируется на основе SELECT-списка. В набор первого типа включаются строки, имеющие одно и то же значение выражений, указанных за ключевым словом BY секции COMPUTE BY.

Второй тип – это набор, состоящий из одной строки, сформированной по тому же принципу, что и при выполнении секции COMPUTE без BY. Агрегатные функции вычисляются только над строками предшествующего набора первого типа.



```
use BSTU
go
select AUDITORIUM_TYPE       [тип аудитории],
       AUDITORIUM_CAPACITY   [вместимость аудитории]
from AUDITORIUM
order by AUDITORIUM_TYPE
compute sum(AUDITORIUM_CAPACITY) by AUDITORIUM_TYPE
```

ЛБ-К	15
ЛБ-К	15
ЛБ-К	90
ЛБ-К	15
ЛБ-К	90

sum
225

тип аудитории	вместимость аудитории
ЛК	90
ЛК	90
ЛК	90
ЛК	90
ЛК	60
ЛК	90
ЛК	90
ЛК	90
ЛК	90
ЛК	90
ЛК	30
ЛК	30
ЛК	90

sum
1020

Рис. 7.115. Пример использования секции COMPUTE BY

В целом результат выполнения COMPUTE BY формирует последовательность чередующихся результирующих наборов первого и второго типов.

7.8. Конструкция SELECT … INTO

Конструкция SELECT … INTO очень удобна при создании новой таблицы на основе существующей таблицы-образца. В секции INTO указывается имя создаваемой таблицы. Имена и типы данных столбцов новой таблицы будут совпадать с именами (или псевдонимами),

заданными в SELECT-списке оператора. Кроме того, в созданную таблицу будут добавлены строки сформированного оператором SELECT результирующего набора.

На рис. 7.116 представлен сценарий, в котором с помощью первого оператора SELECT создается новая таблица с именем **#TEACHER1** (временная таблица) на основе таблицы **TEACHER**, а с помощью второго – выводятся строки новой таблицы.

```
use BSTU
go
select * into #TEACHER1 from TEACHER
select * from #TEACHER1
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
АКНВЧ	Акунович Станислав Иванович	м	ИСиТ
АРС	Арсентьев Виталий Арсентьевич	м	ПОиСОИ
АТР	Атрощенко Олег Александрович	м	ПУ
БЗБРДВ	Безбородов Владимир Степанович	м	ОХ
БЛВ	Беляев Валерий Павлович	м	ПОиСОИ
БРГ	Бурганская Татьяна Минаевна	м	ЛПиСПС
БРКВЧ	Бракович Андрей Игоревич	м	ИСиТ
БРНВСК	Барановский Станислав Иванович	м	ЭТиМ
БРС	Брусенцова Татьяна Палладьевна	ж	ИСиТ

Рис. 7.116. Создание новой таблицы с помощью конструкции SELECT … INTO

Не сложно убедиться, что в таблицу **#TEACHER1** скопированы все строки таблицы **TEACHER** (рис. 7.117).

```
use BSTU
go
select
 (select count(*) from TEACHER)  [количество строк в TEACHER],
 (select count(*) from #TEACHER1)[количество строк в #TEACHER1]
```

количество строк в TEACHER	количество строк в #TEACHER1
39	39

Рис. 7.117. Количество строк исходной и новой таблиц совпадает

Проанализировав структуры и свойства таблиц **TEACHER** и **#TEACHER1** с помощью уже знакомой нам утилиты SP_HELP, заметим, что структуры (количество столбцов, их имена и типы) таблиц совпадают, ограничений целостности типа DEFAULT, CHECK, PRIMARY KEY и FOREIGN KEY, существующих в таблице **TEACHER,** в **#TEACHER1** нет, но свойство столбцов NOT NULL в новой таблице сохранилось (рис. 7.118, 7.119).

```
use BSTU
go
execute sp_help 'TEACHER'
```

Column_name	Type	Computed	Length	Prec	Scale	Nullable
TEACHER	char	no	10			no
TEACHER_NAME	varchar	no	100			yes
GENDER	char	no	1			yes
PULPIT	char	no	20			yes

index_name	index_description	index_keys
TEACHER_PK	clustered, unique, primary key located on FG1	TEACHER

constraint_type	constraint_name
DEFAULT on column GENDER	DF__TEACHER__GENDER__2AA05119
CHECK on column GENDER	TEACHER_GENDER_CH
PRIMARY KEY (clustered)	TEACHER_PK
FOREIGN KEY	TEACHER_PULPIT_FK

Рис. 7.118. Структура и свойства столбцов таблицы **TEACHER**

```
use TEMPDB
go
execute sp_help '#TEACHER1'
```

Column_name	Type	Computed	Length	Prec	Scale	Nullable
TEACHER	char	no	10			no
TEACHER_NAME	varchar	no	100			yes
GENDER	char	no	1			yes
PULPIT	char	no	20			yes

Рис. 7.119. Структура и свойства столбцов таблицы #**TEACHER1**

На рис. 7.120 демонстрируется более сложный пример применения конструкции SELECT … INTO с использованием псевдонимов столбцов и секции WHERE.

```
use BSTU
go
select TEACHER      [Код], TEACHER_NAME [Преподаватель]
into #TEACHER2   from TEACHER where GENDER = 'ж' and PULPIT = 'ИСиТ'
select * from #TEACHER2
```

Код	Преподаватель
БРС	Брусенцова Татьяна Палладьевна
ЖЛК	Жиляк Надежда Александровна
КШК	Кишкурно Татьяна Вадимовна
МРЗ	Мороз Елена Станиславовна
ПСТВЛВ	Пустовалова Наталия Николаевна

Рис. 7.120. Создание таблицы **#TEACHER2**

Обратите внимание на следующее.

1. В новой таблице только два столбца – столько, сколько указано в SELECT-списке.

2. Имена столбцов в новой таблице совпадают с заданными в SELECT-списке псевдонимами.

3. В новую таблицу добавляются только те строки, которые удовлетворяют условию в секции WHERE.

Глава 8. КОМБИНИРОВАНИЕ РЕЗУЛЬТАТОВ SELECT-ЗАПРОСОВ

8.1. Оператор UNION

Оператор UNION позволяет объединить результирующие наборы, сформированные двумя SELECT-запросами в общий результирующий набор. Единственным требованием, предъявляемым к SELECT-запросам, является совместимость типов соответствующих столбцов сформированных результирующих наборов.

На рис. 8.1 приведен простейший пример использования оператора UNION для объединения результирующих наборов двух SELECT-запросов.

```
select  1 A, 2 B, 3 C, 4 D
union
select  5 E, 6 F, 7 G, 8 H
```

A	B	C	D
1	2	3	4
5	6	7	8

Рис. 8.1. Пример простейшего использования оператора UNION

Каждый оператор SELECT (рис. 8.1) формирует результирующий набор, состоящий из одной строки. Заметим следующее.

1. Результирующий набор оператора UNIONсодержит объединение строк результирующих наборов первого и второго SELECT-запроса.

2. Имена столбцов результирующего набора оператора UNION совпадают с именами столбцов результирующего набора, сформированного первым SELECT-запросом.

3. Количество столбцов результирующих наборов первого и второго SELECT-запросов должно быть одинаковое.

4. Типы соответствующих столбцов должны быть совместимы.

5. Оператор UNION является коммутативным: результат не зависит от порядка объединяемых наборов.

Пример на рис. 8.2 демонстрирует случай, когда типы столбцов двух результирующих наборов разные, но могут быть автоматически преобразованы друг к другу.

Пример на рис. 8.3 демонстрирует, что автоматическое преобразование типов не всегда возможно. Второй результирующий набор содержит строку, значения в которой не могут быть преобразованы к типу INT. Попытка выполнения этого оператора приводит к ошибке.

На рис. 8.4 приведен еще один пример ошибочного применения оператора UNION. Ошибка произошла из-за того, что объединяемые оператором UNION результирующие наборы строк имеют разное количество столбцов.

```
select  1 A, 2 B, 3 C, 4 D
union
select  '5' E, '6' F, '7' G, '8' H
```

A	B	C	D
1	2	3	4
5	6	7	8

Рис. 8.2. Автоматическое преобразование типов в операторе UNION

```
select  1 A, 2 B, 3 C, 4 D
union
select  'E' E, 'F' F, 'G' G, 'H' H

Msg 245, Level 16, State 1, Line 1
Conversion failed when converting the varchar value 'E' to data type int.
```

Рис. 8.3. Сообщение об ошибке при попытке объединения результирующих наборов, содержащих разнотипные соответствующие столбцы, значения которых не могут быть приведены к общему типу

```
select  1 A, 2 B, 3 C, 4 D
union
select  5 E, 6 F, 7 G

Msg 205, Level 16, State 1, Line 1
All queries combined using a UNION, INTERSECT or EXCEPT operator must have
an equal number of expressions in their target lists.
```

Рис. 8.4. Сообщение об ошибке при попытке объединения результирующих наборов с разным количеством столбцов

Оператор UNION выполняет теоретико-множественную операцию объединения, т. е. результатом является множество (в смысле теории множеств) строк, в котором строки не могут повторяться. На рис. 8.5 представлен пример объединения двух одинаковых строк. Обратите внимание на то, что результатом объединения является только одна строка.

```
select  1 A, 2 B, 3 C, 4 D
union
select  1 E, 2 F, 3 G, 4 H
```

A	B	C	D
1	2	3	4

Рис. 8.5. Теоретико-множественное объединение одинаковых строк

Если требуется механическое объединение строк, можно применить оператор UNION ALL. На рис. 8.6 приведен пример механического объединения строк двух результирующих наборов.

```
select  1 A, 2 B, 3 C, 4 D
union all
select  1 E, 2 F, 3 G, 4 H
```

A	B	C	D
1	2	3	4
1	2	3	4

Рис. 8.6. Механическое объединение одинаковых строк

Заметим, что оператор UNION может быть выражен через оператор UNION ALL (рис. 8.7) с помощью опции DISTINCT, примененной к результату объединения результирующих наборов с помощью UNION ALL.

```
select distinct t.* from ( select  1 A, 2 B, 3 C, 4 D
                           union all
                           select  1 E, 2 F, 3 G, 4 H)t
```

A	B	C	D
1	2	3	4

Рис. 8.7. SELECT-запрос, выражающий оператор UNION через UNION ALL

Чаще всего операторы UNION и UNION ALL используются для объединения SELECT-запросов, которые выполняются к разным таблицам. Например, пусть существует таблица **TEACHER_ARC**, имеющая структуру аналогичную таблице TEACHER (рис. 8.8).

```
use BSTU
go
select * from TEACHER_ARC
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
ИВН	Иванов Иван Иванович	NULL	МиЭП
ПТР	Петров Петр Петрович	NULL	ОВ
АКС	Алексеев Алексей Алексеевич	NULL	ОВ
СДР	Сидоров Сидор Сидорович	NULL	ЛПиСПС
ВСЛ	Васильев Василий Васильевич	NULL	ЛПиСПС

Рис. 8.8. Содержимое таблицы TEACHER_ARC

Тогда запрос, осуществляющий поиск строк с первой буквой **A** в столбце **TEACHER_NAME** в таблицах **TEACHER** и **TEACHER_ARC**, может быть записан двумя способами (рис. 8.9 и 8.10).

```
use BSTU
go
select TEACHER      [код],
       TEACHER_NAME [преподаватель],
       PULPIT       [кафедра]   from TEACHER
       where TEACHER_NAME like 'A%'
union all
select TEACHER,
       TEACHER_NAME,
       PULPIT  from TEACHER_ARC
       where TEACHER_NAME like 'A%'
```

код	преподаватель	кафедра
АКНВЧ	Акунович Станислав Иванович	ИСиТ
АРС	Арсентьев Виталий Арсентьевич	ПОиСОИ
АТР	Атрощенко Олег Александрович	ЛУ
АКС	Алексеев Алексей Алексеевич	ОВ

Рис. 8.9. Фильтрация строк в двух таблицах (первый способ)

```
use BSTU
select t.TEACHER      [код],
       t.TEACHER_NAME [преподаватель],
       t.PULPIT       [кафедра]
from  (select * from TEACHER
       union all
       select * from TEACHER_ARC)t
where t.TEACHER_NAME like 'A%'
```

код	преподаватель	кафедра
АКНВЧ	Акунович Станислав Иванович	ИСиТ
АРС	Арсентьев Виталий Арсентьевич	ПОиСОИ
АТР	Атрощенко Олег Александрович	ЛУ
АКС	Алексеев Алексей Алексеевич	ОВ

Рис. 8.10. Фильтрация строк в двух таблицах (второй способ)

С точки зрения производительности, оба запроса (рис. 8.9, 8.10) являются равносильными. Вторая (рис. 8.10) форма записи, с точки зрения автора, является предпочтительней.

Операторы UNION и UNION ALL допускают применение секции ORDER BY. Секция ORDER BY в сценарии на рис. 8.11 сортирует результат объединения двух запросов в порядке убывания значений в столбце **PULPIT**.

```
use BSTU
go
select TEACHER      [код],
       TEACHER_NAME [преподаватель],
       PULPIT       [кафедра]   from TEACHER
       where TEACHER_NAME like 'A%'
union all
select TEACHER,
       TEACHER_NAME,
       PULPIT  from TEACHER_ARC
       where TEACHER_NAME like 'A%'
order by  PULPIT desc
```

код	преподаватель	кафедра
АРС	Арсентьев Виталий Арсентьевич	ПОиСОИ
АКС	Алексеев Алексей Алексеевич	ОВ
АТР	Атрощенко Олег Александрович	ЛУ
АКНВЧ	Акунович Станислав Иванович	ИСиТ

Рис. 8.11. Сортировка результата выполнения оператора UNION ALL

Отметим, что после ключевого слова BY в секции ORDER можно указывать псевдонимы, назначенные в первом SELECT-списке, или просто номер элемента списка (рис. 8.12).

```
use BSTU
go
select TEACHER      [код],
       TEACHER_NAME [преподаватель],
       PULPIT       [кафедра]   from TEACHER
       where TEACHER_NAME like 'A%'
union all
select TEACHER,
       TEACHER_NAME,
       PULPIT  from TEACHER_ARC
       where TEACHER_NAME like 'A%'
order by  3 desc
```

код	преподаватель	кафедра
АРС	Арсентьев Виталий Арсентьевич	ПОиСОИ
АКС	Алексеев Алексей Алексеевич	ОВ
АТР	Атрощенко Олег Александрович	ЛУ
АКНВЧ	Акунович Станислав Иванович	ИСиТ

Рис. 8.12. Сортировка результата выполнения оператора UNION ALL по номеру элемента SELECT-списка

8.2. Оператор INTERSECT

Результатом оператора INTERSECT является набор строк, представляющий собой пересечение (в теоретико-множественном смысле) двух исходных наборов строк. Как и для оператора UNION, к SELECT-запросам предъявляется требование совместимости типов соответствующих столбцов.

На рис. 8.13 представлен простейший пример использования оператора INTERSECT.

```
(select  1 A, 1 B
   union all
 select  2, 2
   union all
 select  2, 2
   union all
 select  3, 3)  intersect (select  1, 2
                             union all
                           select  2, 2
                             union all
                           select  2, 2
                             union all
                           select  3, 2)
```

A	B
2	2

Рис. 8.13. Простейший пример использования оператора INTERSECT

Два исходных набора строк оператора INSTERSECT на рис. 8.13 представлены в виде объединения строк (с помощью операторов UNION), формируемых операторами SELECT.Результатом выполнения оператора INTERSECT является набор, состоящий из одной строки.

Заметим следующее.

1. В исходных наборах строк должно быть одинаковое количество столбцов и совместимый тип соответствующих столбцов.

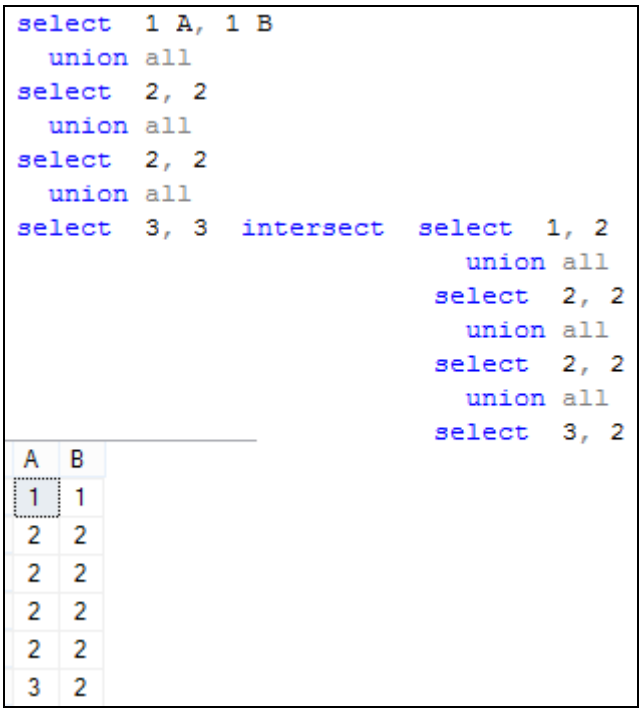
2. Результирующий набор оператора INTERSECT содержит множество строк, являющееся пересечением двух исходных множеств строк. Другими словами, результирующий набор оператора INTERSECT содержит строки входящие одновременно и в первый во второй исходные наборы данных.

3. РезультатомоператораINTERSECTявляется множество неповторяющихся строк. В этом смысле, оператором выполняется теоретико-множественная операция пересечения множеств.

4. Имена и типы столбцов результирующего набора оператора INTERSECTсовпадают с именами и типами столбцов первого результирующего набора.

5. Оператор INTERSECT является коммутативным: результат не зависит от порядка исходных наборов.

Рассмотрим сценарий на рис. 8.14.Его отличие от сценария на рис. 8.13 в отсутствии скобок, регламентирующих порядок выполнения операторов. Заметим: полученный результат отличается от результата на рис. 8.13.



```
select  1 A, 1 B
  union all
select  2, 2
  union all
select  2, 2
  union all
select  3, 3  intersect  select  1, 2
                           union all
                         select  2, 2
                           union all
                         select  2, 2
                           union all
                         select  3, 2
```

A	B
1	1
2	2
2	2
2	2
2	2
3	2

Рис. 8.14. Пример запроса с операторами INTERSECT и UNION

Оператор INTERSECT имеет более высокий приоритет, чем оператор UNION. Поэтому на рис. 8.14 пересечение выполняется для двух наборов строк, каждая из которых состоит только из одной строки.

На рис. 8.15 представлен пример использования оператора INTERSECT, формирующего результирующий набор, являющейся пересечением двух наборов строк, полученных в результате выполнения двух SELECT-запросов. Каждый SELECT-запрос формирует по одному столбцу (строки, состоящие из одного элемента) значений.

```
use BSTU
go
select [SUBJECT] [дисциплина]      -- дисциплины
from PROGRESS                      -- на экзамене
group by [SUBJECT]                 -- средний балл оценок
having  avg(NOTE) > 5              -- по которым > 5
  intersect                     -- пересечь
select [SUBJECT]                   -- все дисциплины
from [SUBJECT] s join PULPIT  p on s.PULPIT = p.PULPIT
               join FACULTY f on f.FACULTY = p.FACULTY
where f.FACULTY = 'ИДиП'           -- на факультете ИДиП
```

дисциплина
БД
КГ
ОАиП

Рис. 8.15. Оператор INTERSECT, формирующий набор строк, являющийся пересечением результатов двух SELECT-запросов

Результатом пересечения является столбец, имя которого заимствовано из первого результирующего набора, для которого в SELECT-запросе указан псевдоним **дисциплина**.

8.3. Оператор EXCEPT

Результатом оператора EXCEPT является набор строк, представляющий собой разность (в теоретико-множественном смысле) двух исходных наборов строк. Как и для операторов UNION и INTERSECT к SELECT-запросам предъявляется требование совместимости типов соответствующих столбцов.

На рис. 8.16 представлен простейший пример использования оператора EXCEPT.

```
(select  1 A, 1 B
  union all
 select  1, 1
  union all
 select  2, 2
  union all
 select  2, 2
  union all
 select  3, 3)  except   (select  1, 2
                            union all
                          select  2, 2
                            union all
                          select  2, 2
                            union all
                          select  3, 2)
```

A	B
1	1
3	3

Рис. 8.16. Оператор EXCEPT, формирующий набор строк, являющийся разностью результатов двух SELECT-запросов

Два исходных набора строк оператора EXCEPT на рис. 8.16 представлены в виде объединения строк (с помощью операторов UNION), формируемых операторами SELECT. Результатом выполнения оператора EXCEPT является набор, состоящий из двух строк.

Заметим следующее.

1. В исходных наборах строк должно быть одинаковое количество столбцов и совместимый тип соответствующих столбцов.
2. Результирующий набор оператора EXCEPT содержит множество строк, являющееся разностью двух исходных множеств строк. Другими словами, результирующий набор оператора EXCEPT содержит строки, которые входят в первый набор строк, но не входят во второй.
3. Результатом оператора EXCEPT является множество неповторяющихся строк. В этом смысле оператором выполняется теоретико-множественная операция вычитания множеств.
4. Имена и типы столбцов результирующего набора оператора EXCEPT совпадают с именами и типами столбцов первого результирующего набора.
5. Оператор EXCEPT не является коммутативным: результат зависит от порядка исходных наборов (рис. 8.17).

```
(select  1 A, 2 B
   union all
 select  2, 2
   union all
 select  2, 2
   union all
 select  3, 2) except (select  1, 1
                        union all
                       select  1, 1
                         union all
                       select  2, 2
                         union all
                       select  2, 2
                         union all
                       select  3, 3)
```

A	B
1	2
3	2

Рис. 8.17. Пример, показывающий не коммутативность оператора EXCEPT

Изменение порядка исходных наборов строк приводит к другому результату оператора EXCEPT (рис.8.17).

Рис. 8.18 представляет пример, демонстрирующий приоритетность операторов UNION, INTERSECT и EXCEPT. В комментариях указывается порядок выполнения операторов. Не сложный анализ результата выполнения сценария позволяет выявить, что наивысшим приоритетом обладает оператор INTERSECT, а операторы UNION и EXCEPT имеют одинаковый приоритет.

```
select  1 A, 1 B
 union all       -- 2
select  2, 2
 except          -- 3
select  1, 1
 union all       -- 4
select  3, 3
 intersect       -- 1
select  2, 2
```

A	B
2	2

Рис. 8.18. Приоритетность операторов UNION, INTERSECT и EXCEPT

```
use BSTU
go
select [SUBJECT]                    -- все дисциплины
from [SUBJECT] s join PULPIT  p on s.PULPIT = p.PULPIT
               join FACULTY f on f.FACULTY = p.FACULTY
where f.FACULTY = 'ИДиП'            -- на факультете ИДиП
   except  --  вычесть
select [SUBJECT] [дисциплина]      -- дисциплины
from PROGRESS                       -- на экзамене
group by [SUBJECT]                  -- средний балл оценок
having  avg(NOTE) > 5               -- по которым > 5
```

SUBJECT
ДМ
ИНФ
КМС
ЛЭВМ
МП
МСОИ
ООП
ОПП
ПЗ
ПИС
ПМАПЛ
ПСП
СУБД

Рис. 8.19. Оператор EXCEPT, формирующий набор строк,
являющийся разностью результатов двух SELECT-запросов

На рис. 8.19 представлен пример использования оператора EXCEPT, формирующего результирующий набор, являющийся разностью двух наборов строк, полученных в результате выполнения двух SELECT-запросов. Каждый SELECT-запрос формирует по одному столбцу значений. Результатом пересечения является столбец, имя которого заимствовано из первого результирующего набора.

Глава 9. ПРИМЕНЕНИЕ ОПЕРАТОРОВ DELETE И TRUNCATE

9.1. Структура оператора DELETE

Оператор DELETE относится к группе операторов DML SQL и предназначен для удаления строк из таблицы.

Уточним описанную ранее (рис. 3.18) структуру оператора DELETE (рис. 9.1).

delete from *целевая таблица*
from *join-выражение*
where *логическое выражение*

Рис. 9.1. Структура оператора DELETE

Синтаксис DELETE предусматривает две секции FROM (обязательной является только первая) и необязательную секцию WHERE. Кроме того, допускается применять секцию OUTPUT и опцию TOP.

Оператор DELETE может удалить ноль, одну или более строк из одной таблицы (целевой таблицы), имя которой указывается в первой секции FROM. Следует отметить, что первое ключевое слов FROM может быть опущено.

Во второй секции FROM может быть указано JOIN-выражение для внутреннего соединения целевой таблицы с другими таблицами. Удалению подлежат только те строки целевой таблицы, для которых выполнилось соединение.

В секции WHERE может быть указано логическое выражение, фильтрующее строки, выбранные второй секцией FROM, подлежащие удалению.

Простейший пример использования оператора DELETE приведен на рис. 9.2.

```
use BSTU
go
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
delete from PROGRESS;  -- удаление всех строк
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
```

количество строк в таблице PROGRESS
34

количество строк в таблице PROGRESS
0

Рис. 9.2. Удаление всех строк таблицы **PROGRESS**

Сценарий включает два оператора SELECT, позволяющих получить количество строк в таблице **PROGRESS** до и после выполнения оператора DELETE. Оператором DELETE используется только единственная (первая) секция FROM, указывающая таблицу (в нашем случае это таблица **PROGRESS**), в которой удаляются строки. Такая форма оператора позволяет удалить все строки таблицы **PROGRESS**.

Заметим, остальные секции (вторая FROM и WHERE) предназначены для выбора строк, подлежащих удалению из таблицы, указанной в первой секции FROM.

9.2. Вторая секция FROM

Вторая секция FROM предназначена для выбора с помощью операции внутреннего соединения строк целевой таблицы, подлежащих удалению.

Прежде чем применим JOIN-соединение в операторе DELETE, выполним сценарий на рис. 9.3, позволяющий предварительно оценить количество отбираемых для удаления этим соединением строк.

```
use BSTU
go
select
(select count(*) from  PROGRESS) [кол. строк в PROGRESS],
(select count(*)
 from  PROGRESS p join STUDENT s on p.IDSTUDENT = s.IDSTUDENT
                  join GROUPS  g on s.IDGROUP   = g.IDGROUP
                                    and g.FACULTY = 'ИДиП') [кол. строк соединилось]
```

кол. строк в PROGRESS	кол. строк соединилось
34	23

Рис. 9.3. Удаление всех строк таблицы **PROGRESS**

Заметим, что количество строк в соединении таблиц **PROGRESS**, **STUDENT** и **GROUPS** составляет 23.

На рис. 9.4 представлен пример оператора DELETE, использующего вторую секцию FROM с JOIN-соединением, определяющим подмножество удаляемых строк таблицы **PROGRESS**. Обратим внимание на то, что JOIN-соединение, указанное в секции FROM, аналогично применяемому в SELECT-запросе сценария на рис. 9.3. Два SELECT-запроса, вычисляющих количество строк в таблице **PROGRESS** до и после выполнения оператора DELETE, позволяют подсчитать, что в результате его выполнения удалено 23 строки.

```
use BSTU
go
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
delete PROGRESS
from  PROGRESS p join STUDENT s on p.IDSTUDENT = s.IDSTUDENT
                 join GROUPS  g on s.IDGROUP   = g.IDGROUP
                                  and FACULTY = 'ИДиП';
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
```

количество строк в таблице PROGRESS
34

количество строк в таблице PROGRESS
11

Рис. 9.4. Применение JOIN-соединения для выбора удаляемых строк из таблицы **PROGRESS**

Обратите внимание: в операторе DELETE (рис. 9.4) опущено ключевое слово FROM для первой секции.

9.3. Секция WHERE

Секция WHERE выполняется после второй секции FROM и содержит логическое выражение для фильтрации ее результирующего набора строк. Если вторая секция FROM не применяется оператором DELETE, то логическое выражение используется для фильтрации строк целевой таблицы. Отобранные таким образом строки удаляются.

На рис. 9.5 представлен пример оператора DELETE, использующего секцию WHERE.

```
use BSTU
go
select count(*) [кол. строк в таблице TEACHER] from TEACHER;
delete TEACHER  where PULPIT = 'ИСиТ'
select count(*) [кол. строк в таблице TEACHER] from TEACHER;
```

кол. строк в таблице TEACHER
39

кол. строк в таблице TEACHER
23

Рис. 9.5. Применение секции WHERE в операторе DELETE

Оператор DELETE (рис. 9.5) удаляет все строки таблицы **TEACHER** со значением **ИСиТ** в столбце **PULPIT**.

Совместное использование секций FROM и WHERE демонстрируется в сценарии на рис. 9.6. Логическое выражение, указанное в секции WHERE, вычисляется для каждой строки результирующего набора, сформированного секцией FROM; все строки таблицы **PROGRESS**, участвующие в JOIN-соединении и для которых логическое выражение принимает значение «истина», удаляются.

```
use BSTU
go
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
delete PROGRESS
from  PROGRESS p join STUDENT s on p.IDSTUDENT = s.IDSTUDENT
                 join GROUPS  g on s.IDGROUP   = g.IDGROUP
where  FACULTY = 'ИДиП';
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
```

количество строк в таблице PROGRESS
34

количество строк в таблице PROGRESS
11

Рис. 9.6. Применение секции WHERE в операторе DELETE

Заметим, что операторы DELETE на рис. 9.4 и 9.6 приводят к одному и тому же результату, но выполняются по-разному. Во втором случае проверка логического условия (в секции WHERE) осуществляется для всех строк сформированного секцией FROM результирующего набора. В первом – результирующий набор секции FROM формируется уже с учетом этого логического условия.

9.4. Опция TOP

Опция TOP в операторе DELETE применяется непосредственно перед самим удалением и предназначена для ограничения количества удаляемых строк. Как и в случае SELECT-оператора, значение в TOP можно указывать в виде абсолютного (рис. 9.7) значения или в процентах (рис. 9.8).

Заметим, что операторы DELETE на рис. 9.7 и 9.8,за исключением опции TOP, полностью повторяют операторы на рис. 9.6 и 9.5 соответственно. Обратите внимание на количество удаленных строк.

```
use BSTU
go
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
delete top(5) PROGRESS
from  PROGRESS p join STUDENT s on p.IDSTUDENT = s.IDSTUDENT
                 join GROUPS  g on s.IDGROUP   = g.IDGROUP
where  FACULTY = 'ИДиП';
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
```

количество строк в таблице PROGRESS
34

количество строк в таблице PROGRESS
29

Рис. 9.7. Применение опции TOP с абсолютным значением в операторе DELETE

```
use BSTU
go
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
delete top (25) percent  from PROGRESS;  -- удаление всех строк
select count(*) [количество строк в таблице PROGRESS] from PROGRESS;
```

количество строк в таблице PROGRESS
34

количество строк в таблице PROGRESS
25

Рис. 9.8. Применение опции TOP со значением, заданным в процентах в операторе DELETE

Следует также отметить, что применение опции TOP в общем случае не позволяет предугадать, какие именно строки будут удалены, так как никакой порядок выбора их в операторе DELETE не гарантируется, а сортировка удаляемых этим оператором строк не предусмотрена. Как правило, эту секцию используют при отладке приложений.

9.5. Секция OUTPUT

В главе 6 уже пояснялось назначение секции OUTPUT в DML-операторах, предназначенных для изменения таблиц БД. При этом отмечалось, что действие оператора отката транзакции (ROLLBACK) не оказывает влияние на формирование этой секцией результирующего набора.

На рис. 9.9 представлен пример использования секции OUTPUT в операторе DELETE, осуществляющем вывод своего результирующего набора в стандартный выходной поток. Обратите внимание в секции OUTPUT на два момента: запись **DELETED.*** в секции OUTPUT, позволяющую вывести значения всех столбцов удаляемых строк; возможность указывать псевдонимы столбцов.

```
use BSTU
go
delete TEACHER
output 'DELETE' [оператор], sysdatetime() [дата удаления], deleted.*
where PULPIT = 'ПОиСОИ'
```

оператор	дата удаления	TEACHER	TEACHER_NAME	GENDER	PULPIT
DELETE	2014-02-16 21:47:52.0866015	АРС	Арсентьев Виталий Арсентьевич	м	ПОиСОИ
DELETE	2014-02-16 21:47:52.0866015	БЛВ	Беляев Валерий Павлович	м	ПОиСОИ
DELETE	2014-02-16 21:47:52.0866015	БРТШВЧ	Барташевич Святослав Александрович	м	ПОиСОИ
DELETE	2014-02-16 21:47:52.0866015	ШМК	Шмаков Михаил Сергеевич	м	ПОиСОИ
DELETE	2014-02-16 21:47:52.0866015	ЮДНКВ	Юденков Виктор Степанович	м	ПОиСОИ

Рис. 9.9. Применение секции OUTPUT в операторе DELETE
для вывода результирующего набора в стандартный поток

На рис. 9.10 представлен сценарий, в котором с помощью секции OUTPUT оператора DELETE во временную таблицу выводится информация об удаляемых строках таблицы **PROGRESS**.

```
use BSTU
go
select top(0)
      '1234567890' [оператор],
      sysdatetime()[дата и время],
      * into #LOG_PROGRESS from PROGRESS; -- создание временной таблицы

 delete PROGRESS
 output 'DELETE' [оператор], sysdatetime() [дата удаления], deleted.* into #LOG_PROGRESS
 from  PROGRESS p join STUDENT s on p.IDSTUDENT = s.IDSTUDENT
                  join GROUPS  g on s.IDGROUP   = g.IDGROUP
 where g.FACULTY = 'ИЭФ';
 select * from  #LOG_PROGRESS;
```

оператор	дата и время	SUBJECT	IDSTUDENT	PDATE	NOTE
DELETE	2014-02-16 23:33:46.6569140	ЭТ	1155	2013-01-15	7
DELETE	2014-02-16 23:33:46.6569140	ЭТ	1156	2013-01-15	8
DELETE	2014-02-16 23:33:46.6569140	ЭТ	1157	2013-01-15	9
DELETE	2014-02-16 23:33:46.6569140	ЭТ	1158	2013-01-15	4
DELETE	2014-02-16 23:33:46.6569140	ЭТ	1159	2013-01-15	5

Рис. 9.10. Применение секции OUTPUT в операторе DELETE
для вывода результирующего набора во временную таблицу

9.6. Оператор TRUNCATE TABLE и его применение

Оператор TRUNCATE TABLE предназначен для удаления всех строк таблицы. В этом смысле действие оператора TRUNCATE TABLE очень похоже на действие оператора DELETE, в котором применяется только первая секция FROM. Но принцип его работы сильно отличается.

Выполнение DELETE осуществляется построчно, с регистрацией каждой операции удаления в журнале транзакции. В отличии от DELETE, оператор TRUNCATE TABLE удаляет все строки одномоментно, а в журнале регистрируется только одна операция. Оператор TRUNCATE TABLE выполняется значительно быстрее равноценного оператора DELETE.

На рис. 9.11 приведен пример использования оператора TRUNCATE TABLE для удаления всех строк таблицы **TEACHER**.

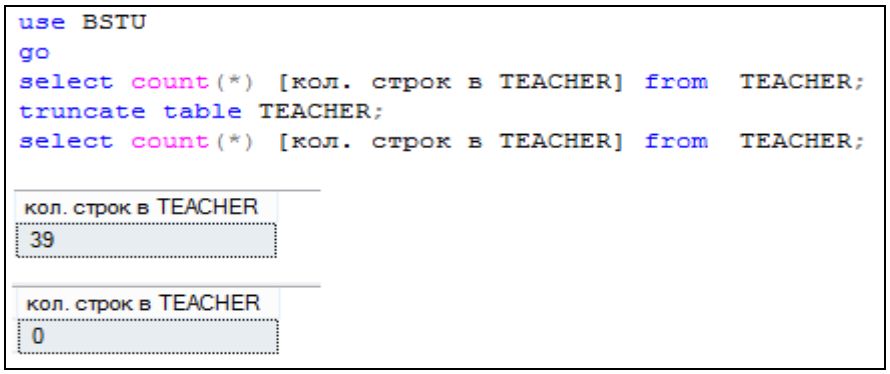


Рис. 9.11. Пример применения оператора TRUNCATE TABLE для удаления строк в таблице **TEACHER**

Глава 10. ПРИМЕНЕНИЕ ОПЕРАТОРА UPDATE

10.1. Структура оператора UPDATE

Оператор UPDATE относится к группе операторов DML SQL и предназначен для изменения строк таблицы. Уточним описанную ранее (см. рис. 3.20 на с. 45) структуру оператора UPDATE (рис. 10.1).

update *целевая таблица*
set *set-выражения*
from *join-выражение*
where *логическое выражение*

Рис. 10.1. Структура оператора UPDATE

Синтаксис UPDATE предусматривает обязательную секцию SET и две необязательные секции FROM и WHERE. Кроме того, допускается применять секцию OUTPUT и опцию TOP.

Логика действия оператора UPDATE похожа на логику оператора DELETE. Оператор UPDATE может изменить ноль, одну или более строк в одной таблице (в целевой таблице), имя которой указывается сразу за ключевым словом UPDATE.

В секции FROM может быть указано JOIN-выражение для внутреннего соединения целевой таблицы с другими таблицами. Изменению подлежат только те строки целевой таблицы, для которых выполнилось соединение. Если секции FROM нет, то предполагается, что в этой секции выбираются все строки таблицы.

В секции WHERE может быть указано логическое выражение, фильтрующее строки, выбранные секцией FROM и подлежащие изменению.

Простейший пример использования оператора UPDATE приведен на рис. 10.2.

```
use BSTU
go
select top(5) * from AUDITORIUM;
update AUDITORIUM set AUDITORIUM_CAPACITY = 100;
select top(5) * from AUDITORIUM;
```

AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
02Б-4	ЛК	90	02Б-4
103-4	ЛК	90	103-4
105-4	ЛК	90	105-4
107-4	ЛК	90	107-4
110-4	ЛК	30	110-4

AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
02Б-4	ЛК	100	02Б-4
103-4	ЛК	100	103-4
105-4	ЛК	100	105-4
107-4	ЛК	100	107-4
110-4	ЛК	100	110-4

Рис. 10.2. Изменение значений в столбце **AUDUTORIUM_CAPACITY** всех строк таблицы **AUDITORIUM**

Сценарий включает два оператора SELECT, позволяющих просмотреть первых пять строк (для экономии места на рисунке) таблицы **AUDITORIUM** до и после выполнения оператора UPDATE. Оператором UPDATE применяется только секция SET, содержащая выражение, интерпретируемое как изменение значения в столбце **AUDITORUM_CAPACITY** на константу 100. Такая форма оператора позволяет изменить все строки таблицы **AUDITORIUM**. В результате выполнения оператора во всех строках столбца **AUDITORUM_CAPACITY** таблицы **AUDITORIUM** будет значение 100.

10.2. Секция SET

Секция SET является обязательной. В ней могут быть записаны выражения, состоящие из трех частей (слева направо): имя столбца, изменяемой таблицы; символ операции (табл. 10.1) и выражение, которое может быть интерпретировано как значение.

Таблица 10.1

Символы операций, применяемых в секции SET

Операция	Назначение
=	Присвоить значение
=default	Присвоить значение по умолчанию
+=	Присвоить сумму (конкатенацию для символьных данных)
-=	Присвоить разницу
*=	Присвоить произведение
/=	Присвоить частное
%=	Присвоить остаток от деления
&=	Присвоить результат побитового логического AND
^=	Присвоить результат побитового логического XOR
|=	Присвоить результат побитового логического OR

На рис. 10.3 приведен пример оператора UPDATE, в секции SET которого содержатся два разделенных запятой выражения. Первое выражение интерпретируется как запись в столбец **AUDITORIUM_CAPACITY** произведения значения столбца **AUDITORIUM_CAPACITY** на число 1.2. Второе – как запись в столбец **AUDITORIUM_NAME** результата конкатенации значения столбца **AUDITORIUM_NAME** со строкой **+20%**.

```
use BSTU
go
select top(3) * from AUDITORIUM;
update AUDITORIUM set AUDITORIUM_CAPACITY *= 1.2,
                      AUDITORIUM_NAME += ' +20%';
select top(3) * from AUDITORIUM;
```

AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
02Б-4	ЛК	90	02Б-4
103-4	ЛК	90	103-4
105-4	ЛК	90	105-4

AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
02Б-4	ЛК	108	02Б-4 +20%
103-4	ЛК	108	103-4 +20%
105-4	ЛК	108	105-4 +20%

Рис. 10.3. Изменение значений в столбцах **AUDUTORIUM_CAPACITY** и **AUDITORIUM_NAME** во всех строках таблицы **AUDITORIUM**

```
use BSTU
go
select *  from TEACHER
update TEACHER set  TEACHER_NAME += (' '+GENDER),
                    PULPIT = default, GENDER = default;
select *  from TEACHER;
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
ГРН	Гурин Николай Иванович	м	ИСиТ
ДЛН	Долинина Татьяна Николаевна	ж	СБУАиА
ДМДК	Демидко Марина Николаевна	м	ЛПиСПС
ДТК	Дятко Александр Аркадьевич	м	ИСиТ
ЕЩНК	Ещенко Людмила Семеновна	ж	ТНВиО...
ЖЛК	Жиляк Надежда Александровна	ж	ИСиТ

TEACHER	TEACHER_NAME	GENDER	PULPIT
ГРН	Гурин Николай Иванович м	м	NULL
ДЛН	Долинина Татьяна Николаевна ж	м	NULL
ДМДК	Демидко Марина Николаевна м	м	NULL
ДТК	Дятко Александр Аркадьевич м	м	NULL
ЕЩНК	Ещенко Людмила Семеновна ж	м	NULL
ЖЛК	Жиляк Надежда Александровна ж	м	NULL

Рис. 10.4. Применение ключевого слова DEFAULT для изменения значений в столбцах **GENDER** и **PULPIT** таблицы **TEACHER**

Пример на рис. 10.4 демонстрирует операцию присвоения значения по умолчанию для столбцов **PULPIT** и **GENDER** таблицы **TEACHER** (рис. 10.5).

```
use BSTU
go
exec sp_help 'TEACHER'
```

Column_name	Type	Computed	Length	Prec	Scale	Nullable
TEACHER	char	no	10			no
TEACHER_NAME	varchar	no	100			yes
GENDER	char	no	1			yes
PULPIT	char	no	20			yes

constraint_type	constraint_name	delete_action	upda...	status_...	status_for...	constraint_keys
DEFAULT on column GENDER	DF__TEACHER__GENDER__2AA05119	(n/a)	(n/a)	(n/a)	(n/a)	('м')
CHECK on column GENDER	TEACHER_GENDER_CH	(n/a)	(n/a)	Enabled	Is_For_R...	([GENDER]='ж' OR
PRIMARY KEY (clustered)	TEACHER_PK	(n/a)	(n/a)	(n/a)	(n/a)	TEACHER
FOREIGN KEY	TEACHER_PULPIT_FK	No Action	No ...	Enabled	Is_For_R...	PULPIT
						REFERENCES BST

Рис. 10.5. Информация о структуре и свойствах таблицы **TEACHER**

10.3. Секция FROM

Принцип применения секции FROM оператора UPDATE аналогичен принципу применения второй секции FROM оператора DELETE: строки целевой таблицы, связанные JOIN-соединением с другими таблицами, являются исходными для выполнения изменений, определенных выражениями в секции SET. В том случае, если в операторе UPDATE секция FROM не используется, это равносильно секции FROM, в которой указывается только одна целевая таблица.

На рис. 10.6, 10.7 приведен пример, демонстрирующий применение секции FROM оператора UPDATE.

Вначале (рис. 10.6) с помощью конструкции SELECT … INTO создается временная таблица с именем **#Механики_с_оценкой<10**, которая заполняется строками, сформированными в результате многотабличного SELECT-запроса с последующей WHERE-фильтрацией. Второй оператор SELECT выводит значения в столбце **NOTE** таблицы **PROGRESS** из строк, связанных JOIN-соединением с строками временной таблицы.

Затем (рис. 10.7) выполняется оператор UPDATE, использующий в секции FROM JOIN-соединение с временной таблицей, аналогичное применяемому во втором SELECT-запросе на рис. 10.6. Выражение в секции SET увеличивает значение в столбце **NOTE** на 1. Второй оператор SELECT на рис. 10.7 совпадает со вторым SELECT-оператором на рис. 10.5 и демонстрирует результат успешного выполнения оператора UPDATE.

```
use BSTU
go
select distinct pr.IDSTUDENT, pr.[SUBJECT], pr.PDATE
into [#Механики_с_оценкой_<_10]
from STUDENT s join GROUPS g      on s.IDGROUP = g.IDGROUP
               join PROGRESS pr   on pr.IDSTUDENT = s.IDSTUDENT
               join PROFESSION p on p.PROFESSION = g.PROFESSION
where p.QUALIFICATION like '%механик%' and pr.NOTE < 10;

select  t.IDSTUDENT, t.[SUBJECT], t.PDATE, pr.NOTE
from PROGRESS pr join [#Механики_с_оценкой_<_10] t
                 on pr.IDSTUDENT = t.IDSTUDENT and
                    pr.[SUBJECT] = t.[SUBJECT] and
                    pr.PDATE = t.PDATE
```

IDSTUDENT	SUBJECT	PDATE	NOTE
1064	ОХ	2013-01-19	6
1065	ОХ	2013-01-19	4
1066	ОХ	2013-01-19	9
1067	ОХ	2013-01-19	5
1068	ОХ	2013-01-19	8
1069	ОХ	2013-01-19	4

Рис. 10.6. Формирование временной таблицы, для применения в секции FROM оператора UPDATE

```
use BSTU
go
update  PROGRESS set NOTE+=1
from PROGRESS pr join [#Механики_с_оценкой_<_10] t
                 on pr.IDSTUDENT = t.IDSTUDENT and
                    pr.[SUBJECT] = t.[SUBJECT] and
                    pr.PDATE = t.PDATE;

select  t.IDSTUDENT, t.[SUBJECT], t.PDATE, pr.NOTE
from PROGRESS pr join [#Механики_с_оценкой_<_10] t
                 on pr.IDSTUDENT = t.IDSTUDENT and
                    pr.[SUBJECT] = t.[SUBJECT] and
                    pr.PDATE = t.PDATE;
```

IDSTUDENT	SUBJECT	PDATE	NOTE
1064	ОХ	2013-01-19	7
1065	ОХ	2013-01-19	5
1066	ОХ	2013-01-19	10
1067	ОХ	2013-01-19	6
1068	ОХ	2013-01-19	9
1069	ОХ	2013-01-19	5

Рис. 10.7. Применение секции FROM оператора UPDATE

10.4. Секция WHERE

Секция WHERE выполняется после секции FROM и содержит логическое выражение для фильтрации ее результирующего набора строк. Если секция FROM не применяется, то

логическое выражение используется для фильтрации строк целевой таблицы. Отобранные таким образом строки изменяются в соответствии с выражениями в секции SET.

На рис. 10.8 представлен пример оператора UPDATE, использующего секцию WHERE. При этом в секции WHERE применяется коррелированный подзапрос к временной таблице, созданной в сценарии, показанном на рис. 10.6. Псевдоним целевой таблицы для применения его в подзапросе можно задать в секции FROM (рис. 10.9).

```
use BSTU
go
update PROGRESS
set NOTE +=1
where exists (select * from [#Механики_с_оценкой_<_10] t
              where PROGRESS.IDSTUDENT = t.IDSTUDENT and
                    PROGRESS.[SUBJECT] = t.[SUBJECT] and
                    PROGRESS.PDATE = t.PDATE);

select  pr.IDSTUDENT, pr.[SUBJECT], pr.PDATE, pr.NOTE
from PROGRESS pr
where exists (select * from [#Механики_с_оценкой_<_10] t
              where pr.IDSTUDENT = t.IDSTUDENT and
                    pr.[SUBJECT] = t.[SUBJECT] and
                    pr.PDATE = t.PDATE);
```

IDSTUDENT	SUBJECT	PDATE	NOTE
1064	ОХ	2013-01-19	7
1065	ОХ	2013-01-19	5
1066	ОХ	2013-01-19	10
1067	ОХ	2013-01-19	6
1068	ОХ	2013-01-19	9
1069	ОХ	2013-01-19	5

Рис. 10.8. Применение секции WHERE с коррелируемым подзапросом в операторе UPDATE

```
use BSTU
go
update PROGRESS
set NOTE +=1
from PROGRESS pr
where exists (select * from [#Механики_с_оценкой_<_10] t
              where pr.IDSTUDENT = t.IDSTUDENT and
                    pr.[SUBJECT] = t.[SUBJECT] and
                    pr.PDATE = t.PDATE);
```

Рис. 10.9. Применение псевдонима целевой таблицы в секции WHERE оператора UPDATE

Обратите внимание на то, что результат выполнения операторов UPDATE, представленных на рис. 10.8 и 10.9 аналогичен результату выполнения оператора UPDATE на рис. 10.7.

Совместное применение секций FROM и HERE (рис. 10.10) позволяет избежать создания временной таблицы (рис. 10.6, 10.7) и получить аналогичный результат.

```
use BSTU
go
update PROGRESS set NOTE+=1
from STUDENT s join GROUPS g      on s.IDGROUP = g.IDGROUP
               join PROGRESS pr   on pr.IDSTUDENT = s.IDSTUDENT
               join PROFESSION p on p.PROFESSION = g.PROFESSION
where p.QUALIFICATION like '%механик%' and pr.NOTE < 10;
```

Рис. 10.10. Совместное использование секций FROM и WHERE оператора UPDATE

10.5. Опция TOP

Как и в других DML-операторах, опция TOP позволяет ограничить количество строк, на которое распространяется действие оператора. На рис. 10.11 приведен пример оператора UPDATE, изменяющего только три первые, выбранные с помощью секций FROM и WHERE, строки.

```
use BSTU
go
update top(3) PROGRESS set NOTE+=1
from STUDENT s join GROUPS g      on s.IDGROUP = g.IDGROUP
               join PROGRESS pr   on pr.IDSTUDENT = s.IDSTUDENT
               join PROFESSION p on p.PROFESSION = g.PROFESSION
where p.QUALIFICATION like '%механик%' and pr.NOTE < 10;

select  pr.IDSTUDENT, pr.[SUBJECT], pr.PDATE, pr.NOTE
from STUDENT s join GROUPS g      on s.IDGROUP = g.IDGROUP
               join PROGRESS pr   on pr.IDSTUDENT = s.IDSTUDENT
               join PROFESSION p on p.PROFESSION = g.PROFESSION
where p.QUALIFICATION like '%механик%'
```

IDSTUDENT	SUBJECT	PDATE	NOTE
1064	ОХ	2013-01-19	7
1065	ОХ	2013-01-19	5
1066	ОХ	2013-01-19	10
1067	ОХ	2013-01-19	5
1068	ОХ	2013-01-19	8
1069	ОХ	2013-01-19	4

Рис. 10.11. Применение опции TOP оператора UPDATE

Как и в случае с оператором DELETE, количество строк может быть задано в процентах. Кроме того, следует помнить, что в общем случае невозможно заранее предугадать, какие строки будут выбраны для изменения.

10.6. Секция OUTPUT

На рис. 10.12 и 10.13 приведены примеры использования секции OUTPUT для формирования результирующего набора в стандартный поток вывода (рис. 10.12) и во временную таблицу (рис. 10.13).

```
use BSTU
go

update AUDITORIUM set AUDITORIUM_CAPACITY*=1.2
output 'UPDATE' [операция], deleted.AUDITORIUM_CAPACITY [до],
                 inserted.AUDITORIUM_CAPACITY [после]
where AUDITORIUM_TYPE = 'ЛБ-К';
```

операция	до	после
UPDATE	15	18
UPDATE	15	18
UPDATE	90	108
UPDATE	15	18
UPDATE	90	108

Рис. 10.12. Применение секции OUTPUT оператора UPDATE

```
use BSTU
go
select top(0) 'XXXXXXXX' [операция],
              AUDITORIUM [аудитория] ,
              AUDITORIUM_CAPACITY [вместимость до],
              AUDITORIUM_CAPACITY [вместимость после]
into [#Регистрация изменений] -- создать таблицу
from AUDITORIUM;

update AUDITORIUM set AUDITORIUM_CAPACITY*=1.2
output 'UPDATE' [операция], inserted.AUDITORIUM [аудитория],
                deleted.AUDITORIUM_CAPACITY [до],
                inserted.AUDITORIUM_CAPACITY [после] into [#Регистрация изменений]
where AUDITORIUM_TYPE = 'ЛБ-К';
select * from [#Регистрация изменений];
```

операция	аудитория	вместимость до	вместимость после
UPDATE	206-1	15	18
UPDATE	301-1	15	18
UPDATE	304-4	90	108
UPDATE	413-1	15	18
UPDATE	423-1	90	108

Рис. 10.13. Применение секции OUTPUT оператора UPDATE с выводом результирующего набора во временную таблицу

Применение секции OUTPUT аналогично ее применению в других DML-операторах. Единственное отличие заключается в возможности использования префиксов INSERTED и DELETED (рис. 10.12. 10.13) в выражениях секции. С помощью INSERTED можно вывести в результирующий набор новые значения (измененные) в столбцах таблицы, а с помощью DELETED – старые (до изменения).

Глава 11. ПРИМЕНЕНИЕ ИНДЕКСОВ

11.1. Назначение и применение индексов

Обработка SQL-запроса, поступившего серверу MSS, проходит в несколько этапов: разбор, разрешение имен, оптимизация, компиляция и выполнение (рис. 11.1).

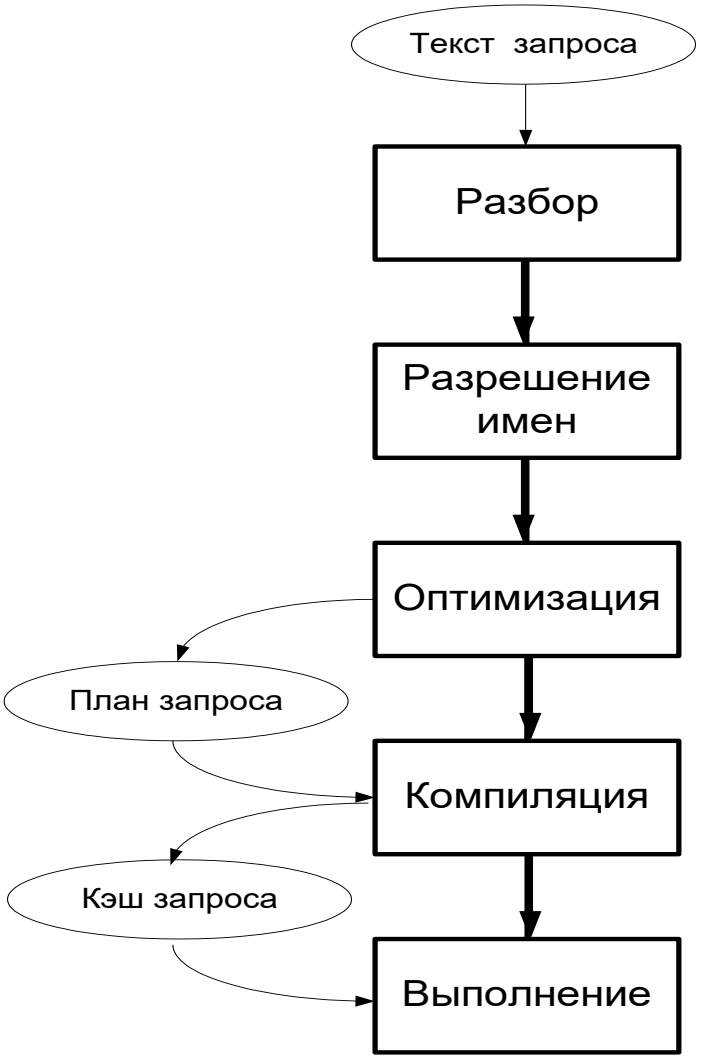


Рис. 11.1. Этапы обработки SQL-запроса

Обработка любого SQL-запроса начинается с синтаксического разбора текста запроса для проверки его на соответствие правилам языка. На этапе разрешения имен осуществляется проверка наличия (возможность доступа) используемых в запросе объектов БД (таблиц, столбцов таблиц, пользовательских и встроенных функций и пр.).

Запрос, прошедший этапы разбора и разрешения имен, поступает на обработку специальной компоненте сервера, называемой оптимизатором. Основная задача оптимизатора – построение плана запроса. План запроса представляет собой алгоритм выполнения SQL-запроса. На каждом шаге алгоритма выполняется элементарное действие сервера СУБД. План любого запроса может быть получен с помощью MSMS. На рис. 11.2 представлен пример графического изображения плана запроса, полученного с помощью MSMS.

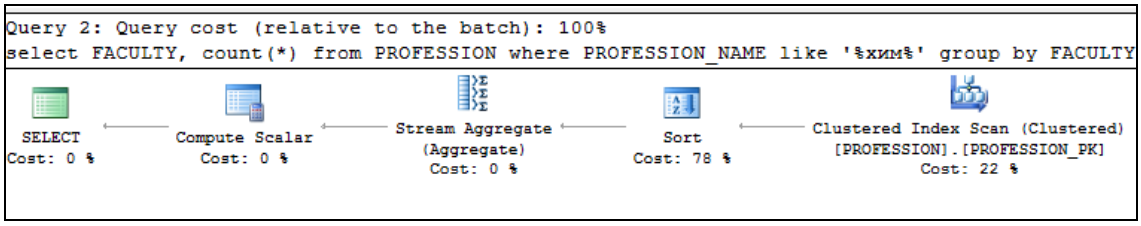


Рис. 11.2. Графическое представление плана запроса

При построении плана запроса для каждого шага вычисляется числовая величина, пропорциональная продолжительности выполнения шага, называемая стоимостью. Суммарная

стоимость шагов плана составляет стоимость всего запроса и, соответственно, является величиной, пропорциональной продолжительности выполнения запроса. На рис. 11.2 показано процентное распределение стоимости запроса (Cost) по шагам плана запроса.

Критерием оптимизации при построении плана является минимизация общей стоимости запроса. Оптимизатор, формирующий план запроса, учитывает статистику, собираемую сервером СУБД, а также наличие специальных объектов БД, предназначенных для ускорения выполнения запросов и называемых индексами.

После построения плана запроса осуществляется его компиляция. Откомпилированный запрос (точнее план запроса), помещается в специальную область памяти, называемую библиотечным кэшем. Кэш потом используется для ускорения выполнения будущих аналогичных запросов. Откомпилированный план выполняется (точнее интерпретируется) сервером СУБД.

Индекс – это объект БД, позволяющий ускорить поиск в определенной таблице. Для получения данных из таблицы наличие индексов не является обязательным. Если нет подходящего индекса, оптимизатор запланирует простой последовательный перебор всех строк таблицы (говорят «просканирует таблицу») для поиска необходимых данных. Иначе говоря, если индекс не позволяет ускорить процесс поиска данных – он является бесполезным.

Как и любой другой объект базы данных, индекс может быть создан с помощью DDL-оператора CREATE, модифицирован с помощью ALTER и удален с помощью оператора DROP. Для одной таблицы может быть построено несколько индексов.

Разработчик SQL-запросов, не выходящий за рамки стандартного SQL, никак непосредственно не может повлиять на процесс формирования оптимизатором плана запроса. Однако существуют специальные средства, выходящие за рамки стандартного языка SQL, позволяющие «подсказать» оптимизатору о необходимости применения индекса. В русскоязычной литературе сложился термин «хинт» (от англ. hint – подсказка) для обозначения инструкций-подсказок, позволяющих повлиять на работу оптимизатора. Применение хинтов выходит за рамки этого пособия, с их перечнем и принципами применения можно ознакомиться в издании [5].

Структура, используемая MSS для хранения индексов в БД, представляет собой сбалансированное дерево, узлами которого являются страницы – стандартные блоки данных размером 8К, используемые MSS для хранения информации. Более подробно с внутренней организацией индексов можно ознакомиться в пособии [3].

MSS поддерживает несколько типов индексов: кластеризованные и некластеризованные индексы, уникальные и неуникальные, XML-индексы, пространственные индексы, а также полнотекстовые индексы.

Пространственные индексы и полнотекстовые индексы в этом пособии не рассматриваются. Для знакомства с этими типами индексов рекомендуется изучить пособия [3, 5].

Индексы XML будут рассматриваться позже в главе, посвященной применению XML в БД MSS.

Наиболее часто применяются кластеризованные и некластеризованные индексы, которые могут быть уникальными и неуникальными.

11.2. Кластеризованные индексы

Обычно кластеризованные индексы формируются автоматически при создании таблицы. Дело в том, что наличие первичного ключа (ограничение PRIMARY KEY) в таблице БД влечет автоматическое создание индекса при создании таблицы. Если при этом в свойствах столбца первичного ключа специально не указано ключевое слово NONCLUSTERED, то автоматически создастся кластеризованный индекс.

С помощью системной процедуры SP_HELPINDEX можно получить перечень индексов, связанных с заданной (параметр процедуры) таблицей. Сценарий, представленный на рис. 11.3, демонстрирует вызов процедуры SP_HELPINDEX, формирующей перечень индексов, связанных с таблицей **TEACHER**.

```
use BSTU
go
exec SP_HELPINDEX 'TEACHER'
```

index_name	index_description	index_keys
TEACHER_PK	clustered, unique, primary key located on FG1	TEACHER

Рис. 11.3. Применение системной процедуры SP_HELPINDEX
для получения перечня индексов, связанных с таблицей **TEACHER**

Каждая строка результирующего набора, сформированного процедурой SP_HELPINDEX, соответствует одному индексу. В столбце **index_name** содержатся имена индексов (на рис. 11.3 – **TEACHER_PK**), в столбце **index_description** – описание индексов (кластеризованный, уникальный, созданный для первичного ключа, расположен в файловой группе **FG1**), в столбце **index_keys** перечислены имена столбцов (**TEACHER**) таблицы, по значениям которых осуществляется построение соответствующего индекса (говорят «индексируемые столбцы»).

Основной особенностью кластеризованного индекса является то, что при его построении таблица становится частью индекса. Другими словами, строки таблиц, имеющих кластеризованные индексы, физически упорядочены в соответствии со значениями индексируемых столбцов. В связи с этим в таблице может быть только один кластеризованный индекс. На рис. 11.4 представлен сценарий, состоящий из трех пакетов (пакет – группа операторов T-SQL, завершающаяся оператором GO). В первом пакете создается временная таблица **#EXPLORE.** Операторы второго пакета добавляют в созданную таблицу 10 000 строк. При этом в столбец **TKEY** с помощью встроенной функции **RAND** вводится случайное числовое значение. Наконец, третий пакет выводит количество строк в таблице **#EXPLORE.**

```
create table #EXPLORE
(
 TKEY   int,
 CC     int identity(1,1),
 TFIELD varchar(100)
);
go
-- добавление в таблицу 10000 строк
set nocount on;      --  не выводить сообщения о вводе строк
declare @i int = 0;
while   @i < 10000
begin
  insert #EXPLORE(TKEY, TFIELD) values(floor(30000*RAND()), replicate('строка ', 10));
  if (@i%100 = 0) print @i;  -- вывести сообщение
  set @i = @i+1;
end;
go
select count(*)[количество строк] from #EXPLORE;
```

количество строк
10000

Рис. 11.4. Создание временной таблицы **#EXPLORE** и добавление в нее 10 000 строк

Заметим: таблица **#EXPLORE** не имеет индексов. Поэтому SELECT-запрос, выполняющий поиск и сортировку строк, с заданным диапазоном значений столбца **TKEY** (рис. 11.5), приводит к полному сканированию таблицы (компонент Table Scan плана запроса) **#EXPLORE** и последующей сортировке (компонент Sort). Общая стоимость запроса (Estimate Subtree Cost) составляет 0,174. При этом на сканирование таблицы пришлось 58% стоимости, на сортировку – 42%.

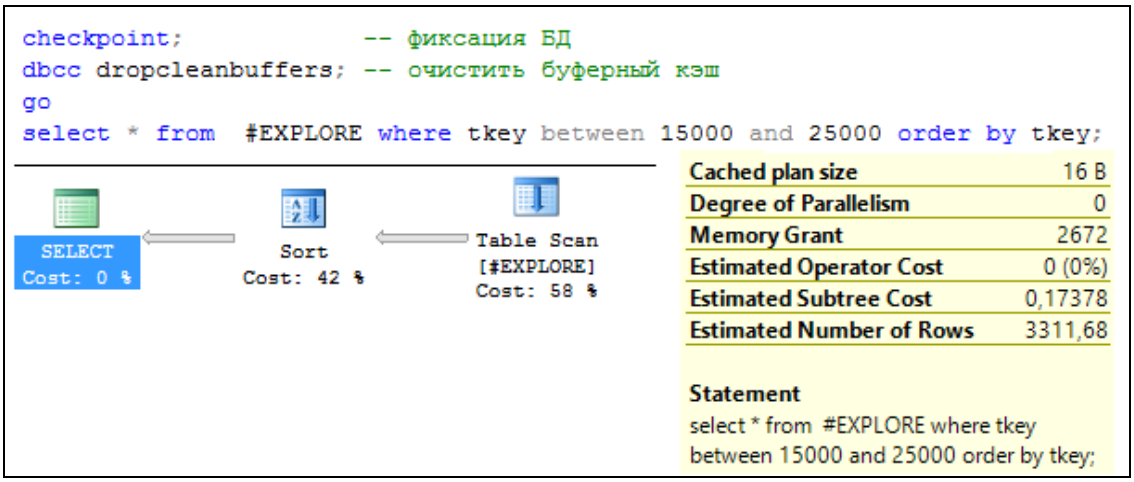


Рис. 11.5. План SELECT-запроса к таблице, не имеющей кластерного индекса

Кратко поясним назначение пакета (рис. 11.5), предшествующего исследуемому оператору SELECT.

Оператор CHECKPOINT языка T-SQL позволяет записать образы страниц из буферного кэша в файлы БД. Сразу после выполнения этого оператора все образы страниц, расположенные в буферном кэше сервера, и соответствующие им оригинальные страницы в файлах БД совпадают.

Оператор DBCC DROPCLEANBUFFER позволяет очистить буферный кэш. Сразу после выполнения этого оператора буферный кэш не содержит никаких образов страниц, и поэтому следующая DML-операция приведет в физическому чтению страниц из файлов БД.

Совместное применение этих двух операторов позволяет устранить эффект, возникающий из-за кэширования страниц, что позволяет более объективно сравнивать время (если оно измеряется) выполнения нескольких запросов. Отметим при этом, что очистка буферного кэша никак не влияет на величину стоимости запроса.

В сценарии на рис. 11.6 создается кластерный индекс (второй пакет) с именем **#EXPLORE_CLU** по столбцу **TKEY** таблицы **#EXPLORE**. Ключевое слова ASC (по возрастанию) указывает порядок, в котором строится индекс. В третьем пакете сценария выполняется SELECT-запрос, аналогичный запросу на рис. 11.5.

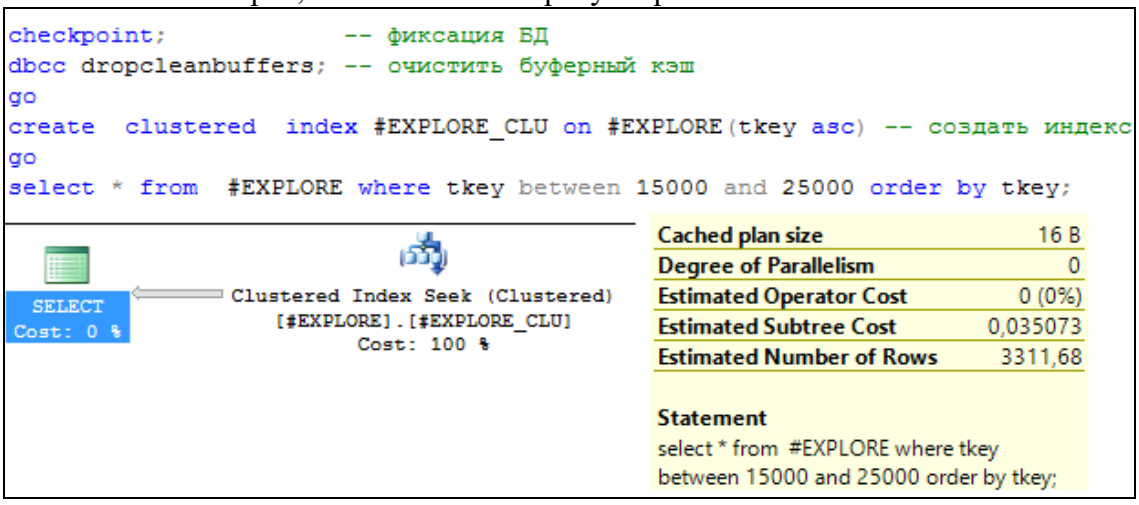


Рис. 11.6. Актуальный план запроса к таблице, имеющей кластерный индекс

План запроса, представленный на рис. 11.6, существенно отличается от плана аналогичного запроса, показанного на рис. 11.5. Во-первых, сканирование осуществляется не по всей таблице, а по части кластеризованного индекса (компонент Clustered Index Seek). Во-вторых, отсутствует компонент сортировки. При этом стоимость запроса (Estimate Subtree Cost) составляет 0,035, что почти в 5 раз меньше, чем при запросе к таблице, не имеющей индекса.

Пятикратное повышение скорости выполнения запроса достигается за счет следующего.

1. Выбор строк таблицы **#EXPLORE** по диапазону значений столбца **TKEY** не требует сканирования всей таблицы. Индекс по столбцу **TKEY** позволяет серверу сразу позиционироваться на первую строку выбираемой группы строк и сканировать только строки, соответствующие заданному диапазону.

2. Строки таблицы **#EXPLORE** уже расположены в порядке возрастания значений столбца **TKEY**, так как она сама является частью индекса. Поэтому сортировка не требуется.

Заметим, что, если бы в SELECT-запросе на рис. 11.6 результат сортировался в порядке убывания значений столбца **TKEY**, то бы план запроса и его стоимость (и соответственно время выполнения) не изменились. Просто вывод строк в результирующий набор будет осуществляться в обратном порядке.

Обратим внимание, что в индексируемом столбце **TKEY** таблицы **#EXPLORE** допускаются повторения. Соответственно индекс **#EXPLORE_CLU** (рис. 11.6) допускает повторение значений индексного ключа (комбинации значений индексируемых столбцов). Такие индексы называются неуникальными.

```
select  t.cc [количество значений], count(*) [количество случаев]
  from (
        select TKEY, count(*) cc
        from #EXPLORE  group by TKEY having  count(*) > 1
        )t
  group by t.cc
  order by [количество случаев] desc
```

количество значений	количество случаев
2	1226
3	142
4	7

Рис. 11.7. Значения в индексируемом столбце **TKEY** не являются уникальными

В сценарии на рис. 11.8 удаляется неуникальный кластерный индекс **#EXPLORE_CLU** и создается уникальный индекс **#EXPLORE_UNIQ_CLU**. При этом возникает ошибка, сообщение о которой приводится на рис. 11.8. Ошибка возникла из-за того, что при создании уникального индекса в индексируемом столбце были обнаружены дубликаты значений.

```
use TEMPDB
go
drop index  #EXPLORE_CLU on #EXPLORE
go
---- создание уникального кластеризованного индекса
create unique clustered  index #EXPLORE_UNIQ_CLU on #EXPLORE(tkey asc) -- создать индекс
```

```
Msg 1505, Level 16, State 1, Line 2
The CREATE UNIQUE INDEX statement terminated because a duplicate
key was found for the object name 'dbo.#EXPLORE__00000000001E' and the index name '#EXPLORE_UNIQ_CLU'.
The duplicate key value is (4).
The statement has been terminated.
```

Рис. 11.8. Ошибка при создании уникального индекса над столбцом таблицы с повторяющимися значениями

Создание уникального индекса по столбцу **CC**, содержащего только уникальные значения (столбец обладает свойством IDENTITY), завершается успешно (рис. 11.9).

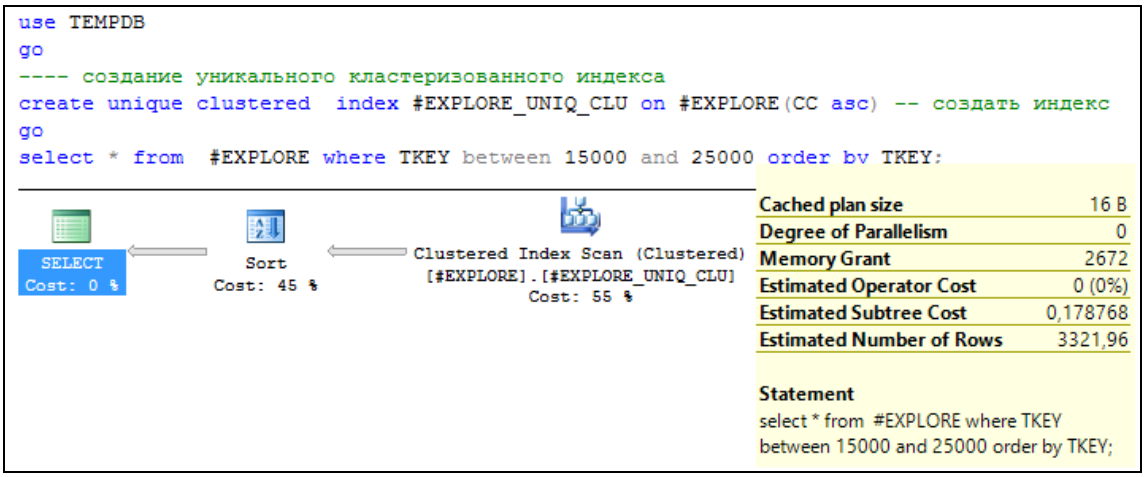


Рис. 11.9. Создание уникального кластерного индекса **#EXPLORE_UNIQ_CLU** над столбцом **CC** таблицы **#EXPLORE**

Обратите внимание на план SELECT-запроса в сценарии на рис. 11.9. Он отличается от плана на рис. 11.6 и похож на план запроса к таблице без индексов (рис. 11.5). При этом во всех трех случаях выполняется один и тот же SELECT-запрос. В соответствии планом (рис. 11.9) выполнение запроса будет осуществляться в два этапа: сначала полное сканирование кластерного индекса **#EXPLORE_UNIQ_CLU** (напомним: кластерный индекс – это и есть таблица) для выбора значений соответствующих WHERE-условию, а затем выполняется сортировка выбранных строк. Заметьте, что стоимость запроса примерно такая же, как и в запросе к неиндексированной таблице (рис. 11.5).

В запросе на рис. 11.10 сортировка выполняется по значениям индексируемого столбца CC. Сравнив этот запрос с планом запроса на рис. 11.9, заметим, что операции сортировки нет. Она не требуется, так как строки таблицы **#EXPLORE** являются частью кластерного индекса **#EXPLORE_UNIQ_CLU** и уже находятся в требуемом порядке. Обратите внимание: стоимость запроса значительно снизилась и вся сосредоточена на операции сканирования кластерного индекса.

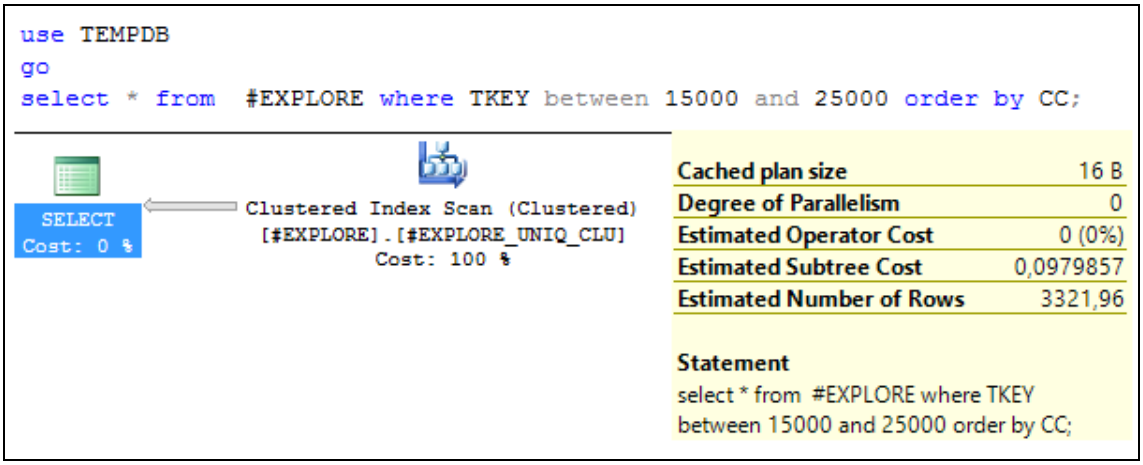
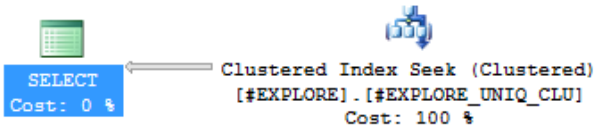


Рис. 11.10. SELECT-запрос, сортирующий результат запроса по значениям индексируемого столбца

Если изменить WHERE-условие таким образом, чтобы фильтрация строк осуществлялась по значениям индексируемого столбца (рис. 11.11), то план становится похожим на план запроса, представленного на рис. 11.6. Кроме того, обратите внимание, что стоимость запроса тоже снизилась.

```
use TEMPDB
go
select * from  #EXPLORE where CC between 4500 and  6500 order by CC;
```



Cached plan size	16 B
Degree of Parallelism	0
Estimated Operator Cost	0 (0%)
Estimated Subtree Cost	0,0217795
Estimated Number of Rows	2001,07

Statement
select * from #EXPLORE where CC between 4500 and 6500 order by CC;

Рис. 11.11. SELECT-запрос, фильтрующий и сортирующий строки по значениям индексируемого столбца

Еще раз напомним, что по умолчанию для таблицы БД со столбцом, имеющим свойство PRIMARY KEY (ограничение целостности – первичный ключ), при создании таблицы автоматически создается кластеризованный уникальный индекс.

11.3. Некластеризованные индексы

Некластеризованные индексы в отличие от кластеризованных никак не влияют на физический порядок строк в таблице. Как и в случае кластеризованных индексов, данные индекса организованы в виде сбалансированного дерева. С таблицей некластеризованный индекс связан указателями на ее строки. Для одной таблицы БД допускается создавать до 1023 некластеризованных индексов.

Индексы, соответствующие ограничению **PRIMARY KEY**, не обязательно должны быть кластеризованными. На рис. 11.12 представлен пример создания таблицы **TEACHER**, имеющей столбец со свойством PRIMARY KEY и соответствующим некластеризованным индексом.

```
use BSTU
go
drop table TEACHER;
go
create table TEACHER
(
 TEACHER       char(10)  constraint TEACHER_PK  primary key nonclustered,
 TEACHER_NAME  varchar(100),
 GENDER        char(1)
               default 'м'
               constraint TEACHER_GENDER_CH check (GENDER in ('м', 'ж')),
 PULPIT        char(20) constraint TEACHER_PULPIT_FK foreign key
                        references PULPIT(PULPIT)
);
go
exec SP_HELPINDEX 'TEACHER'
```

index_name	index_description	index_keys
TEACHER_PK	nonclustered, unique, primary key located on FG1	TEACHER

Рис. 11.12. Создание таблицы **TEACHER** со столбцом **TEACHER**, являющимся первичным ключом с соответствующим ему некластеризованным индексом

Во всех приводимых до сих пор примерах использовались индексы, созданные по одному столбцу таблицы. Такие индексы называются простыми. MSS допускает создавать индексы по

нескольким столбцам – такие индексы называются составными. Составными могут быть кластеризованные и некластеризованные индексы. Например, составному первичному ключу по умолчанию соответствует составной кластерный индекс.

На рис. 11.13 приведен пример создания составного неуникального, некластеризованного индекса **#EXPLORE_CX** по двум столбцам **TKEY** и **CC** таблицы **#EXPLORE** (рис. 11.4).

```
use TEMPDB
go
create   index #EXPLORE_NONCLU on #EXPLORE(TKEY, CC)
go
exec SP_HELPINDEX '#EXPLORE'
```

index_name	index_description	index_keys
#EXPLORE_NONCLU	nonclustered located on PRIMARY	TKEY, CC

Рис. 11.13. Создание составного некластеризованного индекса с именем **#EXPLORE_NONCLU** по столбцам **TKEY и CC** таблицы **#EXPLORE**

Обратите внимание на то, что некластеризованный составной индекс не применяется оптимизатором ни при фильтрации (рис. 11.14), ни при сортировке (рис. 11.15) строк таблицы **#EXPLORE**. В обоих случаях осуществляется сканирование всей таблицы (на плане – Table Scan).

```
select * from  #EXPLORE where  TKEY > 1500 and  CC < 4500;
```

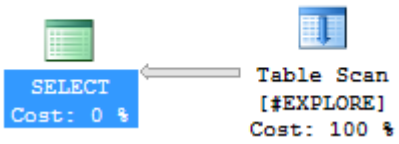


Рис. 11.14. Составной некластеризованный индекс **#EXPLORE_NONCLU**, неиспользуемый при фильтрации по индексируемым столбцам

```
select * from  #EXPLORE order by TKEY, CC;
```



Рис. 11.15. Составной некластеризованный индекс **#EXPLORE_NONCLU**, неиспользуемый при сортировке по индексируемым столбцам

Но если хотя бы одно из индексируемых значений зафиксировать (задать одно значение), то оптимизатор применит индекс (рис. 11.16, 11.17).

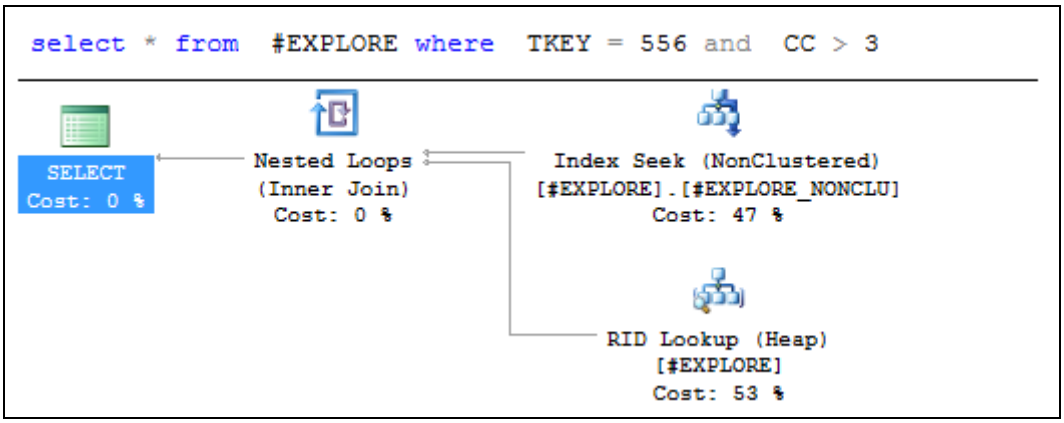


Рис. 11.16. Применение индекса при выполнении запроса (при фиксации в секции WHERE значения индексируемого столбца **TKEY** составного индекса **#EXPLORE_NONCLU**)

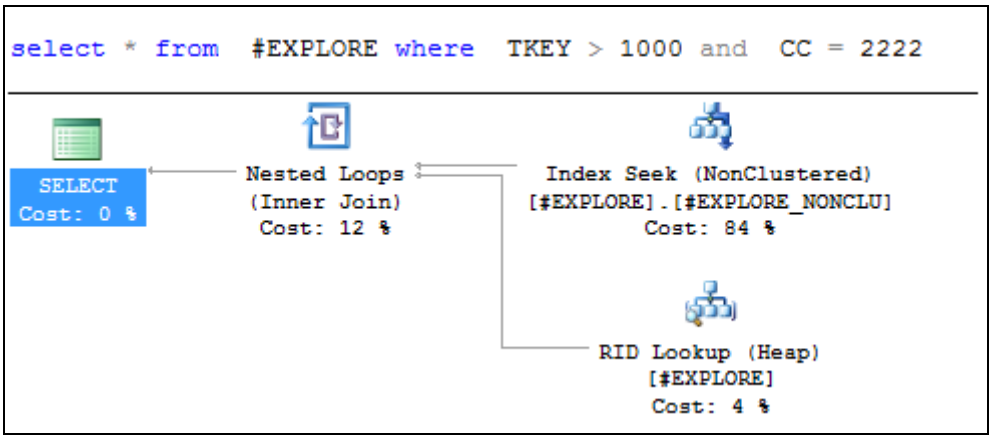


Рис. 11.17. Применение индекса при выполнении запроса (при фиксации в секции WHERE значения индексируемого столбца **CC** составного индекса **#EXPLORE_NONCLU**)

Планы запросов на рис. 11.16 и 11.17 содержат по три компонента. Выполнение запроса начинается с позиционирования (установка текущего указателя на первую строку в группе отобранных строк индекса) в индексе **#EXPLORE_NONCLU** (на плане – Index Seek) и считывание данных из хранилища строк (Heap) с помощью указателя на строку таблицы (RID Lookup). На втором этапе считанные данные из индекса и хранилища строк объединяются (Inner Join).

На рис. 11.18 демонстрируется запрос, аналогичный запросу, представленному на рис. 11.16, но в результирующий набор выводится только два столбца **TKEY** и **CC**. Обратите внимание: **TKEY** и **CC** – индексируемые столбцы (рис. 11.13).

Если в SELECT-списке указываются только индексируемые столбцы (рис. 11.18), то хранилище строк (Heap) в плане запроса не применяется

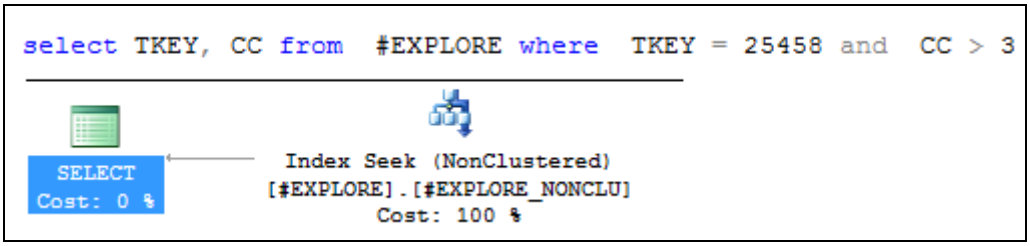


Рис. 11.18. SELECT-запрос с использованием только индексируемых столбцов

Сравнив планы запросов, представленных на рис. 11.16 и 11.18, заметим, что в последнем не используется хранилище строк (Heap). Это происходит потому, что значения столбцов **TKEY** и **CC** хранятся в индексе и считываются именно оттуда.

Наиболее часто составной индекс применяется при группировке и вычислениях по индексируемым столбцам (рис. 11.19).

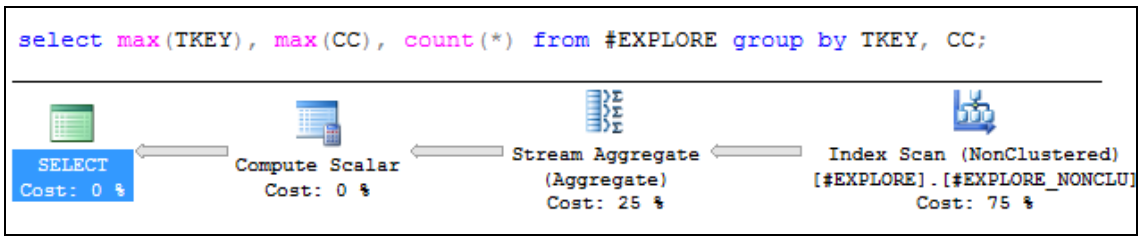


Рис. 11.18. SELECT-запрос с группировкой по индексируемым столбцам

При группировке с вычислениями по двум индексируемым столбцам хранилище строк не используется, выборка данных осуществляется через сканирование индекса.

Удалим составной индекс **#EXPLORE_NONCLU** и создадим два отдельных индекса **#EXPLORE_TKEY**, **#EXPLORE_CC** (рис. 11.19).

```
use TEMPDB
go
drop index #EXPLORE_NONCLU on #EXPLORE; -- удаление индекса
go
create   index #EXPLORE_TKEY on #EXPLORE(TKEY);
go
create   index #EXPLORE_CC on #EXPLORE(CC);
go
```

Рис. 11.19. Удаление индекса **#EXPLORE_NONCLU**, создание двух некластеризованных индексов

Проанализировав запросы на рис. 11.20, приходим к выводу, что во всех трех случаях применяется полное сканирование таблицы.

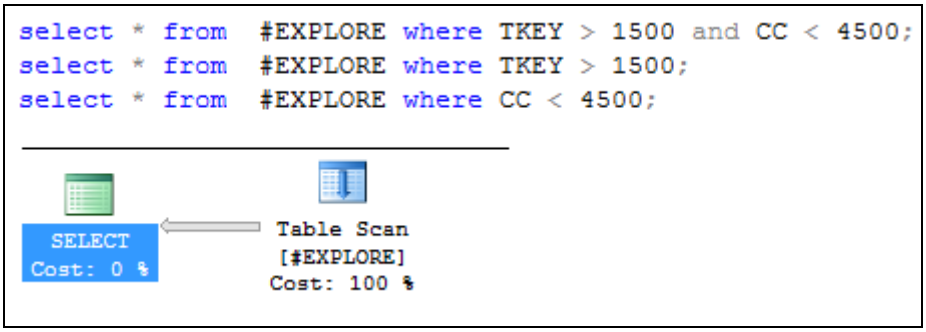


Рис. 11.20. Одинаковый план запроса для всех трех запросов к таблице **#EXPLORE**, имеющей 2 индекса **#EXPLORE_TKEY** и **#EXPLORE_CC** (рис. 11.19)

Индекс применяется лишь в тех случаях, когда в SELECT-списке и в WHERE-выражении используется один и тот же индексируемый столбец (рис. 11.21). Иначе выполняется сканирование всей таблицы (рис. 11.22).

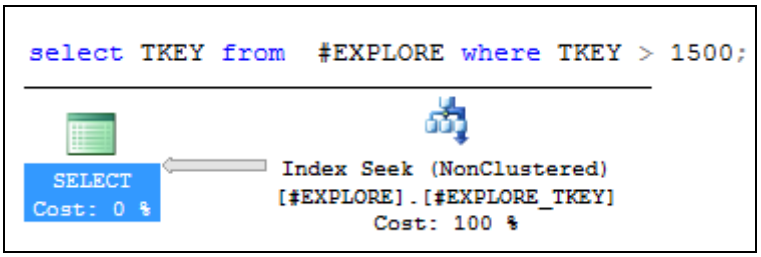


Рис. 11.21. Индексируемый столбец **TKEY** используется
в SELECT-списке и WHERE-выражении

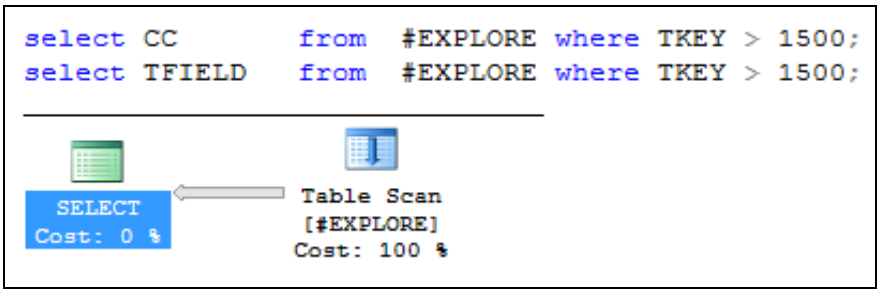


Рис. 11.22. Одинаковый план запроса для двух запросов к таблице **#EXPLORE**

Рассмотрим возможность совместного применения кластеризованного и некластеризованного индексов. Для этого удалим все существующие индексы таблицы **#EXPLORE** и выполним сценарий, представленный на рис. 11.23. Заметим, что кластеризованный индекс построен по столбцу **CC**, а некластеризованный – по столбцу **TKEY**.

```
use TEMPDB
go
create   index #EXPLORE_TKEY on #EXPLORE(TKEY)
go
create  clustered index #EXPLORE_CLU_CC on #EXPLORE(CC)
go
exec SP_HELPINDEX '#EXPLORE'
```

index_name	index_description	index_keys
#EXPLORE_CLU_CC	clustered located on PRIMARY	CC
#EXPLORE_TKEY	nonclustered located on PRIMARY	TKEY

Рис. 11.23. Создание кластеризованного и некластеризованного
индексов таблицы **#EXPLORE**

Выполнив два запроса, представленных на рис. 11.22, получим два разных плана (рис. 11.24, 11.25).

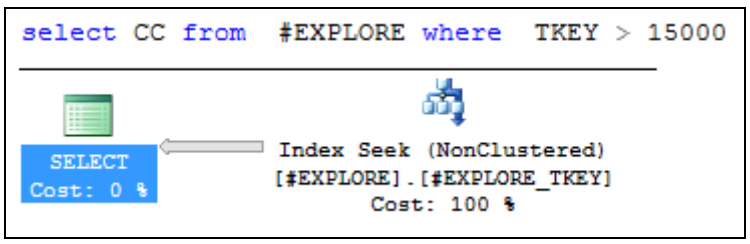


Рис. 11.24. План выполнения первого запроса, представленного на рис. 11.22
при наличии кластеризованного и некластеризованных индексов,
созданных в сценарии на рис. 11.23

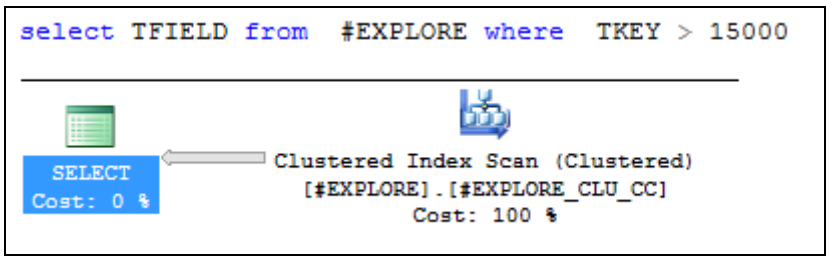


Рис. 11.25. План выполнения второго запроса, представленного на рис. 11.22 при наличии кластеризованного и некластеризованных индексов, созданных в сценарии на рис. 11.23

В первом случае (рис. 11.24) выполнение запроса сводится к позиционированию в некластеризованном индексе **#EXPLORE_TKEY** и считыванию строк в соответствии с диапазоном, заданным во WHERE-выражении. При этом возникает вопрос: каким образом извлекаются данные из столбца **CC**, указанного в SELECT-списке запроса? Дело в том, что при наличии кластеризованного индекса строка таблицы является частью индекса, а некластеризованный индекс хранит не указатели на строки кластеризованного индекса, а значения его индексируемых столбцов (по которым осуществляется поиск строк в кластеризованном индексе). Таким образом, значения столбца **CC** будут храниться в строках некластеризованного индекса **#EXPLORE_TKEY.**

Во втором случае (рис. 11.25) некластеризованный индекс не применяется – для извлечения значений столбца **TFIELD** требуется доступ к кластеризованному индексу (он в данном случае совпадает с таблицей **#EXPLORE**), поэтому оптимизатор «предпочел» сканирование всего индекса.

11.4. Индексы покрытия

Как уже отмечалось, избежать полного сканирования таблицы можно с помощью кластеризованного, некластеризованного индексов или их совместного применения. Рассмотрим еще один способ оптимизации запросов.

Вернемся к запросам в примере на рис. 11.22. Таблица **#EXPLORE** имеет индекс **#EXPLORE_TKEY** по столбцу **TKEY**. Выполнение запросов сводится к сканированию всей таблицы.

Путь необходимо оптимизировать выполнение первого запроса. Для удобства воспроизведем исходный для оптимизации план запроса. Заметим еще раз: в плане запроса не применяется существующий индекс **#EXPLORE_TKEY** потому что, даже если отфильтровать строки таблицы с помощью индекса, все равно придется извлекать значения столбца **CC** из таблицы.

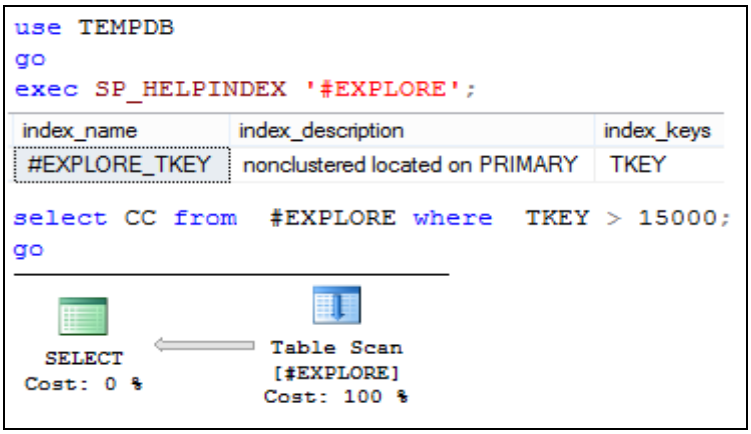


Рис. 11.26. План выполнения запроса, предполагающий сканирование таблицы

Для оптимизации SELECT-запроса, представленного на рис. 11.26 может быть использован другой вид некластеризованного индекса, называемого индексом покрытия запроса или просто индексом покрытия.

Индекс покрытия запроса позволяет включить в состав индексной строки значения одного или нескольких неиндексируемых столбцов. На рис. 11.27 представлен пример создания индекса покрытия. Созданный индекс покрытия **#EXPLORE_TKEY_X** включает значения столбца **CC** (ключевое слово INCLUDE).

```
use TEMPDB
go
drop index #EXPLORE_TKEY on #EXPLORE;
go
 --- создание идекса покрытия --------------------------------
create   index #EXPLORE_TKEY_X  on #EXPLORE(TKEY)  include(CC);
-------------------------------------------------------------------
exec SP_HELPINDEX '#EXPLORE';
```

index_name	index_description	index_keys
#EXPLORE_TKEY_X	nonclustered located on PRIMARY	TKEY

Рис. 11.27. Удаление индекса **#EXPLORE_TKEY** и создание индекса покрытия **#EXPLORE_TKEY_X**

Выполнив запрос, представленный на рис. 11.26 еще раз, получим иной план выполнения запроса, применяющий только некластеризованный индекс для позиционирования и выбора строк, соответствующих WHERE-условию (рис. 11.28).

```
select CC from  #EXPLORE where  TKEY > 15000
```

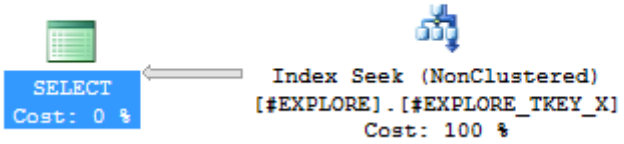


Рис. 11.28. План запроса, использующего индекс покрытия

11.5. Уникальные некластеризованные индексы

Некластеризованные индексы, как и кластеризованные, могут быть уникальными. Например, уникальные индексы создаются автоматически при применении ограничения целостности UNIQUE.

В сценарии на рис. 11.28 с помощью DDL-оператора ALTER столбцу **SUBJECT_NAME** таблицы **SUBJECT** устанавливается свойство (ограничение целостности) UNIQUE. Установка такого свойства влечет за собой автоматическое создание некластеризованного уникального индекса с именем, заданным с ключевым словом CONSTRAINT.

```
use BSTU
go
alter table [SUBJECT]
      add  constraint SUBJECT_UQ unique(SUBJECT_NAME);
go
exec SP_HELPINDEX 'SUBJECT';
```

index_name	index_description	index_keys
SUBJECT_PK	clustered, unique, primary key located on FG1	SUBJECT
SUBJECT_UQ	nonclustered, unique, unique key located on FG1	SUBJECT_NAME

Рис. 11.29. Добавление свойства UNIQUE столбцу **SUBJECT_NAME** таблицы **SUBJECT**

С помощью системной процедуры SP_HELP можем убедиться в наличии ограничения целостности **SUBJECT_UQ** (рис. 11.30).

```
use BSTU
go
exec SP_HELP 'SUBJECT';
```

constraint_type	constraint_name	dele...	upd...	statu...	status...	constraint_keys
PRIMARY KEY (clustered)	SUBJECT_PK	(n/a)	(n/a)	(n/a)	(n/a)	SUBJECT
FOREIGN KEY	SUBJECT_PULPIT_FK	No ...	No ...	Ena...	Is_Fo...	PULPIT
						REFERENCES BSTU.dbo.PULPI...
UNIQUE (non-clustered)	SUBJECT_UQ	(n/a)	(n/a)	(n/a)	(n/a)	SUBJECT_NAME

Рис. 11.30. Ограничение целостности UNIQUE для столбца **SUBJECT_NAME**, отображающееся системной процедурой SP_HELP

Создание некластеризованного уникального индекса влечет за собой запрещение повторных индексируемых значений (рис. 11.31, 11.32), но это не отображается процедурой SP_HELP как свойство UNIQUE столбцов таблицы.

```
use BSTU
go
alter table [SUBJECT]
       drop  constraint SUBJECT_UQ;
go
exec SP_HELPINDEX 'SUBJECT';
```

index_name	index_description	index_keys
SUBJECT_PK	clustered, unique, primary key located on FG1	SUBJECT

Рис. 11.31. Удаление ограничения целостности UNIQUE, вызывающее удаление соответствующего уникального индекса

```
use BSTU
go
create  unique index SUBJECT_EQ on [SUBJECT](SUBJECT_NAME);
go
exec SP_HELPINDEX 'SUBJECT';
```

index_name	index_description	index_keys
SUBJECT_EQ	nonclustered, unique located on FG1	SUBJECT_NAME
SUBJECT_PK	clustered, unique, primary key located on FG1	SUBJECT

```
select count(*) [количество строк SUBJECT_NAME = 'Базы данных']
from [SUBJECT]  where SUBJECT_NAME = 'Базы данных'
go
```

количество строк SUBJECT_NAME = 'Базы данных'
1

```
insert [SUBJECT] values ('XX','Базы данных', 'ИСиТ');
```

```
Msg 2601, Level 14, State 1, Line 1
Cannot insert duplicate key row in object 'dbo.SUBJECT' with unique index 'SUBJECT_EQ'
```

```
exec SP_HELP 'SUBJECT';
```

constraint_type	constraint_name	delete_action	update_action	status_enabled	constraint_keys
PRIMARY KEY (clustered)	SUBJECT_PK	(n/a)	(n/a)	(n/a)	SUBJECT
FOREIGN KEY	SUBJECT_PULPIT_FK	No Action	No Action	Enabled	PULPIT
					REFERENCE...

Рис. 11.32. Примеры с системной процедурой SP_HELP

Создание уникального индекса влечет запрещение повторных индексируемых значений, но не отображается как ограничение целостности UNIQUE системной процедурой SP_HELP

11.6. Фильтруемые некластеризованные индексы

В некоторых случаях бывает важным, чтобы часть запросов выполнялась особенно быстро. Если эти запросы основаны на WHERE-фильтрации строк, то может быть эффективным применение фильтруемых некластеризованных индексов.

```
use TEMPDB
go
select count(*)[количество строк в #EXPLORE] from #EXPLORE;
```

количество строк в #EXPLORE
89154

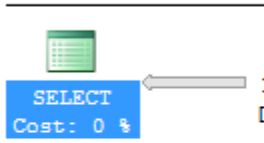
```
create  index #EXPLORE_TKEY  on #EXPLORE(TKEY);
go
exec SP_HELPINDEX '#EXPLORE';
```

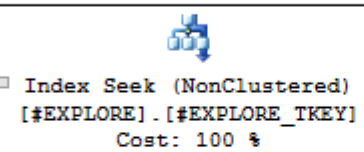
index_name	index_description	index_keys
#EXPLORE_TKEY	nonclustered located on PRIMARY	TKEY

Рис. 11.33. Создание некластеризованного индекса **#EXPLORE_TKEY**

Для исследования эффективности фильтруемых индексов увеличим количество строк в таблице **#EXPLORE** и создадим некластеризованный индекс по столбцу **TKEY** (рис. 11.33). Выполним три запроса (рис. 11.34 –11.36), убедимся, что в плане выполнения применяется созданный (рис. 11.33) индекс и зафиксируем стоимости запросов.

```
select TKEY from #EXPLORE  where TKEY between 15001 and 19999;
```





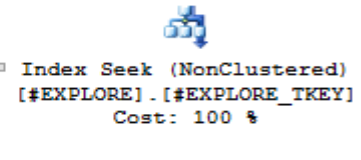
Cached plan size	16 B
Degree of Parallelism	0
Estimated Operator Cost	0 (0%)
Estimated Subtree Cost	0,0440697
Estimated Number of Rows	14857,5

Statement
select TKEY from #EXPLORE where TKEY between 15000 and 20000;

Рис. 11.34. Выполнение запроса со стоимостью 0,044

```
select TKEY from #EXPLORE  where TKEY > 16000 and TKEY < 19000
```





Cached plan size	16 B
Degree of Parallelism	0
Estimated Operator Cost	0 (0%)
Estimated Subtree Cost	0,0280846
Estimated Number of Rows	9079,85

Statement
select TKEY from #EXPLORE where TKEY > 16000 and TKEY < 19000

Рис. 11.35. Выполнение запроса со стоимостью 0,028

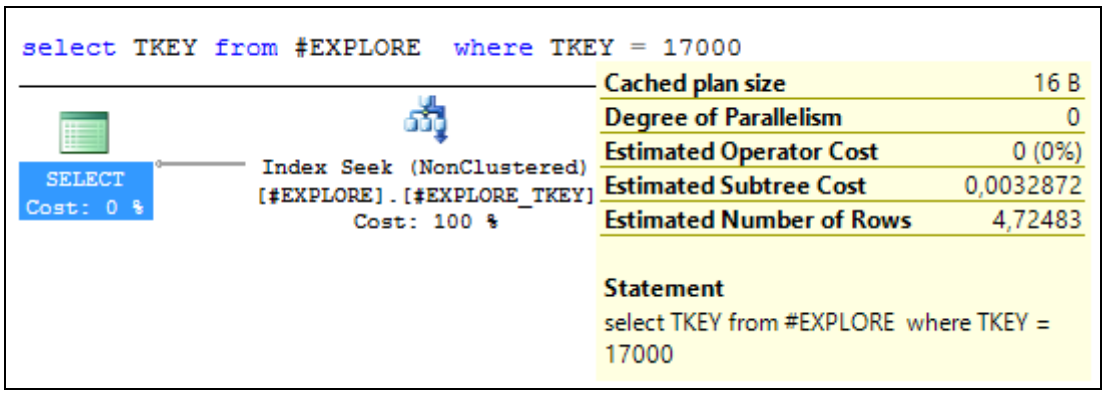
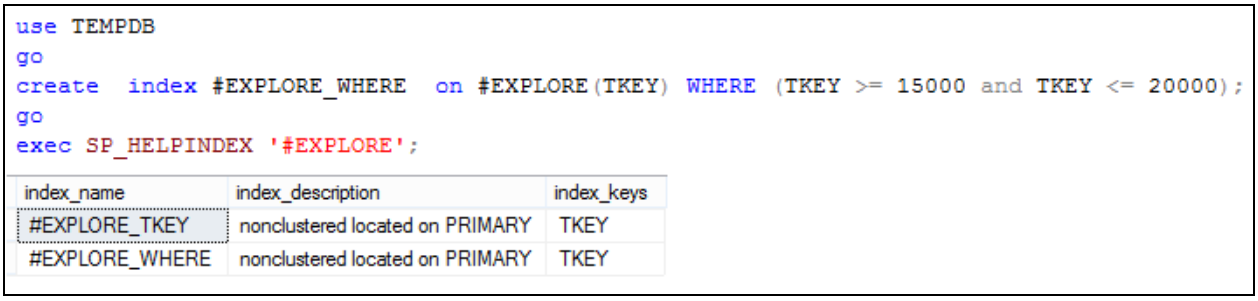


Рис. 11.36. Выполнение запроса со стоимостью 0,033

На рис. 11.37 приведен сценарий, в котором создается фильтрующий индекс с именем **#EXPLORE_WHERE**. Обратите внимание на ключевое слово WHERE и следующее за ним логическое условие в операторе CREATE INDEX: фильтруемый индекс создается только для строк таблицы **#EXPLORE**, которые удовлетворяют этому логическому условию. Кроме того, заметим, что WHERE-выражение при создании индекса описывает подмножество строк, являющееся надмножеством для всех подмножеств строк, выбираемых запросами на рис. 11.34–11.36.



```
use TEMPDB
go
create  index #EXPLORE_WHERE  on #EXPLORE(TKEY) WHERE (TKEY >= 15000 and TKEY <= 20000);
go
exec SP_HELPINDEX '#EXPLORE';
```

index_name	index_description	index_keys
#EXPLORE_TKEY	nonclustered located on PRIMARY	TKEY
#EXPLORE_WHERE	nonclustered located on PRIMARY	TKEY

Рис. 11.37. Создание некластеризованного фильтруемого индекса **#EXPLORE_WHERE**

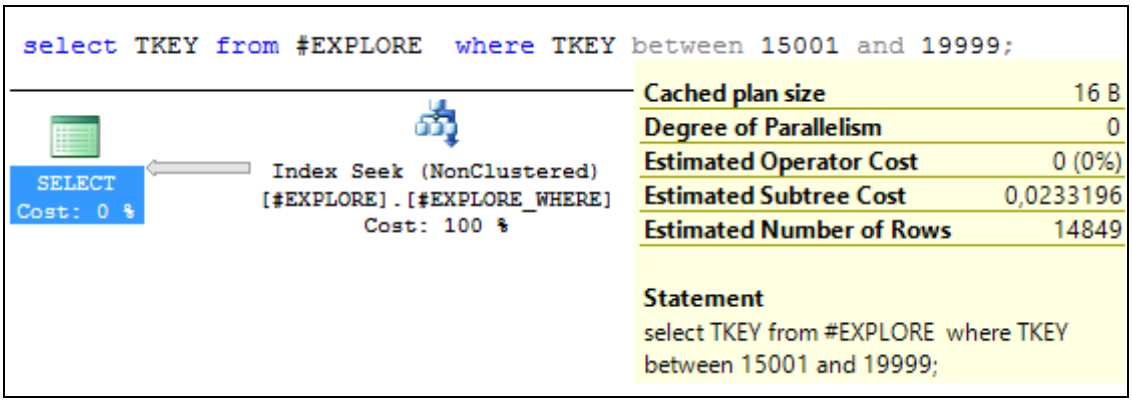


Рис. 11.38. Выполнение запроса со стоимостью 0,023 (меньше стоимости 0,044 запроса на рис. 11.34)

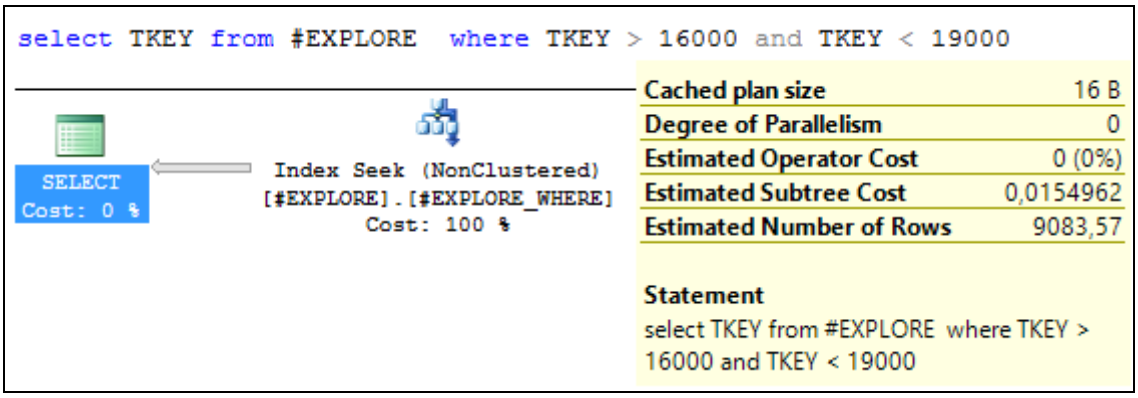


Рис. 11.39. Выполнение запроса со стоимостью 0,015
(меньше стоимости 0,028 запроса на рис. 11.35)

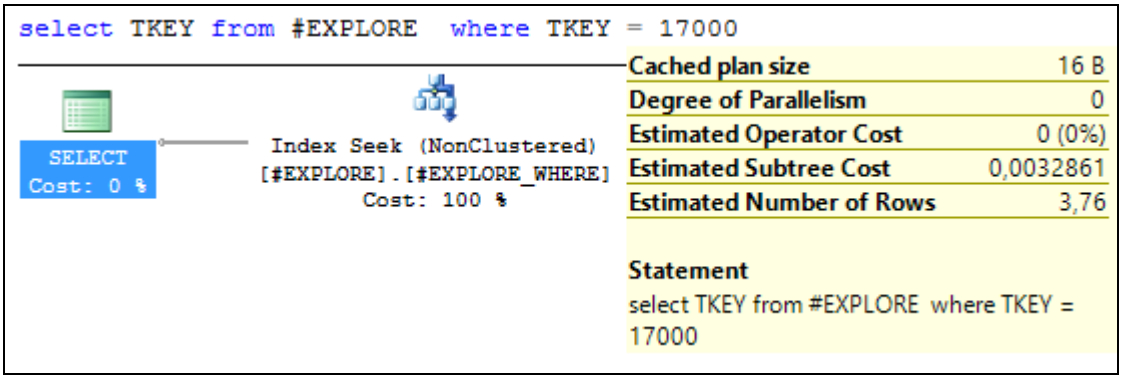


Рис. 11.40. Выполнение запроса со стоимостью 0,0033
(совпадает со стоимостью 0,0033 запроса на рис. 11.36)

Повторим три запроса к таблице **#EXPLORE** (рис. 11.38–11.40). Убедимся, что в плане запросов применяется созданный фильтруемый индекс (рис. 11.37). Зафиксируем стоимости этих запросов. При сравнении стоимости двух серий запросов можно убедиться в том, что в двух из трех запросов стоимость существенно снизилась.

На рис. 11.41 представлен запрос, WHERE-выражение которого описывает множество строк, включающее строки, не входящие во множество строк, соответствующее WHERE-выражению фильтрующего индекса **#EXPLORE_WHERE**. План выполнения этого запроса не применяет фильтрующий индекс, а основывается на позиционировании в индексе **#EXPLORE_TKEY**.

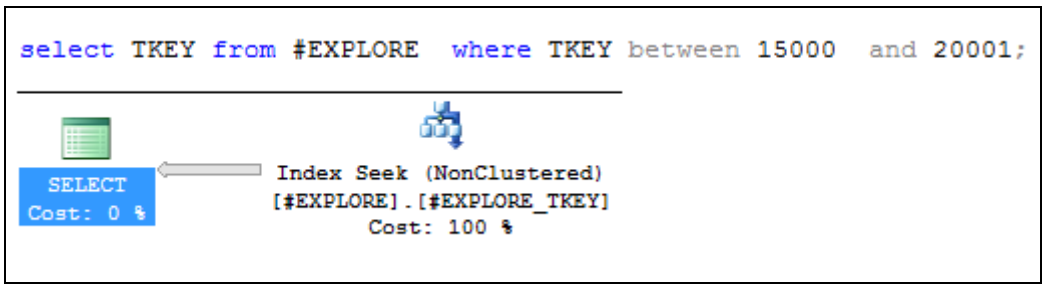


Рис. 11.41. План выполнения запроса, не применяющего фильтрующий индекс

Допускается создание некластеризованных фильтрующих индексов покрытия (рис. 11.42). Обратите внимание: WHERE-выражение в операторе создания индекса содержит логическое выражение, в котором задействованы два столбца таблицы, хотя индексирование выполняется только по одному из них. Как и прежде, если множество строк, выбираемое WHERE-выражением

SELECT-запроса, является подмножеством множества индексируемых фильтрующим индексом строк, то оптимизатор будет его применять в плане запроса (рис. 11.43).

```
use TEMPDB
go
select count(*) [количество строк в #EXPLORE] from #EXPLORE;
```

количество строк в #EXPLORE
90000

```
create  index #EXPLORE_WHERE_I  on #EXPLORE(TKEY) include(CC)  WHERE (TKEY >= 15000 and CC < 45000);
go
exec SP_HELPINDEX '#EXPLORE';
```

index_name	index_description	index_keys
#EXPLORE_WHERE_I	nonclustered located on PRIMARY	TKEY

Рис. 11.42. Создание некластеризованного фильтрующего индекса покрытия

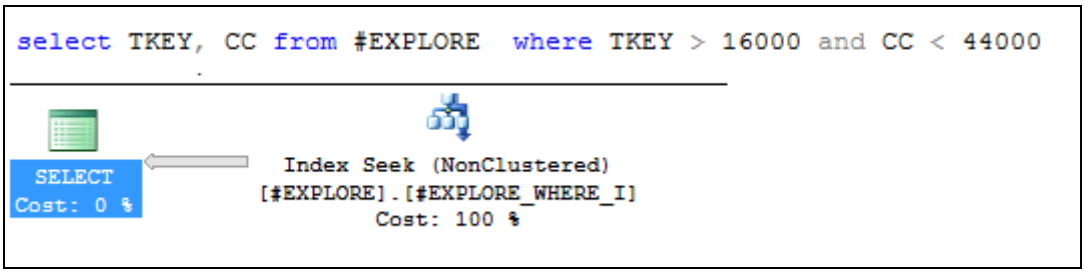


Рис. 11.43. Фильтрующий индекс покрытия, использующийся в плане запроса

11.7. Перестройка и реорганизация индексов

Правильное применение индексов позволяет во многих случаях существенно сократить продолжительность выполнения SELECT-запроса. Однако следует понимать, что эффективность SELECT-запросов достигается снижением эффективности выполнения других DML-операторов. Ясно, что, например, при добавлении строки в таблицу, имеющую некластеризованные индексы, автоматически выполняется изменение в индексах. Аналогично обновляются индексы при выполнении операторов UPDATE и DELETE. Наиболее трудоемкими при этом являются операции, обеспечивающие сбалансированность древовидной структуры индекса.

Чтобы повысить эффективность выполнения операций изменения таблиц БД, алгоритмы изменения индексов реализованы таким образом, чтобы отложить наиболее трудоемкую их часть на более позднее время. Например, удаление строки в таблице БД приводит к необходимости удалить соответствующие записи во всех индексах данной таблицы. Удаление записей индекса не выполняется физически, а данные страницы индекса при этом не уплотняются с целью ликвидации «дырки», образовавшейся после удаления этой записи – запись просто помечается специальным маркером. Аналогично операции добавления и изменения строк БД могут повлечь образование неиспользуемых фрагментов в области памяти индекса. Поэтому индексы таблиц, часто подвергающихся операциям изменения, становятся пористыми, или еще говорят – фрагментированными. Процесс образования неиспользуемых фрагментов памяти называют фрагментацией. Фрагментация индексов снижает эффект от их применения и в некоторых случаях может привести к значительным потерям производительности сервера БД.

Степень фрагментации индекса обычно измеряется в процентах (процентная доля неиспользуемой памяти диска) и может быть получена с помощью запроса, представленного на рис. 11.44.

```
use TEMPDB
go
select count(*)[количество строк] from #EXPLORE;
go
```

количество строк
91915

```
create   index #EXPLORE_TKEY on #EXPLORE(TKEY);
----  данные о фрагментации ------------------------------------
select name [Индекс], avg_fragmentation_in_percent [фрагментация (%)]
from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
     join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
where name is not null;
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	0

Рис. 11.44. Получение информации о степени фрагментации индекса **#EXPLORE_TKEY**

В сценарии на рис. 11.45 в таблицу **#EXPLORE** добавляется 10 000 строк и снова получается информация о степени фрагментации индекса. Заметьте, что уровень фрагментации превысил 99%.

```
use TEMPDB
go
insert top(10000) #EXPLORE(TKEY, TFIELD) select TKEY, TFIELD from #EXPLORE;
go
select count(*)[количество строк] from #EXPLORE;
```

количество строк
101915

```
 ----  данные о фрагментации ------------------------------------
 select name [Индекс], avg_fragmentation_in_percent [фрагментация (%)]
 from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
      join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
 where name is not null;
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	99,2682926829268

Рис. 11.45. Фрагментация индекса после добавления 10 000 строк

От фрагментации индекса можно избавиться с помощью простого пересоздания индекса (рис. 11.46), но таким способом пользуются редко.

```
----  данные о фрагментации ------------------------------------
select name [Индекс], avg_fragmentation_in_percent [фрагментация (%)]
from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
     join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
where name is not null;
go
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	99,2682926829268

```
---- пересоздать индекс ----------------------------------------
drop index #EXPLORE_TKEY on #EXPLORE;
go
create   index #EXPLORE_TKEY on #EXPLORE(TKEY);
go
----  данные о фрагментации ------------------------------------
select name [Индекс], avg_fragmentation_in_percent [фрагментация (%)]
from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
     join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
where name is not null;
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	0

Рис. 11.46. Избавление от фрагментации простым пересозданием индекса

В больших БД пересоздание индекса может выполняться очень долго. Кроме того, эта операция требует эксклюзивного использования индексируемой таблицы, что приводит к блокировке выполнения всех остальных запросов к таблице. В реальных условиях это непозволительно.

Для избавления от фрагментации индекса предусмотрены две специальные операции: реорганизация и перестройка индекса. Обе операции могут выполняться в режиме ONLINE, позволяющем не блокировать таблицу на время построения индекса и допускающем реализацию других запросов к этой таблице. Главное отличие между реорганизацией и перестройкой индекса в глубине перестройки индекса.

Реорганизация (REORGANIZE) выполняется значительно быстрее, но после этой операции фрагментация будет убрана только из листовых узлов индексного дерева (на самом нижнем уровне). Поэтому после реорганизации может оставаться некоторая степень фрагментации индекса. Операцию реорганизации индексов рекомендуется выполнять при уровне фрагментации 10–20%. Реорганизация индекса всегда выполняется в режиме ONLINE.

Операция перестройки (REBUILD) затрагивает все узлы дерева, поэтому после ее выполнения степень фрагментации равна нулю. Перестройку индексов рекомендуется выполнять при уровне фрагментации, превышающем 20%. По умолчанию операция перестройки индекса выполняется в режиме OFFLINE и полностью блокирует индексируемую таблицу для доступа других запросов, но с помощью специального параметра может быть установлен режим ONLINE. Исключением являются индексы локальных временных таблиц, их перестройка всегда выполняется в режиме OFFLINE.

На рис. 11.47 приведен пример выполнения оператора ALTER для реорганизации индекса **#EXPLORE_TKEY.** Заметьте, что после реорганизации небольшой уровень фрагментации сохранился.

```
----  данные о фрагментации ------------------------------------
select name [Индекс], avg_fragmentation_in_percent [Фрагментация (%)]
from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
     join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
where name is not null;
go
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	98,2926829268293

```
alter index #EXPLORE_TKEY on #EXPLORE reorganize;
go
----  данные о фрагментации ------------------------------------
select name [Индекс], avg_fragmentation_in_percent [Фрагментация (%)]
from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
     join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
where name is not null;
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	3,94736842105263

Рис. 11.47. Демонстрация выполнения реорганизации индекса **#EXPLORE_TKEY**

На рис. 11.48 приведен пример выполнения оператора ALTER для перестройки индекса **#EXPLORE_TKEY** в режиме **OFFLINE**.

```
----  данные о фрагментации ------------------------------------
select name [Индекс], avg_fragmentation_in_percent [Фрагментация (%)]
from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
     join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
where name is not null;
go
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	98,0440097799511

```
alter index #EXPLORE_TKEY on #EXPLORE rebuild with (online = off);
go
----  данные о фрагментации ------------------------------------
select name [Индекс], avg_fragmentation_in_percent [Фрагментация (%)]
from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
     join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
where name is not null;
go
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	0

Рис. 11.48. Демонстрация выполнения перестройки индекса **#EXPLORE_TKEY**

Уровнем фрагментации можно в некоторой степени управлять, если при создании или изменении (реорганизации и перестройке) индекса использовать параметры FILLFACTOR и PAD_INDEX.

Параметр FILLFACTOR указывает процент заполнения индексных страниц нижнего уровня. Например, если установлено значение 80, то это означает, что на листовых страницах дерева индекса будет зарезервировано 20% свободной памяти для будущих операций. По умолчанию (значение может быть установлено помощью системной процедуры SP_CONFIGURE) обычно значение этого параметра устанавливается равным 100, т. е. индексные страницы заполняются полностью. Следует иметь в виду, что значение 0 тоже приводит к полному заполнению страницы (трактуется как 100). Отличное от 0 и 100 значение параметра FILLFACTOR позволяет уменьшить фрагментацию, возникающую из-за расщепления полностью заполненных индексных страниц, происходящее при добавлении и корректировке индекса. Но следует помнить, что чем больше зарезервировано на страницах места для будущих операций изменения, тем больше становится страниц индекса, и это снижает эффективность его применения. Считается, что если установить значение FILLFACTOR, равное 50, то это приведет к снижению скорости выполнения операций, базирующихся на применении индекса, примерно в 2 раза. Выбор значения зависит от интенсивности обновления индекса, количества строк в индексируемой таблице, а также от возможности выполнять операции реорганизации и перестройки.

Параметр PAD_INDEX распространяет действие параметра FILLFACTOR на страницы промежуточного и корневого уровня.

На рис. 11.49 демонстрируется фрагментация индекса **#EXPLORE_TKEY** после добавления в таблицу **#EXPLORE** строк. Заметим, что значение параметра FILLFACTOR индекса равно 100.

```
alter index #EXPLORE_TKEY on #EXPLORE rebuild with (online = off);
--- процет фрагментации равен 0 -------------------------------
insert top(50)percent into #EXPLORE(TKEY, TFIELD) select TKEY, TFIELD  from #EXPLORE
----  данные о фрагментации ------------------------------------
select name [Индекс], avg_fragmentation_in_percent [Фрагментация (%)]
from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
     join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
where name is not null;
go
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	99,002493765586

Рис. 11.49. Пример, демонстрирующий процесс фрагментации индекса **#EXPLORE_TKEY** при значении параметра FILLFACTOR, равном 100

В сценарии на рис. 11.50 пересоздается индекс **#EXPLORE_TKEY**. При этом указывается значение параметра FILLFACTOR, равное 65. После добавления строк в таблицу **#EXPLORE** уровень фрагментации вырос только до 40%, что ниже, чем в примере на рис. 11.49

```
drop index #EXPLORE_TKEY on #EXPLORE;
create   index #EXPLORE_TKEY on #EXPLORE(TKEY)with (fillfactor = 65);
 --- процент фрагментации равен 0 -------------------------------
insert top(50)percent into #EXPLORE(TKEY, TFIELD) select TKEY, TFIELD  from #EXPLORE;
----  данные о фрагментации ----------------------------------
select name [Индекс], avg_fragmentation_in_percent [фрагментация (%)]
from sys.dm_db_index_physical_stats(DB_ID(N'TEMPDB'),OBJECT_ID(N'#EXPLORE'), NULL, NULL, NULL) ss
     join sys.indexes ii on ss.object_id = ii.object_id and ss.index_id = ii.index_id
where name is not null;
go
```

Индекс	Фрагментация (%)
#EXPLORE_TKEY	40,1960784313725

Рис. 11.50. Пример, демонстрирующий процесс фрагментации индекса **#EXPLORE_TKEY** при значении параметра FILLFACTOR, равном 65

Для более подробного знакомства с параметрами индексов рекомендуется литература [3, 4, 5].

Глава 12. ПРИМЕНЕНИЕ ПРЕДСТАВЛЕНИЙ

12.1. Назначение и применение индексов

Представление (от англ. View) – это объект БД, представляющий собой поименованный SELECT-запрос. При этом следует подчеркнуть: в БД хранится именно SELECT-запрос, а не результат его выполнения. Поясним на примере. На рис. 12.1 представлен SELECT-запрос, использующий в качестве исходного источника строк результирующий набор, сформированный другим SELECT-запросом.

```
use BSTU
go
select *
from (
      select sb.SUBJECT_NAME [Дисциплина],
             st.NAME         [Студент],
             pr.PDATE        [Дата],
             pr.NOTE         [Оценка]
      from  [SUBJECT] sb join  PROGRESS pr  on sb.[SUBJECT] = pr.[SUBJECT]
                         join  STUDENT  st  on st.IDSTUDENT = pr.IDSTUDENT
     ) [Экзамен];
```

Дисциплина	Студент	Дата	Оценка
Основы алгоритмизации и программирования	Манакова Анастасия Владимировна	2013-01-10	6
Основы алгоритмизации и программирования	Хартанович Екатерина Александровна	2013-01-10	8
Основы алгоритмизации и программирования	Горбач Елизавета Юрьевна	2013-01-10	7
Основы алгоритмизации и программирования	Зыкова Кристина Дмитриевна	2013-01-10	5
Основы алгоритмизации и программирования	Медведева Мария Андреевна	2013-01-10	4
Основы алгоритмизации и программирования	Шенец Екатерина Сергеевна	2013-01-10	4
Основы алгоритмизации и программирования	Шитик Алина Игоревна	2013-01-10	8
Базы данных	Силюк Валерия Ивановна	2013-06-12	4
Базы данных	Сергель Виолетта Николаевна	2013-06-12	9
Базы данных	Добродей Ольга Анатольевна	2013-06-12	5

Рис. 12.1. SELECT-запрос, использующий в качестве исходного источника строк результирующий набор, сформированный другим SELECT-запросом

Создав представление с именем **Экзамен**, запрос, представленный на рис. 12.1, можно записать короче (рис. 12.2). При этом синтаксически применение представления в SELECT-запросах ничем не отличается от использования таблицы с таким же именем (рис. 12.3). На основе представления **Экзамен** можно в свою очередь построить еще одно представление, которое тоже можно использовать как таблицу в других SELECT-запросах (12.4).

```
use BSTU
go
select * from [Экзамен];
```

Дисциплина	Студент	Дата	Оценка
Основы алгоритмизации и программирования	Манакова Анастасия Владимировна	2013-01-10	6
Основы алгоритмизации и программирования	Хартанович Екатерина Александровна	2013-01-10	8
Основы алгоритмизации и программирования	Горбач Елизавета Юрьевна	2013-01-10	7
Основы алгоритмизации и программирования	Зыкова Кристина Дмитриевна	2013-01-10	5
Основы алгоритмизации и программирования	Медведева Мария Андреевна	2013-01-10	4
Основы алгоритмизации и программирования	Шенец Екатерина Сергеевна	2013-01-10	4
Основы алгоритмизации и программирования	Шитик Алина Игоревна	2013-01-10	8
Базы данных	Силюк Валерия Ивановна	2013-06-12	4
Базы данных	Сергель Виолетта Николаевна	2013-06-12	9
Базы данных	Добродей Ольга Анатольевна	2013-06-12	5
Базы данных	Подоляк Мария Сергеевна	2013-06-12	7
Базы данных	Никитенко Екатерина Дмитриевна	2013-06-12	5
Системы управления базами данных	Яцкевич Галина Иосифовна	2013-01-18	6
Системы управления базами данных	Осадчая Эла Васильевна	2013-01-18	5

Рис. 12.2. SELECT-запрос, в секции FROM которого используется представление с именем **Экзамен**

```
use BSTU
go
select [Дисциплина], avg([Оценка]) [Средняя оценка]
from  [Экзамен] group by [Дисциплина] order by [Средняя оценка];
```

Дисциплина	Средняя оценка
Системы управления базами данных	5
Экономическая теория	6
Базы данных	6
Компьютерная геометрия	6
Органическая химия	6
Основы алгоритмизации и программирования	6

Рис. 12.3. Представление **Экзамен** в других SELECT-запросах как аналог таблицы с таким же именем.

```
use BSTU
go
select * from [Средняя оценка] where [Средняя оценка] < 6;
```

Дисциплина	Средняя оценка
Системы управления базами данных	5

Рис. 12.4. Построение представления **Средняя оценка** на основе SELECT-запроса, представленного на рис. 12.3

12.2. Создание, удаление и изменение представлений

Как и любой объект БД, представление создается с помощью оператора CREATE, удаляется с помощью оператора DROP и изменяется с помощью ALTER.

В сценарии, представленном на рис. 12.5, создаются представления **Экзамен** и **Средняя оценка**, примеры использования которых приведены на рис. 12.3 и 12.4.

```
use BSTU
go
----- создание представления  [Экзамен]
create view [Экзамен]
    as select sb.SUBJECT_NAME [Дисциплина],
              st.NAME         [Студент],
              pr.PDATE        [Дата],
              pr.NOTE         [Оценка]
        from  [SUBJECT] sb join  PROGRESS pr  on sb.[SUBJECT] = pr.[SUBJECT]
                           join  STUDENT  st  on st.IDSTUDENT = pr.IDSTUDENT
go
----- создание представления  [Средняя оценка]
create view [Средняя оценка]
as select [Дисциплина], avg([Оценка]) [Средняя оценка]
   from  [Экзамен] group by [Дисциплина];
```

Рис. 12.5. Создание двух представлений **Экзамен** и **Средняя оценка**

В операторе CREATE после ключевого слова VIEW следует имя представления и далее после ключевого слова AS – текст SELECT-запроса, лежащего в основе представления. При этом к SELECT-запросу предъявляются следующие ограничения:

- секцию ORDER BY можно использовать только совместно с опцией TOP;
- не допускается применение секции INTO;
- не допускается применение секций COMPUTE и COMPUTE BY;
- все столбцы результирующего набора, формируемого SELECT-запросом, должны быть поименованы.

Для удаления представления необходимо использовать оператор DROP, для изменения – ALTER. Сценарий на рис. 12.6 демонстрирует удаление представления **Средняя оценка** и изменение представления **Экзамен**.

Обратите внимание: изменение представления практически ничем не отличается от создания. Изменение представления бывает полезным в тех случаях, когда с помощью DCL-предложений пользователю были назначены определенные разрешения относительно этого представления. В случае удаления и создания нового представления с тем же именем все действующие разрешения будут утеряны. Если применить ALTER, то разрешения будут сохранены.

```
use BSTU
go
drop view [Средняя оценка];
go
alter view [Экзамен]
as select sb.SUBJECT_NAME [Дисциплина],
          st.NAME         [Студент],
          pr.PDATE        [Дата],
          pr.NOTE         [Оценка],
          st.IDSTUDENT    [Идентификатор студента]
   from  [SUBJECT] sb join  PROGRESS pr  on sb.[SUBJECT] = pr.[SUBJECT]
                      join  STUDENT  st  on st.IDSTUDENT = pr.IDSTUDENT
```

Рис. 12.6. Удаление представления **Средняя оценка** и изменение представления **Экзамен**

Следует помнить, что, если удалить представление, на основе запроса к которому строится другое представление, то последнее станет неработоспособным. Для того чтобы этого избежать, следует применять специальную опцию SCHEMABINDING.

12.3. Привязка к базовым таблицам

Удаление или изменение таблиц или представлений, используемых в SELECT-запросе, на котором основано представление, может повлечь за собой потерю работоспособности представления. Для предотвращения проблемы применяется специальная опция SCHEMABINDING, после указания которой при создании представления запрещаются операции с таблицами и представлениями, которые могут привести к нарушению его работоспособности.

На рис. 12.7 приведен сценарий, изменяющий (ALTER) представление **Экзамен** и создающий представление **Средняя оценка**. В обоих случаях для представлений указана опция SCHEMABINDING. Обратите внимание: при применении опции требуется использовать в SELECT-запросе для имен таблиц и представлений двухкомпонентный формат. Другими словами, требуется указывать схему БД, к которой принадлежит таблица или представление. Применение и использование схем БД не рассматривается в этом пособии. Здесь только поясним: схема – это поименованный контейнер объектов БД, позволяющий разграничить объекты с одинаковыми именами. Схемы имеют такой же смысл, как пространства имен в языках программирования. Например, в БД допускается использование двух таблиц с одинаковыми именами, но они должны находиться в разных схемах.

```
use BSTU
go
alter view [Экзамен]  with schemabinding
as select sb.SUBJECT_NAME [Дисциплина],
          st.NAME         [Студент],
          pr.PDATE        [Дата],
          pr.NOTE         [Оценка],
          st.IDSTUDENT    [Идентификатор студента]
   from  dbo.[SUBJECT] sb join  dbo.PROGRESS pr  on sb.[SUBJECT] = pr.[SUBJECT]
                       join  dbo.STUDENT  st  on st.IDSTUDENT = pr.IDSTUDENT;
go
create view [Средняя оценка]  with schemabinding
as select [Дисциплина], avg([Оценка]) [Средняя оценка]
   from   dbo.[Экзамен] group by [Дисциплина];
```

Рис. 12.7. Установка опции SHEMABINDING при изменении представления **Экзамен** и создании представления **Средняя оценка**

В таких случаях для однозначной идентификации таблицы будет применяться двухкомпонентное имя, причем первой компонентой указывается имя схемы. Если имя схемы явно не указывается, то предполагается, что применяется стандартная схема с именем DBO. В большинстве случаев это бывает достаточным. В сценарии на рис. 12.8 сделана попытка удаления таблицы **PROGRESS** и представления **Экзамен**. В обоих случаях удаление не выполнено из-за того, что эти объекты связаны с представлениями.

```
use BSTU
go
drop table PROGRESS;
drop view [Экзамен];

Msg 3729, Level 16, State 1, Line 1
Cannot DROP TABLE 'PROGRESS' because it is being referenced by object 'Экзамен'.
Msg 3729, Level 16, State 1, Line 2
Cannot DROP VIEW 'Экзамен' because it is being referenced by object 'Средняя оценка'.
```

Рис. 12.8. Сообщения об ошибках при попытке удаления связанных с представлениями таблицы **PROGRESS** и представления **Экзамен**

Попытка изменить представление **Экзамен** (даже не затрагивающее представление **Средняя оценка**) тоже запрещается (рис. 12.9).

```
use BSTU
go
alter view [Экзамен]  with schemabinding
as select sb.SUBJECT_NAME [Дисциплина],
          st.NAME         [Студент],
          pr.PDATE        [Дата],
          pr.NOTE         [Оценка]
          -- , st.IDSTUDENT    [Идентификатор студента]
   from  dbo.[SUBJECT] sb join  dbo.PROGRESS pr  on sb.[SUBJECT] = pr.[SUBJECT]
                          join  dbo.STUDENT  st  on st.IDSTUDENT = pr.IDSTUDENT;

Msg 3729, Level 16, State 3, Procedure Экзамен, Line 2
Cannot ALTER 'Экзамен' because it is being referenced by object 'Средняя оценка'.
```

Рис. 12.9. Сообщения об ошибках при попытке изменить связанное представление **Экзамен**

12.4. DML-операции с представлениями

Для наиболее простых представлений допустимо выполнение DML-операций INSERT, DELETE и UPDATE. Следует помнить, что любая операция добавления, удаления или изменения строк представления все равно транслируется в соответствующую операцию с базовой таблицей. Как следствие этого, все существующие ограничения целостности базовой таблицы наследуются представлением.

При создании представлений, позволяющих выполнять операции INSERT, DELETE и UPDATE, базовый SELECT-запрос должен удовлетворять следующим условиям:

1) не содержать секцию группировки GROUP BY;
2) не применять агрегатные функции;
3) не использовать опции DISTINCT и TOP;
4) не применять операторы UNION, INTERSECT и EXCEPT;
5) в SELECT-списке запроса не должно быть вычисляемых значений.
6) в секции FROM запроса должна указываться только одна таблица.

Для демонстрации применения операторов INSERT, DELETE, UPDATE на основе запросов к таблицам **AUDITORIUM_TYPE** и **AUDITORIUM** создадим два представления (рис. 12.10) с именами **vAT** и **vAU**. Заметим, что представления удовлетворяют перечисленным выше требованиям. Обратим внимание также, что в обоих случаях в базовых SELECT-запросах применяются секции WHERE для фильтрации строк.

```
use BSTU
go
create view vAT (AT, TN)
as select AUDITORIUM_TYPE, AUDITORIUM_TYPENAME  from AUDITORIUM_TYPE
    where  AUDITORIUM_TYPE !='ЛБ-Х';
go
create view vAU (AU,  AT, AC)
as select AUDITORIUM,  AUDITORIUM_TYPE, AUDITORIUM_CAPACITY  from AUDITORIUM
   where  AUDITORIUM_CAPACITY > 50;
go
select * from vAT;
select * from vAU;
```

AT	TN
ЛБ-К	Компьютерный класс
ЛБ-СК	Спец. компьютерный класс
ЛК	Лекционная
ЛК-К	Лекционная с уст. проектором

AU	AT	AC
105-4	ЛК	80
107-4	ЛК	80
114-4	ЛК-К	75
132-4	ЛК	75
236-1	ЛК	60
313-1	ЛК-К	60
423-1	ЛБ-К	90
429-4	ЛК	90

Рис. 12.10. Создание представлений **vAT** и **vAU**

```
use BSTU
go
insert vAT values ('ЛБ-Х', 'Химическая лаборатория');
insert vAT values ('ЛБ-К', 'Химическая лаборатория');
insert vAT values ('XXXX', 'XXXXXXXXXXXXXXXXXXXXXX');
```

```
Msg 2627, Level 14, State 1, Line 1
Violation of PRIMARY KEY constraint 'AUDITORIUM_TYPE_PK'.
Cannot insert duplicate key in object 'dbo.AUDITORIUM_TYPE'.
The statement has been terminated.
Msg 2627, Level 14, State 1, Line 2
Violation of PRIMARY KEY constraint 'AUDITORIUM_TYPE_PK'.
Cannot insert duplicate key in object 'dbo.AUDITORIUM_TYPE'.
The statement has been terminated.

(1 row(s) affected)
```

Рис. 12.11. Сообщения об ошибках при добавлении строк в представление **vAT**

В сценарии на рис. 12.11 демонстрируется выполнение трех операторов INSERT, добавляющих строки в представление **vAT**. Выполнение первых двух операторов завершается с ошибками. Ошибки возникают в силу нарушения ограничения целостности PRIMARY KEY в базовой для представления таблице **AUDITORIUM_TYPE**. Причем возникает несогласованность: отобразить с помощью оператора SELECT строку со значением **ЛБ-Х** в столбце **AT** невозможно (рис. 12.12), но причиной ошибки (рис. 12.11) она является. Более того, удалить ее тоже невозможно (рис. 12.13).

```
use BSTU
go
select * from vAT where AT in('XXXX', 'ЛБ-Х','ЛБ-К');
```

AT	TN
XXXX	XXXXXXXXXXXXXXXXXXXX
ЛБ-К	Компьютерный класс

Рис. 12.12. Строка представления со значением **ЛБ-Х** в столбце **AT**

```
use BSTU
go
select count(*) [количество строк до удаления] from vAT;
delete  vAT where AT = 'ЛБ-Х';
select count(*) [количество строк после удаления] from vAT;
```

количество строк до удаления
5

количество строк после удаления
5

Рис. 12.13. Удаление «невидимой» строки в представлении,
не приводящее к результату

Кроме того, в представление можно добавить строку, которую не «учтет» функция COUNT (рис. 12.14) и не отобразит оператор SELECT.

```
use BSTU
go
select count(*) [количество строк до INSERT]    from vAU;
insert vAU values ('AAAAA', 'XXXX', 20);
select count(*) [количество строк после INSERT] from vAU;
```

количество строк до INSERT
8

количество строк после INSERT
8

Рис. 12.14. Успешно добавленная строка, которая может стать «невидимой»

Оператор UPDATE может привести к эффекту удаления строки (рис. 12.15).

```
use BSTU
go
select count(*) [количество строк до INSERT]    from vAU;
update  vAU  set AC = 45 where AU = '236-1' ;
select count(*) [количество строк после INSERT] from vAU;
```

количество строк до INSERT
8

количество строк после INSERT
7

Рис. 12.15. Эффект удаления строк в результате применения
оператора UPDATE к представлению

Такое несогласованное поведение DML-операторов может привести к ошибкам при написании запросов. Для того чтобы предотвратить несогласованность, при создании или изменении представления может быть указана специальная опция CHECK OPTION. В представлении с этой опцией помимо унаследованных от базовой таблицы ограничений целостности появляется еще одно ограничение, основанное на логическом выражении в секции WHERE базового SELECT-оператора.

В сценарии, представленном на рис. 12.16, с помощью оператора ALTER изменяются два представления: **vAT** и **vAU**. Заметьте, в обоих случаях применяется предложение WITH CHECK OPTION.

```
use BSTU
go
alter view vAT (AT, TN)
as select AUDITORIUM_TYPE, AUDITORIUM_TYPENAME  from AUDITORIUM_TYPE
    where  AUDITORIUM_TYPE !='ЛБ-Х' with check option;
go
alter view vAU (AU,  AT, AC)
as select AUDITORIUM,  AUDITORIUM_TYPE, AUDITORIUM_CAPACITY  from AUDITORIUM
   where  AUDITORIUM_CAPACITY > 50 with check option;
go
```

Рис. 12.16. Опция CHECK OPTION, указанная в операторе ALTER для представлений **vAT** и **vAU**

Сценарий на рис. 12.17 демонстрирует выполнение операторов INSERT и UPDATE для представлений **vAT** и **vAU** после выполнения операторов ALTER в сценарии на рис. 12.16. Заметим, что если бы опция CHECK OPTION не была установлена, то второй оператор INSERT и оператор UPDATE выполнились бы успешно.

```
use BSTU
go
insert vAT values ('ЛБ-Х', 'Химическая лаборатория');
go
insert vAU values ('BBBB', 'XXXX', 20);
go
update  vAU  set AC = 45 where AU = '114-4';
go
```

```
Msg 2627, Level 14, State 1, Line 1
Violation of PRIMARY KEY constraint 'AUDITORIUM_TYPE_PK'.
Cannot insert duplicate key in object 'dbo.AUDITORIUM_TYPE'.
The statement has been terminated.
Msg 550, Level 16, State 1, Line 1
The attempted insert or update failed because the target
view either specifies WITH CHECK OPTION or spans a view that specifies
WITH CHECK OPTION and one or more rows resulting from the operation did
not qualify under the CHECK OPTION constraint.
The statement has been terminated.
Msg 550, Level 16, State 1, Line 1
The attempted insert or update failed because the target
view either specifies WITH CHECK OPTION or spans a view that specifies
WITH CHECK OPTION and one or more rows resulting from the operation did
not qualify under the CHECK OPTION constraint.
The statement has been terminated.
```

Рис. 12.17. Опция CHECK OPTION, добавляющая представлению дополнительное ограничение

Обратите внимание на следующее.

1. В первом операторе INSERT ограничение целостности PRIMARY KEY, унаследованное представлением **vAT**, «сработало» раньше ограничения, обусловленного опцией CHECK OPTION.
2. Добавление в представление строк данных, не удовлетворяющих логическому WHERE-выражению базового SELECT-запроса, невозможно.
3. Изменить существующие строки представления с помощью оператора UPDATE таким образом, чтобы после изменения строки не удовлетворяли логическому WHERE-выражению в базовом SELECT-запросе, невозможно.

Глава 13. ОСНОВЫ ПРОГРАММИРОВАНИЯ НА T-SQL

13.1. Введение в T-SQL

Язык SQL – является декларативным языком. В нем отсутствуют инструкции, позволяющие управлять последовательностью выполнения операторов. Другими словами, в языке SQL нет операторов перехода, цикла, ветвления и других механизмов, с помощью которых можно было бы управлять ходом выполнения программы (записать алгоритм).

Современный SQL – это язык, мощности которого в большинстве случаев оказывается достаточно для разработки приложений БД. T-SQL – это процедурное расширение языка SQL. Иначе говоря, это надстройка над SQL, дополняющая его возможностями алгоритмического языка программирования.

Применение языка T-SQL обусловлено современной тенденцией в программировании, которая заключается в переносе тяжести программного обеспечения на сторону сервера. Иначе говоря, основной программный код в современных клиент-серверных приложениях размещается на серверной стороне, клиентское программное обеспечение в основном решает задачи ввода и отображения данных. Применительно в MSS можно сказать, что T-SQL является его встроенным языком, позволяющим создавать программные объекты БД. К программным объектам MSS относятся хранимые процедуры, скалярные и табличные функции и триггеры.

Начиная с версии 2005 г. MSS разработчику предоставляется возможность разрабатывать программные объекты БД на языке C#. Более того, в C# допускается разработка собственных типов данных, которые можно применять для столбцов при создании таблиц. Применение C# в этом пособии не рассматривается. Для знакомства с возможностью использования этого языка для разработки программных объектов БД рекомендуется пособие [5].

Необходимо сделать еще одно замечание относительно терминологии: начиная с версии 2008 в документации MSS не различаются языки SQL и T-SQL, их называют T-SQL. Следуя этому правилу, далее будет говорится только о языке T-SQL. Операторы языка T-SQL, как правило, отделяются друг от друга точкой с запятой, но это правило не является строгим. Если операторы расположены на разных строках, то допускается не применять разделители. Синтаксис T-SQL не чувствителен к регистру, поэтому допускается любое сочетание букв верхнего и нижнего регистров. Дальнейшее описание синтаксиса языка T-SQL будет осуществляться по ходу изложения.

13.1.1. Понятие пакета

Пакет – это группа операторов T-SQL, которая обрабатывается сервером СУБД вместе. Понятие пакета необходимо при клиент-серверной архитектуре приложения. В этом смысле, пакет – это группа операторов T-SQL, которую программа клиент отправляет серверу на исполнение.

Стандартные программы-клиенты, входящие в программное обеспечение MSS, такие как MMS и утилита командной строки SQLCMD, разграничивают пакеты с помощью специальной

инструкции GO. Инструкция GO интерпретируется как команда отправки на сервер порции операторов T-SQL. В качестве операнда инструкции GO можно указать целое положительное число, которое будет восприниматься как количество повторений пакета. Следует отметить, что в некоторых случаях разделение инструкций на пакеты является обязательным.

На рис. 13.1 представлен пример сценария, состоящего из пяти пакетов.

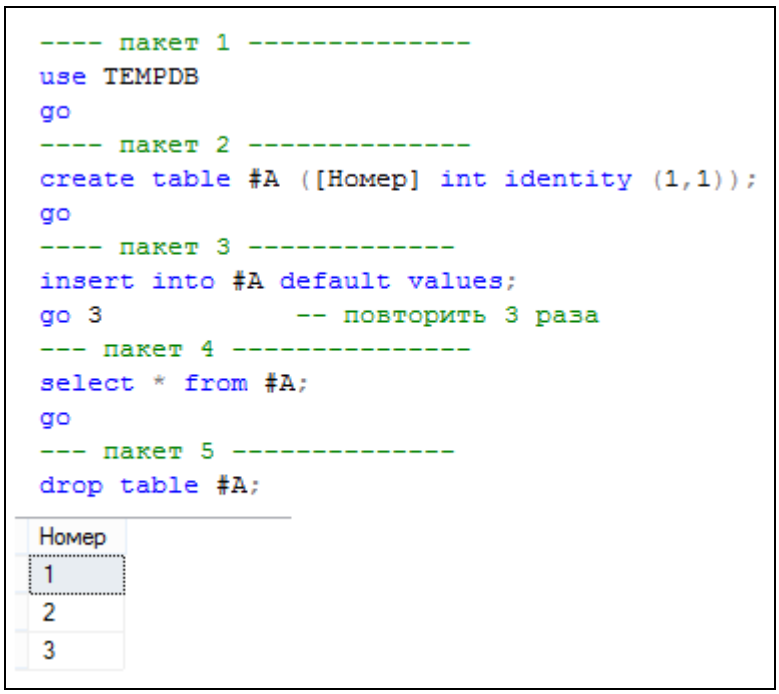


Рис. 13.1. Сценарий, состоящий из пяти пакетов

13.1.2. Контекст выполнения пакета

Каждый пакет выполняется в контексте какой-то БД. Контекст БД переключается с помощью инструкции USE (рис. 13.1). За ключевым словом USE указывается имя БД, в контекст которой осуществляется переключение. Обычно переключение контекста выполняется в отдельном пакете. Как и GO инструкция USE применяется стандартными программами-клиентами, входящими в состав программного обеспечения MSS. Переключение контекста БД позволяет указывать короткие имена объектов БД, используемых в операторах T-SQL. Если в контексте одной БД требуется сослаться на объект другой БД в рамках одного сервера, то необходимо использовать трехкомпонентное имя объекта, включающего имя БД, имя схемы и собственно имя объекта.

13.2. Оператор DECLARE

Оператор DECLARE (рис. 13.2) предназначен для объявления используемых в программе переменных. Для каждой переменной указывается имя и тип. Имя должно обязательно начинаться с символа @, но не с @@ , так как это сочетание символов зарезервировано для системных переменных и функций.

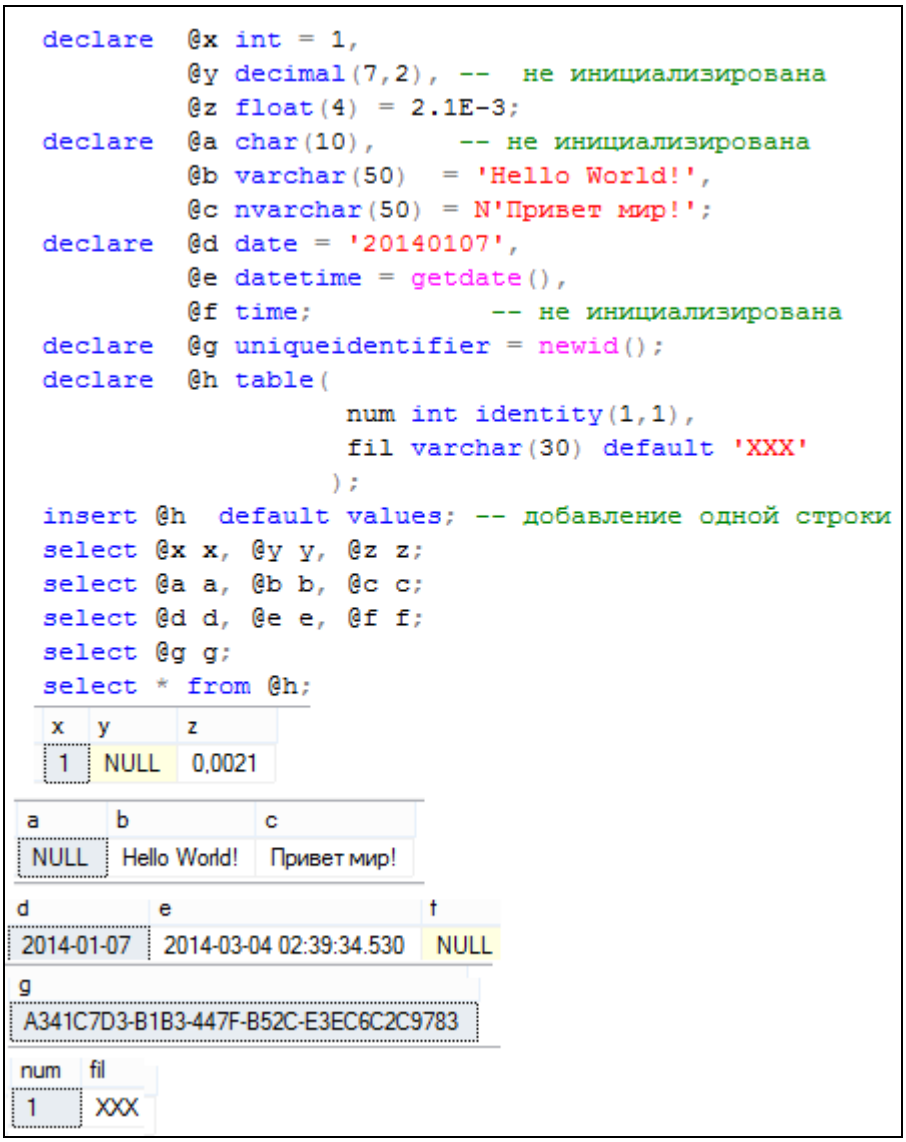


Рис. 13.2. Пример сценария T-SQL, содержащего пять операторов DECLARE

Перечень допустимых типов переменных совпадает с перечнем типов данных БД MSS (табл. 5.1–5.7). Переменная может быть проинициализирована в операторе DECLARE с помощью символа =. В одном сценарии допускается несколько операторов DECLARE.

Обратите внимание на последний оператор DECLARE в сценарии на рис. 13.3. В этом операторе объявлена переменная с типом TABLE. Этот тип позволяет создавать таблицы в памяти и использовать их для хранения промежуточных данных. Жизненный цикл всех объявленных переменных заканчивается вместе с завершением пакета, в котором находится соответствующий оператор DECLARE.

13.3. Операторы присваивания

Переменной, объявленной в операторе DELCLARE, можно присвоить значение тремя способами: инициализировать в операторе DECLARE (рис.13.2), присвоить значение с помощью операторов SET или SELECT (рис 13.3).

```
declare  @y decimal(7,2) = 0.0,
         @a char(10) ;
set @y = 2.55;
set @a = 'Hello';
select @y y, @a a;
go
```

y	a
2.55	Hello

```
declare  @y decimal(7,2) = 0.0,
         @a char(10) ;
select @y = 2.55,  @a = 'Hello';
select @y y, @a a;
go
```

y	a
2.55	Hello

Рис. 13.3. Присваивание значений с помощью операторов SET и SELECT

Единственное отличие операторов присваивания SET и SELECT заключается в возможности SELECT присваивать значения одновременно нескольким переменным.

13.4. Оператор вывода

Вывод данных в T-SQL возможен двумя способами: с помощью оператора SELECT можно сформировать выходной результирующий набор, с помощью оператора PRINT можно вывести строку в стандартный выходной поток. Каждый оператор SELECT формирует отдельный результирующий набор, а каждый оператор PRINT – строку в единственный выходной поток.

Сценарий на рис. 13.4 формирует два результирующих набора и три строки в стандартный выходной поток.

```
use BSTU
go

declare
      @StudentsCount    int  = (select count(*) from STUDENT),
      @AuditoriumCount  int  = (select count(*) from AUDITORIUM),
      @TeacherCount     int  = (select count(*) from TEACHER);

select AUDITORIUM_TYPE [Тип аудитории],
       count(*)        [Количество аудиторий]
       from AUDITORIUM group by AUDITORIUM_TYPE;
select PULPIT          [Кафедра],
       count(*)        [Количество дисциплин]
       from SUBJECT group by PULPIT;
print 'Количество студентов      : ' + cast (@StudentsCount    as varchar(10));
print 'Количество аудиторий      : ' + cast (@AuditoriumCount  as varchar(10));
print 'Количество преподавателей : ' + cast (@AuditoriumCount  as varchar(10));
go
```

Тип аудитории	Количество аудиторий
XXXX	1
ЛБ-К	5
ЛК	13

Кафедра	Количество дисциплин
ИСиТ	14
ЛЗиДВ	1
ЛМиПЗ	1

```
Количество студентов      : 178
Количество аудиторий      : 24
Количество преподавателей : 24
```

Рис. 13.4. Пример сценария, формирующего два результирующих набора и выводящего три строки в выходной поток

Обратите внимание: при формировании строк, выводимых оператором PRINT, применяется операция конкатенации (объединения) строк, для обозначения которой используется символ +.

13.5. Арифметические операции и встроенные математические функции

Для обозначения арифметических операций в T-SQL используется символика, применяемая и устоявшаяся в современных языках программирования. На рис. 13.5 приведен сценарий, демонстрирующий применение основных арифметических операций.

```
use BSTU
go
declare @x  int = 0, @x1 int = 1,
        @y  int = 1, @y1 int = 2,
        @z  numeric(7,2) = 2.0, @z1 numeric(5,1) = 33.0,
        @w  float(8) = 3.0, @w1 float(4) = 5.1E3;
------------------------------------------------------
set @x+=1;   print '@x-=1 =    '+ cast(@y as varchar(10));
set @y-=1;   print '@y-=1 =    '+ cast(@y as varchar(10));
set @z*=3.0; print '@z*=3.0 =  '+ cast(@z as varchar(10));
set @w/=2.0; print '@w/=2.0 =  '+ cast(@w as varchar(10));
set @z%=4;   print '@z/=4   =  '+ cast(@z as varchar(10));
------------------------------------------------------
set @z=1233.2 + 15.01;      print '@z=1233.2 + 15.01 = '+ cast(@z as varchar(10));
set @w=12.33E3 - 15.01E4;   print '@w=12.33E3 - 15.01E2 = '+ cast(@w as varchar(10));
set @x=1233 * 15;           print '@x=1233 * 15 = '+ cast(@x as varchar(10));
set @y=1233.33 / 15;        print '@y=1233.33 / 15 = '+ cast(@y as varchar(10));
set @x=-100%11;             print '@x=100%10 = '+ cast(@x as varchar(10));
------------------------------------------------------
select  @x1 = @x+@y, @y1 =@x-@y;
print '@x1 = @x+@y = '+ cast(@x1 as varchar(10))+ '; @y1 =@x-@y = '+cast(@y1 as varchar(10));
select @z1 = @x*@z1, @w1 =@w1/@y;
print '@z1 = @x*@z1 = '+ cast(@z1 as varchar(10))+ '; @w1 =@w1/@y = '+cast(@w1 as varchar(10));
select @w1 = @z%@x;   print '@w1 = @z%@x = '+ cast(@w1 as varchar(10));
```

```
@x-=1 =    1
@y-=1 =    0
@z*=3.0 =  6.00
@w/=2.0 =  1.5
@z/=4   =  2.00
@z=1233.2 + 15.01 = 1248.21
@w=12.33E3 - 15.01E2 = -137770
@x=1233 * 15 = 18495
@y=1233.33 / 15 = 82
@x=100%10 = -1
@x1 = @x+@y = 81; @y1 =@x-@y = -83
@z1 = @x*@z1 = -33.0; @w1 =@w1/@y = 62.1951
@w1 = @z%@x = 0.21
```

Рис. 13.5. Примеры использования основных арифметических операций

Следует обратить внимание, что в некоторых выражениях использовались переменные разных типов. В таких случаях интерпретатор T-SQL выполняет неявное преобразование типов. По возможности следует избегать смешивания типов в арифметических операциях или выполнять явное преобразование к общему типу.

Кроме того, MSS поддерживает широкий набор встроенных математических функций, которые можно применять в сценариях T-SQL для вычислений. На рис. 13.6 приведены примеры использования некоторых стандартных математических функций.

```
print 'Округление           : '+ cast(round(12345.12345, 2) as varchar(12));
print 'Нижнее целое         : '+ cast(floor(24.5) as varchar(12));
print 'Верхнее целое        : '+ cast(ceiling(24.5) as varchar(12));
print 'Возведение в степень: '+ cast(power(12.0, 2) as varchar(12));
print 'Логарифм по exp      : '+ cast(log(144.0) as varchar(12));
print 'Корень квадратный    : '+ cast(sqrt(144.0) as varchar(12));
print 'Экпонента            : '+ cast(exp(4.96981) as varchar(12));
print 'Абсолютное значение  : '+ cast(abs(-5) as varchar(12));
print 'Радианы              : '+ cast(radians(180.0) as varchar(20));
print 'Число пи             : '+ cast(pi() as varchar(12));
print 'Градусы              : '+ cast(degrees(pi()) as varchar(20));
print 'Синус                : '+ cast(sin(pi()) as varchar(12));
print 'Косинус              : '+ cast(cos(pi()) as varchar(12));
```

```
Округление           : 12345.12000
Нижнее целое         : 24
Верхнее целое        : 25
Возведение в степень: 144.0
Логарифм по exp      : 4.96981
Корень квадратный    : 12
Экпонента            : 144
Абсолютное значение  : 5
Радианы              : 3.141592653589793100
Число пи             : 3.14159
Градусы              : 180
Синус                : 1.22465e-016
```

Рис. 13.6. Примеры использования встроенных математических функций T-SQL

С полным перечнем встроенных математических функций T-SQL можно ознакомиться в изданиях [1, 5].

13.6. Работа со строками

Помимо операции конкатенации строк, упоминаемой ранее, язык T-SQL обеспечивает широкий набор встроенных функций для работы со строковыми данными. На рис.13.7 приведены примеры использования встроенных строковых функций.

```
print 'Подстрока              : '+  substring('1234567890', 3,2);
print 'Удалить пробелы слева  : '+  'X'+ ltrim('     67890');
print 'Удалить пробелы справа : '+  rtrim('12345     ') +'X';
print 'Нижний регистр         : '+  lower ('ВЕРХНИЙ РЕГИСТР');
print 'Верхний регистр        : '+  upper ('нижний регистр');
print 'Заменить               : '+  replace('1234512345', '5', 'X');
print 'Строка пробелов        : '+  'X'+ space(5) +'X';
print 'Повторить строку       : '+  replicate('12', 5);
print 'Найти по шаблону       : '+  cast (patindex ('%Y_Y%', '123456YxY7890') as varchar(5));
```

```
Подстрока              : 34
Удалить пробелы слева  : X67890
Удалить пробелы справа : 12345X
Нижний регистр         : верхний регистр
Верхний регистр        : НИЖНИЙ РЕГИСТР
Заменить               : 1234X1234X
Строка пробелов        : X     X
Повторить строку       : 1212121212
Найти по шаблону       : 7
```

Рис. 13.7. Примеры использования встроенных строковых функций T-SQL

С полным перечнем встроенных строковых функций T-SQL можно ознакомиться в пособиях [1, 5].

13.7. Работа с датами и временем

Для работы с датами и временем в языке T-SQL предусмотрен специальный набор встроенных функций. На рис. 13.8 представлен сценарий, демонстрирующий наиболее часто применяемые функции для работы с датами.

```
declare @dt datetime = getdate(), -- текущая дата
        @ed datetime = dateadd(d, 200, getdate()); -- текущая дата +200 дней
print 'Текущая дата           : '+ convert(varchar(12), @dt, 103 );
print 'Текущая дата+200 дней : '+ convert(varchar(12), @ed, 103 );
print 'День                   : '+ convert(varchar(12), day(@dt));
print 'День недели (1-вс)     : '+ convert(varchar(12), datepart(dw, @dt));
print 'Неделя                 : '+ convert(varchar(12), datepart(wk, @dt));
print 'Месяц                  : '+ convert(varchar(12), month(@dt));
print 'Квартал                : '+ convert(varchar(12), datepart(q, @dt));
print 'Год                    : '+ convert(varchar(12), year(@dt));
print '+1 день                : '+ convert(varchar(12), dateadd(d, 1, @dt), 103);
print '+1 неделя              : '+ convert(varchar(12), dateadd(wk, 1, @dt), 103);
print '+1 месяц               : '+ convert(varchar(12), dateadd(m, 1, @dt), 103);
print '+1 квартал             : '+ convert(varchar(12), dateadd(q, 1, @dt), 103);
print '+1 год                 : '+ convert(varchar(12), dateadd(y, 1, @dt), 103);
print 'Разница в днях         : '+ convert(varchar(12), datediff(d, @dt,@ed));
print 'Разница в неделях      : '+ convert(varchar(12), datediff(wk, @dt,@ed));
print 'Разница в месяцах      : '+ convert(varchar(12), datediff(m, @dt,@ed));
print 'Разница в годах        : '+ convert(varchar(12), datediff(yy, @dt,@ed));
```

```
Текущая дата           : 05/03/2014
Текущая дата+200 дней : 21/09/2014
День                   : 5
День недели            : 4
Неделя                 : 10
Месяц                  : 3
Квартал                : 1
Год                    : 2014
+1 день                : 06/03/2014
+1 неделя              : 12/03/2014
+1 месяц               : 05/04/2014
+1 квартал             : 05/06/2014
+1 год                 : 06/03/2014
Разница в днях         : 200
Разница в неделях      : 29
Разница в месяцах      : 6
Разница в годах        : 0
```

Рис. 13.8. Примеры использования встроенных функций T-SQL для работы с датами

Обратите внимание на третий числовой параметр функции CONVERT, используемой здесь для преобразования даты к строке. Этот параметр определяет строчный формат даты. Возможные его значения описываются в пособии [5].

Сценарий на рис. 13.9 демонстрирует основные встроенные функции T-SQL для работы со временем.

```
declare @t  time(7) =  sysdatetime(),    -- текущее время
        @et time(7) =  dateadd(mi, 12*60, getdate()),
        @dt datetime2 = sysdatetime(); -- текущая дата и время
print 'Текущее время         : '+ convert(varchar(12), @t);
print 'Текущие дата и время  : '+ convert(varchar(20), @dt, 120);
print 'Текущие время + 12час : '+ convert(varchar(20), @et);
print 'Часы                  : '+ convert(varchar(20), datepart(hh, @t));
print 'Минуты                : '+ convert(varchar(20), datepart(mi, @t));
print 'Секунды               : '+ convert(varchar(20), datepart(ss, @t));
print 'Милисекунды           : '+ convert(varchar(20), datepart(ms, @t));
print 'Текущее время + 1час  : '+ convert(varchar(20), dateadd(hh, 1, @t));
print 'Текущее время + 1мин  : '+ convert(varchar(20), dateadd(mi, 1, @t));
print 'Текущее время + 1сек  : '+ convert(varchar(20), dateadd(ss, 1, @t));
print 'Разница в днях        : '+ convert(varchar(20), datediff(d, @t,@et));
print 'Разница в час         : '+ convert(varchar(20), datediff(hh, @t,@et));
print 'Разница в мс          : '+ convert(varchar(20), datediff(ms, @t,@et));
```

```
Текущее время         : 11:11:38.248
Текущие дата и время  : 2014-03-05 11:11:38
Текущие время + 12час : 23:11:38.2470000
Часы                  : 11
Минуты                : 11
Секунды               : 38
Милисекунды           : 248
Текущее время + 1час  : 12:11:38.2489297
Текущее время + 1мин  : 11:12:38.2489297
Текущее время + 1сек  : 11:11:39.2489297
Разница в днях        : 0
Разница в час         : 12
Разница в мс          : 43199999
```

Рис. 13.9. Примеры использования встроенных функций T-SQL для работы со временем

Ознакомиться с полным перечнем встроенных функций T-SQL для работы с датами и временем можно в пособиях [1, 5].

13.8. Оператор RETURN

Немедленно завершить работу пакета можно с помощью оператора RETURN. На рис. 13.10 приведен сценарий, демонстрирующий работу оператора RETURN.

```
print 'Метка 1';
return;          -- выход из пакета
print 'Метка 2'; -- не выполняется
go
print 'Метка 3';
```

Рис. 13.10. Пример использования оператора RETURN

Сценарий на рис. 13.10 состоит из двух пакетов. В первом пакете выполняется оператор RETURN, осуществляющий немедленное завершение этого пакета и переход на выполнение следующего.

Кроме выхода из текущего пакета оператор RETURN применяется для завершения работы процедур, функций и триггеров (все рассматривается позже).

13.9. Операторные скобки BEGIN END

В некоторых случаях в сценарии T-SQL появляется необходимость указать, что группа следующих друг за другом операторов должна выполняться как единое целое. Это делается с помощью оператора BEGIN END. На рис. 13.11 приведен сценарий, демонстрирующий применение оператора BEGIN END.

```
print 'пакет1: начало-------------';
declare  @x0 int = 0;
print 'пакет1:@x0 =  '+ cast(@x0 as char(1));
begin
   declare  @x1 int = 1;
   print 'блок1: начало -----------';
   print 'блок1:@x0 =  '+ cast(@x0 as char(1));
   print 'блок1:@x1 =  '+ cast(@x1 as char(1));
   begin
      declare  @x2 int = 2;
      print '    блок2: начало -------';
      print '    блок2:@x0 =  '+ cast(@x0 as char(1));
      print '    блок2:@x1 =  '+ cast(@x1 as char(1));
      print '    блок2:@x2 =  '+ cast(@x2 as char(1));
      print '    блок2: конец --------';
   end;
   begin
      print '    блок3: начало -------';
      print '    блок3:@x0 =  '+ cast(@x0 as char(1));
      print '    блок3:@x1 =  '+ cast(@x1 as char(1));
      print '    блок3:@x2 =  '+ cast(@x2 as char(1));
      return;   -- выход из пакета
      print '    блок3: конец -------';
      print ''
   end;
   print 'блок1: конец -----------';
end;
print 'пакет1: конец-------------';
go
print 'пакет2: начало-------------';
print 'пакет2: конец--------------';
```

```
пакет1: начало-------------
пакет1:@x0 =  0
блок1: начало -----------
блок1:@x0 =  0
блок1:@x1 =  1
    блок2: начало -------
    блок2:@x0 =  0
    блок2:@x1 =  1
    блок2:@x2 =  2
    блок2: конец --------
    блок3: начало -------
    блок3:@x0 =  0
    блок3:@x1 =  1
    блок3:@x2 =  2
пакет2: начало-------------
пакет2: конец--------------
```

Рис. 13.11. Сценарий, демонстрирующий применение операторных скобок BEGIN END

Сценарий на рис. 13.11 состоит из двух пакетов. В первом пакете с помощью BEGIN END создано три блока операторов. Причем первый блок включает в себя два других: второй и третий. Необходимо обратить внимание на следующее.

1. Оператор RETURN, выполненный во внутреннем блоке (на рис. 13.11 – блок 3), осуществляет выход из пакета (пакет 1).
2. Область видимости переменной, объявленной внутри блока (переменная **@x3** в блоке 2), распространяется на следующие блоки (блок 3) и внешний блок (блок 1).

Другими словами, объединение операторов T-SQL в блоки с помощью BEGIN END никак не влияет на действие оператора RETURN (в любом случае осуществляется выход из пакета) и на область видимости переменных (область видимости распространяется строго сверху вниз).

13.10. Оператор WHILE

Оператор WHILE предназначен для организации программного цикла. Принцип работы оператора WHILE такой же, как в большинстве алгоритмических языков (например C++, Java).

Оператор WHILE содержит две составляющие: логическое выражение и тело цикла. Логическое выражение определяет условие выполнения тела цикла. Тело цикла содержит один

или более операторов, которые выполняются в том случае и до тех пор, пока логическое выражение принимает значение «истина».

На рис. 13.2 приведен сценарий, демонстрирующий работу оператора WHILE. Заметим, что тело цикла содержит только один оператор.

```
declare @i int = 0;
while @i <5         -- логическое выражение
     set @i+=1;     -- тело цикла
print '@i = '+ cast(@i as varchar(3));
```

```
@i = 5
```

Рис. 13.12. Пример использования простейшего оператора WHILE с телом цикла, содержащим один оператор

В том случае, если тело цикла содержит более, чем один оператор, следует применять операторные скобки BEGIN END (рис. 13.13).

На рис. 13.13 предлагается пример более сложного сценария, применяющего WHILE-цикл для итеративного обновления строк таблицы **AUDITORIUM**. В каждой итерации цикла осуществляется увеличение числовых значений столбца **AUDITORIUM_CAPACITY** в строках таблицы **AUDITORIUM**. Условием завершения цикла является достижение суммарного значения, превышающего 1500 в столбце **AUDITORIUM**.

```
declare @i int = 0;
while @i <5         -- логическое выражение
begin               -- тело цикла
     set @i+=1;
     print '@i = '+ cast(@i as varchar(3));
end;
```

```
@i = 1
@i = 2
@i = 3
@i = 4
@i = 5
```

Рис. 13.12. Пример использования простейшего оператора WHILE с телом цикла, содержащим более, чем один оператор

Заметим, без процедурных возможностей T-SQL, используя только «чистый» SQL, реализовать алгоритм, запрограммированный в сценарии на рис. 13.13, невозможно.

```
set nocount on;   -- не выводить:  row(s) affected
declare @ac int  =  (select sum(AUDITORIUM_CAPACITY)
                     from AUDITORIUM
                     where AUDITORIUM_TYPE in ('ЛК','ЛБ-К'));
print 'Общая вместимость аудиторий (ЛК,ЛБ-К) : ' + cast(@ac as varchar(6));
while (@ac < 1500)
begin
  update AUDITORIUM set AUDITORIUM_CAPACITY *= 1.1   where AUDITORIUM_TYPE = 'ЛК';
  update AUDITORIUM set AUDITORIUM_CAPACITY *= 1.05  where AUDITORIUM_TYPE = 'ЛБ-К';
  set @ac  =  (select sum(AUDITORIUM_CAPACITY)
               from AUDITORIUM
               where AUDITORIUM_TYPE in ('ЛК','ЛБ-К'));
  print space(38) +  ': ' + cast(@ac as varchar(6));
end;
print 'Общая вместимость аудиторий (ЛК,ЛБ-К) : ' + cast(@ac as varchar(6));
```

```
Общая вместимость аудиторий (ЛК,ЛБ-К) : 1245
                                      : 1355
                                      : 1465
                                      : 1586
Общая вместимость аудиторий (ЛК,ЛБ-К) : 1586
```

Рис. 13.13. Пример использования оператора WHILE для итеративного изменения строк таблицы **AUDITORUIM**

Обратите внимание на оператор SET NOCOUNT, расположенный в начале сценария на рис. 13.13. Значение ON, указанное в этом операторе, позволяет предотвратить вывод в стандартный выходной поток формируемых по умолчанию информационных сообщений сервера о количестве обработанных DML-операторами строк.

Управление выполнением WHILE-цикла может осуществляться с помощью инструкций BREAK и CONTINUE. На рис. 13.14 представлен пример, демонстрирующий использование этих инструкций.

```
declare @i int = 0, @x int;
while (@i < 5)
begin
  set @x = @i;
  set @i+=1;
  print 'A' + cast(@x as varchar(2));
  if @x = 2 continue;    --  в начало
  if @x = 3 break;       --  выйти
  print 'B' + cast(@x as varchar(2));
end;
```

```
A0
B0
A1
B1
A2
A3
```

Рис. 13.14. Пример использования инструкций BREAK и CONTINUE

Инструкция CONTINUE (рис. 13.14) прерывает выполнение блока операторов в теле WHILE и передает управление в начало цикла. Если в теле цикла выполнилась инструкция BREAK, работа цикла прекращается и управление передается следующему за WHILE оператору.

13.11. Оператор IF

Оператор ветвления IF позволяет в зависимости от значения логического выражения выполнять или не выполнять оператор или блок операторов сценария T-SQL. Примеры использования простейшей конструкции оператора IF приведены в сценарии на рис. 13.15.

```
use BSTU
go
declare @x int = (select count(*) from TEACHER);
if @x > 50   print '@x > 50';
if @x > 25   print '@x > 25';
if @x > 12   print '@x > 12';
print '@x = '+cast(@x as varchar(7));
```

```
@x > 25
@x > 12
@x = 39
```

Рис. 13.15. Пример использования простейшей конструкции оператора IF

За логическим выражением оператора IF располагается оператор, выполняющийся в случае истинности значения логического выражения (рис. 13.15). Если при этом требуется выполнить оператор при альтернативном значении логического выражения, следует применить конструкцию IF ELSE (13.16).

```
use BSTU
go
declare @x int = (select count(*) from TEACHER);
if @x > 50    print '@x > 50';
else          print '@x < 50';
if @x > 25    print '@x > 25';
else          print '@x < 25';
if @x > 12    print '@x > 12';
else          print '@x < 12';
print '@x = '+cast(@x as varchar(7));
```

```
@x < 50
@x > 25
@x > 12
@x = 39
```

Рис. 13.16. Пример использования конструкции IF ELSE

В сценарии на рис. 13.16 последовательно выполняются все операторы IF и соответственно вычисляются все значения логических выражений. В каждом операторе IF в зависимости от принимаемого логическим выражением значения выполняется тот или иной оператор PRINT.

Сократить количество вычислений значений логических выражений позволяет конструкция IF ELSE IF, в которой проверка осуществляется только в ветке ELSE оператора IF (рис. 13.7).

```
use BSTU
go
declare @x int = (select count(*) from TEACHER);
if @x > 50         print '@x > 50';
else if @x > 25    print '@x > 25';
else if @x > 12    print '@x > 12';
else               print '@x < 12';
print '@x = '+cast(@x as varchar(7));
```

```
@x > 25
@x = 39
```

Рис. 13.17. Пример использования конструкции IF ELSE IF

Заметим, что в рассмотренных примерах (рис. 13.16, 13.17) при истинном или альтернативном значении логического выражения оператора IF выполняется только один оператор. Для выполнения нескольких операторов применяются операторные скобки (рис. 13.18).

```
use BSTU
go
declare @x int = (select count(*) from TEACHER);
if (select count(*) from TEACHER) > 50
   begin
        print '@x > 50';
        print 'второй  оператор';
   end;
else
   begin
        print '@x < 50';
        print 'второй  оператор';
   end;
print '@x = '+cast(@x as varchar(7));
```

```
@x < 50
второй  оператор
@x = 39
```

Рис. 13.18. Пример использования операторных скобок BEGIN END в операторе IF

На рис. 13.19 представлен сценарий, демонстрирующий более сложные логические выражения, которые могут применяться в операторе IF.

В первом операторе IF (рис. 13.19) логическое выражение примет истинное значение в том случае, если в таблице **TEACHER** есть хотя бы одна строка.

Логическое выражение второго оператора IF (рис. 13.19) примет истинное значение, если в столбце **SUBJECT** одноименной таблицы есть хотя бы одно значение **БД**.

В третьем операторе IF (рис. 13.19) сравнивается максимальное значение даты из столбца **PDATE** таблицы **PROGRESS** с заданным с помощью операцией BETWEEN отрезком (границы включаются) дат. Причем правая граница отрезка задается значением переменной, проинициализированной функцией GETDATE (возвращает текущую дату и время), а левая – вычисленным с помощью функцией DATEADD значением.

```
declare @cd datetime = getdate()
if exists(select * from TEACHER)
     print 'а) истина';
else print 'а) ложь';
if 'БД' in (select [SUBJECT] from [SUBJECT])
     print 'б) истина';
else print 'б) ложь';
if (select max(PDATE) from PROGRESS) between dateadd(wk,-2,@cd) and @cd
     print 'в) истина';
else print 'в) ложь';
```

```
а) истина
б) истина
в) ложь
```

Рис. 13.19. Примеры логических выражений, используемых в операторе IF

13.12. Выражение CASE

Выражение CASE, как правило, применяется в операторе SELECT (в SELECT-списке, секциях WHERE, ORDER BY, HAVING) и служит для формирования одного из нескольких возможных значений. Может применяться в операторах UPDATE (в секциях SET и WHERE), DELETE (в секции WHERE). Выражение CASE имеет два формата: первый сравнивает результат одного выражения со списком заданных значений, второй – вычисляет набор логических выражений.

На рис. 13.20 представлен составной SELECT-запрос, использующий выражение CASE первого формата в SELECT-списке и в секции GROUP BY внутреннего запроса, а также в секции ORDER BY внешнего запроса.

Сразу за ключевым словом CASE (рис. 13.20) следует вычисляемое выражение (в нашем случае – это просто значения из столбца **NOTE** таблицы **PROGRESS** или из столбца **Категория оценки** результирующего набора внутреннего запроса), а далее одно или более предложений WHEN, содержащих значения для сравнений. Ключевое слово THEN, присутствующее в каждом предложении WHEN, указывает на один из возможных (соответствующих значению) результатов, формируемых CASE. Список WHEN-предложений завершается предложением ELSE, содержащим результат выражения CASE, формируемого в случае безуспешного сравнения значения исходного выражения со всеми предшествующими WHEN-значениями.

Обратите внимание на применение выражения CASE в секциях GROUP BY и ORDER BY.

Применение CASE в GROUP BY позволяет формировать группы строк на основе заданного в этом выражении условия. В примере на рис. 13.20 в зависимости от значений в столбце **NOTE** строки таблицы **PROGRESS** разбиваются на четыре группы (в первой группе нет ни одной строки), соответствующие четырем возможным результатам выражения CASE.

```
select *
from  (select  case NOTE
                when 10 then 'превосходно'
                when 9  then 'отлично'
                when 8  then 'хорошо'
                else         'остальные'
        end  [Категория оценки] , count(*) [Количество]
        from PROGRESS group by case NOTE
                                 when 10 then 'превосходно'
                                 when 9  then 'отлично'
                                 when 8  then 'хорошо'
                                 else         'остальные'
                               end ) t
 order by case [Категория оценки]
           when 'превосходно' then 0
           when 'отлично'     then 1
           when 'хорошо'      then 2
           else                    3
          end
```

Категория оценки	Количество
отлично	5
хорошо	4
остальные	25

Рис. 13.20. Пример использования выражения CASE первого формата

Выражение CASE в ORDER BY позволяет динамически формировать значения ключей сортировки. В примере на рис. 13.20 с помощью CASE в результате сортировки строка результирующего набора внутреннего запроса со значением **превосходно** в столбце **Категория оценки** будет первой (ключ сортировки **0**), со значением **отлично** (ключ сортировка **1**) и т. д.

Пример на рис. 13.21 демонстрирует применение выражения CASE второго формата в SELECT-списке и секции GROUP BY запроса SELECT.

```
use BSTU
go
 select  case
        when NOTE between 9 and 10 then 'отлично'
        when NOTE between 7 and 8  then 'хорошо'
        when NOTE between 4 and 6  then 'удовлетворительно'
        else   'неудовлетворительно'
        end  [Категория оценки] , count(*) [Количество]
 from PROGRESS group by case
                       when NOTE between 9 and 10 then 'отлично'
                       when NOTE between 7 and 8  then 'хорошо'
                       when NOTE between 4 and 6  then 'удовлетворительно'
                       else   'неудовлетворительно'
                       end
```

Категория оценки	Количество
отлично	5
удовлетворительно	20
хорошо	9

Рис. 13.21. Пример использования выражения CASE второго формата

Обратите внимание: каждое предложение WHEN содержит логическое выражение. Логические выражения проверяются на истинность сверху вниз, и при первом успешном сравнении формируется результирующее значение, указанное за ключевым словом THEN. В том случае, если ни одно из логических WHEN-выражений не принимает истинного значения, в качестве результата CASE формируется значение, указанное в предложении ELSE.

13.13. Оператор WAITFOR

С помощью оператора WAITFOR можно приостановить выполнение пакета. Оператор работает в двух режимах: DELAY и TIME.

В режиме DELAY оператор WAITFOR приостанавливает выполнение пакета на заданный интервал времени. В сценарии, представленном на рис. 13.22, демонстрируется выполнение оператора WAITFOR в режиме DELAY.

```
declare @t datetime = '00:00:03';
print sysdatetime();
waitfor delay '00:00:02';  -- приостановить на 2 сек
print sysdatetime();
waitfor delay @t;          -- приостановить на @t
print sysdatetime();
```

```
2014-03-18 12:09:26.0570469
2014-03-18 12:09:28.0580234
2014-03-18 12:09:31.0590000
```

Рис. 13.22. Пример использования оператора WAITFOR в режиме DELAY

В сценарии (рис. 13.22) последовательно выполняются два оператора WAITFOR DELAY. Первый приостанавливает выполнение пакета на 2 с, второй на 3 с. Обратите внимание на то, что во втором операторе время задается значением переменной, имеющей тип DATETIME и проинициализированной в операторе DECLARE с помощью строкового литерала. При анализе (рис. 13.23) значения переменной заметим, что ее значение соответствует дате 01.01.1900 и времени 0 ч 0 мин и 3 с. Следует помнить, что при вычислении интервала задержки в операторе WAITFOR DELAY используется разница между текущим и значением по умолчанию типа DATETIME (01.01.1900 00:00:00).

```
declare @t datetime = '00:00:03';
print 'год  : '+  cast(datepart(yy, @t) as varchar(4));
print 'месяц: '+  cast(datepart(m, @t)as varchar(4));
print 'день : '+  cast(datepart(d, @t)as varchar(4));
print 'час  : '+  cast(datepart(hh, @t)as varchar(4));
print 'мин  : '+  cast(datepart(mi, @t)as varchar(4));
print 'сек  : '+  cast(datepart(ss, @t)as varchar(4));
```

```
год  : 1900
месяц: 1
день : 1
час  : 0
мин  : 0
сек  : 3
```

Рис. 13.23. Представление интервала времени с помощью типа DATETIME

В примере на рис. 13.24 используется оператор WAIFOR в режиме TIME. В этом режиме можно приостановить выполнение потока до наступления заданного значения системного времени на сервере.

```
declare @t1 datetime =  dateadd(ss, 2, sysdatetime()),
        @t2 datetime =  dateadd(ss, 3, sysdatetime());
print sysdatetime();
waitfor time @t1;    -- приостановить до @t1
print sysdatetime();
waitfor time @t2;    -- приостановить до @t2
print sysdatetime();
```

```
2014-03-18 12:05:49.2650547
2014-03-18 12:05:51.2689609
2014-03-18 12:05:52.2699375
```

Рис. 13.24. Пример использования оператора WAITFOR в режиме TIME

В сценарии на рис. 13.24 последовательно выполняются два оператора WAITFOR TIME. Каждый из них приостанавливает выполнение пакета до момента времени, указанного значением переменной. Обратите внимание: переменным при инициализации устанавливается значение с помощью встроенной функции SYSTEMDATETIME, возвращающей текущее системное время сервера MSS. При этом переменной, применяемой в первом операторе, устанавливается время со смещением в 2 с относительно текущего времени, для переменной, используемой вторым оператором, – со смещением в 3 с.

13.14. Обработка ошибок

Для обработки ошибок выполнения в сценарии T-SQL предусмотрена конструкция, состоящая их двух блоков: TRY и CATCH. При этом блок TRY содержит код T-SQL, в котором могут возникнуть ошибки (говорят – охраняемый код), а блок CATCH – код, предназначенный для обработки ошибок. Ошибка, возникающая в охраняемом коде, приводит к немедленной передаче управления в блок обработки ошибок.

На рис. 13.25 представлен пример сценария, демонстрирующего применение механизма TRY/CATCH для обработки ошибки, возникающей при делении на нуль. С помощью трех операторов PRINT осуществляется трассировка хода выполнения сценария.

```
use BSTU
go
declare @x int = 3, @y int = 0, @z int;
begin try                   -- охраняемый код
    print 'begin try:A'
    set @z = @x/@y;         -- деление на 0
    print 'begin try:B'     -- если ошибка - не выполняется
end try
begin catch                 -- обработка ошибки
    print 'begin catch:C'
end  catch
```

```
begin try:A
begin catch:C
```

Рис. 13.25. Обработка ошибки с помощью механизма TRY/CATCH

Обратите внимание: второй оператор PRINT не выполняется, так как ошибка, возникшая при попытке деления на нуль, привела к немедленной передаче управления в блок обработки ошибок.

В блоке CATCH можно использовать несколько (табл. 13.1) системных функций, позволяющих диагностировать возникшую в охраняемом коде ошибку.

Таблица 13.1

Функции диагностики ошибок в сценарии T-SQL

Наименование функции	Функция возвращает
ERROR_NUMBER	Код (целочисленное значение) последней ошибки
ERROR_MESSAGE	Сообщение (символьная строка) об ошибке
ERROR_SEVERITY	Уровень серьезности (целочисленное значение) ошибки
ERROR_STATE	Метку (целочисленное значение) ошибки
ERROR_LINE	Код (целочисленное значение) последней ошибки
ERROR_PROCEDURE	Имя (символьная строка) процедуры или NULL

Пример на рис. 13.25 демонстрирует использование системных функций (табл. 13.1) при обработке ошибки, возникшей при нарушении ссылочной целостности.

```
use BSTU
go
begin try                -- охраняемый код
   delete PULPIT where PULPIT = 'ИСиТ'; -- нарушение FK
end try
begin catch              -- обработка ошибки
    print 'номер ошибки  : ' + cast(error_number()    as varchar(6));
    print 'сообщение     : ' + error_message();
    print 'уровень       : ' + cast(error_severity()  as varchar(6));
    print 'метка         : ' + cast(error_state()     as varchar(8));
    print 'номер строки  : ' + cast(error_line()      as varchar(8));
    if error_procedure() is not  null
      print 'имя процедуры : ' + error_procedure();
end  catch;
```

```
номер ошибки  : 547
сообщение     : The DELETE statement conflicted with the REFERENCE constraint "SUBJECT_PULPIT_FK".
                The conflict occurred in database "BSTU", table "dbo.SUBJECT", column 'PULPIT'.
уровень       : 16
метка         : 0
номер строки  : 2
```

Рис. 13.26. Применение системных функций для диагностирования ошибки

Разработчик сценария на языке T-SQL может сам сгенерировать ошибку с помощью специальной инструкции RAISERROR. В простейшем случае, при вызове инструкции можно передать три параметра: текстовое сообщение об ошибке, уровень серьезности ошибки и метку.

На рис. 13.27 приведен пример, демонстрирующий использование инструкции RAISERROR.

```
use BSTU
go
declare @x int = 3, @y int = 0, @z int;
begin try                  -- охраняемый код
    if @y = 0
          raiserror ('Деление на нуль', 10, 1);
    else  set @z = @x/@y;          -- деление на 0
    if exists(select * from PULPIT where PULPIT = 'ИСиТ')
          raiserror ('Нарушение РК', 11, 2);
    else  insert  PULPIT(PULPIT) values('ИСиТ');
    print 'begin try:B'
end try
begin catch                -- обработка ошибки
    print 'номер ошибки  : ' + cast(error_number()    as varchar(6));
    print 'сообщение     : ' + error_message();
    print 'уровень       : ' + cast(error_severity()  as varchar(6));
    print 'метка         : ' + cast(error_state()     as varchar(8));
    print 'номер строки  : ' + cast(error_line()      as varchar(8));
    if error_procedure() is not  null
      print 'имя процедуры : ' + error_procedure();
end  catch
```

```
Деление на нуль
номер ошибки  : 50000
сообщение     : Нарушение РК
уровень       : 11
метка         : 2
номер строки  : 7
```

Рис. 13.27. Применение инструкции RAISERROR

Инструкция RAISERROR в сценарии (рис. 13.27) применяется дважды: в первом случае для предупреждения об ошибке при делении на нуль, во втором – об ошибке нарушения ограничения PRIMARY KEY. Обратите внимание: инструкция ведет себя по-разному в зависимости от заданного параметром уровня серьезности ошибки. В первом случае (уровень серьезности 10) выполнение инструкции свелось к выводу в стандартный поток сообщения и выполнение охраняемого кода продолжилось. Во втором (уровень серьезности 11) – управление было передано в блок обработки ошибок.

Для более детального знакомства с методами обработки ошибок в сценариях T-SQL рекомендуется изучить источники [4, 5].

13.15. Применение курсора

При пояснении понятия курсора используют два непротиворечащих друг другу определения. Первое определение: курсор – это механизм обработки результирующего набора SELECT-запроса. Второе определение: курсор – это поименованная область памяти, содержащая результирующий набор SELECT-запроса.

Следует отметить, что при объявлении курсора допускается применения синтаксиса стандарта ISO и расширенного синтаксиса T-SQL. Дальнейшее описание будет ориентировано на синтаксис T-SQL.

13.15.1. Общая схема работы с курсором

Работа с любым курсором осуществляется в следующей последовательности.

1. Курсор объявляется в операторе DECLARE.
2. Курсор открывается с помощью оператора OPEN.

3. С помощью оператора FETCH считывается одна или несколько строк результирующего набора, связанного с курсором SELECT-оператора. Результат каждого считывания проверяется с помощью системной функции @@FETCH_STATUS.

4. Курсор закрывается с помощью оператора CLOSE.

5. Если курсор глобальный (поясняется позже), то курсор должен быть освобожден с помощью оператора DEALLOCATE.

На рис. 13.28 приведен сценарий, демонстрирующий общую схему работы с курсором. С помощью комментариев на рисунке отмечено соответствие операторов сценария перечисленным выше пунктам общей схемы. Поясним пример, приведенный на рис. 13.28.

1. В операторе DECLARE объявлен курсор с именем **C_TEACHER**. За ключевым словом CURSOR может следовать один или несколько атрибутов курсора, определяющих его возможности. После ключевого слова FOR следует оператор SELECT, связанный с данным курсором.

2. Перед тем как курсор будет использоваться, он должен быть открыт с помощью оператора OPEN. В зависимости от атрибутов курсора выполнение OPEN осуществляется по-разному и будет пояснено позже, но в любом случае становится доступным для чтения результирующий набор строк, соответствующий связанному с курсором оператору SELECT.

```
use BSTU
go
declare @tid char(10), @tnm char(40), @tgn char(1);
declare c_teacher cursor global                                  -- 1
                  for select TEACHER, TEACHER_NAME, GENDER       -- 1
                      from TEACHER where PULPIT = 'ПОиСОИ';      -- 1
open c_teacher;                                                  -- 2
fetch  c_teacher into @tid, @tnm, @tgn;                          -- 3
while @@fetch_status = 0                                         -- 3
begin
   print @tid + ' '+ @tnm + ' '+ @tgn;
   fetch  c_teacher into @tid, @tnm, @tgn;                       -- 3
end;
close  c_teacher;                                                -- 4
deallocate c_teacher;                                            -- 5
```

```
БРТШВЧ      Барташевич Святослав Александрович       м
ЮДНКВ       Юденков Виктор Степанович                м
ШМК         Шмаков Михаил Сергеевич                  м
АРС         Арсентьев Виталий Арсентьевич            м
БЛВ         Беляев Валерий Павлович                  м
```

Рис. 13.28. Пример, демонстрирующий общую схему работы с курсором

3. После выполнения оператора OPEN указатель текущей строки курсора установлен на первую строку результирующего набора. Оператор FETCH считывает одну строку из результирующего набора и продвигает указатель на следующую строку. В операторе FETCH после ключевого слова INTO следует список переменных. Количество переменных в этом списке должно быть равно количеству столбцов результирующего набора строк, а порядок их должен соответствовать порядку перечисления столбцов в SELECT-списке связанного запроса. После выполнения FETCH следует проверить значение системной функции **@@FETCH_STATUS**, которая может вернуть одной значений, перечисленных в табл. 13.2.

4. После завершения работы с курсором он должен быть закрыт с помощью оператора CLOSE. Выполнение этого оператора освобождает ресурсы, выделенные курсору при выполнении оператора OPEN. Для возобновления работы с курсором в рамках пакета необходимо его открыть с помощью оператора OPEN.

Таблица 13.2

Коды возврата функции @@FETCH_STATUS

Возвращаемое значение	Описание
0	Оператор FETCH выполнен успешно. Строка считана из результирующего набора
–1	Достигнут конец результирующего набора. Строка не считывается
–2	Выбранная строка отсутствует в БД

5. В тех случаях, когда курсор является глобальным (при объявлении указан атрибут GLOBAL) и не предполагается его дальнейшее применение, следует выполнить оператор DEALLOCATE, выполняющий освобождение всех ресурсов, выделенных курсору при его объявлении в операторе DECLARE.

13.15.2. Локальные и глобальные курсоры

Курсоры могут быть глобальными и локальными.

Локальный курсор может применяться в рамках одного пакета и ресурсы, выделенные ему при объявлении, освобождаются сразу после завершения работы пакета. Признаком того, что курсор является локальным, служит атрибут LOCAL, указанный при объявлении курсора (рис. 13.29).

```
declare @tid char(10), @tnm char(40), @tgn char(1);
declare c_teacher cursor local
                  for select TEACHER, TEACHER_NAME, GENDER
                      from TEACHER where PULPIT = 'ПОиСОИ';
open c_teacher;
fetch  c_teacher into @tid, @tnm, @tgn;
print '1. '+ @tid + ' '+ @tnm + ' '+ @tgn;
go ---------------------------------------
declare @tid char(10), @tnm char(40), @tgn char(1);
fetch  c_teacher into @tid, @tnm, @tgn;
print '2. '+@tid + ' '+ @tnm + ' '+ @tgn;
go ---------------------------------------
```

```
1. БРТШВЧ     Барташевич Святослав Александрович       м
Msg 16916, Level 16, State 1, Line 0
A cursor with the name 'c_teacher' does not exist.
```

Рис. 13.29. Пример, демонстрирующий работу с локальным курсором

Обратите внимание (рис. 13.29): локальный (атрибут LOCAL) курсор с именем **C_TEACHER** доступен для использования только в том пакете, в котором был объявлен. Следующий пакет уже не «видит» курсора.

Глобальный курсор может быть объявлен, открыт и использован в разных пакетах. Выделенные ему при объявлении ресурсы освобождаются только после выполнения оператора DEALLOCATE или при завершении сеанса пользователя. Для того чтобы объявить глобальный курсор, следует применить атрибут GLOBAL (13.30).

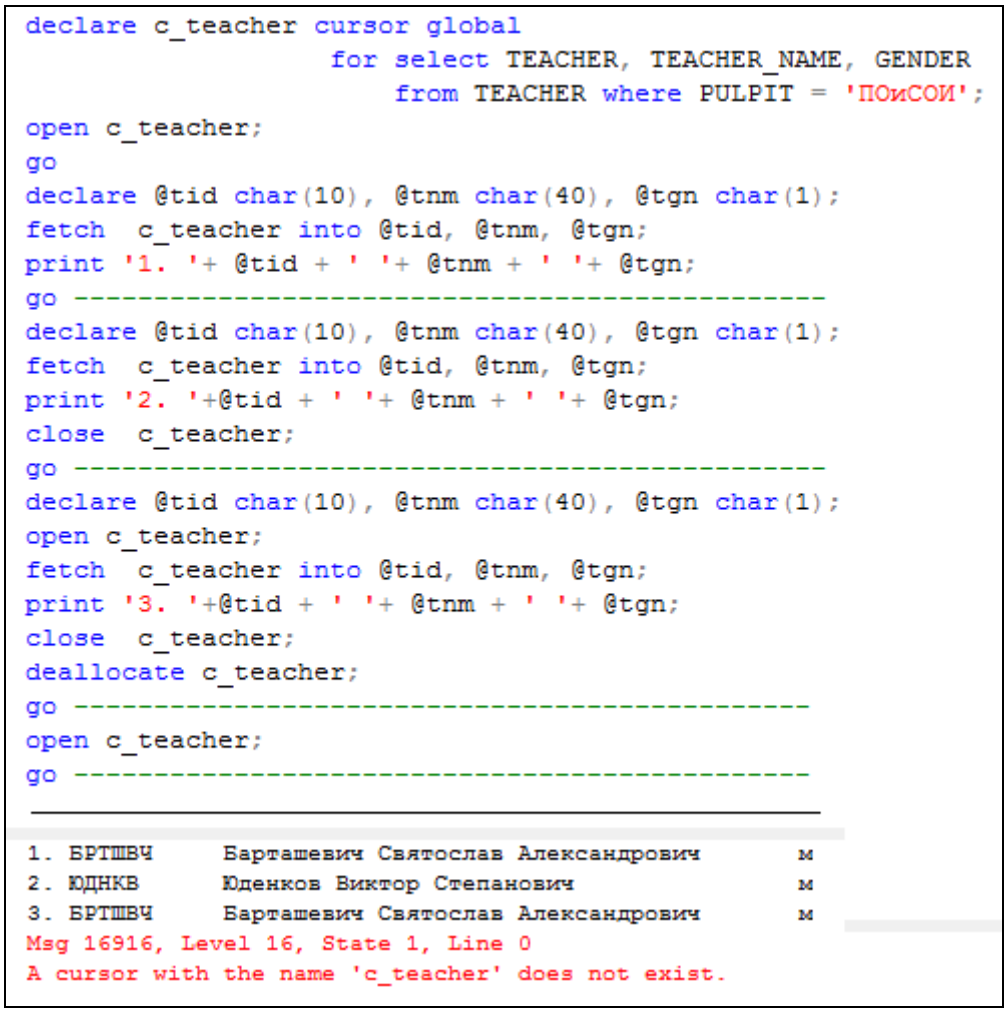


Рис. 13.30. Пример, демонстрирующий работу с глобальным курсором

В примере на рис. 13.30 последовательно выполняются четыре пакета. При этом глобальный (атрибут GLOBAL) курсор с именем **C_TEACHER** объявлен и открыт в первом, а использован во втором и третьем пакетах. В третьем пакете выполнен оператор DEALLOCATE, сделавший невозможным использование курсора в четвертом пакете.

Обычно по умолчанию курсор является глобальным. Однако значение этого атрибута по умолчанию может быть установлено с помощью оператора ALTER DATABASE SET CURSOR_DEFAULT [5].

13.15.3. Статические и динамические курсоры

Курсоры могут быть статическими и динамическими.

При объявлении статического курсора должен указываться атрибут STATIC. Открытие статического курсора приводит к выгрузке результирующего набора строк в динамически созданную временную таблицу системной БД **TEMPDB**, и все дальнейшие курсорные операции осуществляются с этой таблицей. После открытия курсора изменения в исходных таблицах, которые осуществляются в рамках этого или других сеансов, не будут отражаться в курсорном результирующем наборе.

Рис. 13.31–13.33 демонстрируют особенности работы со статическим курсором.

```
use BSTU
go
select *  from TEACHER where PULPIT = 'ПОиСОИ';
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
БРТШВЧ	Барташевич Святослав Александрович	м	ПОиСОИ
ЮДНКВ	Юденков Виктор Степанович	м	ПОиСОИ
ШМК	Шмаков Михаил Сергеевич	м	ПОиСОИ
АРС	Арсентьев Виталий Арсентьевич	м	ПОиСОИ
БЛВ	Беляев Валерий Павлович	м	ПОиСОИ

Рис. 13.31. Результирующий набор SELECT-запроса, выполненного до исполнения сценария на рис. 13.32

На рис. 13.31 представлен результат SELECT-запроса, отражающий содержимое результирующего набора, который будет сформирован в сценарии на рис. 13.32 при открытии курсора с именем **C_TEACHER.**

В сценарии на рис. 13.32 объявлен и затем отрыт курсор **C_TEACHER**. При этом сформированный результирующий набор (он будет идентичен набору, представленному на рис. 13.31) выгружается во временную таблицу системной БД **TEMPDB**. Заметьте: все изменения (операторы UPDATE, DELETE и INSERT ограничеы с сверху и снизу линиями комментарий), выполненные в исходной таблице **TEACHER**, никак не отразились на результате выборки строк курсора с помощью оператора FETCH. В то же время, если снова выполнить SELECT-запрос, аналогичный представленному на рис. 13.31 и курсорному запросу, то получим ожидаемый результат, отражающий изменения в таблице **TEACHER** (рис. 13.3).

```
use BSTU
go
declare @tid char(10), @tnm char(40), @tgn char(1);
declare c_teacher cursor local static
                    for select TEACHER, TEACHER_NAME, GENDER
                        from TEACHER where PULPIT = 'ПОиСОИ';
open c_teacher;
print   'Количество строк : '+cast(@@CURSOR_ROWS as varchar(5));
-----------------------------------------------------------------
update TEACHER set TEACHER_NAME = 'XXX' where TEACHER = 'АРС';
delete TEACHER where TEACHER = 'ШМК';
insert TEACHER(TEACHER, PULPIT) values ('AAA', 'ПОиСОИ');
-----------------------------------------------------------------
fetch  c_teacher into @tid, @tnm, @tgn;
while @@fetch_status = 0
begin
   print @tid + ' '+ @tnm + ' '+ @tgn;
   fetch  c_teacher into @tid, @tnm, @tgn;
end;
close  c_teacher;
-----------------------------------------------------------------

Количество строк : 5
БРТШВЧ     Барташевич Святослав Александрович       м
ЮДНКВ      Юденков Виктор Степанович                м
ШМК        Шмаков Михаил Сергеевич                  м
АРС        Арсентьев Виталий Арсентьевич            м
БЛВ        Беляев Валерий Павлович                  м
```

Рис. 13.32. Пример, демонстрирующий особенности работы статического курсора

```
use BSTU
go
select *  from TEACHER where PULPIT = 'ПОиСОИ';
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
БРТШВЧ	Барташевич Святослав Александрович	м	ПОиСОИ
ЮДНКВ	Юденков Виктор Степанович	м	ПОиСОИ
ААА	NULL	м	ПОиСОИ
АРС	XXX	м	ПОиСОИ
БЛВ	Беляев Валерий Павлович	м	ПОиСОИ

Рис. 13.33. Результирующий набор SELECT-запроса,
выполненного после исполнения сценария на рис. 13.32

Обратите внимание на применение в сценарии на рис. 13.2 системной функции **@@CURSOR_ROWS**, с помощью которой можно получить количество строк в результирующем наборе статического курсора.

Вернем таблицу **TEACHER** в состояние, предшествующее выполнению сценария на рис. 13.32 и соответствующее запросу на рис. 13.31. После этого выполним сценарий, представленный на рис. 13.34, в котором применяется динамический курсор.

```
use BSTU
go
declare @tid char(10), @tnm char(40), @tgn char(1);
declare c_teacher cursor local dynamic
                    for select TEACHER, TEACHER_NAME, GENDER
                        from TEACHER where PULPIT = 'ПОиСОИ';
open c_teacher;
print   'Количество строк : '+cast(@@CURSOR_ROWS as varchar(5));
-------------------------------------------------------------
update TEACHER set TEACHER_NAME = 'XXX' where TEACHER = 'АРС';
delete TEACHER where TEACHER = 'ШМК';
insert TEACHER(TEACHER, PULPIT) values ('ААА', 'ПОиСОИ');
-------------------------------------------------------------
fetch  c_teacher into @tid, @tnm, @tgn;
while @@fetch_status = 0
begin
   print @tid + ' '+ @tnm + ' '+ @tgn;
   fetch  c_teacher into @tid, @tnm, @tgn;
end;
close  c_teacher;
select @@fetch_status
use BSTU
go
```

```
Количество строк : -1
БРТШВЧ      Барташевич Святослав Александрович       м
ЮДНКВ       Юденков Виктор Степанович                м

АРС         XXX                                      м
БЛВ         Беляев Валерий Павлович                  м
```

Рис. 13.34. Пример, демонстрирующий особенности работы
динамического курсора

Заметим: изменение, выполненное оператором UPDATE, отразилось в результирующем наборе; удаленной (DELETE) строке соответствует строка со значением NULL во всех столбцах, а добавленная (INSERT) в исходную таблицу строка в курсорный результирующий набор не введена.

Обратите внимание: значение функции **@@CURSOR_ROWS** после открытия динамического курсора всегда равно **–1**.

Следует отметить, что кроме STATIC и DYNAMIC-курсоров допускается использовать курсоры с атрибутом KEYSET, занимающие промежуточное положение между STATIC и DYNAMIC. Для знакомства с KEYSET- курсорами рекомендуется изучить пособие [5].

13.15.4. Курсоры с атрибутами FORWARD_ONLY и SCROLL

По умолчанию для курсора установлен атрибут SCROLL, позволяющий применять оператор FETCH с дополнительными опциями позиционирования.

Для демонстрации возможностей позиционирования в SCROLL- курсорах предварительно рассмотрим запрос на рис. 13.35, применяющий функцию ROW_NUMBER.

```
use BSTU
go
select  row_number() over (order by TEACHER) RN, TEACHER
from TEACHER where PULPIT = 'ИСиТ'
```

RN	TEACHER
1	АКНВЧ
2	БРКВЧ
3	БРС
4	ГРМН
5	ГРН
6	ДТК
7	ЖЛК
8	КБЛ
9	КЛСНВ
10	КШК
11	ЛЩНК
12	МРЗ
13	ПСТВЛВ
14	РМНК
15	СМЛВ
16	УРБ

Рис. 13.35. Применение оконной функции ROW_NUMBER для сортировки нумерации строк

Функция ROW_NUMBER предназначена для нумерации строк результирующего набора и относится к классу оконных встроенных функций T-SQL. Основным признаком того, что применяется оконная функция, является ключевое слово OVER, за которым в скобках следуют опции, разбивающие результирующий набор на подмножества (окна) строк, а также сортирующие эти строки внутри подмножества. В приведенном примере разбиение не осуществляется, поэтому все строки результирующего набора только сортируются. Результатом выполнения функции является номер строки в результирующем наборе. Более подробно с оконными функциями можно ознакомиться в пособии [5].

```
use BSTU
go
declare @rn int, @tc char(10);
declare c_teacher cursor local dynamic scroll
                  for select  row_number() over (order by TEACHER) RN, TEACHER
                      from TEACHER where PULPIT = 'ИСиТ'
open c_teacher;
fetch  c_teacher into @rn, @tc;
print 'следующая         : ' +  cast(@rn as varchar(3))+ '['+rtrim(@tc)+']';
fetch  last from  c_teacher into @rn, @tc;
print 'последняя         : ' +  cast(@rn as varchar(3))+ '['+rtrim(@tc)+']';
fetch  first from  c_teacher into @rn, @tc;
print 'первая            : ' +  cast(@rn as varchar(3))+ '['+rtrim(@tc)+']';
fetch  absolute 10 from  c_teacher into @rn, @tc;
print 'с начала 10я      : ' +  cast(@rn as varchar(3))+ '['+rtrim(@tc)+']';
fetch  absolute -10 from  c_teacher into @rn, @tc;
print 'с конца 10я       : ' +  cast(@rn as varchar(3))+ '['+rtrim(@tc)+']';
fetch  relative  5  from  c_teacher into @rn, @tc;
print 'с тек. вперед  5я : ' +  cast(@rn as varchar(3))+ '['+rtrim(@tc)+']';
fetch  relative  -5  from  c_teacher into @rn, @tc;
print 'с тек. назад  5я  : ' +  cast(@rn as varchar(3))+ '['+rtrim(@tc)+']';
fetch  next from  c_teacher into @rn, @tc;
print 'следующая         : ' +  cast(@rn as varchar(3))+ '['+rtrim(@tc)+']';
fetch  prior from  c_teacher into @rn, @tc;
print 'предыдущая        : ' +  cast(@rn as varchar(3))+ '['+rtrim(@tc)+']';
close c_teacher;
```

```
следующая         : 1[АКНВЧ]
последняя         : 16[УРБ]
первая            : 1[АКНВЧ]
с начала 10я      : 10[КШК]
с конца 10я       : 7[ЖЛК]
с тек. вперед  5я : 12[МРЗ]
с тек. назад  5я  : 7[ЖЛК]
следующая         : 8[КБЛ]
предыдущая        : 7[ЖЛК]
```

Рис. 13.36. Пример, демонстрирующий опции позиционирования оператора FETCH, доступные в SCROLL-курсорах

Курсоры с атрибутом FORWARD_ONLY допускают чтение строк результирующего набора только в одном порядке – сверху вниз (FETCH NEXT).

13.15.5. Курсоры с атрибутами FOR UPDATE и READ_ONLY

Курсор дает возможность программисту последовательно обрабатывать строки результирующего набора строк, полученного в результате выполнения связанного с курсором SELECT-запроса.

Для курсоров с установленным свойством FOR UPDATE, помимо чтения данных из строк с помощью оператора FETCH, можно эти строки изменять или удалять с помощью специального формата операторов UPDATE и DELETE.

На рис. 13.37–13.38 представлен пример использования курсора со свойством FOR UPDATE.

```
use BSTU
go
select * from TEACHER where PULPIT = 'ПОиСОИ';
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
БРТШВЧ	Барташевич Святослав Александрович	м	ПОиСОИ
ЮДНКВ	Юденков Виктор Степанович	м	ПОиСОИ
ШМК	Шмаков Михаил Сергеевич	м	ПОиСОИ
АРС	Арсентьев Виталий Арсентьевич	м	ПОиСОИ
БЛВ	Беляев Валерий Павлович	м	ПОиСОИ

Рис. 13.37. Результирующий набор SELECT-запроса, выполненного до запуска сценария на рис. 13.38

Обратите внимание: при объявлении курсора **C_TEACHER** (рис. 13.38) используется ключевая фраза FOR UPDATE, дающая возможность применять операторы DELETE и UPDATE для удаления или изменения строки таблицы.

В секции WHERE операторы DELETE и UPDATE (рис. 13.38) используют операцию CURRENT OF, для которой указывается имя курсора (в примере **C_TEACHER**). Такой формат операторов позволяет удалять или изменять строки в таблице (**TEACHER**), соответствующие текущей позиции курсора в результирующем наборе.

```
use BSTU
go
declare @tid char(10), @tnm char(40), @tgn char(1);
declare c_teacher cursor local dynamic
                  for select TEACHER,TEACHER_NAME,GENDER
                       from TEACHER where PULPIT = 'ПОиСОИ' for update;
open c_teacher;
fetch  c_teacher into @tid, @tnm, @tgn;
delete TEACHER where current of c_teacher;
fetch  c_teacher into @tid, @tnm, @tgn;
update TEACHER set TEACHER_NAME = 'XXXX' where current of c_teacher;
close c_teacher;
select * from TEACHER where PULPIT = 'ПОиСОИ';
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
ЮДНКВ	XXXX	м	ПОиСОИ
ШМК	Шмаков Михаил Сергеевич	м	ПОиСОИ
АРС	Арсентьев Виталий Арсентьевич	м	ПОиСОИ
БЛВ	Беляев Валерий Павлович	м	ПОиСОИ

Рис. 13.38. Сценарий, демонстрирующий применение курсора со свойством FOR UPDATE

Свойством FOR UPDATE могут обладать только динамические (обладают по умолчанию) и ключевые (в пособии не рассматриваются) курсоры.

Курсору может быть установлено свойство READ_ONLY, запрещающее применение операции CURRENT OF в секции WHERE операторов DELETE и UPDATE. Однако свойство READ_ONLY не мешает применять эти операторы без использования CURRENT OF (рис. 13.39).

```
use BSTU
go
declare @tid char(10), @tnm char(40), @tgn char(1);
declare c_teacher cursor local dynamic read_only
                  for select TEACHER,TEACHER_NAME,GENDER
                       from TEACHER where PULPIT = 'ПОиСОИ';
open c_teacher;
fetch  c_teacher into @tid, @tnm, @tgn;
delete TEACHER where TEACHER = @tid;
fetch  c_teacher into @tid, @tnm, @tgn;
update TEACHER set TEACHER_NAME = 'XXXX' where TEACHER = @tid;
close c_teacher;
select * from TEACHER where PULPIT = 'ПОиСОИ';
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
ЮДНКВ	XXXX	м	ПОиСОИ
ШМК	Шмаков Михаил Сергеевич	м	ПОиСОИ
АРС	Арсентьев Виталий Арсентьевич	м	ПОиСОИ
БЛВ	Беляев Валерий Павлович	м	ПОиСОИ

Рис. 13.39. Сценарий, демонстрирующий применение курсора со свойством READ_ONLY

В сценарии на рис. 13.39 при объявлении курсора **C_TEACHER** используется ключевое слово READ_ONLY. Для удаления и изменения строк таблицы **TEACHER** применяются операторы DELETE и UPDATE, не применяющие операцию CURRENT OF в секции WHERE.

Следует отметить, что атрибут READ_ONLY имеет смысл только для динамических и ключевых курсоров, статические курсоры по определению применяются только для чтения.

13.16. Применение транзакций

Состояние БД называют согласованным, если хранящиеся данные удовлетворяют всем ограничениям целостности и не противоречат друг другу. Первая часть определения очевидна, вторая требует пояснения. Приведем пример.

Рассмотрим БД банковской системы, учитывающей операции на текущем счете клиента в таблице **Текущий счет**, а кассовые операции (выдача и прием наличных денежных средств) – в таблице **Касса** (рис. 13.40).

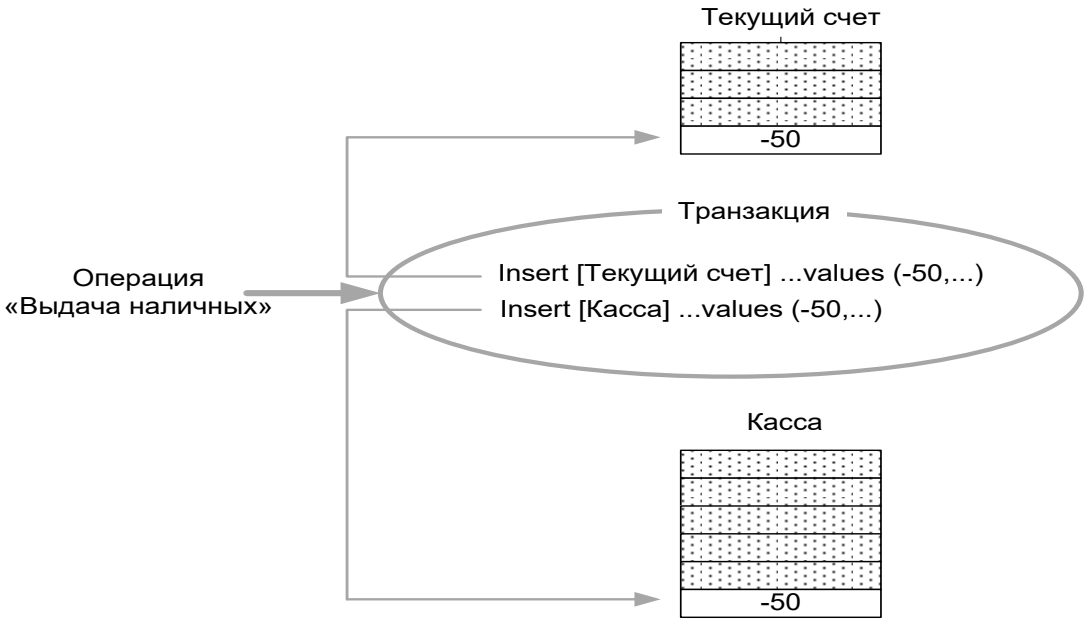


Рис. 13.40. Схема выполнения операции «Выдача наличных»

Если клиент банка получает 50 единиц денежных средств со своего текущего счета в виде наличности, то этому событию в гипотетической банковской системе будет соответствовать банковская операция «Выдача наличных», которая в БД представляется в виде результата выполнения двух операторов INSERT, добавляющих по строке в таблицы **Текущий счет** и **Касса**. Добавленная строка в таблицу **Текущий счет** будет отражать тот факт, что обязательства банка перед клиентом теперь уменьшатся на 50 единиц (уменьшился остаток на текущем счете), а строка в таблице **Касса** – факт уменьшения наличных денег в кассе банка на те же 50 единиц. Если два оператора INSERT выполнились успешно, то БД перешла из одного согласованного состояния в другое, тоже согласованное.

Предположим теперь, что после успешного выполнения первого оператора INSERT, добавившего строку в таблицу **Текущий счет**, произошел системный сбой, не позволивший выполнить второй оператор INSERT и, соответственно, клиенту получить причитающиеся ему наличные денежные средства. Произойдет рассогласование, которое заключается в том, что обязательства банка перед клиентом уменьшились, а кассовый остаток остался прежним – клиент не получил наличные. В этом случае БД перешла из согласованного состояния в несогласованное.

Как видно из приведенного примера, понятие «согласованное состояние БД» зависит от той предметной области, состояние которой отражается в БД. Очевидно, что если бы рассматривалась БД, хранящая, например, данные о ходе технологического процесса, то согласованность бы понималась иначе.

С другой стороны, рассогласование в любом случае будет происходить в тогда, когда при необходимости одновременно выполнить несколько изменений в БД, выполняется только их часть, а другая часть не выполнится вообще. Тогда, по всей видимости, если бы был механизм, позволяющий объединять несколько операторов изменения БД в одну атомарную операцию, которая либо полностью выполнялась, либо полностью не выполнялась, то эта проблемы была бы решена.

В первом приближении транзакцией будем назвать механизм БД, позволяющий объединять несколько операторов, изменяющих БД таким образом, чтобы при выполнении этой совокупности операторов они или все выполнились, или все не выполнились.

13.16.1. Основные свойства транзакций

Обычно при определении понятия транзакции используют перечень требований к этому механизму, сформулированный в конце 70-х гг. Джимом Греем в виде аббревиатуры ACID: Atomicity – атомарность, Consistency – согласованность, Isolation – изолированность, Durability – долговечность.

Атомарность транзакции уже пояснялась выше на примере банковской операции. Она заключается в гарантии того, что никакая транзакция не будет зафиксирована в БД частично: операторы изменения БД, включенные в транзакцию, либо выполнятся все, либо не выполнится ни один. В БД это свойство реализуется с помощью механизма отката, позволяющего отменить выполненные, но незафиксированные изменения БД.

Свойство согласованности заключатся в том, что транзакция должна фиксировать новое согласованное состояние БД. Это свойство тоже пояснялось на примере банковской операции. Здесь лишь еще раз отметим, что понятие согласованности зависит от предметной области, состояние которой отражается в БД.

Современные СУБД допускают одновременную (часто говорят – параллельную) работу нескольких транзакций. При этом параллельные транзакции могут изменять одни и те же данные в БД. Свойство изолированности гарантирует с точностью до определенного уровня отсутствие взаимного влияния параллельных транзакций на результаты их выполнения. Современные СУБД могут обеспечивать несколько уровней изолированности транзакций. Уровни изолированности транзакций, поддерживаемые MSS, будут рассматриваться позже.

Любая транзакция завершается либо фиксацией (оператор COMMIT), либо откатом (оператор ROLLBACK). Откат – это отмена всех изменений в БД, выполненных в рамках

транзакции. Фиксация – подтверждение успешного завершения транзакции. Свойство долговечности заключается в том, что изменения в БД, выполненные и зафиксированные транзакцией, не могут быть отменены, кроме как с помощью новой транзакции.

13.16.2. Автоматическая фиксация транзакций

Режим автоматической фиксации (часто просто автофиксация) транзакции используется MSS по умолчанию. После завершения каждый оператор T-SQL фиксируется или откатывается. Если оператор выполняется без ошибок, то автоматически выполняется фиксация. В противном случае состояние БД откатывается назад.

Действие режима автофиксации продолжается до тех пор, пока не запускается явная транзакция или не включается режим неявной транзакции. Если явная транзакция завершается (фиксируется или откатывается) или выключается режим неявной транзакции, то автоматически осуществляется возврат в режим автофиксации.

13.16.3. Неявные транзакции

Режим неявной транзакции может быть включен для текущего соединения с сервером БД с помощью специальной инструкции T-SQL: SET IMPLICIT_TRANSACTIONS ON. После выполнения этой инструкции соединение переключается из режима автофиксации в режим неявной транзакции. Обратное переключение осуществляется с помощью аналогичной инструкции, но с использованием ключевого слова OFF вместо ON.

После того как для соединения установлен режим неявной транзакции, она автоматически будет начинаться, если выполняется один из следующих операторов T-SQL:

- CREATE, DROP;
- ALTER TABLE;
- INSERT, DELETE, UPDATE, SELECT, TRUNCATE TABLE;
- OPEN, FETCH;
- GRANT, REVOKE.

Неявная транзакция продолжается до тех пор, пока не будет выполнен оператор фиксации (COMMIT) или оператор отката (ROLLBACK) транзакции. На рис. 13.41, 13.42 приведены примеры, демонстрирующие использование режима неявной транзакции.

```
-- действует режим автофиксации
set nocount on
if  exists (select * from  SYS.OBJECTS        -- таблица X есть?
            where OBJECT_ID= object_id(N'DBO.X') and TYPE in (N'U'))
     drop table X;           -- начало транзакции
declare @c int, @flag char = 'c';    -- commit/rollback?
set implicit_transactions  on  -- включить режим неявной транзакции
    create table X(K int );     -- начало транзакции
    insert X values (1),(2),(3);
    set @c = (select count(*) from X);
    print 'количество строк в таблице X: ' + cast( @c as varchar(2));
    if @flag = 'c'  commit;     -- завершение транзакции: фиксация
    else            rollback;  -- завершение транзакции: откат
set implicit_transactions  off -- выключить режим неявной транзакции
-- действует режим автофиксации
if  exists (select * from  SYS.OBJECTS       -- таблица X есть?
            where OBJECT_ID= object_id(N'DBO.X') and TYPE in (N'U'))
     print 'таблица X есть';
else print 'таблицы X нет'
```

```
количество строк в таблице X: 3
таблица X есть
```

Рис. 13.4.1 Пример использования режима неявной транзакции
с фиксацией изменений

В примере на рис. 13.41 с помощью инструкции SET осуществляется переключение соединения в режим неявной транзакции. Неявная транзакция стартует при выполнении оператора CREATE TABLE и завершается фиксацией изменений с помощью оператора COMMIT. После этого осуществляется возврат в режим автофиксации (инструкция SET OFF). Заметим: созданная таблица с именем X сохранилась после выполнения сценария.

Сценарий на рис. 13.42 отличается от сценария на рис. 13.41 только значением переменной с именем **@flag**, установленным при инициализации. Следствием этого является выполнение оператора ROLLBACK, приведшее к отмене всех изменений в БД, осуществленных последней транзакцией (созданная таблица **X** не сохранилась).

```
-- действует режим автофиксации
set nocount on
if  exists (select * from  SYS.OBJECTS        -- таблица X есть?
            where OBJECT_ID= object_id(N'DBO.X') and TYPE in (N'U'))
     drop table X;            -- начало транзакции
declare @c int, @flag char = 'r';   -- commit/rollback?
set implicit_transactions  on  -- включить режим неявной транзакции
    create table X(K int );    -- начало транзакции
    insert X values (1),(2),(3);
    set @c = (select count(*) from X);
    print 'количество строк в таблице X: ' + cast( @c as varchar(2));
    if @flag = 'c'  commit;    -- завершение транзакции: фиксация
    else            rollback;  -- завершение транзакции: откат
set implicit_transactions  off -- выключить режим неявной транзакции
-- действует режим автофиксации
if  exists (select * from  SYS.OBJECTS        -- таблица X есть?
            where OBJECT_ID= object_id(N'DBO.X') and TYPE in (N'U'))
     print 'таблица X есть';
else print 'таблицы X нет'
```

```
количество строк в таблице X: 3
таблицы X нет
```

Рис. 13.42. Пример использования режима неявной транзакции с откатом изменений

13.16.4. Явные транзакции

Переключение в режим явной транзакции осуществляется с помощью оператора BEGIN TRANSACTION. Транзакцию должен завершать один из операторов: COMMIT TRANSACTION или ROLLBACK TRANSACTION. Ключевое слово TRANSACTION в перечисленных операторах может быть сокращено до TRAN. После завершения явной транзакции происходит возврат в исходный режим (автофиксации или неявной транзакции).

На рис. 13.43 приведен пример использования явной транзакции. Транзакция начинается внутри TRY-блока и в случае успешного выполнения завершатся здесь же (оператор COMMIT). Если при выполнении транзакции возникает ошибка, в CATH-блоке формируется и выводится соответствующее сообщение, а затем выполняется откат (ROLLBACK).

```
use BSTU
go
begin try
 begin tran  -- начало  явной транзакции
   delete [SUBJECT] where [SUBJECT]='ДМ';
   insert [SUBJECT] values ('ТГ','Теория графов',  'ИСиТ'),
                           ('КБ', 'Комбинаторика', 'ИСиТ'),
                           ('МЛ', 'Мат. логика', 'ВМ');
 commit tran;  -- фиксация транзакции
end try
begin catch
  print 'ошибка: '+ case
                       when error_number() = 547 and
                            patindex('%SUBJECT_PULPIT_FK%', error_message()) > 0
                            then 'нет кафедры с таким кодом'
                       when error_number() = 2627 and
                            patindex('%SUBJECT_PK%', error_message()) > 0
                            then 'дублирование кода дисциплины'
                       else 'неизвестная ошибка: '+
                            cast(error_number() as  varchar(5))
                            + error_message()
                     end;
  if @@trancount > 0 rollback tran; -- откат  транзакции
end catch;
```

```
ошибка: нет кафедры с таким кодом
```

Рис. 13.43. Пример использования явной транзакции

Обратите внимание на применение системной функции @@TRANCOUNT. Она возвращает уровень вложенности транзакции. Значение, превышающее нуль, означает, что транзакция не завершена.

Кроме того, в сценарии на рис. 13.43 применяется встроенная функция PATINDEX, позволяющая определить в строке позицию первого символа подстроки, заданную шаблоном. С помощью этой функции в тексте сообщения об ошибке отыскивается имя ограничения целостности.

Если транзакция состоит из нескольких относительно независимых блоков операторов T-SQL, изменяющих БД, то может быть использован оператор SAVE TRANSACTION, формирующий контрольную точку транзакции. Контрольная точка – это метка в явной транзакции, определяющая момент времени, к которому может быть выполнен откат.

На рис. 13.44 приведен сценарий, демонстрирующий применение трех контрольных точек в явной транзакции. Контрольные точки с именами **p1**, **p2**, и **p3** с помощью операторов SAVE TRANSACTION установлены перед операторами INSERT. Если при выполнении INSERT формируется исключение, в блоке CATCH осуществляется откат к последней контрольной пройденной точке, а также фиксация изменений, выполненных до контрольной точки.

```
use BSTU
go
declare @point varchar(32); -- макс. длина имени 32
begin try
 begin tran  -- начало  явной транзакции
   delete [SUBJECT] where [SUBJECT]='ДМ';
 set @point = 'p1'; save tran @point;   -- контрольная точка p1
   insert [SUBJECT] values ('ТГ','Теория графов',  'ИСиТ');
 set @point = 'p2'; save tran @point;   -- контрольная точка p2
   insert [SUBJECT] values ('КБ', 'Комбинаторика', 'ИСиТ');
 set @point = 'p3'; save tran @point;   -- контрольная точка p3
   insert [SUBJECT] values ('МЛ', 'Мат. логика', 'ВМ');
 commit tran;  -- фиксация транзакции
end try
begin catch
  print 'ошибка: '+ case
                       when error_number() = 547 and
                            patindex('%SUBJECT_PULPIT_FK%', error_message()) > 0
                            then 'нет кафедры с таким кодом'
                       when error_number() = 2627 and
                            patindex('%SUBJECT_PK%', error_message()) > 0
                            then 'дублирование кода дисциплины'
                       else 'неизвестная ошибка: '+
                            cast(error_number() as  varchar(5))
                            + error_message()
                  end;
 if @@trancount > 0
 begin
    print 'котрольная точка: '+ @point;
    rollback tran @point; -- откат к контрольной точке
    commit tran;          -- фиксация изменений выполненных до контрольной точки
 end;
end catch;
```

```
ошибка: нет кафедры с таким кодом
котрольная точка: p3
```

Рис. 13.44. Пример использования явной транзакции с контрольными точками

При выполнении сценария (рис. 13.44) последний оператор INSERT вызвал исключение с кодом 547. В блоке CATCH откат выполнен к контрольной точке **p3**. Это позволило сохранить и зафиксировать (оператор COMMIT) выполненные до этой контрольной точки изменения в таблице **SUBJECT** (удаление одной строки и добавление двух новых). Запрос на рис. 13.45 демонстрирует изменения в таблице SUBJECT после выполнения сценария на рис. 13.44.

```
use BSTU
go
select * from [SUBJECT] where [SUBJECT] in  ('ДМ', 'ТГ','КБ', 'МЛ');
```

SUBJECT	SUBJECT_NAME	PULPIT
КБ	Комбинаторика	ИСиТ
ТГ	Теория графов	ИСиТ

Рис. 13.45. Результат выполнения сценария на рис. 13.44

13.16.5. Уровни изолированности транзакций

Одним из основных свойств транзакции является изолированность. Под изолированностью подразумеваются ограничения на одновременное чтение или изменение данных в параллельно работающих транзакциях.

MSS поддерживает пять уровней изолированности транзакций: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SNAPSHOT и SERIALIZABLE. Для установки уровня изолированности текущей для транзакций в текущем подключении служит инструкция SET TRANSACTION ISOLATION LEVEL.

Обычно уровни изолированности транзакций рассматриваются относительно трех проблем параллельного доступа: неподтвержденное чтение, неповторяющееся чтение, фантомное чтение.

На рис. 13.46 схематически изображена проблема неподтвержденного чтения. Двумя прямоугольниками с метками **A** и **B** на схеме изображены параллельные транзакции. Направленная линия между прямоугольниками представляет ось времени с двумя отметками t_1 и t_2. До момента t_1 транзакцией **B** выполняется два оператора: INSERT и UPDATE. Эти операторы изменяют таблицы БД, но до момента времени t_2 не фиксируют и не откатывают эти изменения.

После момента t_1 транзакция **A** считывает содержимое таблиц, измененных транзакцией **B**, и «видит» измененные или добавленные транзакцией **B** строки. При этом изменения остаются до момента t_2 в неподтвержденном состоянии, т. е. могут быть как зафиксированными, так и отмененными. Другими словами: неподтвержденное чтение фактически свидетельствует об отсутствии изолированности транзакций.

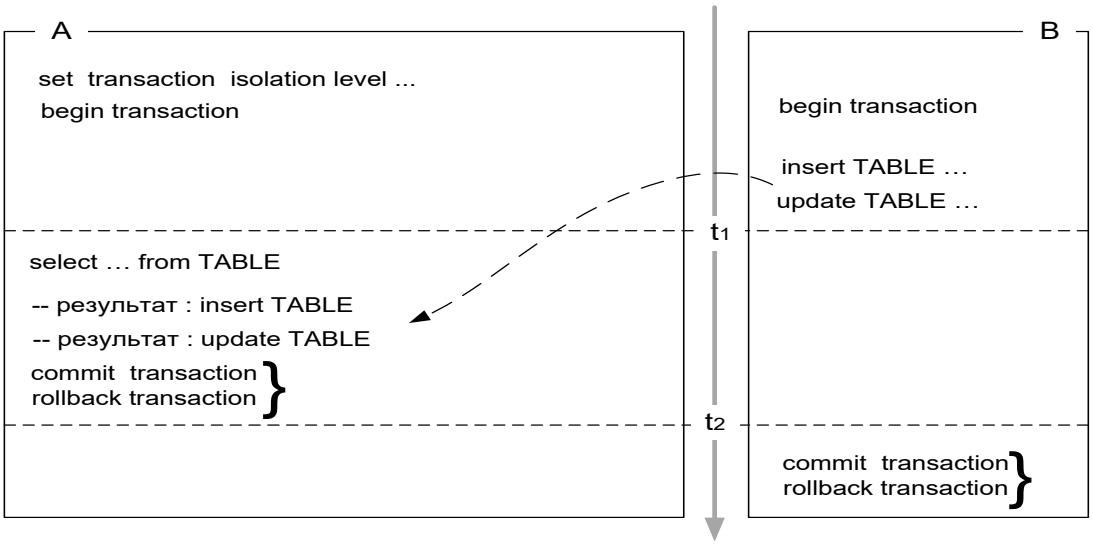


Рис. 13.46. Схема, демонстрирующая неподтвержденное чтение

Схема на рис. 13.47 демонстрирует проблему неповторяющегося чтения. Как и ранее, на схеме изображены две параллельно работающие транзакции **A** и **B**, выполняющие операции с БД в рамках трех интервалов времени.

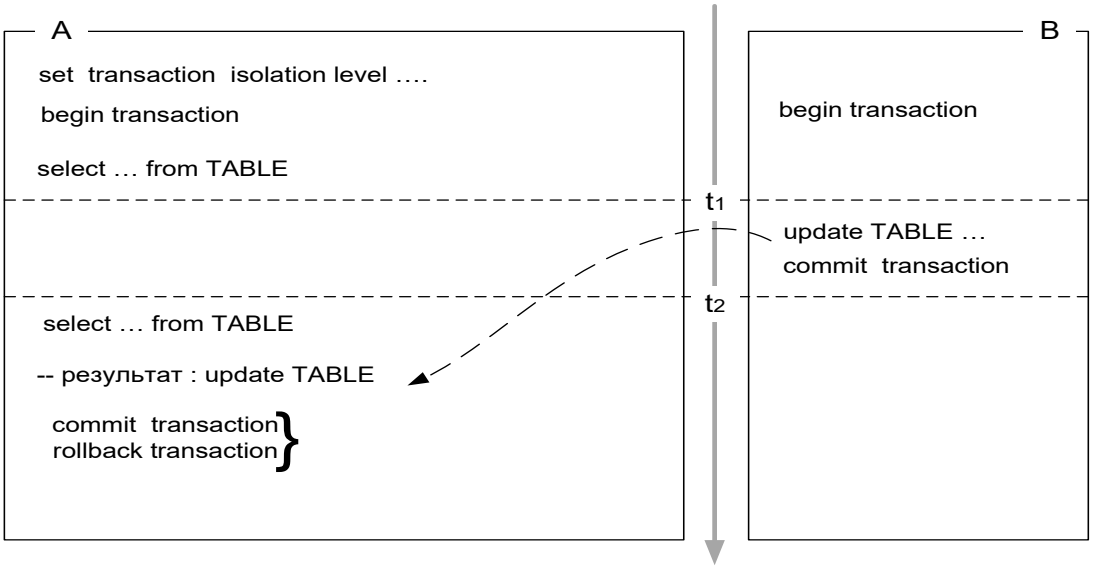


Рис. 13.47. Схема, демонстрирующая неповторяющееся чтение

До момента t_1 транзакция **A** выполняет чтение строк из таблиц БД с помощью оператора SELECT.

После момента t_1, но до момента t_2 транзакция **B** выполняет изменение строк, прочитанных в транзакции **A**, и фиксирует эти изменения (COMMIT).

После момента t_2 транзакция **A** повторяет чтение тех же строк и «видит» изменения, выполненные транзакцией **B**. Другими словами: при повторном чтении данных транзакцией не гарантируется, что они не изменены или не удалены другой транзакцией.

Схема на рис. 13.48 демонстрирует проблему фантомного чтения. Проблема похожа на предыдущий случай. Транзакцией **B** выполняется добавление с фиксацией строк в таблицу, предварительно прочитанную транзакцией **A**. При повторном чтении транзакция «видит» добавленные транзакцией **B** строки.

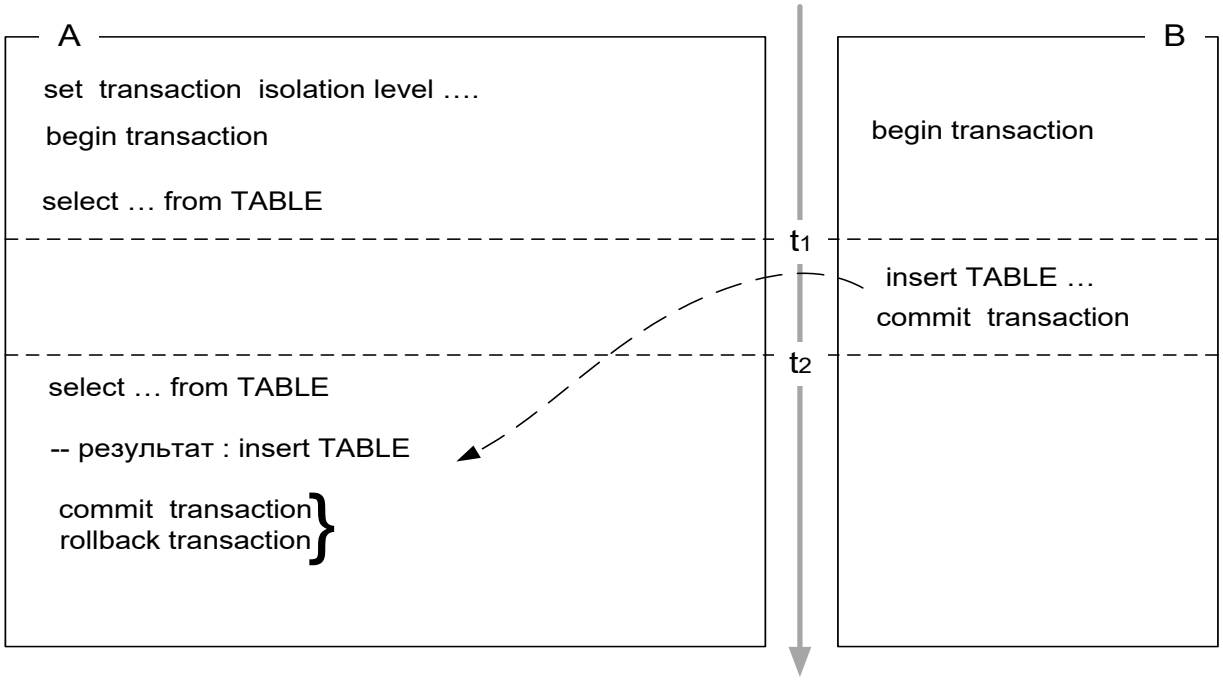


Рис. 13.48. Схема, демонстрирующая фантомное чтение

В табл. 13.3 перечислены все пять уровней изолированности транзакций, поддерживаемых MSS. Для каждого уровня символом «плюс» отмечается наличие соответствующей проблемы. Например, уровень READ COMMITTED (действует по умолчанию) не допускает неподтвержденного чтения, но при этом возможно неповторяющееся и фантомное чтение, а уровень SERIALIZABLE решает все три проблемы.

Анализ таблицы 13.3 может привести к неверному заключению: выбрав уровень SERIALIZABLE, программист решает все проблемы изолированности транзакции. Это действительно так, но решив проблему изолированности, разработчик столкнется с проблемой производительности.

Дело в том, что изолированность транзакций достигается с помощью механизма блокировок. Например, пусть для транзакции A установлен уровень REPEATABLE READ (рис. 13.47), а параллельно работающая транзакция **B** пытается изменить (оператор UPDATE) предварительно считанную (SELECT) транзакцией A строку.

Таблица 13.3

Свойства уровней изоляции транзакций

Уровень изоляции	Неподтвержденное чтение	Неповторяющееся чтение	Фантомное чтение
READ UNCOMMTITTED	+	+	+
READ COMMITED	–	+	+
REPEATABLE READ	–	–	+
SNAPSHOT	–	–	–
SERIALIZABLE	–	–	–

Для того чтобы транзакции **A** гарантировать заданный уровень изолированности, сервер установит на прочитанные этой транзакцией строки блокировку, запрещающую выполнять UPDATE в рамках другой транзакции. Это приведет к тому, что при попытке транзакцией **B** выполнить UPDATE для блокированной строки, транзакция **B** будет переведена в состояние ожидания до тех пор, пока транзакцией **A** не будет выполнен один из операторов завершения транзакции (COMMIT или ROLLBACK).

С одной стороны, чем выше уровень изолированности, тем меньше строк кода потребуется программисту при разработке приложения, но это приведет к увеличению времени выполнения параллельно работающих транзакций, при условии, что они работают с общими данными. С другой стороны, при низком уровне изолированности (например, READ COMMITTED) программисту придется предусматривать проблемы неповторяющегося и фантомного чтения.

Для исследования взаимодействия двух параллельных транзакций следует их выполнить в двух разных сеансах.

На рис. 13.49, 13.50 представлены два параллельно работающих сценария, содержащих явные транзакции **A** и **B**.

```
--- A ------------------------------------------
use BSTU
go
set transaction isolation level READ UNCOMMITTED
begin transaction
------------------------- t1 -----------------
  select @@SPID, 'insert FACULTY' 'результат', *
  from FACULTY where FACULTY = 'ИТ';
  select @@SPID, 'update PULPIT'  'результат', *
  from PULPIT  where FACULTY = 'ИТ';
commit;
------------------------- t2 -----------------
```

(No column name)	результат	FACULTY	FACULTY_NAME
54	insert FACULTY	ИТ	Информационных технологий

(No column name)	результат	PULPIT	PULPIT_NAME	FACULTY
54	update PULPIT	ИСиТ	Информационн...	ИТ

Рис. 13.49. Неподтвержденное чтение: транзакция **A**

```
--- B ----------------------------------------
use BSTU
go
begin transaction
  select @@SPID
  insert FACULTY values ('ИТ', 'Информационных технологий');
  update PULPIT set FACULTY = 'ИТ' where PULPIT = 'ИСиТ'
------------------------- t1 --------------------

------------------------- t2 --------------------
rollback;
```

(No column name)
55

Рис. 13.50. Неподтвержденное чтение: транзакция **B**

С помощью комментариев (рис. 13.49, 13.50) в обоих сценариях обозначены моменты времени t_1 и t_2, соответствующие одноименным временным засечкам на рис. 13.46. Кроме того, применяется системная встроенная функция @@SPID, возвращающая системный идентификатор процесса, назначенный сервером текущему подключению.

Для моделирования схемы, представленной на рис. 13.46, сценарии следует выполнять в три этапа:

1) последовательно оба сценария до отметки t_1;
2) последовательно оба сценария с отметки t_1 до отметки t_2;
3) последовательно оба сценария с отметки t_2 и до конца.

Обратите внимание:

1) для транзакции **A** установлен уровень изоляции READ UNCOMMITTED;
2) транзакцией **A** прочитаны незафиксированные изменения строк таблиц **FACULTY** и **PULPIT**.

Сценарии на рис. 13.51 и 13.52 демонстрируют неповторяющееся чтение (рис. 13.47) в транзакции с уровнем изолированности READ COMMITTED.

```
--- A ----------------------------------------
use BSTU
go
set transaction isolation level READ COMMITTED
begin transaction
select count(*) from PULPIT
where FACULTY = 'ИТ';
------------------------- t1 -------------------

------------------------- t2 -------------------
select  'update PULPIT'  'результат', count(*)
from PULPIT  where FACULTY = 'ИТ';
commit;
```

(No column name)
0

результат	(No column name)
update PULPIT	1

Рис. 13.51. Неповторяющееся чтение: транзакция **A**

```
--- B ------------------------------------------
use BSTU
go
begin transaction
------------------------- t1 --------------------
  update PULPIT set FACULTY = 'ИТ' where PULPIT = 'ИСиТ';
commit;
------------------------- t2 --------------------
```

Рис. 13.52. Неповторяющееся чтение: транзакция **B**

Обратите внимание:

1) для транзакции **A** установлен уровень изоляции READ COMMITTED;

2) при повторном чтении в транзакции **A** (второй оператор SELECT) прочитаны зафиксированные изменения строк таблицы **PULPIT**.

Сценарии на рис. 13.53 и 13.54 демонстрируют фантомное чтение (рис. 13.48) в транзакции с уровнем изолированности READ REAPETABLE.

```
--- A ------------------------------------------
 use BSTU
 go
 set transaction isolation level REPEATABLE READ
 begin transaction
 select TEACHER from TEACHER
 where PULPIT = 'ПОиСОИ';
------------------------- t1 ----------------

------------------------- t2 ----------------
 select case
        when TEACHER = 'ПТР' then  'insert TEACHER'
        else ' '
        end 'результат', TEACHER
 from TEACHER  where PULPIT = 'ПОиСОИ';
 commit;
```

TEACHER
БРТШВЧ
ШМК
АРС
БЛВ
ЮДНКВ

результат	TEACHER
	БРТШВЧ
insert TEACHER	ПТР
	ШМК
	АРС
	БЛВ
	ЮДНКВ

Рис. 13.53. Фантомное чтение: транзакция **A**

```
--- B ------------------------------------------
use BSTU
go
begin transaction
------------------------- t1 --------------------
  insert TEACHER values ('ПТР', 'Петров Петр Петрович','м', 'ПОиСОИ');
commit;
------------------------- t2 --------------------
```

Рис. 13.54. Фантомное чтение: транзакция **B**

Обратите внимание:

1) для транзакции **A** установлен уровень изоляции REPEATABLE READ;

2) при повторном чтении в транзакции **A** (второй оператор SELECT) прочитаны добавленная и зафиксированная строка таблицы **TEACHER**.

13.16.6. Уровень изолированности SNAPSHOT

Уровень изолированности SNAPSHOT (снимок) обеспечивает полную изолированность (см. табл. 13.3) и при этом не блокирует строки таблицы. Транзакция с этим уровнем изолированности «видит» строки таблиц в версии, в которой она застала их при первом чтении. При последующих чтениях транзакция будет «видеть» только те изменения БД, которые произвела сама.

Применение уровня изолированности SNAPSHOT требует установки опции ALLOW_SNAPSHOT_ISOLATION для БД на уровне сервера (рис. 13.55).

```
use MASTER
go
-- разрешить применять уровень изолированности SNAPSHOT
alter database BSTU set allow_snapshot_isolation on
```

Рис. 13.55. Установка опции, разрешающей в БД **BSTU** применять уровень изолированности SNAPSHOT

На рис. 13.56 и 13.57 демонстрируется взаимодействие двух транзакций: **A** и **B**. При этом для транзакции **A** установлен уровень изолированности SNAPSHOT.

```
- A ----------------------------------------
  use BSTU
  go
  set transaction isolation level SNAPSHOT
  begin transaction
     select TEACHER from TEACHER where PULPIT = 'ПОиСОИ';
----------------------- t1 -----------------
     delete TEACHER where TEACHER = 'АРС';
     insert TEACHER values ('ИВН', 'Иванов Сергей Борисович','м', 'ПОиСОИ');
     update TEACHER set TEACHER='ШМКВ' where TEACHER = 'ШМК';
----------------------- t2 -----------------
     select TEACHER from TEACHER  where PULPIT = 'ПОиСОИ';
  commit;
```

TEACHER
БРТШВЧ
ЮДНКВ
ШМК
АРС
БЛВ

TEACHER
БРТШВЧ
ЮДНКВ
ШМКВ
БЛВ
ИВН

Рис. 13.56. Уровень изолированности SNAPSHOT: транзакция **A**

```
--- B ----------------------------------------
use BSTU
go
begin transaction
------------------------- t1 --------------------
  insert TEACHER values ('ПТР', 'Петров Петр Петрович','м', 'ПОиСОИ');
  delete TEACHER where TEACHER = 'ЮДНКВ';
  update TEACHER set TEACHER = 'БРТ' where TEACHER = 'БРТШВЧ';
commit;
------------------------- t2 --------------------
```

Рис. 13.57. Уровень изолированности SNAPSHOT: транзакция **B**

Обратите внимание:

1) для транзакции **A** установлен уровень изоляции SNAPSHOT;

2) при повторном чтении в транзакции **A** (второй оператор SELECT) отражаются только те изменения, которые она выполнила сама.

Ошибка в SNAPSHOT-транзакции возникает при попытке изменить или удалить строку, которая к этому времени была уже удалена или изменена другой транзакцией. Сценарии на рис. 13.58 и 13.59 демонстрируют две параллельных транзакции: **A** и **B**. Для транзакции **A** установлен уровень изолированности SNAPSHOT.

```
------ A -------------------------------------------------------
        use BSTU
        go
        set transaction isolation level SNAPSHOT;
        begin try
            begin tran
               select TEACHER from TEACHER where PULPIT = 'ПОиСОИ'
------------------------- t1 -------------------------------
               waitfor delay '00:00:10';  -- задержка 10 сек
------------------------- t2 -------------------------------
               delete TEACHER where TEACHER = 'БРТШВЧ';
               select TEACHER from TEACHER  where PULPIT = 'ПОиСОИ';
             commit;
         end try
         begin catch
            print 'код ошибки : '+cast(error_number() as varchar(10));
            print 'сообщение  : '+error_message();
            rollback;
         end catch;
```

TEACHER
ЮДНКВ
ШМК
АРС
БПВ
БРТШВЧ

```
код ошибки : 3960
сообщение  : Snapshot isolation transaction aborted due to update conflict.
             You cannot use snapshot isolation to access table 'dbo.TEACHER'
             directly or indirectly in database 'BSTU' to update, delete, or
             insert the row that has been modified or deleted by another transaction.
             Retry the transaction or change the isolation level for the update/delete statement.
```

Рис. 13.58. Уровень изолированности SNAPSHOT: транзакция **A**

```
-- B ----------------------------------------------

------------------------ t1 --------------------
 use BSTU
 go
 begin transaction
    update TEACHER set TEACHER = 'БРТ' where TEACHER = 'БРТШВЧ';
 commit;
------------------------- t2 --------------------
```

Рис. 13.59. Уровень изолированности SNAPSHOT: транзакция **B**

Обратите внимание:

1) для транзакции **A** установлен уровень изоляции SNAPSHOT;

2) при попытке транзакцией **A** удалить строку (оператор DELETE), которая была изменена (UPDATE) транзакцией **B**, возникает ошибка с кодом **3960**;

3) в транзакции **A** применяется оператор WAITFOR DELAY, позволяющий приостановить выполнение сценария с транзакцией **A** для того, чтобы выполнить после момента времени t_1 сценарий с транзакцией **B**.

Применение уровня изолированности SNAPSHOT, с одной стороны, позволяет быть уверенным в полной изоляции транзакции, с другой – возлагает на разработчика обязанность

разрешать конфликты (обрабатывать ошибки с кодом **3960**), возникающие при одновременном изменении строк в нескольких транзакциях.

13.16.7. Вложенные транзакции

Транзакция, выполняющаяся в рамках другой транзакции, называется вложенной. При работе с вложенными транзакциями следует учитывать следующие свойства.

1. COMMIT вложенной транзакции действует только на внутренние операции вложенной транзакции.
2. ROLLBACK внешней транзакции отменяет зафиксированные операции внутренней транзакции.
3. ROLLBACK вложенной транзакции действует на операции внешней и внутренней транзакции, а также завершает обе транзакции.
4. Уровень вложенности транзакции можно определить с помощью встроенной системной функции @@TRANCOUT.

На рис. 13.60 представлен пример, демонстрирующий свойство 2 при работе с вложенной транзакцией.

```
use BSTU
go
begin tran                      --  внешняя транзакция
  insert FACULTY values ('ИТ', 'Информационных технологий');
  begin tran     --  внутренняя транзакция
     update PULPIT set FACULTY = 'ИТ' where PULPIT = 'ИСиТ'
  commit;          --  внутренняя транзакция
if @@trancount > 0  rollback;     -- внешняя транзакция
select
 (select count(*) from PULPIT   where PULPIT  = 'ИТ') 'PULPIT',
 (select count(*) from FACULTY  where FACULTY = 'ИТ') 'FACULTY';
```

PULPIT	FACULTY
0	0

Рис. 13.60. Отмена оператором ROLLBACK внешней транзакции зафиксированных операций внутренней транзакции

Обратите внимание:

1) внутренняя транзакция завершается фиксацией своих операций;
2) ROLLBACK внешней транзакции отменяет зафиксированные операции внутренней транзакции.

На рис. 13.61 представлен пример, демонстрирующий свойство 3 при работе с вложенной транзакцией.

```
use BSTU
go
begin tran                      --  внешняя транзакция
  insert FACULTY values ('ИТ', 'Информационных технологий');
  begin tran     --  внутренняя транзакция
     update PULPIT set FACULTY = 'ИТ' where PULPIT = 'ИСиТ'
  rollback;          --  внутренняя транзакция
if @@trancount > 0
  begin
  print '@@trancount = '+ cast(@@trancount as varchar(2));
  commit;     -- внешняя транзакция
 end
else print '@@trancount = '+ cast(@@trancount as varchar(2));
select
 (select count(*) from PULPIT   where PULPIT  = 'ИТ') 'PULPIT',
 (select count(*) from FACULTY  where FACULTY = 'ИТ') 'FACULTY';
```

```
@@trancount = 0
```

PULPIT	FACULTY
0	0

Рис. 13.61. Действие оператора ROLLBACK вложенной транзакции на операции внешней и внутренней транзакции

Обратите внимание:

1) внутренняя транзакция завершается откатом своих операций;

2) ROLLBACK внутренней транзакции отменяет действия внутренней и внешней транзакций;

3) ROLLBACK внутренней транзакции завершает внутреннюю и внешнюю транзакции;

4) функция @@TRANCOUNT возвращает **0**, если она выполнена вне транзакции.

13.17. Пользовательские хранимые процедуры

Хранимая процедура – это объект БД, представляющий собой поименованный код T-SQL. Как и любой объект БД, хранимая процедура может быть создана с помощью CREATE, изменена с помощью ALTER и удалена с помощью оператора DROP. К именованию процедуры предъявляются такие же требования, как и к именам объектов БД. Не рекомендуется использовать имена процедур, начинающиеся с символов **SP** – обычно так именуются системные хранимые процедуры. Хранимая процедура может принимать входные и формировать выходные параметры, а результатом ее выполнения может быть целочисленное значение, возвращаемое к точке вызова с помощью оператора RETURN один или более результирующих наборов, сформированных операторами SELECT, а также содержимое стандартного выходного потока, полученного при выполнении операторов PRINT. Вызов процедуры осуществляется с помощью оператора EXECUTE. Кроме того, результирующий набор хранимой процедуры может быть использован в качестве исходного источника строк для оператора INSERT.

На рис. 13.62 приведен пример создания простейшей хранимой процедуры с именем **GET_PULPITS**. Процедура не имеет параметров. В результате выполнения процедуры будет сформирован результирующий набор.

```
use BSTU
go
create procedure GET_PULPITS
as  begin                  -- начало кода
 select * from PULPIT;     -- код процедуры
end;                       -- конец кода
```

Рис. 13.62. Создание процедуры с именем GET_PULPITS

В сценарии на рис. 13.63 осуществляется вызов хранимой процедуры **GET_PULPITS**. Кроме того, на рисунке приведен фрагмент результирующего набора, сформированного процедурой. Обратите внимание, что ключевое слово EXECUTE может быть сокращено EXEC.

```
use BSTU
go
-- допускается сокращение ключевого
-- слова EXECUTE
exec GET PULPITS; -- вызов процедуры
```

PULPIT	PULPIT_NAME	FACULTY
БФ	Белорусской филологии	ИДиП
ИСиТ	Информационных систем и технологий	ИДиП
ЛВ	Лесоводства	NULL
ЛЗиДВ	Лесозащиты и древесиноведения	NULL
ЛКиП	Лесных культур и почвоведения	NULL
ЛМиЛЗ	Лесных машин и технологии лесозаготовок	ТТЛП
ЛПиСПС	Ландшафтного проектирования и садово-паркового с...	NULL
ЛУ	Лесоустройства	NULL

Рис. 13.63. Вызов процедуры **GET_PULPITS**

В сценарии на рис. 13.64 сначала удаляется процедура (оператор DROP PROCEDURE), а затем создается снова (оператор CREATE PROCEDURE). Новая процедура имеет то же имя **GET_PULPITS**.

Новая версия процедуры с помощью оператора RETURN возвращает к точке вызова количество строк в таблице **PULPIT**.

```
use BSTU
go
drop procedure  GET_PULPITS;  -- удаление процедуры
go
create procedure GET_PULPITS  -- создание процедуры
as  begin
 declare @rc int = (select count(*) from PULPIT) ;
 select * from PULPIT;
 return @rc;  -- код возврат
end;
```

Рис. 13.64. Удаление и создание новой процедуры

На рис. 13.65 приведен пример сценария, в котором вызывается вновь созданная хранимая процедура **GET_PULPITS**, возвращающая к точке вызова количество строк в таблице **PULPIT** и формирующая результирующий набор.

```
use BSTU
go
declare @n int = 0;
exec @n = GET_PULPITS;
print 'код возврата: ' + cast(@n as varchar(3));
```

код возврата: 26

PULPIT	PULPIT_NAME	FACULTY
БФ	Белорусской филологии	ИДиП
ИСиТ	Информационных систем и технологий	ИДиП

Рис. 13.65. Вызов процедуры, возвращающей целочисленное значение к точке вызова

На рис. 13.66 приведен пример сценария, модифицирующего процедуру **GET_PULPITS**. Модифицированная процедура принимает два параметра: входной параметр **@f** и выходной (указано ключевое слово OUTPUT) параметр **@c**. В результате своей работы процедура формирует значение выходного параметра (равно значению встроенной функции @@rowcount), выводит в стандартный поток сообщение (оператор PRINT), а также, как прежде, формирует результирующий набор и код возврата к точке вызова.

```
use BSTU
go
alter procedure GET_PULPITS  -- модификация процедуры
  @f varchar(20),
  @c int output
as  begin
 declare @rc int = (select count(*) from PULPIT) ;
 print 'параметры: @f = '+@f +', @c = '+ cast(@c as varchar(3));
 select * from PULPIT where  FACULTY = @f;
 set @c = @@rowcount;
 return @rc;  -- код возврат
end;
```

Рис. 13.66. Модификация процедуры **GET_PULPITS**

Пример вызова последней версии процедуры **GET_PULPITS** приведен в сценарии на рис. 13.67. При вызове хранимой процедуры применяется параметрическая форма передачи параметров (задаются имена параметров и их значения в произвольном порядке).

Для входных параметров можно задать значения по умолчанию. В этом случае при вызове процедуры можно не указывать значение этого параметра (рис. 13.68).

```
use BSTU
go
declare @n int = 0, @r int = 0;
exec @n = GET_PULPITS @f = 'ТТЛП', @c = @r output;
print 'код возврата     : ' + cast(@n as varchar(3));
print 'выходной параметр: ' + cast(@r as varchar(3));
go
```

```
параметры: @f = ТТЛП, @c = 0
код возврата     : 26
выходной параметр: 4
```

PULPIT	PULPIT_NAME	FACULTY
ЛМиЛЗ	Лесных машин и технологии лесозаготовок	ТТЛП
ТДП	Технологий деревообрабатывающих производств	ТТЛП
ТиДИД	Технологии и дизайна изделий из древесины	ТТЛП
ТЛ	Транспорта леса	ТТЛП

Рис. 13.67. Вызов процедуры **GET_PULPITS** после модификации (рис. 13.66) и результаты ее выполнения

```
use BSTU
go
alter procedure GET_PULPITS  -- модификация процедуры
  @f varchar(20) = 'ТТЛП'    -- значение по умолчанию
 as  begin
    select * from PULPIT where  FACULTY = @f;
 end;
 ----------------------------------------------------
 go
 exec GET_PULPITS;
```

PULPIT	PULPIT_NAME	FACULTY
ЛМиЛЗ	Лесных машин и технологии лесозаготовок	ТТЛП
ТДП	Технологий деревообрабатывающих производств	ТТЛП
ТиДИД	Технологии и дизайна изделий из древесины	ТТЛП
ТЛ	Транспорта леса	ТТЛП

Рис. 13.68. Установка значения входного параметра по умолчанию

Процедуры, формирующие результирующий набор и не имеющие выходных параметров, могут быть применены в операторе INSERT в качестве источника строк для добавления в целевую таблицу (рис. 13.69).

```
use BSTU
go
select top(0) * into #xxx from PULPIT;
insert  #xxx execute GET_PULPITS  @f ='ТТЛП';
```

Рис. 13.69. Применение в операторе INSERT хранимой процедуры, формирующей результирующий набор

В хранимых процедурах допускается применение основных DDL и всех DML и TCL-операторов, конструкций TRY/CATCH, курсоров, использование временных таблиц.

На рис. 13.70 приведен пример хранимой процедуры, создающей и заполняющей временную таблицу данными, формирующимися на основе значений входных параметров.

```
use BSTU
go
create procedure GET_TEACHER  -- модификация процедуры
  @f varchar(20) = null,  -- факультет
  @p varchar(20) = null   -- кафедра
as
declare @rc int = 0;
begin try
 begin tran
   if not exists (select * from  TEMPDB.SYS.OBJECTS
                  where OBJECT_ID = object_id(N'TEMPDB.DBO.#tch') and TYPE = N'U')
       create table #tch (teacher varchar(20), pulpit  varchar(20));
   else delete #tch;
   insert #tch select t.TEACHER, t.PULPIT
               from TEACHER t join PULPIT p on t.PULPIT = p.PULPIT
               where p.PULPIT  = isnull(@p, p.PULPIT) and
                     p.FACULTY = isnull(@f, p.FACULTY);
   set @rc = @@rowcount;
 commit;
 return @rc;
end try
begin catch
 print 'ошибка: ' + cast(error_number() as varchar(10));
 if (@@TRANCOUNT > 0)rollback;
 return -1;
end catch;
```

Рис. 13.70. Пример хранимой процедуры, использующий механизм транзакций, конструкцию TRAY/CATCH, временную таблицу

В процедуре используются механизм транзакций и конструкция для структурной обработки ошибок.

13.18. Пользовательские функции

Функция – это объект БД, представляющий собой поименованный код T-SQL. Для создания, удаления и изменения функций необходимо применять CREATE, DROP и ALTER соответственно. Отличие функций от хранимых процедур в ограничениях, накладываемых на код функции, в форме представления результата работы, а также в способе вызова.

В функции не допускается применение DDL-операторов, DML-операторов, изменяющих БД (INSERT, DELETE, UPDATE), конструкций TRY/CATCH, а также использование транзакций.

Результатом выполнения функции является возвращаемое к точке вызова значение. Если функция возвращает единственное значение (число, строка, дата, время и пр.), то она называется скалярной. Функция, возвращающая таблицу, называется табличной. В зависимости от структуры кода различают встроенные (inline) функции и многооператорные табличные функции.

13.18.1. Скалярные функции

Скалярная функция возвращает единственное значение и принимает любое количество параметров. Допускается возвращать и принимать все типы данных, за исключением TIMESTAMP и устаревших IMAGE, TEXT и NTEXT.

На рис. 13.71 представлен пример создания функции с именем **COUNT_STUDENTS**.

```
use BSTU
go

create function COUNT_STUDENTS(@faculty varchar(20)) returns int
as begin
 declare @rc int = 0;
 set @rc = (
           select count(idstudent)
           from FACULTY f join GROUPS  g on f.FACULTY = g.FACULTY
                          join STUDENT s on s.IDGROUP =  g.IDGROUP
           where f.FACULTY = @faculty
           );
 return @rc;
end;
```

Рис. 13.71. Создание скалярной функции с именем **COUNT_STUDENTS**, принимающей один параметр и возвращающей значение типа INT

Функция **COUNT_STUDENTS** принимает один параметр типа VARCHAR с именем **@faculty** и возвращает значение типа INT. В теле функции объявляется переменная **@rc**. Затем ей присваивается значение, вычисленное в результате выполнения оператора SELECT. С помощью оператора RETURN значение переменной возвращается к точке вызова.

На рис. 13.71–13.75 приведены сценарии, демонстрирующие способы вызова скалярной функции **COUNT_STUDENTS**.

```
use BSTU
go
declare @c int = dbo.COUNT_STUDENTS('ИДиП');
print 'количество студентов на факультете ИДиП: '+
       cast(@c  as varchar(4));
```

```
количество студентов на факультете ИДиП: 64
```

Рис. 13.72. Применение скалярной функции для инициализации или присвоения значения переменных

```
use BSTU
go
print 'количество студентов на факультете ИДиП: '+
      cast(dbo.COUNT_STUDENTS('ИДиП')  as varchar(4));
```

```
количество студентов на факультете ИДиП: 64
```

Рис. 13.73. Встроенный вызов скалярной функции

```
use BSTU
go
select FACULTY                          'факультеты',
       dbo.COUNT_STUDENTS(FACULTY)  'количество студентов' from FACULTY;
```

факультеты	количество студентов
ИДиП	64
ИЭФ	29
ЛХФ	28
ТОВ	21
ТТЛП	15
ХТиТ	21

Рис. 13.74. Применение скалярной функции в списке SELECT

```
use BSTU
go
select FACULTY 'факультеты' from FACULTY
       where dbo.COUNT_STUDENTS(FACULTY) > 20
```

факультеты
ИДиП
ИЭФ
ЛХФ
ТОВ
ХТиТ

Рис. 13.75. Применение скалярной функции в секции WHERE оператора SELECT

Обратите внимание: при вызове функции необходимо указывать ее имя с точностью до схемы БД. При создании (рис. 13.71) функции имя схемы не указывалось, что привело ее размещение в схеме по умолчанию – **DBO**.

При создании или изменении функции можно указать для параметров значения по умолчанию (рис. 13.76).

```
use BSTU
go
alter function COUNT_STUDENTS(@faculty varchar(20) = null,
                              @profession varchar(20) = null) returns int
as begin
   return  (
           select count(IDSTUDENT)
           from FACULTY f join PROFESSION p on f.FACULTY= p.FACULTY
                          join GROUPS g     on p.PROFESSION = g.PROFESSION
                          join STUDENT s    on g.IDGROUP =  s.IDGROUP
           where f.FACULTY =    isnull(@faculty, f.FACULTY)  and
                 p.PROFESSION = isnull(@profession, p.PROFESSION)
           );
end;
```

Рис. 13.76. Изменение функции **COUNT_STUDENTS**: функция принимает два параметра с установленными значениями по умолчанию

На рис. 13.77–13.79 приведены примеры вызова скалярной функции **COUNT_STUDENTS** без применения (13.77) и с применением значений по умолчанию для ее параметров (13.78, 13.79).

```
use BSTU
go
select f.FACULTY                                    'факультеты',
       p.PROFESSION                                 'специальности',
       dbo.COUNT_STUDENTS(f.FACULTY, p.PROFESSION) 'количество студентов'
       from FACULTY f join PROFESSION  p on f.FACULTY= p.FACULTY
       group by f.FACULTY, p.PROFESSION order by 'количество студентов' desc
```

факультеты	специальности	количество студентов
ИДиП	1-36 06 01	24
ИДиП	1-40 01 02	24
ИЭФ	1-25 01 07	17
ИДиП	1-47 01 01	16
ЛХФ	1-75 02 01	12
ИЭФ	1-25 01 08	12
ХТиТ	1-36 01 08	11
ТТЛП	1-36 05 01	10

Рис. 13.77. Вызов в SELECT-списке функции **COUNT_STUDENTS** с заданными значениями для двух параметров

```
use BSTU
go
select FACULTY                          'факультеты',
       dbo.COUNT_STUDENTS(FACULTY, default) 'количество студентов' from FACULTY;
```

факультеты	количество студентов
ИДиП	64
ИЭФ	29
ЛХФ	28
ТОВ	21
ТТЛП	15
ХТиТ	21

Рис. 13.78. Вызов функции **COUNT_STUDENTS** с заданным значением для первого и значением по умолчанию для второго параметров

```
use BSTU
go
print 'всего студентов: '+
      cast(dbo.COUNT_STUDENTS(default, default)  as varchar(4));
```

```
всего студентов: 178
```

Рис. 13.79. Вызов функции **COUNT_STUDENTS** со значениями по умолчанию для двух параметров

Обратите внимание: при использовании значения по умолчанию параметра необходимо указывать ключевое слово default в соответствующей этому параметру позиции в списке параметров функции.

13.18.2. Встроенные табличные функции

Встроенные табличные функции возвращают к точке вызова результирующий набор SELECT-запроса и по своим возможностям очень напоминают представления. На рис. 13.80 представлен пример создания простейшей встроенной функции с именем **fnPROFESSION**.

```
use BSTU
go
create function dbo.fnPROFESSION()  returns table
    as return select PROFESSION, FACULTY, QUALIFICATION
                from PROFESSION;
```

Рис. 13.80. Пример создания простейшей встроенной функции

На рис. 13.81 приведен пример SELECT-запроса, использующего встроенную функцию **fnPROFESSION** (рис. 13.80).

```
use BSTU
go
select * from fnPROFESSION()
```

PROFESSION	FACULTY	QUALIFICATION
1-25 01 07	ИЭФ	экономист-менеджер
1-25 01 08	ИЭФ	экономист
1-36 01 08	ХТиТ	инженер-механик
1-36 05 01	ТТЛП	инженер-механик
1-36 06 01	ИДиП	инженер-электромеханик
1-36 07 01	ХТиТ	инженер-механик
1-40 01 02	ИДиП	инженер-программист-системотехник
1-46 01 01	ТТЛП	инженер-технолог
1-47 01 01	ИДиП	редактор-технолог
1-48 01 02	ТОВ	инженер-химик-технолог
1-48 01 05	ТОВ	инженер-химик-технолог

Рис. 13.81. Пример использования встроенной функции **fnPROFESSION**

Функция **fnPROFESSION** не имеет параметров и ее использование (рис. 13.81) ничем не отличается от применения представления, основанного на аналогичном SELECT-запросе. Преимущества функции проявляются при использовании параметров (рис. 13.82)

```
use BSTU
go
alter function dbo.fnPROFESSION(@f varchar(20) = null, @like_q varchar(30) = null)
      returns table
      as return select PROFESSION, FACULTY, QUALIFICATION
                from PROFESSION
                where FACULTY = isnull(@f,FACULTY) and
                      1 = case
                              when @like_q is null              then 1
                              when  QUALIFICATION like @like_q  then 1
                              else 0
                          end;
```

Рис. 13.82. Встроенная функция **fnPROFESSION** с двумя параметрами

На рис. 13.83–13.86 приведены примеры вызова функции **fnPROFESSION** с разными наборами параметров.

```
use BSTU
go
select * from fnPROFESSION(default, default);
```

PROFESSION	FACULTY	QUALIFICATION
1-25 01 07	ИЭФ	экономист-менеджер
1-25 01 08	ИЭФ	экономист
1-36 01 08	ХТиТ	инженер-механик
1-36 05 01	ТТЛП	инженер-механик
1-36 06 01	ИДиП	инженер-электромеханик
1-36 07 01	ХТиТ	инженер-механик
1-40 01 02	ИДиП	инженер-программист-системотехник
1-46 01 01	ТТЛП	инженер-технолог
1-47 01 01	ИДиП	редактор-технолог
1-48 01 02	ТОВ	инженер-химик-технолог
1-48 01 05	ТОВ	инженер-химик-технолог

Рис. 13.83. Вызов встроенной функции **fnPROFESSION** с заданными по умолчанию параметрами

```
use BSTU
go
select * from fnPROFESSION('ИЭФ', default);
```

PROFESSION	FACULTY	QUALIFICATION
1-25 01 07	ИЭФ	экономист-менеджер
1-25 01 08	ИЭФ	экономист

Рис. 13.84. Вызов встроенной функции **fnPROFESSION** с заданным по умолчанию вторым параметром

```
use BSTU
go
select * from fnPROFESSION(default, '%экономист%');
```

PROFESSION	FACULTY	QUALIFICATION
1-25 01 07	ИЭФ	экономист-менеджер
1-25 01 08	ИЭФ	экономист

Рис. 13.85. Вызов встроенной функции **fnPROFESSION** с заданным по умолчанию первым параметром

```
use BSTU
go
select * from fnPROFESSION('ХТиТ', '%механик%');
```

PROFESSION	FACULTY	QUALIFICATION
1-36 01 08	ХТиТ	инженер-механик
1-36 07 01	ХТиТ	инженер-механик

Рис. 13.86. Вызов встроенной функции **fnPROFESSION** с заданными значениями параметров

Встроенные функции допускается применять в операциях внутреннего (INNER JOIN) и внешнего (OUTER JOIN) соединения таблиц (18.87 и 18.88).

```
use BSTU
go
select count(*) 'кол. механиков на ХТиТ'
from GROUPS g join fnPROFESSION('ХТиТ', '%механик%') p
                    on g.PROFESSION = p.PROFESSION
               join STUDENT s
                    on s.IDGROUP = g.IDGROUP
```

кол. механиков на ХТиТ
21

Рис. 13.87. Применение встроенной функции **fnPROFESSION** в операции INNER JOIN

```
use BSTU
go
select f.FACULTY, p.QUALIFICATION
     from FACULTY f left outer join  fnPROFESSION(default, '%механик%') p
                  on f.FACULTY = p.FACULTY
```

FACULTY	QUALIFICATION
ИДиП	инженер-электромеханик
ИЭФ	NULL
ЛХФ	NULL
ТОВ	NULL
ТТЛП	инженер-механик
ХТиТ	инженер-механик
ХТиТ	инженер-механик

Рис. 13.88. Применение встроенной функции **fnPROFESSION** в операции LEFT OUTER JOIN

Встроенные функции не дают никакого выигрыша в производительности ни в сравнении с явными SELECT-запросами, ни в сравнении с представлениями. Главная ценность встроенных функций в их повторной применимости, а также в возможности существенно сократить размер SELECT-запроса.

13.18.3. Многооператорные табличные функции

Многооператорные табличные функции возвращают к точке вызова результирующий набор и допускают более сложную логику работы.

На рис. 13.89 представлен пример создания табличной многооператорной функции, а на рис. 13.90 – использования.

```
use BSTU
go
create function FACULTY_REPORT(@c int)
   returns @fr table
              (
               [Факультет]                 varchar(50),
               [Количество кафедр]         int,
               [Количество групп]          int,
               [Количество студентов]      int,
               [Количество специальностей] int
              )
     as begin
       declare cc cursor static for
                  select FACULTY from FACULTY
                  where dbo.COUNT_STUDENTS(FACULTY, default)> @c;
       declare @f varchar(30);
       open cc;
       fetch cc into @f;
       while @@fetch_status = 0
       begin
         insert @fr values(
                           @f,
                          (select count(PULPIT) from PULPIT where FACULTY = @f),
                          (select count(IDGROUP) from GROUPS where FACULTY = @f),
                           dbo.COUNT_STUDENTS(@f, default),
                          (select count(PROFESSION) from PROFESSION where FACULTY = @f)
                         );
          fetch cc into @f;
       end;
       return;
    end;
```

Рис. 13.89. Создание многооператорной табличной функции **FACULTY_REPORT**

```
use BSTU
go
select * from FACULTY_REPORT(28)
```

Факультет	Количество кафедр	Количество групп	Количество студентов	Количество специальностей
ИДиП	5	11	64	3
ИЭФ	3	5	29	2

Рис. 13.90. Использование многооператорной табличной функции **FACULTY_REPORT**

Обратите внимание:

1) в функции применяется курсор;
2) в функции дважды применяется скалярная функция **COUNT_STUDENTS**;
3) в функции используется переменная табличного типа (переменная **@fr**);
4) к точке вызова возвращается переменная табличного типа;
5) вызов многооператорной функции осуществляется аналогично вызову встроенных табличных функций.

13.19. Применение триггеров

Триггер – это особый вид хранимой процедуры, предназначенной для обработки событий в БД. MSS поддерживает два типа триггеров: DDL-триггеры и DML-триггеры. Для каждого типа определено свое семейство событий, обработку которых триггер этого типа может выполнять.

13.19.1. DDL-триггеры

Различают два вида DDL-триггеров: триггеры уровня сервера (ALLSERVER), триггеры уровня БД (DATABASE). Триггеры уровня сервера могут обрабатывать события сервера СУБД

(создание, изменение или удаление объектов сервера, подключение к серверу и пр.). Триггеры уровня базы данных предназначены для обработки событий, происходящих в рамках БД.

События DDL-триггеров являются иерархическими. Их иерархия имеет пять уровней. В табл. 13.4 приведены первые три уровня событий. С полным их перечнем можно ознакомиться в пособии [6].

Таблица 13.4

Первые три уровня событий DDL-триггеров

События уровня 1	События уровня 2	События уровня 3
DDL_EVENTS	DDL_DATABASE_LEVEL_EVENTS	DDL_ASSEMBLY_EVENTS
		DDL_DATABASE_SECURITY_EVENTS
		DDL_DEFAULT_EVENTS
		DDL_EVENT_NOTIFICATION_EVENTS
		DDL_EXTENDED_PROPERTY_EVENTS
		DDL_FULLTEXT_CATALOG_EVENTS
		DDL_FULLTEXT_STOPLIST_EVENTS
		DDL_FUNCTION_EVENTS
		DDL_PRTITION_EVENTS
		DDL_PLAN_GUIDE_EVENTS
		DDL_PROCEDURE_EVENTS
		DDL_RULE_EVENTS
		DDL_SSB_EVENTS
		DDL_SYNONYM_EVENTS
		DDL_TABLE_VIEW_EVENTS
		DDL_TYPE_EVENTS
		RENAME
		ALTER_INSTANCE
	DDL_SERVER_LEVEL_EVENTS	DLL_DATABASE_EVENTS
		DLL_ENDPOINTS_LEVEL
		DLL_EVENT_SESSION_EVENTS
		DLL_MESSAGE_EVENTS
		DLL_RESOURCE_GOVERNOR_LEVEL

На втором уровне иерархии (табл. 13.4) осуществляется разделение на события уровня сервера (DDL_SERVER_LEVEL_EVENTS) и события уровня базы данных (DDL_DATABASE_LEVEL_EVENTS).

В рамках любого DDL-триггера доступна для применения встроенная функция EVENTDATA, с помощью которой можно получить исчерпывающую информацию о событии, активизировавшем триггер. Результатом работы функции является значение XML-типа.

DDL-триггеры уровня сервера. Триггеры уровня сервера предназначаются, прежде всего, для отслеживания настроек сервера, а также для обеспечения его безопасности. Условно триггеры уровня сервера можно разбить на две группы: триггеры для обработки событий, связанных с созданием, изменением или удалением объектов сервера, и триггеры, обрабатывающие событие подключения к серверу.

Для хранения результатов выполнения функции EVENTDATA создадим в системной БД MASTER таблицу с именем SERVER_EVENT_LOG (рис. 13.91).

```
use MASTER
go
create table SERVER_EVENT_LOG
(
  id int identity,   -- идентификатор
  ep xml             -- описание события
);
```

Рис. 13.91. Таблица для хранения результатов выполнения функции **EVENTDATA**

Второй столбец таблицы SERVER_EVENT_LOG (рис. 13.91) предназначен для хранения данных в формате XML. Более подробно о типе данных XML и принципах работы с ним поясняется в 14 главе этого пособия.

На рис. 13.92 приведен пример создания DLL-триггера с именем CREATE_DB уровня сервера (ALLSERVER), предназначенного для обработки события с именем CREATE_DATABASE. Событие CREATE_DATABASE находится на четвертом уровне иерархии (под DLL_DATABASE_EVENTS) и возникает при выполнении DLL-оператора CREATE DATABASE, предназначенного для создания новой БД.

```
use MASTER
go

create trigger CREATE_DB
on all server
for CREATE_DATABASE
as
    print  'создание базы данных';
    insert SERVER_EVENT_LOG values( EVENTDATA());
go
```

Рис. 13.92. Создание триггера уровня сервера для обработки события CREATE_DATABASE

Триггер с именем DROP_DB (13.93) реагирует на выполнение оператора DROP DATABASE (удаление БД).

```
use MASTER
go
create trigger DROP_DB
on all server
for DROP_DATABASE
as
    print  'удаление базы данных';
    insert SERVER_EVENT_LOG values(EVENTDATA());
go
```

Рис. 13.93. Создание триггера уровня сервера для обработки события DROP_DATABASE

Триггеры CREATE_DB и DROP_DB (рис. 13.92, 13.93) содержат похожий код, выполнение которого приводит к выдаче сообщения в стандартный поток вывода (оператор PRINT) и

добавлению строки (оператор INSERT) в таблицу SERVER_EVENT_LOG. В столбец с именем EP записывается результат выполнения функции EVENTDATA.

При выполнении операторов CREATE DATABASE и DROP DATABASE (рис. 13.94, 13.95) в результате выполнения триггеров в стандартный выходной поток выводятся соответствующие сообщения и добавляются строки в таблицу SERVER_EVENT_LOG (рис. 13.96).

```
use MASTER
go
create database [DB1] on  primary
(
  NAME = N'DB1', FILENAME = N'C:\DB_BSTU\DB1.mdf' ,
  SIZE = 3072KB, MAXSIZE = UNLIMITED, FILEGROWTH = 1024KB
 )
 LOG ON
(
  NAME = N'DB1_log',
  FILENAME = N'C:\DB_BSTU\DB1_log.ldf',
  SIZE = 1024KB, MAXSIZE = 2048GB, FILEGROWTH = 10%
 )
go
```

```
создание базы данных
```

Рис. 13.94. Сообщение, формируемое в результате выполнения триггера **CREATE_DB** (рис. 13.92) при создании БД

```
use MASTER
go
drop database [DB1];
```

```
удаление базы данных
```

Рис. 13.95. Сообщение, формируемое в результате выполнения триггера **DROP_DB** (рис.13.92) при удалении БД

```
use MASTER
go
select * from SERVER_EVENT_LOG
```

id	ep
1	<EVENT_INSTANCE><EventType>CREATE_DATABASE...
2	<EVENT_INSTANCE><EventType>DROP_DATABASE</...

Рис. 13.96. Добавление двух строк в таблицу **SERVER_EVENT_LOG** в результате выполнения триггеров **CREATE_DB** и **DROP_DB**

На рис. 13.97, 13.98 демонстрируется содержимое столбца с именем EP таблицы SERVER_EVENT_LOG. Значением столбца является XML-структура, сформированная функцией EVENTDATA при выполнении триггеров CREATE_DB и DROP_DB (рис. 13.92, 13.93).

```
<EVENT_INSTANCE>
  <EventType>CREATE_DATABASE</EventType>
  <PostTime>2014-06-09T15:02:13.703</PostTime>
  <SPID>53</SPID>
  <ServerName>WIN-FPSVQQ7K4B3</ServerName>
  <LoginName>sa</LoginName>
  <DatabaseName>DB1</DatabaseName>
  <TSQLCommand>
    <SetOptions ANSI_NULLS="ON" ANSI_NULL_DEFAULT="ON"
                ANSI_PADDING="ON" QUOTED_IDENTIFIER="ON"
                ENCRYPTED="FALSE" />
    <CommandText>create database [DB1] on  primary
  (
    NAME = N'DB1', FILENAME = N'C:\DB_BSTU\DB1.mdf' ,
    SIZE = 3072KB, MAXSIZE = UNLIMITED, FILEGROWTH = 1024KB
   )
   LOG ON
  (
    NAME = N'DB1_log',
    FILENAME = N'C:\DB_BSTU\DB1_log.ldf',
    SIZE = 1024KB, MAXSIZE = 2048GB, FILEGROWTH = 10%
   )
</CommandText>
  </TSQLCommand>
</EVENT_INSTANCE>
```

Рис. 13.97. Результат выполнения функции **EVENTDATA** в триггере **CREATE_DB** (рис.13. 92) при выполнении оператора CREATE DATABASE (рис. 13.94)

```
<EVENT_INSTANCE>
   <EventType>DROP_DATABASE</EventType>
   <PostTime>2014-06-09T15:05:54.027</PostTime>
   <SPID>53</SPID>
   <ServerName>WIN-FPSVQQ7K4B3</ServerName>
   <LoginName>sa</LoginName>
   <DatabaseName>DB1</DatabaseName>
   <TSQLCommand>
     <SetOptions ANSI_NULLS="ON" ANSI_NULL_DEFAULT="ON"
                 ANSI_PADDING="ON" QUOTED_IDENTIFIER="ON"
                 ENCRYPTED="FALSE" />
     <CommandText>drop database [DB1]; </CommandText>
   </TSQLCommand>
 </EVENT_INSTANCE>
```

Рис. 13.98. Результат выполнения функции **EVENTDATA** в триггере **DROP_DB** (рис. 13.93) при выполнении оператора DROP DATABASE (рис. 13.95)

В табл. 13.5 приведено описание XML-элементов, составляющих результат выполнения функции EVENTDATA.

Таблица 13.4

Описание XML-элементов, формируемых функцией EVENTDATA

Наименование элемента	Назначение элемента
EVENT_INSTANCE	Корневой элемент
EventType	Наименование события, вызвавшего триггер
PostTime	Штамп времени, соответствующий моменту возникновения события
SPID	Значение системного идентификатора процесса
ServerName	Имя сервера
LoginName	Имя Login от лица которого выполняется триггер
DatabaseName	Контекст (имя текущей БД) выполнения триггера
TSQLCommand	Корневой элемент описания текущей команды
SetOption	Значение текущих опций БД
CommandText	Текст оператора, вызвавшего событие, обрабатываемое триггером

На уровне сервера (ALLSERVER) может быть создан LOGON-триггер, предназначенный для обработки подключения к серверу (рис. 13.99).

```
use MASTER
go
create trigger LOGON_SERVER
on all server
with execute as 'sa'
for LOGON
as
   insert SERVER_EVENT_LOG values(EVENTDATA());
go
```

Рис. 13.99. Пример создания Logon-триггера

Если после создания Logon-триггера выполнить подключение к серверу (рис. 13.100), то в результате выполнения триггера (рис. 13.99) содержимое таблицы SERVER_EVENT_LOG пополнится строкой (рис. 13.101), содержащей XML-структуру, представленную на рис. 13.102.



Рис. 13.100. Подключение к серверу с помощью MSMS

```
use MASTER
go
select *  from SERVER_EVENT_LOG
where ep.value('(/EVENT_INSTANCE/LoginName)[1]', 'varchar(50)')= 'smw'
```

id	ep
2	<EVENT_INSTANCE><EventType>LOGON</EventType><Pos...
3	<EVENT_INSTANCE><EventType>LOGON</EventType><Pos...

Рис. 13.101. Строки таблицы **SERVER_EVENT_LOG**, добавленные в результате выполнения Logon-триггера (рис. 13.99)

```
<EVENT_INSTANCE>
  <EventType>LOGON</EventType>
  <PostTime>2014-06-09T20:53:19.510</PostTime>
  <SPID>56</SPID>
  <ServerName>WIN-FPSVQQ7K4B3</ServerName>
  <LoginName>smw</LoginName>
  <LoginType>SQL Login</LoginType>
  <SID>XXBAqoGtbUOpWPCPifiLXQ==</SID>
  <ClientHost>192.168.0.200</ClientHost>
  <IsPooled>0</IsPooled>
</EVENT_INSTANCE>
```

Рис. 13.102. XML-структура, описывающая событие подключения (Logon) к MSS

Обратите внимание, что XML-структура (рис. 13.102) содержит несколько новых элементов: **SID** – идентификатор сессии, **ClientHost –** хост клиента.

DDL-триггеры уровня базы данных. На рис. 13.103 представлен пример триггера с именем DDL_DB уровня БД, предназначенного для обработки группы событий с именем DLL_DATABASE_LEVEL_EVENTS (табл. 13.4).

```
use BSTU
go
create trigger DDL_DB
on database
for DDL_DATABASE_LEVEL_EVENTS
as
   insert MASTER.DBO.SERVER_EVENT_LOG values(EVENTDATA());
go
```

Рис. 13.103. Создание DDL-триггера уровня БД, обрабатывающего группу событий **DLL_DATABASE_LEVEL_EVENTS**

После выполнения сценария на рис. 13.104 в таблицу SERVER_EVENT_LOG добавится три строки, соответствующие выполнению трех DDL-операторов CREATE TABLE, ALTER TABLE и DROP TABLE.

```
use BSTU
go
create table t1 ( x int);
go
alter table t1 add y float;
go
drop table t1;
go
select *  from  MASTER.DBO.SERVER_EVENT_LOG
```

id	ep
395	<EVENT_INSTANCE><EventType>CREATE_TABLE</EventTy...
396	<EVENT_INSTANCE><EventType>ALTER_TABLE</EventType...
397	<EVENT_INSTANCE><EventType>DROP_TABLE</EventType...

Рис. 13.104. Строки, добавляемые в таблицу **SERVER_EVENT_LOG** в результате выполнения триггера DDL_DB при создании, изменении и удалении таблиц

Как и в случае триггера уровня сервера, в триггере уровня БД функция EVENTDATA формирует XML-структуру, описывающую обрабатываемое триггером событие (рис. 13.105).

```
<EVENT_INSTANCE>
   <EventType>CREATE_TABLE</EventType>
   <PostTime>2014-06-09T22:01:30.650</PostTime>
   <SPID>51</SPID>
   <ServerName>WIN-FPSVQQ7K4B3</ServerName>
   <LoginName>sa</LoginName>
   <UserName>dbo</UserName>
   <DatabaseName>BSTU</DatabaseName>
   <SchemaName>dbo</SchemaName>
   <ObjectName>t1</ObjectName>
   <ObjectType>TABLE</ObjectType>
   <TSQLCommand>
     <SetOptions ANSI_NULLS="ON" ANSI_NULL_DEFAULT="ON"
                 ANSI_PADDING="ON"  QUOTED_IDENTIFIER="ON"
                 ENCRYPTED="FALSE" />
     <CommandText>create table t1 ( x int); </CommandText>
   </TSQLCommand>
 </EVENT_INSTANCE>
```

Рис. 13.105. Описание события **CREATE_TABLE** в XML-структуре, формируемой функцией **EVENTDATA**

Обратите внимание на несколько новых элементов в XML-структуре (рис. 13.105): UserName – имя пользователя БД, SchemaName – имя схемы БД, ObjectName и ObjectType – имя и тип объекта БД.

В примере на рис. 13.106 триггер с именем DLL_DB_DROP_TABLE делает невозможным удаление таблицы с именем TEACHER.

```
use BSTU
go
create trigger DDL_DB_DROP_TABLE
on database
for DROP_TABLE
as
 declare @t varchar(50)=
 EVENTDATA().value('(/EVENT_INSTANCE/ObjectName)[1]', 'varchar(50)');
 if @t = 'TEACHER'
 begin
   raiserror( N'удаление таблицы TEACHER запрещено', 16, 1);
   rollback;
 end;
```

Рис. 13.106. Пример триггера уровня БД, запрещающего удалять таблицу с именем **TEACHER**

Обратите внимание на следующее: 1) для выбора значения элемента ObjectName из XML-структуры, сформированной функцией EVENTDATA, применяется метод VALUE, принципы использования которого будут описаны позже; 2) для формирования сообщения об ошибке применяется встроенная функция RAISERROR; 3) для отмены выполнения оператора DROP TABLE осуществляется откат с помощью оператора ROLLBACK; 4) в результате выполнения триггера (рис. 13.106) формируется системное сообщение с номером 50 000 и сообщение, указанное в функции RAISERROR (рис. 13.107).

```
use BSTU
go
drop table TEACHER;
```

```
(1 row(s) affected)
Msg 50000, Level 16, State 1, Procedure DDL_DB_DROP_TABLE, Line 9
удаление таблицы TEACHER запрещено
Msg 3609, Level 16, State 2, Line 1
The transaction ended in the trigger. The batch has been aborted.
```

Рис. 13.107. Сообщение, формируемое триггером **DDL_DB_DROP_TABLE** (рис. 13.106) при попытке удаления таблицы с именем **TEACHER**

Удаление DDL-триггеров. Удалить DDL-триггеры можно с помощью оператора DROP TRIGGER (рис. 13.108). При этом следует указывать уровень действия удаляемого триггера (ON ALL SERVER или ON DATABASE).

```
use MASTER
go
drop trigger DROP_DB on all server;
drop trigger LOGON_SERVER on all server;
go
use BSTU
go
drop trigger DDL_DB on database;
drop trigger DDL_DB_DROP_TABLE on database;
```

Рис. 13.108. Пример сценария, удаляющего DDL-триггеры

13.19.2. DML-триггеры

Каждый DML-триггер связан с таблицей или представлением и предназначен для обработки одного или нескольких событий, соответствующих трем операторам, изменяющим содержимое таблицы: INSERT, UPDATE и DELETE. MSS поддерживает два типа DML-триггеров: AFTER и INSTEAD OF.

AFTER-триггеры. Триггеры типа AFTER исполняются после выполнения оператора, вызвавшего соответствующее событие. На рис. 13.109 представлен сценарий, создающий три триггера с именами: AUD_AFTER_INSERT, AUD_AFTER_DELETE и AUD_AFTER_UPDATE.

```
use BSTU
go

create trigger AUD_AFTER_INSERT
on AUDITORIUM after INSERT
as
 print 'AUD_AFTER_INSERT';
 return;
go

create trigger AUD_AFTER_DELETE
on AUDITORIUM after DELETE
as
 print 'AUD_AFTER_DELETE';
 return;
go

create trigger AUD_AFTER_UPDATE
on AUDITORIUM after UPDATE
as
 print 'AUD_AFTER_UDATE';
 return;
go
```

Рис. 13.109. Сценарий, создающий три DML-триггера

Обратите внимание (рис. 13.109) при создании триггера указывается: 1) имя таблицы (после ключевого слова ON), с которой этот триггер связан; 2) событие, обработку которого осуществляет триггер.

На рис. 13.110–13.112 представлены примеры сценариев, выполнение которых приводит к вызову и исполнению триггеров, созданных сценарием на рис. 13.109. Следует обратить внимание на то, что DML-триггеры являются триггерами операторного типа. Триггеры операторного типа вызываются один раз для каждого оператора, генерирующего соответствующее событие, независимо от количества строк таблицы, обработанных этим оператором. Например, оператор INSERT в сценарии на рис. 13.110 добавляет две строки в таблицу AUDITORIUM. При этом триггер с именем AUD_AFTER_INSERT выполняется только один раз. Аналогично при изменении и удалении строк (рис. 13.111, 13.112) осуществляется однократный вызов соответствующих триггеров.

```
use BSTU
go
insert  AUDITORIUM  values ('301-4', 'ЛК', 75, '301-4'),
                           ('401-4', 'ЛК', 80, '301-4')
AUD_AFTER_INSERT
```

Рис. 13.110. Выполнение триггера с именем **AUD_AFTER_INSERT** после добавления строк в таблицу **AUDITORIM**

```
use BSTU
go
update AUDITORIUM  set AUDITORIUM_CAPACITY += 5
where  AUDITORIUM in ('301-4','401-4');
AUD_AFTER_DELETE
```

Рис. 13.111. Выполнение триггера с именем **AUD_AFTER_UPDATE** после изменения строк в таблице **AUDITORIM**

```
use BSTU
go
delete AUDITORIUM  where  AUDITORIUM in ('301-4','401-4');
AUD_AFTER_UPDATE
```

Рис. 13.112. Выполнение триггера с именем **AUD_AFTER_DELETE** после удаления строк в таблице **AUDITORIM**

Один AFTER-триггер может реагировать на несколько событий. В этом случае после ключевого слова AFTER должны быть перечислены все необходимые события.

На рис. 13.113 приведен сценарий, создающий AFTER-триггер с именем AUD_AFTER. Этот триггер будет вызван после выполнения одного из операторов INSERT, UPDATE или DELETE.

В рамках выполнения триггера разработчику доступны две псевдотаблицы с именами INSERTED и DELETED. В зависимости от типа события, активизировавшего триггер, содержимое таблиц разное.

Событие INSERT приводит к тому, что в таблицу INSERTED помещаются строки, добавленные оператором INSERT, вызвавшим это событие. При этом таблица DELETED остается пустой.

При событии DELETE в таблицу DELETED копируются удаленные строки, а таблица INSERTED остается пустой.

При изменении строк таблицы с помощью оператора UPDATE заполняются обе псевдотаблицы. При этом INSERTED содержит обновленные версии строк, а таблица DELETED версию строк до их изменения.

```
use BSTU
go
delete AUDITORIUM  where  AUDITORIUM in ('301-4','401-4');

go
create trigger AUD_AFTER
on AUDITORIUM after INSERT, DELETE, UPDATE
as
 declare @ins int = (select count(*) from INSERTED),
         @del int = (select count(*) from DELETED);

 if       @ins > 0 and  @del = 0  print 'Событие: INSERT';
 else  if @ins = 0 and  @del > 0  print 'Событие: DELETE';
 else  if @ins > 0 and  @del > 0  print 'Событие: UPDATE';
 return;
go
```

Рис. 13.113. Создание AFTER-триггера, реагирующего на все DML-события таблицы AUDITORIUM

Триггер, созданный в сценарии на рис. 13.113, с помощью содержимого псевдотаблиц INSERTED и UPDATED определяет тип события, вызвавшего данный триггер, и формирует соответствующее сообщение в стандартный поток вывода.

Сценарий на рис. 13.114 демонстрирует работу триггера AUD_AFTER (рис. 13.113). Обратите внимание: независимо от триггера AUD_AFTER, выполняются триггеры AUD_AFTER_INSERT, AUD_AFTER_DELETE и AUD_AFTER_UPDATE, созданные ранее (рис. 13.109). С одной таблицей или представлением могут быть связаны несколько триггеров.

Как любой объект БД, триггер может быть изменен с помощью DDL-оператора ALTER. На рис. 13.115 представлен сценарий изменяющий триггеры AUD_AFTER_INSERT, AUD_AFTER_DELETE и AUD_AFTER_UPDATE. После изменения каждый триггер будет формировать результирующий набор, содержащий объединение строк таблиц INSERTED и DELETED.

```
use BSTU
go
insert  AUDITORIUM  values ('301-4', 'ЛК', 75, '301-4'),
                           ('401-4', 'ЛК', 80, '301-4');
update AUDITORIUM  set AUDITORIUM_CAPACITY += 5
where  AUDITORIUM in ('301-4','401-4');
delete AUDITORIUM  where  AUDITORIUM in ('301-4','401-4');
```

```
AUD_AFTER_INSERT
Событие: INSERT

AUD_AFTER_UPDATE
Событие: UPDATE

AUD_AFTER_DELETE
Событие: DELETE
```

Рис. 13.114. Выполнение триггеров, созданных в сценариях на рис. 13.109 и 13.113

```
use BSTU
go

alter trigger AUD_AFTER_INSERT
on AUDITORIUM after INSERT
as
 select 'insert:INSERTED', *  from INSERTED
 union
 select 'insert:DELETED',  *  from DELETED;
 return;
go

alter trigger AUD_AFTER_DELETE
on AUDITORIUM after DELETE
as
 select 'delete:INSERTED', *  from INSERTED
 union
 select 'delete:DELETED',  *  from DELETED;
 return;
go

alter trigger AUD_AFTER_UPDATE
on AUDITORIUM after UPDATE
as
 select 'update:INSERTED', *  from INSERTED
 union
 select 'update:DELETED',  *  from DELETED;
 return;
go
```

Рис. 13.115. Изменение триггеров **AUD_AFTER_INSERT**, **AUD_AFTER_UPDATE** и **AUD_AFTER_DELETE**

На рис. 13.116 представлен сценарий, активизирующий измененные триггеры (рис. 13.115), а также сформированные ими результирующие наборы.

```
use BSTU
go
insert  AUDITORIUM  values ('301-4', 'ЛК', 75, '301-4'),
                           ('401-4', 'ЛК', 80, '301-4');
update AUDITORIUM  set AUDITORIUM_CAPACITY += 5
where  AUDITORIUM in ('301-4','401-4');
delete AUDITORIUM  where  AUDITORIUM in ('301-4','401-4');
```

(No column name)	AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
insert:INSERTED	401-4	ЛК	80	301-4
insert:INSERTED	301-4	ЛК	75	301-4

(No column name)	AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
update:INSERTED	401-4	ЛК	85	301-4
update:INSERTED	301-4	ЛК	80	301-4
update:DELETED	401-4	ЛК	80	301-4
update:DELETED	301-4	ЛК	75	301-4

(No column name)	AUDITORIUM	AUDITORIUM_TYPE	AUDITORIUM_CAPACITY	AUDITORIUM_NAME
delete:DELETED	401-4	ЛК	85	301-4
delete:DELETED	301-4	ЛК	80	301-4

Рис. 13.116. Результаты выполнения триггеров **AUD_AFTER_INSERT**, **AUD_AFTER_UPDATE** и **AUD_AFTER_DELETE**

Важной особенностью AFTER-триггера является то, что он вызывается после выполнения активизирующего его оператора. Поэтому, если оператор нарушает ограничение целостности, то возникшая ошибка не допускает выполнения этого оператора и соответствующих триггеров (рис. 13.117).

```
use BSTU
go
alter table AUDITORIUM
add constraint CH_AUDITORIUM  check(AUDITORIUM_CAPACITY >=15)
go
update AUDITORIUM set AUDITORIUM_CAPACITY = 10 where AUDITORIUM = '206-1';
```

```
Msg 547, Level 16, State 0, Line 1
The UPDATE statement conflicted with the CHECK constraint "CH_AUDITORIUM".
The conflict occurred in database "BSTU",
table "dbo.AUDITORIUM", column 'AUDITORIUM_CAPACITY'.
The statement has been terminated.
```

Рис. 13.117. Блокировка вызова соответствующего AFTER-триггера при нарушении DML-оператором ограничения целостности

В AFTER-триггере допускается применения функции RAISERROR и TCL-операторов COMMIT и ROLLBACK (рис. 13.118, 13.119).

```
use BSTU
go
alter trigger AUD_AFTER
on AUDITORIUM after  DELETE, UPDATE
as
 declare @c int = (select sum(AUDITORIUM_CAPACITY) from AUDITORIUM);
 if (ISNULL(@c,0) < 1900)
 begin
  raiserror('Общая вместимость аудиторий не может быть < 1900',10,1);
  rollback;
 end;
 return;          ;
```

Рис. 13.118.Вызов системной функции RAISERROR и применение TCL-операторов COMMIT и ROLLBACK в AFTER-триггере

```
use BSTU
go
delete AUDITORIUM where AUDITORIUM_TYPE = 'ЛБ-К';
Общая вместимость аудиторий не может быть < 1900
Msg 3609, Level 16, State 1, Line 1
The transaction ended in the trigger. The batch has been aborted.
```

Рис. 13.119. Сообщение об ошибке и аварийном завершении транзакции в результате выполнения триггера **AUD_AFTER** (рис. 13.118)

INSTEAD OF-триггеры. Триггеры типа INSTEAD OF исполняются вместо оператора, вызвавшего соответствующее событие. На рис. 13.120 представлен сценарий, создающий INSTEAD OF-триггер с именем AUDTYPE_INSTEAD.

```
use BSTU
go
create trigger AUDTYPE_INSTED
on AUDITORIUM_TYPE instead of INSERT, DELETE, UPDATE
as
  raiserror (N'изменение данных запрещено!!!', 10, 1);
  return;
```

Рис. 13.120. Сценарий, создающий INSTEAD OF-триггер

Сценарий на рис. 13.121 демонстрирует выполнение INSTEAD OF-триггера, созданного с помощью сценария, представленного на рис. 13.120.

```
use BSTU
go
insert AUDITORIUM_TYPE values('ЛК', 'XXX');
update AUDITORIUM_TYPE  set AUDITORIUM_TYPENAME = 'YYY'
       where AUDITORIUM_TYPE = 'ЛК';
delete  AUDITORIUM_TYPE  where AUDITORIUM_TYPE = 'ЛК';
```

```
изменение данных запрещено!!!
изменение данных запрещено!!!
изменение данных запрещено!!!
```

Рис. 13.121. Сценарий, демонстрирующий выполнение INSTEAD OF-триггера **AUDTYPE_INSTEAD** (рис. 13.120)

Следует отметить, для таблицы или представления допускается создание только по одному INSTEAD OF-триггеру, реагирующему на каждое событие. Кроме того, выполнение INSTEAD OF-триггера предшествует проверке установленных для таблицы ограничений целостности.

Порядок выполнения DML-триггеров. Если для таблицы или представления созданы INSTEADOF и AFTER-триггеры, реагирующие на одно и то же событие, то выполниться только INSTEAD OF- триггер.

Для нескольких AFTER-триггеров, реагирующих на одно и то же событие, имеется возможность частично упорядочить их выполнение с помощью системной процедуры SP_SETTRIGGERORDER.

Процедура SP_SETTRIGGERORDER принимает три параметра: имя триггера (triggername), позиция в последовательности вызова (order) и тип события (stmttype).

Параметр order может принимать только два значения: First и Last. При этом с помощью значения First обозначается первая позиция триггера, а с помощью Last – последняя. Параметр stmttype может принимать одно из трех следующих значений: INSERT, UPDATE и DELETE.

Таким образом, для заданного с помощью параметра stmttype события с помощью процедуры SP_SETTRIGGERORDER могут быть указаны только первый и последний триггеры таблицы. Порядок выполнения других триггеров, реагирующих на это же событие той же таблицы, не гарантируется.

В сценарии на рис. 13.122 создается три AFTER-триггера, реагирующих на событие UPDATE таблицы AUDITORIUM. Вместе с триггерами, созданными ранее, общее количество AFTER-триггеров таблицы AUDITORIUM составляет пять.

```
use BSTU
go
create trigger AUD_AFTER_UPDATEA
on AUDITORIUM after UPDATE
as
 print 'AUD_AFTER_UPDATE_A';
 return;
go
create trigger AUD_AFTER_UPDATEB
on AUDITORIUM after UPDATE
as
 print 'AUD_AFTER_UPDATE_B';
 return;
go
create trigger AUD_AFTER_UPDATEC
on AUDITORIUM after UPDATE
as
 print 'AUD_AFTER_UPDATE_C';
 return;
go
```

Рис. 13.122. Создание трех AFTER-триггеров, реагирующих на событие **UPDATE** таблицы **AUDITORIUM**

```
use BSTU
go
select t.name, e.type_desc
from sys.triggers  t join  sys.trigger_events e
                     on t.object_id = e.object_id
where OBJECT_NAME(t.parent_id)='AUDITORIUM' and
      e.type_desc = 'UPDATE' ;
```

name	type_desc
AUD_AFTER_UPDATE	UPDATE
AUD_AFTER	UPDATE
AUD_AFTER_UPDATEA	UPDATE
AUD_AFTER_UPDATEB	UPDATE
AUD_AFTER_UPDATEC	UPDATE

Рис. 13.123. Получение перечня триггеров БД С помощью системных представлений **SYS.TRIGGERS** и **SYS.TRIGGER_EVENTS**

Сценарий на рис.13.124 дважды вызывает функцию SP_SETTRIGGERORDER. В первом случае триггер AUD_AFTER_UPDATEB устанавливается первым в порядке вызова при событии UPDATE таблицы AUDITORIUM. Второй вызов SP_SETTIRGGERORDER делает триггер AUD_AFTER_UPDATEA замыкающим.

```
use BSTU
go

exec SP_SETTRIGGERORDER @triggername = 'AUD_AFTER_UPDATEB',
                        @order='First', @stmttype = 'UPDATE';
exec SP_SETTRIGGERORDER @triggername = 'AUD_AFTER_UPDATEA',
                        @order='Last', @stmttype = 'UPDATE';
```

Рис. 13.124. Порядок вызова триггеров **AUD_AFTER_UPDATEB** и **AUD_AFTER_UPDATEB** при событии **UPDATE** таблицы **AUDITORIUM**

На рис. 13.125 приведен пример сценария, демонстрирующего порядок выполнения триггеров, реагирующих на событие UPDATE таблицы AUDITORIUM.

```
use BSTU
go
update AUDITORIUM set AUDITORIUM_CAPACITY = 80
where AUDITORIUM = '103-4'

AUD_AFTER_UPDATE_B
AUD_AFTER_UPDATE
Событие: UPDATE
AUD_AFTER_UPDATE_C
AUD_AFTER_UPDATE_A
```

Рис. 13.125. Пример, демонстрирующий порядок вызова AFTER-триггеров, реагирующих на событие **UPDATE** таблицы **AUDITORIM**

Удаление DML-триггеров. Как и DDL-триггеры, DML- триггеры могут быть удалены с помощью оператора DROP TRIGGER (рис.13.126). Обратите внимание: имена DML-триггеров в рамках одной БД уникальны и поэтому в операторе DROP не требуется указания таблицы с которой связан удаляемый триггер.

```
use BSTU
go
drop trigger AUD_INSTED, AUDTYPE_INSTED, AUD_AFTER_INSERT,
             AUD_AFTER_DELETE, AUD_AFTER_UPDATE, AUD_AFTER,
             AUD_AFTER_UPDATEA,AUD_AFTER_UPDATEB, AUD_AFTER_UPDATEC
```

Рис. 13.126. Удаление DML-триггеров

Глава 14. ПРИМЕНЕНИЕ XML

14.1. Стандарт XML

XML (Extensible Markup Language) – расширяемый язык разметки, разработка которого началась в 1996 г. консорциумом W3C [6]. В 1998 г. была выпущена первая спецификация XML 1.0. Эта версия постоянно совершенствуется, на сегодняшний день действует ее третья редакция. Параллельно идет работа над новой версией XML 1.1, но окончательный ее выпуск постоянно откладывается.

Язык XML задает общие правила, по которым создаются теги (элементы языка разметки, предназначенные для разделения данных в текстовых файлах) и оформляются XML-документы. При этом XML-документ представляет собой текстовый файл, содержащий данные, разделенные (часто говорят – размеченные) тегами, составленными по правилам XML и расположенными в правильном (в соответствии с правилами XML) порядке. Совокупность XML-тегов, предназначенных для разметки документов определенного типа называют словарями XML или XML-реализациями. Например, XHTML является XML-реализацией языка гипертекстовой разметки текста, CML – реализация языка записи химических формул, а VoxML – языка записи звуков.

На рис. 14.1 представлен пример XML-документа, содержащего информацию о двух преподавателях кафедры.

```
<?xml version="1.0" encoding="utf-8"?>

<Кафедра код="ИСиТ">
    <Преподаватель фамилия="Пацей" имя="Наталья" отчетсво="Владимирована" должность ="доцент">
        <Дисциплина>Основы алгоритмизации и программирования</Дисциплина>
        <Дисциплина>Объектно-ориентированное программирование</Дисциплина>
        <Дисциплина>Тестирование программного обеспечения</Дисциплина>
    </Преподаватель>
    <Преподаватель фамилия="Шиман" имя="Дмитрий" отчетсво="Васильевич" должность ="доцент">
      <Дисциплина>Программирование сетевых приложений</Дисциплина>
      <Дисциплина>Распределнные системы</Дисциплина>
    </Преподаватель>
</Кафедра>
```

Рис.14.1. Пример XML-документа

В XML-документе (рис. 14.1) применяется три тега с именами: **Кафедра**, **Преподаватель** и **Дисциплина**. Тег **Кафедра** имеет один атрибут с именем **код**, тег **Преподаватель** – четыре атрибута (**фамилия**, **имя**, **отчество** и **должность**), а тег **Дисциплина** не имеет атрибутов.

Каждый записанный в XML-документе тег представляет элемент этого документа. Например, первый тег **Преподаватель** задает элемент, описывающий информацию о преподавателе с фамилией **Пацей**.

Обратите внимание на следующее: 1) XML-документ начинается специальным тегом с именем **?xml** (называется – объявление XML), содержащим информацию о версии XML и используемой кодировке; 2) каждый элемент начинается открывающим тегом и завершается закрывающим тегом (имя начинается с наклонной); 3) между открывающим и закрывающим тегами некоторых элементов (например, соответствующим тегам **Преподаватель**) могут находиться другие теги; 4) XML-документ является содержимым единственного элемента (в примере соответствует тегу **Кафедра**), называемого корневым; 5) применение XML-формата влечет к значительному увеличению избыточности данных; 6) XML-формат позволяет создавать самодокументированные данные.

Для более подробного знакомства с языком XML рекомендуется изучить пособие [7].

14.2. Обмен XML-данными

XML-формат часто используется для обмена данными между компонентами одной информационной системы или на межсистемном уровне. Здесь проявляются главные преимущества XML: 1) все программные платформы «понимают» XML одинаково; 2) практически все программные системы имеют встроенные механизмы, позволяющие эффективно обрабатывать XML-данные.

Для разработчиков программного обеспечения важными являются две задачи: преобразование табличных данных в XML-структуры; преобразование XML-структур в строки реляционной таблицы.

14.2.1. Преобразование реляционных данных в XML-структуры

Для преобразования результата SELECT-запроса в формат XML в операторе SELECT применяется секция FOR XML. При этом можно использовать один из четырех режимов: RAW, AUTO, PATH и EXPLICIT. В рамках этого пособия рассматриваются только первые три режима, для знакомства с принципами применения режима EXPLICIT рекомендуется изучить издание [5].

14.2.1.1. Режим RAW. По умолчанию в режиме RAW в результате SELECT-запроса создается XML-фрагмент, состоящий из последовательности элементов с именем **row**. Каждый элемент **row** соответствует строке результирующего набора, имена его атрибутов совпадают с именами столбцов результирующего набора (из списка SELECT), а значения атрибутов равны их значениям.

На рис. 14.2 представлен пример SELECT-запроса, формирующего результирующий набор в реляционном виде. В SELECT-запросе на рис. 14.3 применена инструкция FOR XML RAW, позволяющая получить результирующий набор в виде XML-фрагмента в режиме RAW.

```
use BSTU
go
select * from TEACHER where PULPIT = 'ЛВ'
```

TEACHER	TEACHER_NAME	GENDER	PULPIT
ЛБХ	Лабоха Константин Валентинович	м	ЛВ
РЖК	Рожков Леонид Николаевич	м	ЛВ

Рис. 14.2. Результат SELECT-запроса в реляционном виде

```
use BSTU
go
select * from TEACHER where PULPIT = 'ЛВ' for xml raw
```

```
<row TEACHER="ЛБХ" TEACHER_NAME="Лабоха Константин Валентинович" GENDER="м" PULPIT="ЛВ" />
<row TEACHER="РЖК" TEACHER_NAME="Рожков Леонид Николаевич " GENDER="м" PULPIT="ЛВ" />
```

Рис. 14.3. Применение инструкции FOR XML RAW

Обратите внимание: 1) результатом выполнения SELECT-запроса является XML-фрагмент, а не XML-документ; 2) данные из столбцов таблицы представляются в виде значений атрибутов (такая форма представления данных называется атрибутной).

Если в SELECT-списке запроса указать столбцы, то сформированные элементы имеют только соответствующие им столбцы (рис. 14.4).

```
use BSTU
go
select  TEACHER, GENDER from TEACHER where PULPIT = 'ЛВ' for xml raw
```

```
<row TEACHER="ЛБХ" GENDER="м" />
<row TEACHER="РЖК" GENDER="м" />
```

Рис. 14.4. XML-элементы, имеющие атрибуты, соответствующие столбцам, указанным в SELECT-списке запроса

Для того чтобы изменить стандартное имя элемента **raw** на заданное, достаточно его указать в скобках после ключевого слова RAW (рис. 14.5).

```
use BSTU
go
select  TEACHER, GENDER from TEACHER where PULPIT = 'ЛВ' for xml raw('MYTAG');
```

```
<MYTAG TEACHER="ЛБХ" GENDER="м" />
<MYTAG TEACHER="РЖК" GENDER="м" />
```

Рис. 14.5. Формирование имени элемента, соответствующего строке результирующего набора, после ключевого слова RAW

Построить корневой элемент для XML-фрагмента можно с помощью ключевого слова ROOT, которое может быть применено в инструкции FOR XML (рис. 14.6).

```
use BSTU
go
select  TEACHER, GENDER from TEACHER where PULPIT = 'ЛВ' for xml raw('MYTAG'),root('MYROOT');
```

```
<MYROOT>
  <MYTAG TEACHER="ЛБХ" GENDER="м" />
  <MYTAG TEACHER="РЖК" GENDER="м" />
</MYROOT>
```

Рис. 14.6. Создание корневого элемент с помощью ключевого слова ROOT

Если в сформированных XML-элементах требуется, чтобы имена атрибутов отличались от имен столбцов исходных таблиц, следует в SELECT-списке для столбцов указать псевдонимы, которые будут использованы в качестве имен атрибутов (рис. 14.7).

```
use BSTU
go
select p.PULPIT_NAME 'наименование_кафедры', t.TEACHER 'преподаватель', t.GENDER 'пол'
from  PULPIT p join TEACHER t on p.PULPIT = t.PULPIT
where t.PULPIT = 'ЛВ' for xml raw('MYTAG'),root('MYROOT');
```

```
<MYROOT>
  <MYTAG наименование_кафедры="Лесоводства" преподаватель="ЛБХ" пол="м" />
  <MYTAG наименование_кафедры="Лесоводства" преподаватель="РЖК" пол="м" />
</MYROOT>
```

Рис. 14.7. Использование псевдонимов столбцов в качестве имен атрибутов

Обратите внимание, что во всех приведенных выше примерах в результате использования режима RAW (рис. 14.3–14.7) формируется результирующий набор в атрибутном виде (данные – значение атрибута). Для того чтобы сформировать результирующий набор в альтернативном, называемом элементном виде, необходимо в инструкции FOR XML применить ключевое слово ELEMENTS (рис. 14.8).

```
use BSTU
go
select p.PULPIT_NAME 'наименование_кафедры', t.TEACHER 'преподаватель', t.GENDER 'пол'
from  PULPIT p join TEACHER t on p.PULPIT = t.PULPIT
where t.PULPIT = 'ЛВ' for xml raw('Преподаватель'),root('Список_преподавателей'), elements;
```

```
<Список_преподавателей>
   <Преподаватель>
     <наименование_кафедры>Лесоводства</наименование_кафедры>
     <преподаватель>ЛБХ</преподаватель>
     <пол>м</пол>
   </Преподаватель>
   <Преподаватель>
     <наименование_кафедры>Лесоводства</наименование_кафедры>
     <преподаватель>РЖК</преподаватель>
     <пол>м</пол>
   </Преподаватель>
 </Список преподавателей>
```

Рис. 14.8. Использование ключевого слова ELEMENTS для формирования результирующего набора в элементном виде

Обратите внимание (рис. 14.8): 1) имена столбцов (или их псевдонимы) являются именами вложенных элементов; 2) данные представлены как содержимое элементов; 3) применение JOIN в секции FROM оператора SELECT никак не влияет на структуру формируемого XML-фрагмента.

14.2.1.2. Режим AUTO. Результат, полученный в режиме AUTO для простых SELECT-запросов, похож на результат, полученный в режиме RAW. Основное отличие – в качестве имени элемента, соответствующего строке исходной таблицы, используется ее имя (рис. 14.9).

```
use BSTU
go
select TEACHER, GENDER  from TEACHER where PULPIT = 'ЛВ' for xml auto
```

```
<TEACHER TEACHER="ЛБХ" GENDER="м" />
<TEACHER TEACHER="РЖК" GENDER="м" />
```

Рис. 14.9. Применение инструкции FOR XML AUTO

Особенность режима AUTO проявляется в многотабличных запросах. В этом случае режим AUTO позволяет построить XML-фрагмент с применением вложенных элементов. Причем порядок вложенности основывается на порядке столбцов (слева направо), указанных в SELECT-списке.

На рис. 14.10 и 14.11 приведены примеры многотабличных SELECT-запросов, формирующих результирующие наборы в режиме AUTO.

```
use BSTU
go
select p.PULPIT, t.TEACHER, t.GENDER
from  PULPIT p join TEACHER t on p.PULPIT = t.PULPIT
where t.PULPIT in('ЛВ','ПОиСОИ') order by p.PULPIT
for xml auto
```

```
<p PULPIT="ЛВ">
  <t TEACHER="ЛБХ" GENDER="м" />
  <t TEACHER="РЖК" GENDER="м" />
</p>
<p PULPIT="ПОиСОИ">
  <t TEACHER="БРТШВЧ" GENDER="м" />
  <t TEACHER="ЮДНКВ" GENDER="м" />
  <t TEACHER="ШМК" GENDER="м" />
  <t TEACHER="АРС" GENDER="м" />
  <t TEACHER="БЛВ" GENDER="м" />
</p>
```

Рис. 14.10. Формирование XML-фрагментов с вложенными элементами для многотабличных запросов

```
use BSTU
go
select  [Факультет].FACULTY [факультет],
        [Кафедра].PULPIT [кафедра],
        [Преподаватель].TEACHER [преподаватель],
        [Преподаватель].GENDER [пол]
from  FACULTY [Факультет] join PULPIT [Кафедра] on  [Факультет].FACULTY = [Кафедра].FACULTY
                          join TEACHER [Преподаватель]  on [Кафедра].PULPIT = [Преподаватель].PULPIT
where [Преподаватель].PULPIT in('ЛВ','ПОиСОИ') order by [факультет],[кафедра]
for xml auto, root ('Список_преподавателей')
```

```
<Список_преподавателей>
   <Факультет факультет="ИДиП">
     <Кафедра кафедра="ПОиСОИ">
       <Преподаватель преподаватель="ЮДНКВ" пол="м" />
       <Преподаватель преподаватель="ШМК" пол="м" />
       <Преподаватель преподаватель="АРС" пол="м" />
       <Преподаватель преподаватель="БЛВ" пол="м" />
       <Преподаватель преподаватель="БРТШВЧ" пол="м" />
     </Кафедра>
   </Факультет>
   <Факультет факультет="ЛХФ">
     <Кафедра кафедра="ЛВ">
       <Преподаватель преподаватель="ЛБХ" пол="м" />
       <Преподаватель преподаватель="РЖК" пол="м" />
     </Кафедра>
   </Факультет>
 </Список_преподавателей>
```

Рис. 14.11. Три уровня вложенности XML-элементов

Как и в режиме RAW, здесь допускается применение ключевых слов ROOT (указывает имя корневого элемента) и ELEMENTS (формирование элементного вида XML-фрагмента). На рис. 14.11 используется ROOT для формирования корневого элемента XML-фрагмента в атрибутной форме. На рис. 14.12 применяется ROOT и ELEMEMTS.

```
use BSTU
go
select  [Факультет].FACULTY [факультет],
        [Кафедра].PULPIT [кафедра],
        [Преподаватель].TEACHER [преподаватель],
        [Преподаватель].GENDER [пол]
from  FACULTY [Факультет] join PULPIT [Кафедра] on  [Факультет].FACULTY = [Кафедра].FACULTY
                          join TEACHER [Преподаватель]  on [Кафедра].PULPIT = [Преподаватель].PULPIT
where [Преподаватель].PULPIT in('ЛВ','ПОиСОИ') order by [факультет],[кафедра]
for xml auto, root ('Список_преподавателей'), elements
```

```
<Список_преподавателей>
   <Факультет>
     <факультет>ИДиП</факультет>
     <Кафедра>
       <кафедра>ПОиСОИ</кафедра>
       <Преподаватель>
         <преподаватель>ЮДНКВ</преподаватель>
         <пол>м</пол>
       </Преподаватель>
       <!--.....  и т.д. .....-->
     </Кафедра>
   </Факультет>
   <Факультет>
     <факультет>ЛХФ</факультет>
     <Кафедра>
       <кафедра>ЛВ</кафедра>
       <Преподаватель>
         <преподаватель>ЛБХ</преподаватель>
         <пол>м</пол>
       </Преподаватель>
       <!--.....  и т.д. .....-->
     </Кафедра>
   </Факультет>
 </Список_преподавателей>
```

Рис. 14.12. Формирование XML-фрагмента в элементной форме
с помощью ключевого слова ELEMENTS

14.2.1.3. Режим PATH. Режим PATH позволяет разработчику наиболее полным образом управлять процессом формирования XML-структуры. Каждый столбец конфигурируется независимо с помощью заданного в формате XPATH [7] имени псевдонима этого столбца. В таблице 14.1 приведено несколько шаблонов, которые могут быть использованы при записи псевдонимов столбцов в режиме PATH.

Если псевдонимы задавать традиционным способом (это соответствует первому шаблону в таблице 14.1), то сформируется XML-фрагмент, состоящий из элементов с именем **row**, соответствующих строкам результирующего набора. Каждый **row**-элемент включает в себя все значения столбцов одной строки в элементной форме. При этом элементы будут иметь имена, совпадающие с заданными в SELECT-списке псевдонимами (рис. 14.13).

Таблица 14.1

Условные обозначения применяемы в режиме PATH

Шаблон	Описание
eName	Создание XML-элемента с именем eName с содержимым соответствующего столбца
@aName	Создание XML-атрибута с именем aName с содержимым соответствующего столбца
eName1/eName2	Создание XML-элемента с именем eName1 и вложенного элемента с именем eName2 с содержимым соответствующего столбца
eName/@aName	Создание XML-элемента с именем eName и атрибут с именем aName с содержимым соответствующего столбца
text()	Создание текстового содержимого тега
data()	Создание атомарного значения текстового содержимого тега

```
use BSTU
go
select  [Факультет].FACULTY [факультет],
        [Кафедра].PULPIT [кафедра],
        [Преподаватель].TEACHER [преподаватель],
        [Преподаватель].GENDER [пол]
from  FACULTY [Факультет] join PULPIT [Кафедра] on  [Факультет].FACULTY = [Кафедра].FACULTY
                          join TEACHER [Преподаватель]  on [Кафедра].PULPIT = [Преподаватель].PULPIT
where [Преподаватель].PULPIT in('ЛВ','ПОиСОИ') order by [факультет],[кафедра]
for xml path, root ('Список преподавателей')
```

```
<Список_преподавателей>
   <row>
     <факультет>ИДиП</факультет>
     <кафедра>ПОиСОИ</кафедра>
     <преподаватель>ЮДНКВ</преподаватель>
     <пол>м</пол>
   </row>
   <row>
     <факультет>ИДиП</факультет>
     <кафедра>ПОиСОИ</кафедра>
     <преподаватель>ШМК</преподаватель>
     <пол>м</пол>
   </row>
   <!--.................. и т.д. .........................-->
 </Список_преподавателей>
```

Рис. 14.13. Простейший XML-фрагмент, сформированный в режиме PATH

На рис. 14.14 и 14.15 приведен пример SELECT-запроса в режиме PATH, использующего несколько шаблонов из табл. 14.1. Следует обратить внимание, что в этом режиме при формировании результата у разработчика появляется возможность сочетать атрибутную и элементную форму.

```
use BSTU
go
select  [Факультет].FACULTY [факультет/@код],
        [Кафедра].PULPIT [факультет/кафедра/@код],
        [Преподаватель].TEACHER [факультет/кафедра/преподаватель/@код],
        [Преподаватель].TEACHER_NAME [факультет/кафедра/преподаватель/data()],
        [Преподаватель].GENDER [факультет/кафедра/преподаватель/data()]
from  FACULTY [Факультет] join PULPIT [кафедра] on  [Факультет].FACULTY = [кафедра].FACULTY
                          join TEACHER [Преподаватель]  on [кафедра].PULPIT = [Преподаватель].PULPIT
where [Преподаватель].PULPIT in('ЛВ','ПОиСОИ') order by [факультет/@код],[факультет/кафедра/@код]
for xml path(''), root ('Список_преподавателей')
```

Рис. 14.14. Применение шаблонов для псевдонимов столбцов в режиме PATH

```
<Список_преподавателей>
  <факультет код="ИДиП">
    <кафедра код="ПОиСОИ">
      <преподаватель код="ЮДНКВ">Юденков Виктор Степанович м</преподаватель>
    </кафедра>
  </факультет>
  <факультет код="ИДиП">
    <кафедра код="ПОиСОИ">
      <преподаватель код="ШМК">Шмаков Михаил Сергеевич м</преподаватель>
    </кафедра>
  </факультет>
  <!--.............. и т.д.............-->
  <факультет код="ЛХФ">
    <кафедра код="ЛВ">
      <преподаватель код="ЛБХ">Лабоха Константин Валентинович м</преподаватель>
    </кафедра>
  </факультет>
  <факультет код="ЛХФ">
    <кафедра код="ЛВ">
      <преподаватель код="РЖК">Рожков Леонид Николаевич  м</преподаватель>
    </кафедра>
  </факультет>
</Список_преподавателей>
```

Рис. 14.15. Результат выполнения SELECT-запроса,
представленного на рис. 14.14, режиме PATH

В примере на рис. 14.16 приведен более сложный запрос, формирующий четырехуровневую XML-структуру. При этом в SELECT-списке внешнего запроса используются два подзапроса. Первый подзапрос формирует скалярное значение, которое потом записывается как текстовое содержимое тега **количество_кафедр**. В результате выполнения второго подзапроса формируется значение атрибута с именем **код** элемента **кафедра**, а также XML-фрагмент, который помещается в тег **преподаватели**.

Обратите внимание на применение ключевого слова **TYPE** во внутреннем подзапросе. Использование **TYPE** позволяет указать на то, что формируемый XML-фрагмент следует рассматривать как вложенный.

```
select f.FACULTY [@код],
       (
        select count(*)
        from PULPIT p1 where p1.FACULTY = f.FACULTY
       ) [количество_кафедр/text()],
       (
        select p.PULPIT [@код],
              (
               select TEACHER [@код],TEACHER_NAME [data()] from TEACHER t
               where  t.PULPIT = p.PULPIT  for xml path('преподаватель'),type
               ) [преподаватели]
        from  PULPIT p
        where f.FACULTY = p.FACULTY for xml path('кафедра'),type
       ) [кафедры]
        from   FACULTY f for xml path('факультет'),root('университет')
```

Рис. 14.16. Пример использования вложенных подзапросов в режиме PATH

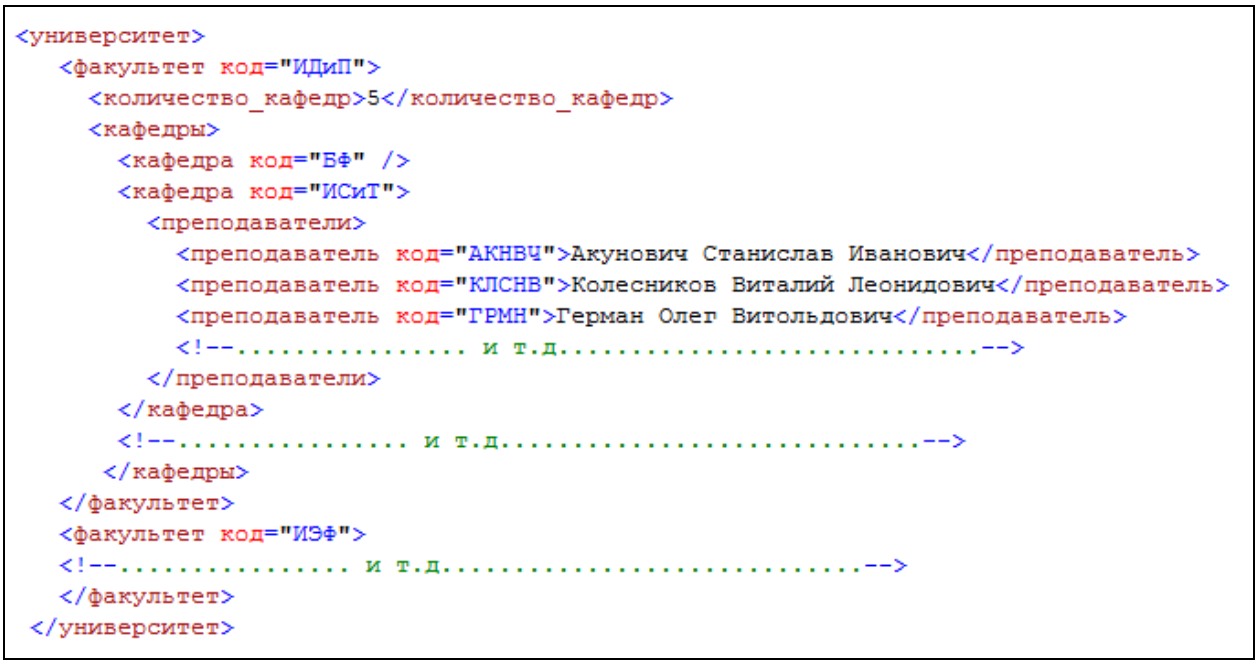


Рис. 14.17. Результат выполнения SELECT-запроса,
представленного на рис. 14.16, в режиме PATH

14.2.2. Преобразование XML-документа в табличный вид

Для преобразования XML-данных в строки таблицы предназначена функция OPENXML. Она принимает три входных параметра: дескриптор, сформированный системной хранимой процедурой с именем SP_XML_PREPAREDOCUMENT; выражение XPATH [7] и целое положительное число, определяющее режим работы функции.

Процедура SP_XML_PREPAREDOCUMENT должна быть выполнена до SELECT-запроса, применяющего OPENXML. Процедура принимает в качестве входного параметра XML-документ (в формате строки) и возвращает дескриптор, который впоследствии применяется функцией OPENXML.

Выражение XPATH, принимаемое функцией OPENXML в качестве второго параметра, предназначено для выбора требуемых данных из исходного (введенного процедурой SP_XML_PREPAREDOCUMENT) XML-документа.

Последний, третий, параметр функции OPENXML указывает на тип преобразования (режим). Основные базовые типы преобразования приведены в табл. 14.2.

Кроме того, с помощью выражения WITH должна быть указана структура формируемого результирующего набора.

Таблица 14.2

Режимы работы функции OPENXML

Режим	Описание
0	Применяется атрибутивная модель сопоставления. Каждый XML-атрибут преобразовывается в столбец таблицы
1	Аналогично режиму 0, но для необработанных столбцов применяется сопоставление на основе элементов XML-документа
2	Применяется сопоставление на основе элементов. Каждый элемент преобразовывается в столбец таблицы

На рис. 14.8 приведен пример использования функции OPENXML для формирования реляционного результирующего набора.

```
use BSTU
go
declare @h int = 0,
        @x varchar(2000) = ' <?xml version="1.0" encoding="windows-1251" ?>
                            <аудитории>
                             <аудитория номер="309-1" тип="ЛБ-К" вместимость="20" />
                             <аудитория номер="310-1" тип="ЛБ-К" вместимость="15" />
                             <аудитория номер="114-4" тип="Л"    вместимость="80" />
                            </аудитории>';
exec sp_xml_preparedocument @h output, @x;          -- подготовка документа
select * from openxml(@h, '/аудитории/аудитория', 0)
with(
     [номер] char(10),
     [тип] char(10),
     [вместимость] int
    )
exec sp_xml_removedocument @h;                      -- удаление документа
```

номер	тип	вместимость
309-1	ЛБ-К	20
310-1	ЛБ-К	15
114-4	Л	80

Рис. 14.18. Пример использования функции OPENXML

Обратите внимание (рис. 14.18): 1) из XML-фрагмента с помощью XPATH-выражения выделяется множество тегов с именами **аудитория**; 2) применяется режим работы **0**, подразумевающий атрибутивное сопоставление; 3) имена столбцов в выражении WITH совпадают с именами атрибутов в XML-структуре; 4) после применения функции OPENXML должна быть выполнена системная процедура SP_XML_REMOVEDOCUMENT, освобождающая использованные OPENXML ресурсы.

Для того, чтобы явно указать сопоставление XML-структуры со структурой стоки результирующего набора, можно в выражении WITH каждого столбца результирующего набора указать XPATH-выражение, определяющее содержимое этого столбца (рис. 14.19).

```
use BSTU
go
declare @h int = 0;
declare @x varchar(2000) =
  '<?xml version="1.0" encoding="windows-1251" ?>
   <аудитории  дата= "2014.05.15" >
     <аудитория номер="309-1" тип="ЛБ-К" вместимость="20" />
     <аудитория номер="310-1" тип="ЛБ-К" вместимость="15" />
     <аудитория номер="114-4" тип="Л"    вместимость="80" />
   </аудитории>';
exec sp_xml_preparedocument @h output, @x;
select * from openxml(@h, '/аудитории/аудитория', 0)
with(
     loaddate       datetime    '../@дата',
     auditorium     char(10)    '@номер',
     auditorium_type char(10)   '@тип',
     auditorium_capacity int    '@вместимость'
    )
exec sp_xml_removedocument @h;
```

loaddate	auditorium	auditorium_type	auditorium_capacity
2014-05-15 00:00:00.000	309-1	ЛБ-К	20
2014-05-15 00:00:00.000	310-1	ЛБ-К	15
2014-05-15 00:00:00.000	114-4	Л	80

Рис. 14.19. Явное указание XPATH-выражения, определяющего источник данных

На рис. 14.20 приведен пример использования OPENXML в режиме 2, предназначенном для обработки XML-фрагмента в элементном формате XML-данных.

```
use BSTU
go
declare @h int = 0;
declare @x varchar(2000) =
'<?xml version="1.0" encoding="windows-1251" ?>
<аудитории> <дата> 2014.05.15 </дата>
<аудитория>
  <номер>309-1</номер><тип>ЛБ-К</тип><вместимость>20</вместимость>
 </аудитория>
<аудитория>
  <номер>310-1</номер><тип>ЛБ-К</тип><вместимость>15</вместимость>
</аудитория>
<аудитория>
  <номер>114-1</номер><тип>Л</тип><вместимость>80</вместимость>
 </аудитория>
 </аудитории>';
exec sp_xml_preparedocument @h output, @x;
select * from openxml(@h, '/аудитории/аудитория', 2)
with(
      [дата]          datetime '../дата' ,
      [номер]         char(10),
      [тип]           char(10),
      [вместимость]   int
    )
exec sp_xml_removedocument @h;
```

loaddate	auditorium	auditorium_type	auditorium_capacity
2014-05-15 00:00:00.000	309-1	ЛБ-К	20
2014-05-15 00:00:00.000	310-1	ЛБ-К	15
2014-05-15 00:00:00.000	114-4	Л	80

Рис. 14.20. Применение режима 2 функций OPENXML
для обработки XML-фрагмента в элементном формате

14.3. Тип данных XML

В большинстве своем данные, хранимые в таблицах БД, имеют фиксированный формат. Например, дата рождения. Очевидно, что для хранения этой информации можно воспользоваться типом DATE. При этом, если потребуется указать, что эта дата не известна, то для столбца можно разрешить значение NULL и интерпретировать при обработке это значение соответствующим образом.

В тех случаях, если данные являются составными (состоящими из нескольких компонент), и при этом не имеют фиксированный формат, удобно использовать XML для их структуризации. Например, если в таблице хранится информация о гражданах разных стран, то структура их почтового адреса может значительно отличаться в зависимости от страны проживания. Или, например, информация об образовании людей (учебные заведения, даты начала и завершения обучения, специальности, квалификации и пр.) будут различными у разных людей.

В таблице БД допускается использование столбцов типа XML. В примере на рис. 14.21 создается таблица **STUDENT**, содержащая XML-столбец с именем **INFO**.

```
use BSTU
go
create table STUDENT
(
     IDSTUDENT   integer  identity(1000,1)
                 constraint STUDENT_PK  primary key,
     IDGROUP     integer
                 constraint STUDENT_GROUP_FK
                 foreign key  references GROUPS(IDGROUP),
     NAME      nvarchar(100),
     BDAY      date,
     STAMP     timestamp,
     INFO      xml,          -- столбец XML-типа
     FOTO      varbinary
 );
```

Рис. 14.21. Создание таблицы, содержащей XML-столбец

XML-данные могут быть добавлены в таблицу с помощью оператора INSERT. В простейшем случае сами XML-данные задаются в виде строкового литерала (рис. 14.22), который автоматически будет преобразован в XML-тип при записи

```
insert into STUDENT (IDGROUP,NAME, BDAY, INFO)
values(1, 'Манакова Анастасия Владимировна','09.12.1995',
          '<студент>
             <паспорт серия="MP" номер="22223333" дата="11.01.2000" />
             <телефон>44445555</телефон>
             <адрес>
                  <страна>Беларусь</страна>
                  <город>Минск</город>
                  <улица>Кирова</улица>
                  <дом>52</дом>
                  <квартира>76</квартира>
             </адрес>
          </студент>' );
```

Рис. 14.22. Добавление в таблицу строки, содержащей данные XML-типа

С помощью оператора SELECT данные XML-типа могут быть найдены и извлечены из БД. В простейшем случае извлекается полностью XML-структура (рис. 14.23).

```
use BSTU
go
select IDSTUDENT, INFO from  STUDENT
```

IDSTUDENT	INFO
1001	<студент><паспорт серия="MP" номер="22223333" дата="11.01.2000" /><телефон>444455...

Рис. 14.23. Извлечение данных XML-столбца

Часто требуется извлечь не полностью хранящуюся в БД XML-структуру, а лишь ее подмножество. Для этого можно воспользоваться методами XML-типа (табл. 14.3). В качестве параметра методы принимают строку, содержащую выражение на языках XPATH и XQUERY [7],

позволяющие либо выделить, либо изменить фрагменты хранящегося в столбце XML-документа (рис. 14.24–14.26).

Таблица 14.3

Методы типа данных XML

Метод	Описание
query	Исполняет выражения XPATH и XQUERY и возвращает XML-фрагмент
value	Исполняет выражения XPATH и XQUERY и возвращает скалярное значение, которое может быть преобразовано в тип SQL
exist	Исполняет выражения XPATH и XQUERY и возвращает 1, если узел, заданный выражением, найден
modify	Изменение XML-содержимого
nodes	Исполняет выражения XPATH и XQUERY и возвращает XML-фрагмент

```
use BSTU
go
select IDSTUDENT,
       INFO.value('(/студент/паспорт/@серия)[1]','varchar(10)') [серия паспорта],
       INFO.value('(/студент/паспорт/@номер)[1]','varchar(10)') [номер паспорта],
       INFO.value('(/студент/телефон)[1]','varchar(10)')        [телефон],
       INFO.query('/студент/адрес')                            [адрес]
 from  STUDENT
```

IDSTUDENT	серия паспорта	номер паспорта	телефон	адрес
1001	MP	22223333	44445555	<адрес><страна>Беларусь</страна><город>Минск</го...

Рис. 14.24. Применение метода **value** XML-типа для извлечения значений атрибутов и текстовых данных, метода **query** для извлечения фрагмента XML-документа

Обратите внимание (рис. 14.24): 1) XML-тип является объектным типом (имеет методы и свойства); 2) в первых двух случаях применения метода **value** извлекаются значения атрибутов **серия** и **номер**; 3) в третьем случае применения **value** извлекается текстовое значение элемента **телефон**; 4) во всех трех случаях применения метода **value** в конце заданного выражения стоит число в квадратных скобках, обозначающее номер выбираемого экземпляра; дело в том, что выражения, указанные в качестве параметров, позволяют выбрать более чем одно значение; указанная в квадратных скобках единица позволяет получить только первый экземпляр; 5) второй параметр метода **value** указывает на тип данных, к которому должно быть преобразовано выбранное значение; 6) метод **query**, применяемый в четвертом элементе SELECT-списка, позволяет выбрать XML-фрагмент.

В примере на рис. 14.25 метод **value** используется в выражении секции FROM оператора UPDATE. В секции SET задается новое значение для столбца **INFO** в выбранных строках таблицы **STUDENT**.

```
use BSTU
go
update STUDENT
set INFO = '<студент>
              <паспорт серия="MP" номер="7777777" дата="11.01.2000" />
              <телефон>44445555</телефон>
              <телефон>66666666</телефон>
            </студент>'
where  INFO.value('(/студент/паспорт/@номер)[1]','varchar(10)') = '22223333';
select   INFO.value('(/студент/паспорт/@номер)[1]','varchar(10)') [номер паспорта]
from STUDENT;
```

номер паспорта
7777777

Рис. 14.25. Поиск и корректировка данных XML-типа в таблице

T-SQL допускает применение переменных XML-типа в сценариях. Работа с такими переменными практически ничем не отличается от работы с переменными традиционных типов.

На рис. 14.26 приведен пример сценария, в котором с помощью оператора DECLARE объявлены две переменные XML-типа.

```
declare @x xml, @x1 xml;
select @x = INFO from STUDENT
       where INFO.value('(/студент/паспорт/@номер)[1]','varchar(10)') = '7777777';
set @x1 = @x.query('for $n in /студент/телефон return $n');

print cast(@x as varchar(200));
print cast(@x1 as varchar(200));
if  @x.exist('/студент/телефон') = 1  print 'есть тег /студент/телефон';
else  print 'нет тега /студент/телефон';
```

```
<студент><паспорт серия="MP" номер="7777777" дата="11.01.2000"/><телефон>44445555</телефон><телеф
<телефон>44445555</телефон><телефон>66666666</телефон>
есть тег /студент/телефон
```

Рис. 14.26. Применение XML-типа в сценарии T-SQL

С помощью операторов SELECT и SET этим переменным присваиваются значения. С помощью оператора PRINT предварительно преобразованные значения XML-переменных в строки выводятся в выходной поток. В логическом выражении оператора IF применяется метод **exists**, позволяющий определить наличие заданного XML-узла в значении XML-переменной.

14.4. Типизированные XML-данные

Главной особенностью данных XML-типа является их сложность. Значение XML-переменной или XML-столбца таблицы, как правило, является составным и может содержать в себе несколько других значений, которые сами могут представлять собой XML-документы, XML-фрагменты, просто числовые, символьные или другие данные. Сложность усугубляется еще тем, что некоторые элементы, атрибуты XML-документа или значения могут быть не обязательными. Для упрощения работы XML-тип реализован в виде объектного типа. Иными словами, XML-тип имеет методы (табл. 14.3), облегчающие их применение.

Сложность XML-данных требует иного подхода к механизму ограничения целостности для этого типа данных. В семействе XML-технологий существует и активно используется технология, основанная на языке XML-Schema.

XML-Schema – это одна из реализаций языка XML, поддерживаемая консорциумом W3C и предназначенная для описания структуры XML-документа. С помощью языка XML-Schema можно описать правила, которым должен подчиниться XML-документ. Файл, содержащий XML-Schema, обычно имеет расширение XSD (XML Schema definition). Большинство современных систем программирования предусматривают встроенные механизмы, позволяющие с помощью заданного XSD-файла проверять на корректность XML-документы. Для знакомства с языком XML-Schema рекомендуются источники [6, 7].

Для хранения документов XML-Schema в БД MSS предусмотрен специальный объект – XML SCHEMA COLLECTION. Каждый такой объект может содержать один или более XML-SCHEMA-документов.

На рис. 14.27 приведен пример создания объекта XML SCHEMACOLLECTION (далее коллекция схем) с именем **Student**, содержащего один документ.

```
use BSTU
go
create xml schema collection Student as
 N'<?xml version="1.0" encoding="utf-16" ?>
   <xs:schema attributeFormDefault="unqualified"
              elementFormDefault="qualified"
              xmlns:xs="http://www.w3.org/2001/XMLSchema">
   <xs:element name="студент">
       <xs:complexType><xs:sequence>
       <xs:element name="паспорт" maxOccurs="1" minOccurs="1">
           <xs:complexType>
           <xs:attribute name="серия" type="xs:string" use="required" />
           <xs:attribute name="номер" type="xs:unsignedInt" use="required"/>
           <xs:attribute name="дата"  use="required"  >
               <xs:simpleType>
               <xs:restriction base ="xs:string">
                   <xs:pattern value="[0-9]{2}.[0-9]{2}.[0-9]{4}"/>
               </xs:restriction>
               </xs:simpleType>
          </xs:attribute>
           </xs:complexType>
       </xs:element>
       <xs:element maxOccurs="3" name="телефон" type="xs:unsignedInt"/>
       <xs:element name="адрес">
           <xs:complexType><xs:sequence>
           <xs:element name="страна" type="xs:string" />
           <xs:element name="город" type="xs:string" />
           <xs:element name="улица" type="xs:string" />
           <xs:element name="дом" type="xs:string" />
           <xs:element name="квартира" type="xs:string" />
           </xs:sequence></xs:complexType>
       </xs:element>
       </xs:sequence></xs:complexType>
   </xs:element></xs:schema>';
```

Рис. 14.27. Создание объекта XML SCHEMA COLLECTION, содержащего один документ XML-Schema

Документ XML-Schema, размещенный в коллекции **Student**, описывает XML-документ с корневым элементом **студент** (первый тег **element**).

На втором уровне (внутри тега **студент**) может быть расположено три тега: **паспорт**, **телефон** и **адрес** (вложенные теги **element**). Причем тег **паспорт** должен быть ровно один

(атрибуты **maxOccurs** и **minOccurs**); тег **телефон** является обязательным и может быть в количестве не более трех (атрибут **maxOccurs**); тег **адрес** тоже является обязательным, и количество таких тегов не должно быть более одного.

Элементы третьего уровня присутствуют только внутри элемента **адрес**. Это элементы: **страна**, **город**, **улица**, **дом**, **квартира**. Все эти элементы являются обязательными и должны присутствовать ровно один раз.

Данные в документе размещаются как значения атрибутов (теги **attribute**) или как значения, размещенные в теле элементов (**телефон**, **страна**, **город**, **улица**, **дом**, **квартира**). Тип данных, размещаемых в атрибутах или теле элементов данных, определяется значением атрибута **type**.

При создании таблицы для XML-столбца можно указать имя коллекции схем (рис. 14.28). Такой столбец называется типизированным столбцом.

```
use BSTU
go
drop table STUDENT;
go
create table STUDENT
(
     IDSTUDENT    integer  identity(1000,1)
                  constraint STUDENT_PK  primary key,
     IDGROUP      integer
                  constraint STUDENT_GROUP_FK
                  foreign key  references GROUPS(IDGROUP),
     NAME      nvarchar(100),
     BDAY      date,
     STAMP     timestamp,
     INFO      xml(STUDENT),     -- типизированный столбец XML-типа
     FOTO      varbinary
 );
```

Рис. 14.28. Создание таблицы, содержащей типизированный XML-столбец

Типизированный XML-столбец должен удовлетворять хотя бы одной схеме из связанной с ним коллекции.

На рис. 14.29 приведен пример попытки добавления строки в таблицу **STUDENT**, содержащую типизированный XML-столбец. При этом возникла ошибка из-за несоответствия вводимого XML-документа схеме (рис. 14.27). Обратите внимание, что в значении атрибута **дата** элемента **паспорт** допущена ошибка. В схеме (рис. 14.27) для атрибута **дата** указано регулярное выражение, задающее правильный формат значения.

В примере на рис. 14.29 возникла ошибка из-за отсутствия обязательного элемента **страна**, который должен присутствовать внутри элемента **адрес**.

```
use BSTU
go
insert into STUDENT (IDGROUP,NAME, BDAY, INFO)
   values (1, 'Горбач Елизавета Юрьевна',  '07.12.1995',
   '<студент>
    <паспорт серия="МР" номер="22223333" дата="x1.01.2000" />
    <телефон>44445555</телефон>
    <адрес>
         <город>Минск</город>
         <улица>Кирова</улица>
         <дом>52</дом>
         <квартира>76</квартира>
    </адрес>
    </студент>' );
```

```
XML Validation: Invalid simple type value: 'x1.01.2000'.
Location: /*:студент[1]/*:паспорт[1]/@*:дата
```

Рис. 14.29. Ошибка при вводе данных в типизированный XML-столбец (атрибут **дата** в теге **паспорт**)

```
use BSTU
go
insert into STUDENT (IDGROUP,NAME, BDAY, INFO)
   values (1, 'Горбач Елизавета Юрьевна',  '07.12.1995',
   '<студент>
    <паспорт серия="МР" номер="22223333" дата="01.01.2000" />
    <телефон>44445555</телефон>
    <адрес>
         <город>Минск</город>
         <улица>Кирова</улица>
         <дом>52</дом>
         <квартира>76</квартира>
    </адрес>
    </студент>' );
```

```
Msg 6965, Level 16, State 1, Line 1
XML Validation: Invalid content.
 Expected element(s): 'страна'.
Found: element 'город' instead.
Location: /*:студент[1]/*:адрес[1]/*:город[1].
```

Рис. 14.30. Ошибка при вводе данных в типизированный XML-столбец (отсутствует обязательный тег **страна**)

На рис. 14.31 приведен пример добавления в типизированный XML-столбец **INFO** таблицы **STUDENT** XML-документа, полностью удовлетворяющего схеме, содержащейся в коллекции схем с именем **STUDENT**.

```
use BSTU
go
insert into STUDENT (IDGROUP,NAME, BDAY, INFO)
   values (1, 'Горбач Елизавета Юрьевна',  '07.12.1995',
   '<студент>
    <паспорт серия="MP" номер="22223333" дата="01.01.2000" />
    <телефон>44445555</телефон>
    <адрес>
         <страна>Беларусь</страна>
         <город>Минск</город>
         <улица>Кирова</улица>
         <дом>52</дом>
         <квартира>76</квартира>
    </адрес>
    </студент>' );
 select NAME, INFO from STUDENT
 where IDSTUDENT = (select max(IDSTUDENT) from STUDENT);
```

NAME	INFO
Горбач Елизавета Юрьевна	<студент><паспорт серия="MP" номер="22223333" дата="01...

Рис. 14.31. XML-документ, удовлетворяющий схеме (рис. 14.27)

Для более детального знакомства с возможностями применения XML-технологий в MSS рекомендуется воспользоваться изданиями [5, 7].

ЛИТЕРАТУРА

1. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – М.: Вильямс, 2005. – 1316 с.
2. Жилинский, А. А. Самоучитель Microsoft SQL Server / А. А. Жилинский. – СПб.: БХВ-Петербург, 2009. – 240 с.
3. Хотек, М. Microsoft SQL Server 2008: Реализация и обслуживание. Учебный курс Microsoft / М. Хотек. – М.: Русская редакция, 2012. – 576 с.
4. Microsoft SQL Server 2008: руководство администратора для профессионалов / Б. Найт [и др]. – М.: Вильямс, 2010. – 944 с.
5. Хабибуллин, И. Ш. Самоучитель XML / И. Ш. Хабибуллин. – СПб.: БХВ-Петербург, 2003. – 336 с.
6. Браст, Э. Дж. Разработка приложений на основе Microsoft SQL Server 2005. Мастер-класс / Э. Дж. Браст, С. Форте. – М.: Русская редакция, 2007. – 880 с.
7. Библиотека MSDN [Электронный ресурс]. – 2012. – Режим доступа: http://msdn.microsoft.com/ru-ru/library / Microsoft – Дата доступа: 12.09.2013.

Приложение. Таблицы базы данных BSTU

Таблица П.1

Таблицы базы данных BSTU

Имя таблицы	Столбцы таблицы	Наименования столбцов, свойства столбцов
FACULTY (Факультеты)	FACULTY	код факультета ,PK, char(10). not null
	FACULTY_NAME	наименование факультета, varchar(50), default '???'
PROFESSION (Специальности)	PROFESSION	код специальности, PK, char(20). not null
	FACULTY	код факультета, FK(FACULTY), char(10). not null
	PROFESSION_NAME	наименование специальности, varchar(100), null
	QUALIFICATION	квалификация, varchar(50), null
PULPIT (Кафедры)	PULPIT	код кафедры, PK, char(20). not null
	PULPIT_NAME	наименование кафедры, varchar(100), null
	FACULTY	код факультета, FK(FACULTY), char(10). not null
TEACHER (Преподаватели)	TEACHER	код преподавателя ,PK, char(10). not null
	TEACHER_NAME	фамилия, имя, отчество преподавателя, varchar(100), null
	GENDER	пол, char(1), GENDER in ('м','ж')
	PULPIT	код кафедры, FK(PULPIT), char(10). not null
SUBJECT (Дисциплины)	SUBJECT	код дисциплины, PK, char(10). not null
	SUBJECT_NAME	наименование дисциплины, varchar(100), null, unique
	PULPIT	код кафеды, FK(PULPIT), char(20). not null
AUDITORIUM_TYPE (Типы учебных аудиторий)	AUDITORIUM_TYPE	код типа аудитории, PK, char(10). not null
	AUDITORIUM_TYPENAME	наименование типа аудитории, varchar(30), null
AUDITORIUM (Учебные аудитории)	AUDITORIUM	код аудитории, PK, char(20). not null
	AUDITORIUM_TYPE	код типа аудитории, FK(AUDITORIUM_TYPE), char(10). not null
	AUDITORIUM_CAPACITY	вместимость аудитории, int, default 1, check between 1 and 300
	AUDITORIUM_NAME	наименование аудитории, varchar(50), null
GROUPS (Студенческие группы)	IDGROUP	идентификатор группы, PK, int, not null
	FACULTY	код факультета, FK(FACULTY), char(10). not null
	PROFESSION	код специальности, FK(PROFESSION) , char(20). not null
	YEAR_FIRST	год поступления, smallint, YEAR_FIRST < текущий год +2
	COURSE	курс, вычисляемое поле, tinyint, вычисляется на основе текущей даты и значения YEAR_FIRST
STUDENT (Студенты)	IDGROUP	идентификатор группы, FK(GROUP), int, not null
	NAME	фамилия, имя, отчество, nvarchar(100)

	BDAY	дата рождения, date
	STAMP	штамп времени, timestamp
	INFO	дополнительная информация, xml, по умолчанию null
	FOTO	фотография, varbinary(max), по умолчанию null
PROGRESS (Успеваемость)	SUBJECT	предмет
	IDSTUDENT	
	PDATE	дата экзамена, date
	NOTE	оценка, integer check between 1 and 10

Таблица П.2

Содержимое таблицы FACULTY

FACULTY	FACULTY_NAME
ТТЛП	Технология и техника лесной промышленности
ТОВ	Технология органических веществ
ХТиТ	Химическая технология и техника
ИЭФ	Инженерно-экономический факультет
ЛесХоз	Лесохозяйственный факультет
ИДиП	Издательское дело и полиграфия

Таблица П.3

Содержимое таблицы PROFESSION

FACULTY	PROFESSION	PROFESSION_NAME	QUALIFICATION
ИДиП	1-40 01 02	Информационные системы и технологии	Инженер-программист-системотехник
ИДиП	1-47 01 01	Издательское дело	Редактор-технолог
ИДиП	1-36 06 01	Полиграфическое оборудование и системы обработки информации	Инженер-электромеханик
ХТиТ	1-36 01 08	Конструирование и производство изделий из композиционных материалов	Инженер-механик
ХТиТ	1-36 07 01	Машины и аппараты химических производств и предприятий строительных материалов	Инженер-механик
ЛХФ	1-75 01 01	Лесное хозяйство	Инженер лесного хозяйства
ЛХФ	1-75 02 01	Садово-парковое строительство	Инженер садово-паркового строительства
ЛХФ	1-89 02 02	Туризм и природопользование	Специалист в сфере туризма
ИЭФ	1-25 01 07	Экономика и управление на предприятии	Экономист-менеджер

ИЭФ	1-25 01 08	Бухгалтерский учет анализ и аудит	Экономист
ТТЛП	1-36 05 01	Машины и оборудование лесного комплекса	Инженер-механик
ТТЛП	1-46 01 01	Лесоинженерное дело	Инженер-технолог
ТОВ	1-48 01 02	Химическая технология органических веществ материалов и изделий	Инженер-химик-технолог
ТОВ	1-48 01 05	Химическая технология переработки древесины	Инженер-химик-технолог
ТОВ	1-54 01 03	Физико-химические методы и приборы контроля качества продукции	Инженер по сертификации

Таблица П.4

Содержимое таблицы PULPIT

PULPIT	PULPIT_NAME	FACULTY
ИСиТ	Информационных систем и технологий	ИДиП
ПОиСОИ	Полиграфического оборудования и систем обработки информации	ИДиП
БФ	Белорусской филологии	ИДиП
РИТ	Редакционно-издательских тенологий	ИДиП
ПП	Полиграфических производств	ИДиП
ЛВ	Лесоводства	ЛХФ
ОВ	Охотоведения	ЛХФ
ЛУ	Лесоустройства	ЛХФ
ЛЗиДВ	Лесозащиты и древесиноведения	ЛХФ
ЛКиП	Лесных культур и почвоведения	ЛХФ
ТиП	Туризма и природопользования	ЛХФ
ЛПиСПС	Ландшафтного проектирования и садово-паркового строительства	ЛХФ
ТЛ	Транспорта леса	ТТЛП
ЛМиЛЗ	Лесных машин и технологии лесозаготовок	ТТЛП
ТДП	Технологий деревообрабатывающих производств	ТТЛП
ТиДИД	Технологии и дизайна изделий из древесины	ТТЛП
ОХ	Органической химии	ТОВ
ТНХСиППМ	Технологии нефтехимического синтеза и переработки полимерных материалов	ТОВ
ХПД	Химической переработки древесины	ТОВ
ТНВиОХТ	Технологии неорганических веществ и общей химической технологии	ХТиТ
ХТЭПиМЭЕ	Химии, технологии электрохимических производств и материалов электронной техники	ХТиТ
МиАХиСП	Машин и аппаратов химических и силикатных производств	ХТиТ
ПиАХП	Процессов и аппаратов химических производств	ХТиТ
ЭТиМ	Экономической теории и маркетинга	ИЭФ
МиЭП	Менеджмента и экономики природопользования	ИЭФ

Таблица П.5

Содержимое таблицы TEACHER

TEACHER	TEACHER_NAME	GENDER	PULPIT
СМЛВ	Смелов Владимир Владиславович	м	ИСиТ
АКНВЧ	Акунович Станислав Иванович	м	ИСиТ
КЛСНВ	Колесников Виталий Леонидович	м	ИСиТ
ГРМН	Герман Олег Витольдович	м	ИСиТ
ЛЩНК	Лащенко Анатолий Павлович	м	ИСиТ
БРКВЧ	Бракович Андрей Игоревич	м	ИСиТ
ДТК	Дятко Александр Аркадьевич	м	ИСиТ
КБЛ	Кабайло Александр Серафимович	м	ИСиТ
УРБ	Урбанович Павел Павлович	м	ИСиТ
РМНК	Романенко Дмитрий Михайлович	м	ИСиТ
ПСТВЛВ	Пустовалова Наталия Николаевна	ж	ИСиТ
ГРН	Гурин Николай Иванович	м	ИСиТ
ЖЛК	Жиляк Надежда Александровна	ж	ИСиТ
БРС	Брусенцова Татьяна Палладьевна	ж	ИСиТ
КШК	Кишкурно Татьяна Вадимовна	ж	ИСиТ
МРЗ	Мороз Елена Станиславовна	ж	ИСиТ
БРТШВЧ	Барташевич Святослав Александрович	м	ПОиСОИ
ЮДНКВ	Юденков Виктор Степанович	м	ПОиСОИ
ШМК	Шмаков Михаил Сергеевич	м	ПОиСОИ
АРС	Арсентьев Виталий Арсентьевич	м	ПОиСОИ
БЛВ	Беляев Валерий Павлович	м	ПОиСОИ
БРНВСК	Барановский Станислав Иванович	м	ЭТиМ
НВРВ	Неверов Александр Васильевич	м	МиЭП
РВКЧ	Ровкач Андрей Иванович	м	ОВ
ДМДК	Демидко Марина Николаевна	ж	ЛПиСПС
БРГ	Бурганская Татьяна Минаевна	ж	ЛПиСПС
МШКВСК	Машковский Владимир Петрович	м	ЛУ
АТР	Атрощенко Олег Александрович	м	ЛУ
ЛБХ	Лабоха Константин Валентинович	м	ЛВ
РЖК	Рожков Леонид Николаевич	м	ЛВ
ЗВГЦВ	Звягинцев Вячеслав Борисович	м	ЛЗиДВ
БЗБРДВ	Безбородов Владимир Степанович	м	ОХ
ПРКПЧК	Прокопчук Николай Романович	м	ТНХСиППМ
НСКВЦ	Насковец Михаил Трофимович	м	ТЛ
МХВ	Мохов Сергей Петрович	м	ЛМиЛЗ
ЕЩНК	Ещенко Людмила Семеновна	м	ТНВиОХТ
ЖРСК	Жарский Иван Михайлович	м	ХТЭПиМЭЕ
ЧРН	Черная Наталья Виеторовна	ж	ХПД
ДЛН	Долинина Татьяна Николаевна	ж	СБУАиА

Таблица П.6

Содержимое таблицы SUBJECT

SUBJECT	SUBJECT_NAME	PULPIT
СУБД	Системы управления базами данных	ИСиТ
БД	Базы данных	ИСиТ
ИНФ	Информационные технологии	ИСиТ
ОАиП	Основы алгоритмизации и программирования	ИСиТ
ПЗ	Представление знаний в компьютерных системах	ИСиТ
ПСП	Программирование сетевых приложений	ИСиТ
МСОИ	Моделирование систем обработки информации	ИСиТ
ПИС	Проектирование информационных систем	ИСиТ
КГ	Компьютерная геометрия	ИСиТ
ПМАПЛ	Полиграфические машины, автоматы и поточные линии	ПОиСОИ
КМС	Компьютерные мультимедийные системы	ИСиТ
ОПП	Организация полиграфического производства	ПОиСОИ
ДМ	Дискретная математика	ИСиТ
МП	Математическое программирование	ИСиТ
ЛЭВМ	Логические основы ЭВМ	ИСиТ
ООП	Объектно-ориентированное программирование	ИСиТ
ЭП	Экономика природопользования	МиЭП
ЭТ	Экономическая теория	ЭТиМ
БЛЗиПсОО	Биология лесных зверей и птиц с осн. охотов.	ОВ
ОСПиЛПХ	Основы садовопаркового и лесопаркового хозяйства	ЛПиСПС
ИГ	Инженерная геодезия	ЛУ
ЛВ	Лесоводство	ЛЗиДВ
ОХ	Органическая химия	ОХ
ТРИ	Технология резиновых изделий	ТНХСиППМ
ВТЛ	Водный транспорт леса	ТЛ
ТиОЛ	Технология и оборудование лесозаготовок	ЛМиЛЗ
ТОПИ	Технология обогащения полезных ископаемых	ТНВиОХТ
ПЭХ	Прикладная электрохимия	ХТЭПиМЭЕ

Таблица П.7

Содержимое таблицы AUDITORIUM

AUDITORIUM	AUDITORIUM_ TYPE	AUDITORIUM_ CAPACITY	AUDITORIUM_ NAME
206-1	ЛБ-К	15	206-1
301-1	ЛБ-К	15	301-1
236-1	ЛК	60	236-1
313-1	ЛК-К	60	313-1
324-1	ЛК-К	50	324-1
413-1	ЛБ-К	15	413-1
423-1	ЛБ-К	90	423-1
408-2	ЛК	90	408-2
103-4	ЛК	90	103-4
105-4	ЛК	90	105-4
107-4	ЛК	90	107-4
110-4	ЛК	30	110-4
111-4	ЛК	30	111-4

114-4	ЛК-К	90	114-4
137-4	ЛК-К	60	132-7
132-4	ЛК	90	132-4
02Б-4	ЛК	90	02Б-4
229-4	ЛК	90	229-4
304-4	ЛБ-К	90	304-4
305-4	ЛК-К	90	305-5
314-4	ЛК	90	314-4
320-4	ЛК	90	320-4

Таблица П.8

Содержимое таблицы AUDITORIUM_TYPE

AUDITORIUM_TYPE	AUDITORIUM_TYPENAME
ЛБ-Х	Химическая лаборатория
ЛБ-К	Компьютерный класс
ЛБ-СК	Спец. компьютерный класс
ЛК	Лекционная
ЛК-К	Лекционная с уст. проектором

Таблица П.9

Содержимое таблицы GROUPS

FACULTY	PROFESSION	YEAR_FIRST
ИДиП	1-40 01 02	2013
ИДиП	1-40 01 02	2012
ИДиП	1-40 01 02	2011
ИДиП	1-40 01 02	2010
ИДиП	1-47 01 01	2013
ИДиП	1-47 01 01	2012
ИДиП	1-47 01 01	2011
ИДиП	1-36 06 01	2010
ИДиП	1-36 06 01	2013
ИДиП	1-36 06 01	2012
ИДиП	1-36 06 01	2011
ХТиТ	1-36 01 08	2013
ХТиТ	1-36 01 08	2012
ХТиТ	1-36 07 01	2011
ХТиТ	1-36 07 01	2010
ТОВ	1-48 01 02	2012
ТОВ	1-48 01 02	2011
ТОВ	1-48 01 05	2013
ТОВ	1-54 01 03	2012
ЛХФ	1-75 01 01	2013
ЛХФ	1-75 02 01	2012
ЛХФ	1-75 02 01	2011
ЛХФ	1-89 02 02	2012
ЛХФ	1-89 02 02	2011
ТТЛП	1-36 05 01	2013
ТТЛП	1-36 05 01	2012
ТТЛП	1-46 01 01	2012
ИЭФ	1-25 01 07	2013

ИЭФ	1-25 01 07	2012
ИЭФ	1-25 01 07	2010
ИЭФ	1-25 01 08	2013
ИЭФ	1-25 01 08	2012

Таблица П.10

Содержимое таблицы STUDENT

1	Манакова Анастасия Владимировна	09.12.1995
1	Хартанович Екатерина Александровна	11.03.1995
1	Горбач Елизавета Юрьевна	07.12.1995
1	Зыкова Кристина Дмитриевна	12.16.1995
1	Борисевич Ольга Анатольевна	09.11.1995
2	Силюк Валерия Ивановна	12.17.1994
2	Сергель Виолетта Николаевна	06.23.1994
2	Добродей Ольга Анатольевна	09.11.1994
2	Подоляк Мария Сергеевна	04.17.1994
2	Никитенко Екатерина Дмитриевна	08.31.1994
3	Яцкевич Галина Иосифовна	02.28.1993
3	Осадчая Эла Васильевна	07.12.1993
3	Акулова Елена Геннадьевна	02.16.1993
3	Муковозчик Надежда Вячеславовна	09.11.1993
3	Войтко Елена Андреевна	04.07.1993
4	Плешкун Милана Анатольевна	08.23.1992
4	Буянова Мария Александровна	02.26.1992
4	Мацкевич Надежда Валерьевна	01.19.1992
5	Логинова Мария Вячеславовна	08.27.1995
5	Белько Наталья Николаевна	02.15.1995
5	Дрозд Анастасия Андреевна	04.18.1995
6	Козловская Елена Евгеньевна	08.11.1994
6	Потапнин Кирилл Олегович	02.23.1994
6	Равковская Ольга Николаевна	04.16.1994
6	Ходоронок Александра Вадимовна	09.11.1994
6	Рамук Владислав Юрьевич	04.27.1994
7	Неруганенок Мария Владимировна	03.01.1993
7	Цыганок Анна Петровна	12.09.1993
7	Масилевич Оксана Игоревна	12.06.1993
7	Алексиевич Елизавета Викторовна	09.22.1993
7	Ватолин Максим Андреевич	04.17.1993
8	Синица Валерия Андреевна	08.01.1992
8	Кудряшова Алина Николаевна	12.05.1992
8	Ершов Юрий Олегович	12.27.1992
9	Астафьев Игорь Александрович	10.18.1995
9	Гайтюкевич Андрей Игоревич	02.15.1995
9	Рученя Наталья Александровна	08.21.1995
9	Тарасевич Анастасия Ивановна	11.09.1995
9	Скурат Наталья Ивановна	08.14.1995
9	Волосюк Николай Александрович	09.16.1995
10	Жоглин Николай Владимирович	08.01.1994
10	Санько Андрей Дмитриевич	11.09.1994
10	Гамеза Денис Валерьевич	11.02.1994)

11	Лавренчук Владислав Николаевич	07.11.1993
11	Власик Евгения Викторовна	04.06.1993
11	Савинко Павел Андреевич	08.21.1993
11	Бакун Алексей Викторович	02.09.1993
12	Бань Сергей Анатольевич	11.12.1995
12	Сечейко Илья Александрович	10.06.1995
12	Зизюк Ольга Олеговна	12.09.1995
13	Шарапо Мария Владимировна	08.10.1994
13	Касперович Вадим Викторович	10.02.1994
13	Чупрыгин Арсений Александрович	11.11.1994
13	Воеводская Ольга Леонидовна	10.02.1994
13	Метушевский Денис Игоревич	12.21.1994
14	Ловецкая Валерия Александровна	11.19.1993
14	Дворак Антонина Николаевна	01.12.1993
14	Фомичевская Елена Эрнестовна	01.07.1993
15	Бесараб Маргарита Вадимовна	07.14.1992
15	Бадуро Виктория Сергеевна	10.12.1992
15	Чуйкевич Ирина Дмитриевна	04.06.1992
16	Брель Алеся Алексеевна	08.21.1994
16	Кузнецова Анастасия Андреевна	07.02.1994
16	Томина Карина Геннадьевна	12.26.1994
16	Дуброва Павел Игоревич	03.17.1994
16	Шпаков Виктор Андреевич	04.07.1994
17	Шнейдер Анастасия Дмитриевна	08.11.1993
17	Шыгина Елена Викторовна	02.14.1993
17	Клюева Анна Ивановна	03.16.1993
17	Доморад Марина Андреевна	05.11.1993
17	Линчук Михаил Александрович	03.07.1993
18	Васильева Дарья Олеговна	08.21.1995
18	Щигельская Екатерина Андреевна	06.09.1995
18	Сазонова Екатерина Дмитриевна	08.23.1995
18	Бакунович Алина Олеговна	07.08.1995
18	Тарасова Дарья Николаевна	08.21.1995
18	Матиевская Анна Сергеевна	02.15.1995
19	Урбан Наталья Евгеньевна	08.16.1994
19	Никитенко Диана Валерьевна	08.07.1994
19	Черканович Дарья Леонидовна	03.10.1994
19	Торговцева Елена Михайловна	02.20.1994
19	Прокопчук Юлия Васильевна	01.30.1994
20	Протосюк Вероника Николаевна	07.03.1995
20	Нагорский Алексей Олегович	03.09.1995
20	Петренко Анна Андреевна	01.24.1995
20	Хилько Анастасия Андреевна	05.19.1995
20	Архипова Янина Игоревна	07.04.1995
20	Воробей Елена Сергеевна	08.16.1995)
21	Смагур Роман Витальевич	10.12.1994
21	Михнюк Светлана Николаевна	03.24.1994
21	Подобаева Кристина Анатольевна	09.11.1994
21	Казарская Лилия Владимировна	11.08.1994
21	Репикова Ольга Сергеевна	08.21.1994
21	Трушко Елена Георгиевна	12.02.1994

22	Стрельчик Яна Олеговна	08.13.1993
22	Трухин Станислав Алексеевич	11.03.1993
22	Успенская София Андреевна	09.17.1993
22	Календа Анна Николаевна	12.07.1993
22	Стефанович Екатерина Олеговна	08.23.1993
22	Зубарик Анна Викторовна	12.14.1993
23	Любченко Игорь Геннадьевич	06.10.1994
23	Бедняков Александр Викторович	06.12.1994
23	Сокольчик Андрей Александрович	07.19.1994
23	Шулинский Антон Олегович	04.12.1994
23	Селюжицкий Илья Сергеевич	01.01.1994
24	Грузинов Олег Александрович	09.28.1993

Таблица П.11

Содержимое таблицы PROGRESS

SUBJECT	IDSTUDENT	PDATE	NOTE
ОАиП	1000	01.10.2013	6
ОАиП	1001	01.10.2013	8
ОАиП	1002	01.10.2013	7
ОАиП	1003	01.10.2013	5
ОАиП	1005	01.10.2013	4
ОАиП	1006	01.10.2013	4
ОАиП	1007	01.10.2013	8
БД	1008	06.12.2013	4
БД	1009	06.12.2013	9
БД	1010	06.12.2013	5
БД	1011	06.12.2013	7
БД	1012	06.12.2013	5
СУБД	1013	01.18.2013	6
СУБД	1014	01.18.2013	5
СУБД	1015	01.18.2013	9
СУБД	1016	01.18.2013	5
СУБД	1017	01.18.2013	4
КГ	1018	06.15.2013	4
КГ	1019	06.15.2013	7
КГ	1020	06.15.2013	7
КГ	1021	06.15.2013	9
КГ	1022	06.15.2013	5
КГ	1023	06.15.2013	6
ОХ	1064	01.19.2013	6
ОХ	1065	01.19.2013	4
ОХ	1066	01.19.2013	9
ОХ	1067	01.19.2013	5
ОХ	1069	01.19.2013	4
ЭТ	1155	01.15.2013	7
ЭТ	1156	01.15.2013	8
ЭТ	1157	01.15.2013	9
ЭТ	1158	01.15.2013	4
ЭТ	1159	01.15.2013	5